Ceph Architecture and Installation



Foreword

- Ceph is the only cloud storage platform that can provide object, file, and block storage in one single system. This course introduces Ceph, its usage and architecture, and explains how to install and configure the Ceph cluster.

Objectives

- On completion of this course, you will understand:
  - Ceph architecture
  - Ceph components
  - Ceph installation
  - Ceph common component operations

HUAWEI

Contents

1. **Ceph Architecture**
2. Ceph and Component Installation
3. Ceph Cluster Management

HUAWEI

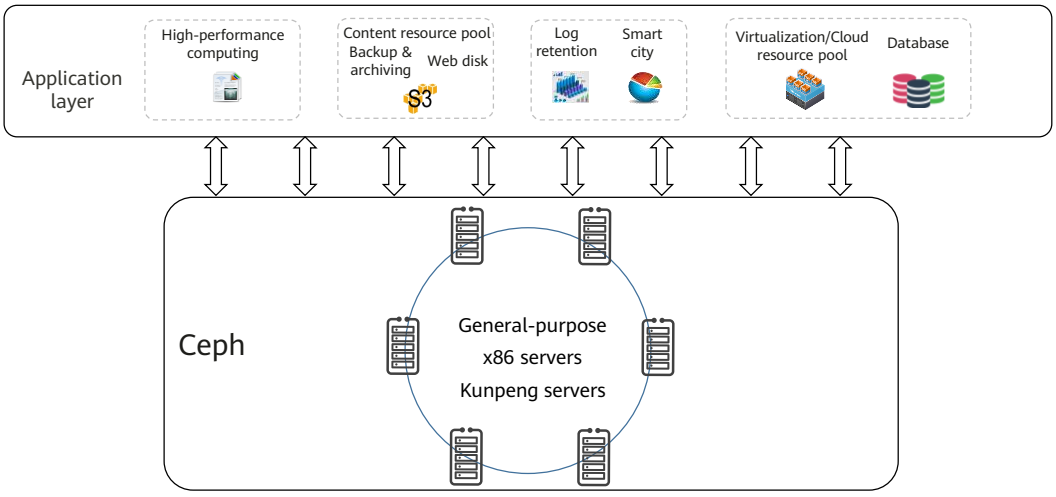
Ceph Overview

- Ceph is an open source SDS system for distributed storage. With high availability, scalability, and usability, Ceph is well suited for mass data storage.
- Ceph storage nodes can be deployed on general-purpose servers with x86 or Arm architecture CPUs, and used to replicate and reallocate stored data.

HUAWEI

- SDS refers to software-defined storage.
- Ceph can be deployed on Huawei TaiShan servers with Kunpeng processors. Kunpeng processors are typical Arm CPUs.

Ceph Application Scenarios



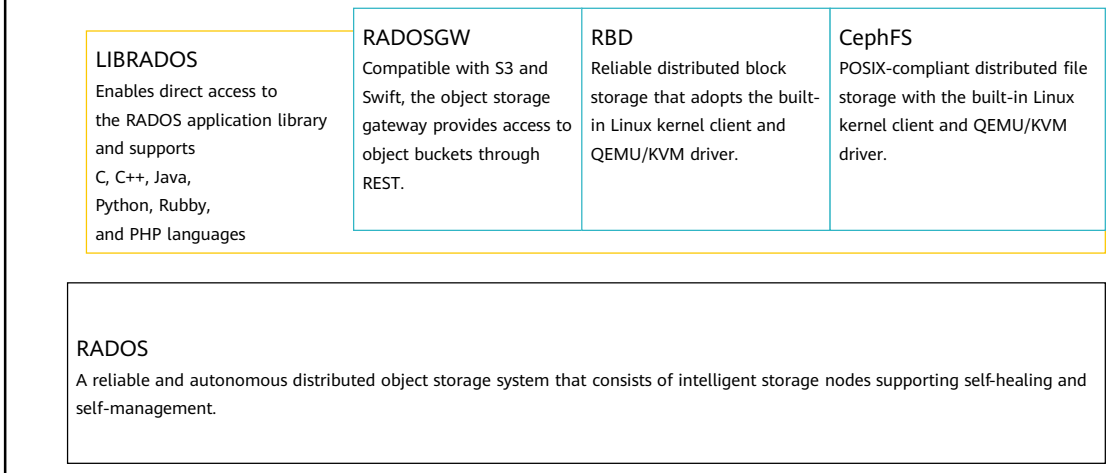
HUAWEI

Technical Advantages of Ceph

- The RADOS system stores all data as objects in the storage pool.
- The CRUSH algorithm calculates storage locations of objects.
- Its decentralization design enables the client end to calculate the object storage location using the CRUSH algorithm and then read and write data.
- It also enables automatic cluster expansion, data rebalancing, and data recovery.

HUAWEI

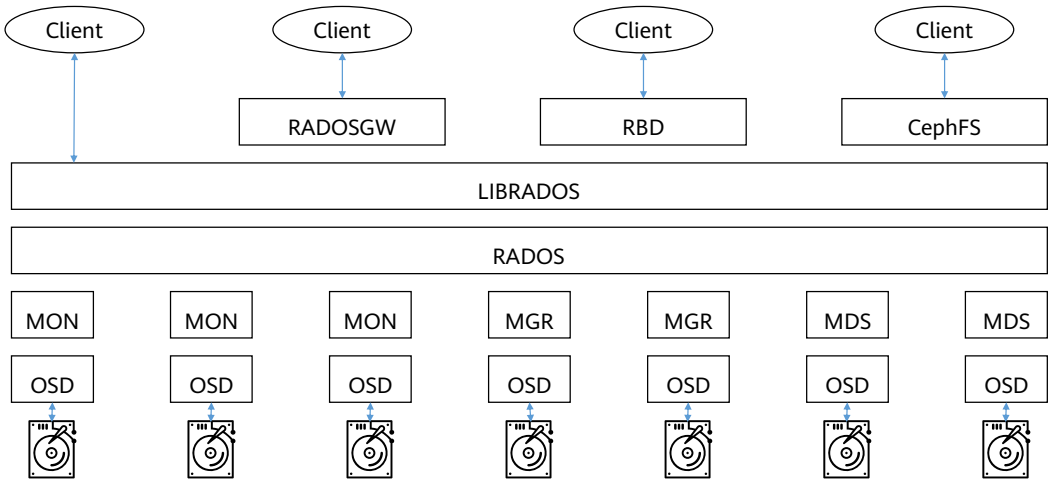
Ceph Architecture



HUAWEI

- RADOS is short for **R**eliable **A**utonomic **D**istributed **O**bject **S**tore. It is the base of Ceph. Its intelligent storage nodes achieve self-healing and self-management.
- Ceph provides multiple interfaces.
  - LIBRADOS: RADOS library.
  - RADOSGW: RADOS gateway, which provides external object storage interfaces.
  - RBD: RADOS block device, which provides external block storage interfaces.
  - CephFS: provides external file storage interfaces.

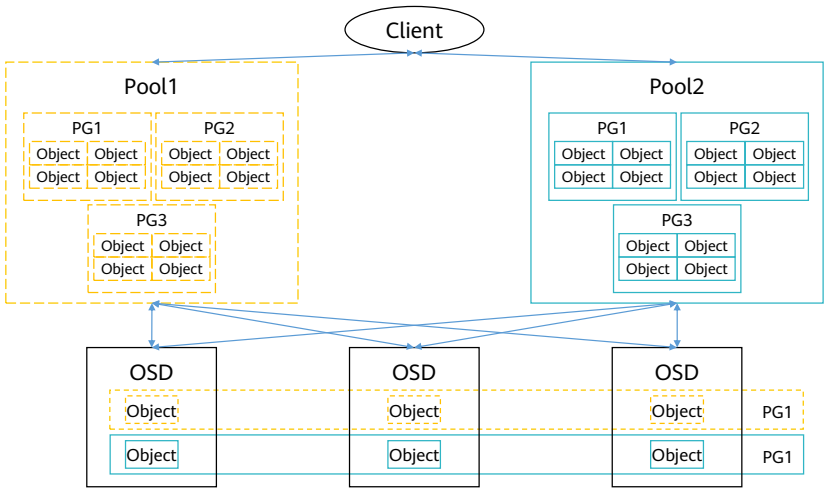
Ceph Component Overview



HUAWEI

- MON, MGR, MDS, and OSD are backend storage components of Ceph.
- RADOSGW, RBD, CephFS, and LIBRADOS are Ceph access modes.

Ceph Data Storage Mechanism



HUAWEI

Ceph Component – Monitor

- Monitors are used to maintain the "Cluster Map" of the Ceph cluster. The cluster map guides Ceph component interaction.
- Cluster map enables Ceph clients to find all Ceph monitors, Ceph object storage daemons (OSDs), and Ceph metadata servers (MDSs). The cluster map includes:
  - Monitor map
  - OSD map
  - PG map
  - CRUSH map
  - MDS map
- To ensure high availability, monitors are deployed in a cluster. In addition, certain algorithms and the majority rule are used to maintain the data consistency among monitors.

HUAWEI

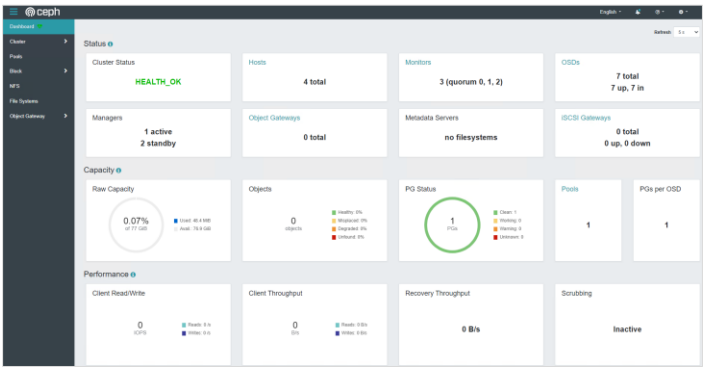
- Monitors ensure data consistency by complying with majority rule, for example, two out of three, three out of five, and four out of six.
- The process corresponding to the monitor is called MON.
- In production environment, the number of MON nodes is an odd number in order to use the voting arbitration.
- Monitor map contains:
  - Cluster fsid. fsid is an automatically generated and unique UUID (universally unique identifier) used to distinguish the Ceph cluster.
  - Location, name, address, and port number of each MON
  - Timestamp
  - Run **ceph mon dump** to view the monitor map information.
- OSD map contains:
  - Cluster fsid
  - Storage pool list
  - Number of copies
  - Number of PGs
  - OSD list and status

- Timestamp
- Run **ceph osd dump** to view the OSD map information.

- PG map contains:
  - PG version
  - Space usage
  - PG details, such as the PG ID, PG status, and resource pool data usage
  - Timestamp
  - Run **ceph pg dump** to view the PG map information.
- CRUSH map contains:
  - Storage device list
  - Failure domain level (device level, host level, cabinet level, cabinet column level, and equipment room level)
  - Data restoration rules
  - Run **ceph osd crush** to view the CRUSH Map information.
- MDS map contains:
  - Metadata storage pool information
  - Metadata server list
  - Metadata server status
  - Timestamp
  - Run **ceph fs dump** to view the FS Map information.

Ceph Component – Manager

- The Ceph manager daemon (ceph-mgr) provides a graphical UI that displays the cluster running status and connects to third-party management plug-ins, such as Prometheus and Zabbix.
- Managers are not used to read and write data, but cluster data cannot be queried without managers. You are advised to configure at least two managers for each cluster.



HUAWEI

Ceph Component – MDS

- Metadata server is also called ceph-mds or just MDS. CephFS uses MDS to store the file system metadata.
- MDS uses RADOS to store metadata. Therefore, during MDS configuration, the system requires to create two storage pools, metadata and data storage pools.
- MDS loads metadata to the memory to accelerate file system access.

Ceph Component – OSD

- Ceph Object Storage Daemon (OSD) is the building unit and data storage location of the Ceph storage cluster. Multiple OSDs run on Ceph storage nodes and store data in raw disks on these storage nodes. Generally, one OSD corresponds to one raw disk.
- Ceph uses the CRUSH algorithm to store objects to OSDs and optimize performance.
- CRUSH, or Controlled Replication Under Scalable Hashing, calculates object storage locations and associates these objects with placement groups (PGs). Specifically, it distributes objects in different PGs using a pseudo-random algorithm, and calculates the mapping between OSDs and PGs. In the event of a failure, Ceph maps PGs to healthy OSDs and synchronizes data based on the configured data protection policy.
- Replication policies are used to store data to different OSDs to ensure data security. Only one OSD is selected as the primary OSD, and other OSDs are replica OSDs. They have different functions in Ceph. When a client reads or writes data, the client communicates only with the primary OSD.
- An OSD can be in any of the following states:
  - Up and in: The OSD is healthy and carries at least one PG.
  - Up and out: The OSD is healthy but does not carry any PG. The OSD newly added to the cluster is in this state.
  - Down and in: The OSD is not running properly but carries at least one PG. The OSD in this state may be restored to the up and in state if treated properly.
  - Down and out: The OSD is not running properly and does not carry any PG.

HUAWEI

- In early Ceph versions, OSDs use BlueStore to store data in raw disks or FileStore to store data in file systems of non-raw disks. Since Ceph Luminous, BlueStore has become the default storage mode. After Ceph Reef, FileStore is no longer supported. Now, data can only be stored in raw disks, which brings OSD storage performance close to that of raw devices.
- The primary OSD has the following functions:
  - Responds to all I/O requests.
  - Replicates and protects data.
  - Checks data consistency.
  - Re-balances and restores data.
- The replica OSD has the following functions:
  - Receives instructions from the primary OSD.
  - Upgrades as a new primary OSD if the old one is faulty.

Contents

1. Ceph Architecture
2. **Ceph and Component Installation**
3. Ceph Cluster Management

HUAWEI

Ceph Installation Methods

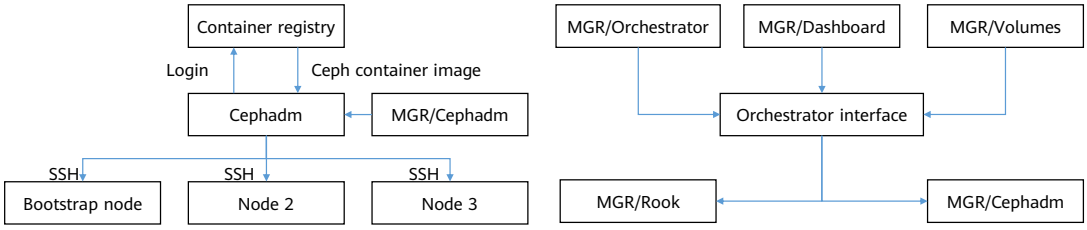
- Officially Recommended Methods
  - Cephadm: deploys Ceph on containers, on which you can manage the Ceph cluster via the command line interface (CLI) or graphical user interface (GUI). It supports Ceph Octopus and later versions.
  - Rook: deploys Ceph based on Kubernetes and gives you a CLI or GUI management platform to use Kubernetes to manage the pod containing the Ceph component. It supports Ceph Nautilus and later versions.
- Other Methods
  - ceph-ansible
  - ceph-deploy
  - DeepSea
  - Manual installation

HUAWEI

- ceph-ansible uses Ansible to deploy Ceph.
- ceph-deploy is no longer maintained and is therefore not recommended.
- DeepSea uses SaltStack to deploy Ceph.

Cephadm Overview

- Cephadm is a Ceph lifecycle management tool. It creates a single-node cluster that contains one MON and one MGR by bootstrapping. After that, the built-in orchestrator interface can be used to expand the cluster capacity, add hosts, and deploy services.
- Cephadm provides the following functions:
  - Adds a container-based Ceph cluster.
  - Removes a Ceph container from a Ceph cluster.
  - Updates the Ceph container.



Ceph Cluster Minimum Hardware Specifications

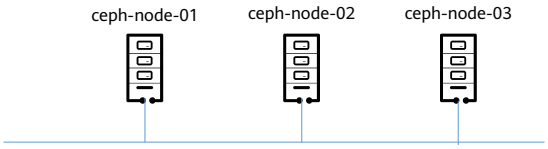
Component	Resource	Specifications
ceph-osd	CPU	At least one core
	Memory	4 GB
	Storage space	One whole drive
	High-speed and ultra-high-speed space	One SSD partition
	Network	At least 1 GB. 10 GB is recommended.
ceph-mon	CPU	At least two cores
	Memory	2 to 4 GB
	Drive space	60 GB
	Network bandwidth	More than 1 GB
ceph-mds	CPU	At least two cores
	Memory	At least 2 GB
	Drive space	1M
	Network bandwidth	More than 1 GB

HUAWEI

- The minimum hardware specifications refer to specifications used in actual production. The specifications in the Lab Guide are more relaxed than those in this table.
- About the OSD resources in this table, please note:
  - Each time a service is added, the listed resources need to be added.
  - The OSD CPU needs to be at least one core. CPU resources can be adjusted to meet bandwidth or IOPS demand. One core is added for every 200 MB/s to 500 MB/s or every 1000 IOPS to 3000 IOPS.
  - The OSD memory can be a value between 2 GB and 4 GB. Basic functions can be implemented, but sometimes stuttering occurs.

Lab: Ceph Cluster Topology

- Three Elastic Cloud Servers (ECSs) are used in this lab.
- ceph-node-*x* indicates the host name of each ECS. ceph-node-01, ceph-node-02, and ceph-node-03 are used to deploy management hosts. ceph-node-01 functions as the bootstrap node.
- Three 10 GB Elastic Volume Service (EVS) disks are attached to all hosts except the system disk.



HUAWEI

Using Cephadm to Deploy a Ceph Cluster

- Preparing for installation
- Installing Cephadm
- Downloading the Ceph Yum source file
- Creating a Ceph cluster
- Adding nodes
- Adding components

Preparation – Configuring Host Names and Host Name Resolution

- All hosts in a cluster must be in the same region. The host name format is *<Host name>.<Domain name>*.
  - The host name can be set as node-01, ceph-node-01 or ceph01.
  - The host name and domain name are separated by a period (.), and periods cannot be used in the host name.
- You can add a host by using the host name or by using the host name and IP address.
- In the production environment, you are advised to use the internal DNS to resolve host names.

HUAWEI

- Adding a host requires a complete host name. A complete host name should be resolved to a corresponding address.

Preparation – Setting the Firewall and SELinux

Service	Port	Description
MON	6789, 3300	Used for internal communication in the Ceph cluster.
OSD	6800–7300	Each OSD occupies four ports, one to communicate with the client and MON over the public network, one to communicate with other OSDs over the cluster or public network, and two for exchanging heartbeat data packets over the cluster or public network.
MDS	6800–7300	Used to communicate with the metadata server.
MGR	8443	Used to log in to the Ceph GUI in SSL mode.
RESTful management module	8003	Used to communicate with the RESTful management module in SSL mode.
Prometheus management module	9283	Used to communicate with the Ceph Prometheus plug-in.
Prometheus alarm management	9093	Used to communicate with the alarm management service of Prometheus.
Prometheus node exporter	9100	Used to communicate with the Prometheus node exporter process.
Grafana server	3000	Used to communicate with the Grafana service.
RGW	80	Used for RADOSGW communication. If **client.rgw** is left blank, Cephadm uses the default port 80.
Ceph iSCSI gateway	9287	Used to communicate with the Ceph iSCSI gateway.

HUAWEI

- In the Lab Guide, firewalld and SELinux are disabled to streamline the installation.
- All ports use the TCP protocol.

Preparation – Synchronizing System Clock

- Use chrony or NTP to synchronize the system clocks on the Ceph cluster nodes.
- You are advised to deploy a clock server on the intranet to prevent time asynchronization caused by network delay when the time is being synchronized with the Internet clock server.

```
[root@ceph-node-01 ~]# chronyc sources
MS Name/IP address         Stratum Poll Reach LastRx Last sample
===============================================================================
^- tock.ntp.infomaniak.ch        1   6    17    81  -1700us[+3047us] +/-   94ms
^- stratum2-1.ntp.mow01.ru.>     2   6   225    74  -4825us[-1257us] +/-   67ms
^+ ntp5.flashdance.cx            2   6    37    17  +3474us[+7042us] +/-  137ms
^* ntp8.flashdance.cx            2   6    37    15  -1363us[+2205us] +/-  133ms
```


HUAWEI

- In the Lab Guide, chrony is used to synchronize time. The lab is deployed on Huawei Cloud to avoid network delay, and the time server is not set up.

Installing and Configuring Cephadm

- Cephadm is an orchestration tool used to install and manage Ceph. It is developed with Python 3, and thus requires Python 3 to be installed on the Ceph node.
- The Yum source provided by openEuler can directly install Cephadm.
- Currently, Cephadm does not support openEuler. You need to download the specified Cephadm script to enable and use openEuler.

HUAWEI

Configuring the Yum Source

- Yum source required by Ceph can be manually configured or automatically configured using Cephadm. The latter method is recommended and its command is **cephadm add-repo --release** *version*.

```
[root@ceph-node-01 ~]# cephadm add-repo --release pacific
Writing repo to /etc/yum.repos.d/ceph.repo...
Enabling EPEL...
Completed adding repo.
[root@ceph-node-01 ~]# yum repolist
repo id                                                 repo name
Ceph                                                    Ceph x86_64
Ceph-noarch                                             Ceph noarch
Ceph-source                                             Ceph SRPMS
EPOL                                                    EPOL
OS                                                      OS
debuginfo                                               debuginfo
docker-ce-stable                                        Docker CE Stable
everything                                              everything
source                                                  source
update                                                  update
```


HUAWEI

- In openEuler, the EPEL is replaced by the built-in EPOL. An error is reported when the EPEL is pulled, which can be ignored.

Creating a Ceph Cluster

- Cephadm uses the **bootstrap** option to deploy clusters and sets deployment specifications via parameters. Common parameters are as follows:
  - **--docker**: The Docker container is used as the container engine that carries Ceph services. The default value is **podman**.
  - **--mon-ip**: specifies the IP address of the node where the MON daemon is deployed.
  - **--initial-dashboard-user**: sets the GUI user.
  - **--initial-dashboard-password**: sets password for logging in to the GUI.
  - **--dashboard-password-noupdate**: cancels the function of forcibly initializing password upon the first GUI login.
  - **--allow-fqdn-hostname**: sets the host name as the login address of the GUI.
  - **--orphan-initial-daemons**: cancels the automatic deployment of the MON and MGR daemons.

```
[root@ceph-node-01 ~]# cephadm --docker bootstrap --mon-ip 192.168.0.21 --allow-fqdn-hostname --initial-dashboard-user admin --initial-dashboard-password Huawei@123 --dashboard-password-noupdate
```


HUAWEI

- Note: **--docker** is an option of the **cephadm** command, not **bootstrap**.

Adding Nodes

- Ceph uses the shared key for identity authentication. You can run **ceph cephadm get-pub-key** to obtain the SSH public key required for connecting the host to the cluster.
- After obtaining the public key, use it to manage nodes in password-free SSH mode.
- Run **ceph orch host add** to add a host.
- Ceph supports hybrid deployment of x86 and Kunpeng hosts.

```
[root@ceph-node-01 ~]# ceph cephadm get-pub-key > ~/ceph.pub
[root@ceph-node-01 ~]# ssh-copy-id -f -i ~/ceph.pub root@ceph-node-05
......
[root@ceph-node-01 ~]# ceph orch host add ceph-node-05.novalocal 192.168.0.25
Added host 'ceph-node-05.novalocal' with addr '192.168.0.25'
[root@ceph-node-01 ~]# ceph orch host ls
HOST                    ADDR          LABELS  STATUS
ceph-node-01.novalocal  192.168.0.21  _admin
ceph-node-02.novalocal  192.168.0.22  _admin
ceph-node-03.novalocal  192.168.0.23  _admin
ceph-node-05.novalocal  192.168.0.25
4 hosts in cluster
```


HUAWEI

- Ceph supports both x86 and Kunpeng hosts in the same cluster.
- In this slide, ceph-node-01, ceph-node-02 and ceph-node-0 are x86 hosts, and ceph-node-05 is a Kunpeng host.

Adding Services

- You can use the Ceph orchestrator to add components in either of the following ways:
  - ceph orch apply *<service_type> [--placement=<placement_string>] [--unmanaged]*
  - ceph orch daemon add *<daemon_type> <placement>*

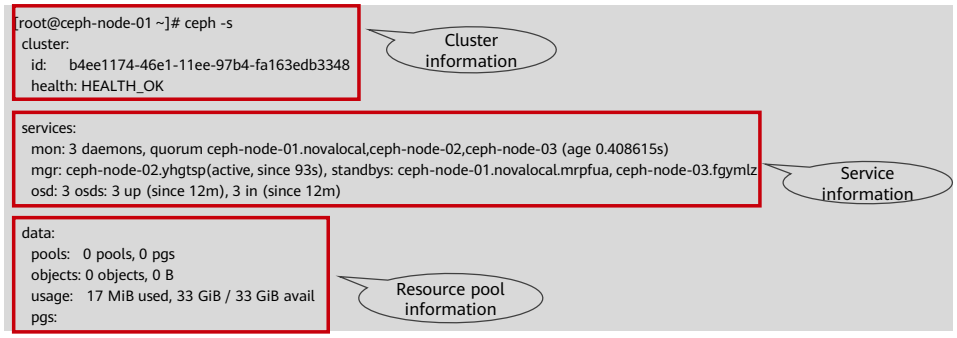
```
[root@ceph-node-01 ~]# ceph orch ls
......
mon                        3/5  2m ago     23h  count:5
[root@ceph-node-01 ~]# ceph orch daemon add mon --placement="ceph-node-05.novalocal"
Deployed mon.ceph-node-05 on host 'ceph-node-05.novalocal'
[root@ceph-node-01 ~]# ceph orch ls
NAME           PORTS        RUNNING  REFRESHED  AGE  PLACEMENT
.............
mon                        4/5  8m ago     14s  count:5
[root@ceph-node-01 ~]# ceph orch apply mon --placement="label:_admin"
Scheduled mon update...
[root@ceph-node-01 ~]# ceph orch ls
NAME           PORTS        RUNNING  REFRESHED  AGE  PLACEMENT
.....
mon                        3/3  10s ago    7s   label:_admin
```


HUAWEI

- **apply** defines the service status and the orchestrator automatically find nodes for service deployment accordingly; **daemon add** directly deploys services based on parameters in the command.
- In this slide, only ceph-node-01, ceph-node-02 and ceph-node-0 are configured with the **_admin** label. After the **ceph orch apply** command is used to configure related policies, the number and the recommended number of MONs are changed to **3**.
- The **orch apply osd [--all-available-devices]** command can be used to configure all available devices on a node as OSDs.

Checking the Cluster Status

- Run **ceph -s** to check the cluster status.



HUAWEI

- **ceph -s** corresponds to **ceph --status**.
- The health status of a cluster can be classified into the following types:
  - **HEALTH_OK**: The health status is good.
  - **HEALTH_WARN**: There is a warning in the cluster. Rectify the fault and then the status changes to **HEALTH_OK**.
  - **HEALTH_ERR**: There is a critical error in the cluster that must be handled immediately.
- In the service Information section, you can check the status of MON, MGR, and OSD, including the quantity, locations, and running time.

Contents

1. Ceph Architecture
2. Ceph and Component Installation
3. **Ceph Cluster Management**

HUAWEI

Configuring Ceph Configuration Nodes

- A configuration node has the permission to modify Ceph nodes.
- To be a Ceph configuration node, the node must:
  - Run the Ceph client configuration tool ceph-common.
  - Have the **ceph.conf** and **ceph.client.admin.keyring** files of the managed cluster in the **/etc/ceph** directory on the node.

```
# minimal ceph.conf for b4ee1174-46e1-11ee-97b4-fa163edb3348
[global]
        fsid = b4ee1174-46e1-11ee-97b4-fa163edb3348
        mon_host = [v2:192.168.0.21:3300/0,v1:192.168.0.21:6789/0] [v2:192.168.0.22:3300/0,v1:192.168.0.22:6789/0]
[v2:192.168.0.23:3300/0,v1:192.168.0.23:6789/0]
```

```
[client.admin]
        key = AQCDse5kyngBJBAAapN5s2wMhJnyzYBU85q/Bw==
        caps mds = "allow *"
        caps mgr = "allow *"
        caps mon = "allow *"
        caps osd = "allow *"
```


HUAWEI

- Ceph cluster management is implemented by giving commands to MONs. The **ceph.conf** file stores all MON host information, and the **ceph.client.admin.keyring** file stores authentication and authorization information.

Viewing Cluster Hosts

- Run **ceph orch host ls** to query the hosts in the current cluster.

```
[root@ceph-node-01 ceph]# ceph orch host ls
HOST                        ADDR          LABELS  STATUS
ceph-node-01.novalocal  192.168.0.21  _admin
ceph-node-02.novalocal  192.168.0.22
ceph-node-03.novalocal  192.168.0.23
ceph-node-04.novalocal  192.168.0.24
4 hosts in cluster
```

Host label

- Run **ceph orch host label add** *<hostname>* *<label>* to add a label to the host.

```
[root@ceph-node-01 ceph]# ceph orch host label add ceph-node-02.novalocal _admin
Added label _admin to host ceph-node-02.novalocal
[root@ceph-node-01 ceph]# ceph orch host label add ceph-node-03.novalocal _admin
Added label _admin to host ceph-node-03.novalocal
[root@ceph-node-01 ceph]# ceph orch host ls
HOST                ADDR          LABELS  STATUS
ceph-node-01.novalocal  192.168.0.21  _admin
ceph-node-02.novalocal  192.168.0.22  _admin
ceph-node-03.novalocal  192.168.0.23  _admin
......
```


HUAWEI

- **ceph orch** provides a CLI for the orchestrator module. The orchestrator module is a ceph-mgr plug-in and interacts with external orchestration services.
- **ceph orch** defines multiple external orchestrators.
  - host: the host name of the physical node
  - service type: the type of the service, for example, nfs, mds, osd, mon, rgw, mgr, and iscsi
  - service: a logical service
  - daemon: a service instance
- Special labels of Cephadm:
  - **_no_schedule**: No service is deployed or scheduled on nodes with this label.
  - **_no_autotune_memory**: Memory tuning is not performed on nodes with this label.
  - **_admin**: **ceph.conf** and **ceph.client.admin.keyring** are automatically sent to nodes with this label.

Viewing Cluster Services

- Run **ceph orch ls** to list the current cluster services.

```
[root@ceph-node-02 ~]# ceph orch ls
NAME                       PORTS          RUNNING  REFRESHED  AGE  PLACEMENT
alertmanager               ?:9093,9094     1/1      5m ago     5d   count:1
crash                                      4/4      8m ago     4d   *
grafana                    ?:3000          1/1      5m ago     5d   count:1
mgr                                        3/3      6m ago     4d   label:_admin
mon                                        3/3      6m ago     3d   label:_admin
node-exporter              ?:9100          4/4      8m ago     4d   *
osd                                        3        6m ago     -    <unmanaged>
osd.all-available-devices                  0        -          4h   <unmanaged>
prometheus                 ?:9095          1/1      5m ago     5d   count:1
```

- The key parameters in the command output are as follows:
  - **RUNNING**: service running status. In terms of its values, the first digit is the number of running services, and the second digit is the number of services recommended by the system based on policies or configurations.
  - **PLACEMENT**: a parameter provided by the service orchestrator during service deployment. The orchestrator can determine the node where the service is deployed based on this parameter. Common placements are as follows.
    - Node name, for example, **--placement=ceph-node-02**
    - Label, for example, **--placement="label:mylabel"**
    - Quantity, for example, **--placement="2 host1 host2 host3"**
    - **unmanaged** indicates that the service is not automatically deployed. You can run **--unmanaged=true** to enable this function and **--unmanaged=false** to disable it.

HUAWEI

Case: Deploying Services on Demand

- Requirement: ceph-node-01 is the bootstrapping node. Configure ceph-node-02 and ceph-node-03 as the configuration nodes, and deploy MON and MGR only on ceph-node-01, ceph-node-02, and ceph-node-03. The following figures show the cluster status before the deployment.
- Implementation process:
  - 1. Check the cluster status.

```
[root@ceph-node-01 ~]# ceph orch ls
NAME                       PORTS        RUNNING  REFRESHED  AGE  PLACEMENT
alertmanager               ?:9093,9094      1/1  66s ago    5d   count:1
crash                                       5/5  8m ago     5d   *
grafana                    ?:3000           1/1  66s ago    5d   count:1
mgr                                         2/2  6m ago     19m  <unmanaged>
mon                                         5/5  8m ago     19m  <unmanaged>
node-exporter              ?:9100           5/5  8m ago     5d   *
osd                                           3  5m ago     -    <unmanaged>
osd.all-available-devices                     7  8m ago     9m   *
prometheus                 ?:9095           1/1  66s ago    5d   count:1
```

  - 2. Check the host status in the current cluster.

```
[root@ceph-node-01 ~]# ceph orch host ls
HOST                   ADDR          LABELS  STATUS
ceph-node-01.novalocal  192.168.0.21  _admin
ceph-node-02.novalocal  192.168.0.22
ceph-node-03.novalocal  192.168.0.23
ceph-node-04.novalocal  192.168.0.24
ceph-node-05.novalocal  192.168.0.25
5 hosts in cluster
```

Continued in the next slide.

HUAWEI

Case: Deploying Services on Demand

- 3. Add the **_admin** label to ceph-node-02 and ceph-node-03.

```
[root@ceph-node-01 ~]# ceph orch host label add ceph-node-02.novalocal _admin
Added label _admin to host ceph-node-02.novalocal
[root@ceph-node-01 ~]# ceph orch host label add ceph-node-03.novalocal _admin
Added label _admin to host ceph-node-03.novalocal
```

- 4. Run **ceph orch apply** *<server-name>* **--placement= "label:_admin"** to deploy the service on the specified node.

```
[root@ceph-node-01 ~]# ceph orch apply mon --placement="label:_admin"
Scheduled mon update...
[root@ceph-node-01 ~]# ceph orch apply mgr --placement="label:_admin"
Scheduled mgr update...
```

- 5. Check the host status and cluster status after the deployment.

```
[root@ceph-node-01 ~]# ceph orch host ls
HOST                  ADDR           LABELS  STATUS
ceph-node-01.novalocal  192.168.0.21  _admin
ceph-node-02.novalocal  192.168.0.22  _admin
ceph-node-03.novalocal  192.168.0.23  _admin
ceph-node-04.novalocal  192.168.0.24
ceph-node-05.novalocal  192.168.0.25
```

```
[root@ceph-node-01 ~]# ceph orch ls
NAME                   PORTS        RUNNING  REFRESHED  AGE  PLACEMENT
alertmanager           ?:9093,9094      1/1  35s ago    5d   count:1
crash                                   5/5  66s ago    5d   *
grafana                ?:3000           1/1  35s ago    5d   count:1
mgr                                     3/3  53s ago    70s  label:_admin
mon                                     3/3  53s ago    81s  label:_admin
node-exporter          ?:9100           5/5  66s ago    5d   *
osd                                       3  53s ago    -    <unmanaged>
osd.all-available-devices                 7  66s ago    18m  *
prometheus             ?:9095           1/1  35s ago    5d   count:1
```


HUAWEI

- Question: How do you verify that ceph-node-02 and ceph-node-03 have become configuration nodes?
- Answer: Install ceph-common on the two nodes, and then run **ceph -s** to confirm the command can be executed.

Querying Cluster Service Details

- Run **ceph orch ps** to list the current cluster services.

```
[root@ceph-node-01 ~]# ceph orch ps
NAME                                HOST                  PORTS        STATUS         REFRESHED  AGE  MEM USE  MEM LIM  VERSION  IMAGE ID      CONTAINER ID
alertmanager.ceph-node-01           ceph-node-01.novalocal  *:9093,9094  running (2h)     9m ago   5d    23.3M        -  0.23.0   ba2b418f427c  c85e173bde6e
crash.ceph-node-01                  ceph-node-01.novalocal               running (3h)     9m ago   4d    15.8M        -  16.2.13  9e29efbb67d5  201f03258773
crash.ceph-node-02                  ceph-node-02.novalocal               running (3h)    29s ago   5d    9.85M        -  16.2.13  9e29efbb67d5  bfff37bad266
crash.ceph-node-03                  ceph-node-03.novalocal               running (5h)    29s ago   4d    16.2M        -  16.2.13  9e29efbb67d5  dab0da0075b4
crash.ceph-node-04                  ceph-node-04.novalocal               running (4h)     2m ago   4h    6336k        -  16.2.13  9e29efbb67d5  f9f929648d39
grafana.ceph-node-01                ceph-node-01.novalocal  *:3000       running (3h)     9m ago   5d     100M        -  8.3.5    dad864ee21e9  f1007c15959c
mgr.ceph-node-01.novalocal.mrpfua   ceph-node-01.novalocal  *:9283       running (3h)     9m ago   5d     468M        -  16.2.13  9e29efbb67d5  3183508aa331
mgr.ceph-node-02.rmbckr             ceph-node-02.novalocal  *:8443,9283  running (2h)    29s ago   2h     379M        -  16.2.13  9e29efbb67d5  d12a50f8eefb
mgr.ceph-node-03.yzeypq             ceph-node-03.novalocal  *:8443,9283  running (2h)    29s ago   2h     379M        -  16.2.13  9e29efbb67d5  1bda2a4daac0
mon.ceph-node-01.novalocal          ceph-node-01.novalocal               running (3h)     9m ago   5d     683M    2048M  16.2.13  9e29efbb67d5  dcbee8d39d5b
mon.ceph-node-02                    ceph-node-02.novalocal               running (2h)    29s ago   2h     288M    2048M  16.2.13  9e29efbb67d5  130ebb47d859
mon.ceph-node-03                    ceph-node-03.novalocal               running (5h)    29s ago   4d     903M    2048M  16.2.13  9e29efbb67d5  3755fd97aad6
node-exporter.ceph-node-01          ceph-node-01.novalocal  *:9100       running (3h)     9m ago   4d    32.4M        -  1.3.1    1dbe0e931976  22ba1d710797
node-exporter.ceph-node-02          ceph-node-02.novalocal  *:9100       running (3h)    29s ago   5d    32.3M        -  1.3.1    1dbe0e931976  9f83a575d8bd
node-exporter.ceph-node-03          ceph-node-03.novalocal  *:9100       running (5h)    29s ago   4d    29.6M        -  1.3.1    1dbe0e931976  a450c67f2691
node-exporter.ceph-node-04          ceph-node-04.novalocal  *:9100       running (4h)     2m ago   4h    22.4M        -  1.3.1    1dbe0e931976  4b8672d27f3b
osd.0                               ceph-node-02.novalocal               running (3h)    29s ago   4h    32.7M    4096M  16.2.13  9e29efbb67d5  611108b49a57
osd.1                               ceph-node-02.novalocal               running (3h)    29s ago   4h    33.2M    4096M  16.2.13  9e29efbb67d5  2d27f2876666
osd.2                               ceph-node-02.novalocal               running (3h)    29s ago   3h    32.7M    4096M  16.2.13  9e29efbb67d5  2838a95b788a
prometheus.ceph-node-01             ceph-node-01.novalocal  *:9095       running (2h)     9m ago   5d     101M        -  2.33.4   514e6a882f6e  ca52965f0da2
```

- After the service details are listed, further operations can be performed on a specified service.
  - Run **ceph orch daemon start|stop|restart|redeploy|reconfig** *<service_name>* to start, stop, or restart a specified service.
  - Run **ceph orch daemon rm** *<service_name>* **[--force]** to delete a specified service.

HUAWEI

Checking the Status of Storage Devices on a Node

- Before adding an OSD, run **orch device ls** *[<hostname>...]* to check the storage device status of the node.

```
[root@ceph-node-01 ~]# ceph orch device ls
HOST                        PATH  TYPE  DEVICE ID                   SIZE  AVAILABLE  REFRESHED  REJECT REASONS
ceph-node-01.novalocal  /dev/vdb  hdd   09b7efb8-6c7f-4834-9  11.8G              24m ago    Insufficient space (<10 extents) on vgs, LVM detected, locked
ceph-node-01.novalocal  /dev/vdc  hdd   20f3926e-c7a8-4bfd-8  11.8G              24m ago    Insufficient space (<10 extents) on vgs, LVM detected, locked
ceph-node-01.novalocal  /dev/vdd  hdd   6c29f68e-144a-4a3f-9  11.8G              24m ago    Insufficient space (<10 extents) on vgs, LVM detected, locked
ceph-node-02.novalocal  /dev/vdb  hdd   9d4607a7-046d-4e4d-a  11.8G              10m ago    Insufficient space (<10 extents) on vgs, LVM detected, locked
ceph-node-02.novalocal  /dev/vdc  hdd   f27ed38e-bb72-4a19-8  11.8G              10m ago    Insufficient space (<10 extents) on vgs, LVM detected, locked
ceph-node-02.novalocal  /dev/vdd  hdd   a44eb87f-3d74-44b9-9  11.8G              10m ago    Insufficient space (<10 extents) on vgs, LVM detected, locked
ceph-node-03.novalocal  /dev/vdb  hdd   000ef854-a93f-4cee-b  11.8G    Yes       24m ago
ceph-node-03.novalocal  /dev/vdc  hdd   619b5d33-6d9e-4a5b-a  11.8G    Yes       24m ago
ceph-node-03.novalocal  /dev/vdd  hdd   2c3a815a-34c4-4c77-9  11.8G    Yes       24m ago
ceph-node-04.novalocal  /dev/vdb  hdd   6cf8334e-9d48-44af-a  11.8G    Yes       10m ago
ceph-node-04.novalocal  /dev/vdc  hdd   d4e4b702-a4b7-46e6-9  11.8G    Yes       10m ago
ceph-node-04.novalocal  /dev/vdd  hdd   2dfa0b74-f528-4e07-b  11.8G    Yes       10m ago
```

- Run **ceph orch apply osd --all-available-devices** to add all available devices as OSDs.

HUAWEI

Case: Deleting a Host from a Cluster

- Requirement: Remove ceph-node-04 from the cluster.
- Implementation process:
  - 1. Check the services running on the host to be deleted.

```
[root@ceph-node-01 ~]# ceph orch ps ceph-node-04.novalocal
NAME                    HOST                    PORTS   STATUS         REFRESHED  AGE  MEM USE  MEM LIM  VERSION  IMAGE ID      CONTAINER ID
crash.ceph-node-04      ceph-node-04.novalocal          running (6h)      3m ago   6h     6336k        -  16.2.13  9e29efbb67d5  f9f929648d39
node-exporter.ceph-node-04  ceph-node-04.novalocal  *:9100  running (6h)      3m ago   6h     21.4M        -  1.3.1    1dbe0e931976  4b8672d27f3b
osd.3                   ceph-node-04.novalocal          running (52m)     3m ago  52m    37.6M    4096M  16.2.13  9e29efbb67d5  267b0374f2e5
osd.6                   ceph-node-04.novalocal          running (52m)     3m ago  52m    34.6M    4096M  16.2.13  9e29efbb67d5  121a6f860567
osd.8                   ceph-node-04.novalocal          running (52m)     3m ago  52m    35.4M    4096M  16.2.13  9e29efbb67d5  74229ad4c804
```

  - 2. Stop all services on it.

```
[root@ceph-node-01 ~]# ceph orch daemon stop crash.ceph-node-04
Scheduled to stop crash.ceph-node-04 on host 'ceph-node-04.novalocal'
[root@ceph-node-01 ~]# ceph orch daemon stop node-exporter.ceph-node-04
Scheduled to stop node-exporter.ceph-node-04 on host 'ceph-node-04.novalocal'
[root@ceph-node-01 ~]# ceph orch daemon stop osd.3
Scheduled to stop osd.3 on host 'ceph-node-04.novalocal'
...
```


HUAWEI

Case: Deleting a Host from a Cluster

- 3. Disable the automatic installation function of the involved services.

```
[root@ceph-node-01 ~]# ceph orch apply node-exporter --unmanaged=true
Scheduled node-exporter update...
[root@ceph-node-01 ~]# ceph orch apply node-exporter --unmanaged=true
Scheduled node-exporter update...
[root@ceph-node-01 ~]# ceph orch apply osd --all-available-devices --unmanaged=true
Scheduled osd.all-available-devices update...
```

- 4. Stop all services on the host to be deleted.

```
[root@ceph-node-01 ~]# ceph orch daemon rm crash.ceph-node-04
Removed crash.ceph-node-04 from host 'ceph-node-04.novalocal'
[root@ceph-node-01 ~]# ceph orch daemon rm node-exporter.ceph-node-04
Removed node-exporter.ceph-node-04 from host 'ceph-node-04.novalocal'
[root@ceph-node-01 ~]# ceph orch daemon rm osd.3 --force
Removed osd.3 from host 'ceph-node-04.novalocal'
...
```

- 5. Remove the OSD mapping in CRUSH.

```
[root@ceph-node-01 ~]# ceph osd rm osd.3
removed osd.3
[root@ceph-node-01 ~]# ceph osd crush remove osd.3
removed item id 3 name 'osd.3' from crush map
...
```


HUAWEI

Case: Deleting a Host from a Cluster

- 6. Remove data from the OSD disk.

```
[root@ceph-node-01 ~]# ceph orch device zap ceph-node-04.novalocal /dev/vdb --force
zap successful for /dev/vdb on ceph-node-04.novalocal
[root@ceph-node-01 ~]# ceph orch device zap ceph-node-04.novalocal /dev/vdc --force
zap successful for /dev/vdc on ceph-node-04.novalocal
...
```

- 7. Check the services on the host to be deleted.

```
[root@ceph-node-01 ~]# ceph orch ps ceph-node-04.novalocal
No daemons reported
```

- 8. Delete the host from the cluster.

```
[root@ceph-node-01 ~]# ceph orch host rm ceph-node-04.novalocal
Removed  host 'ceph-node-04.novalocal'
[root@ceph-node-01 ~]# ceph orch host ls
HOST                    ADDR          LABELS  STATUS
ceph-node-01.novalocal  192.168.0.21  _admin
ceph-node-02.novalocal  192.168.0.22  _admin
ceph-node-03.novalocal  192.168.0.23  _admin
ceph-node-05.novalocal  192.168.0.25
4 hosts in cluster
```


HUAWEI

Quiz

1. (Single-answer question) Which of the following records the Ceph failure domain level?
   A. OSD map
   B. MON map
   C. PG map
   D. CRUSH map

HUAWEI

- D

Summary

- This course introduces the theoretical knowledge of Ceph, including its architecture, components, and key processes, and a lab guide is provided for your practice.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling
RADOS	Reliable Autonomic Distributed Object Store

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

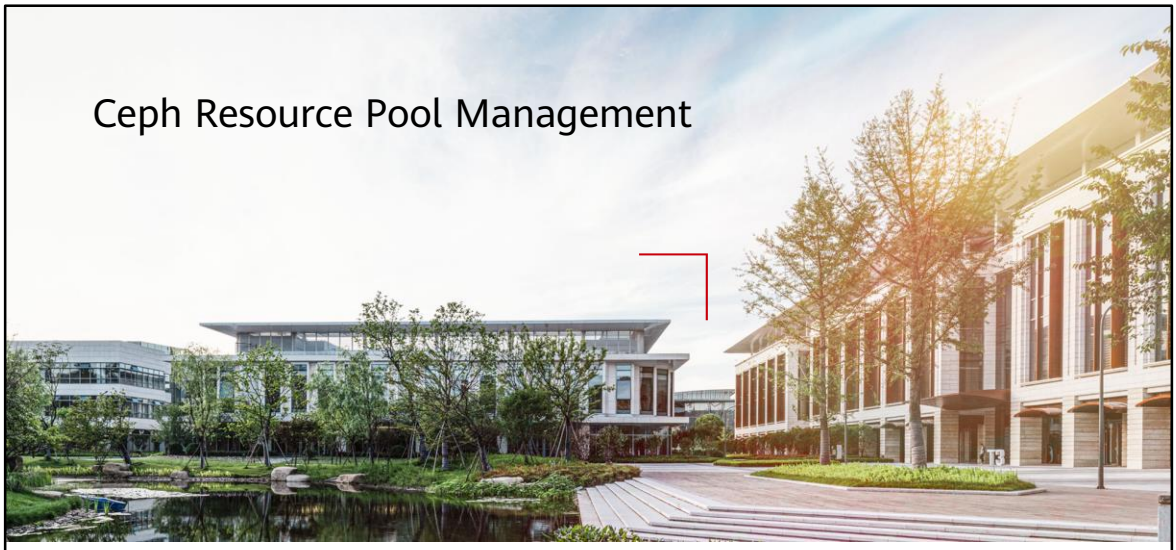
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Ceph Resource Pool Management



Foreword

- After Ceph is installed and initialized, the cluster will undergo lifecycle management. One essential phase is to create and manage resource pools. This course focuses on resource pool management.

HUAWEI

Objectives

On completion of this course, you will understand:

- Working principles of Ceph resource pools
- Management and configuration of replicated pools and erasure-coded pools

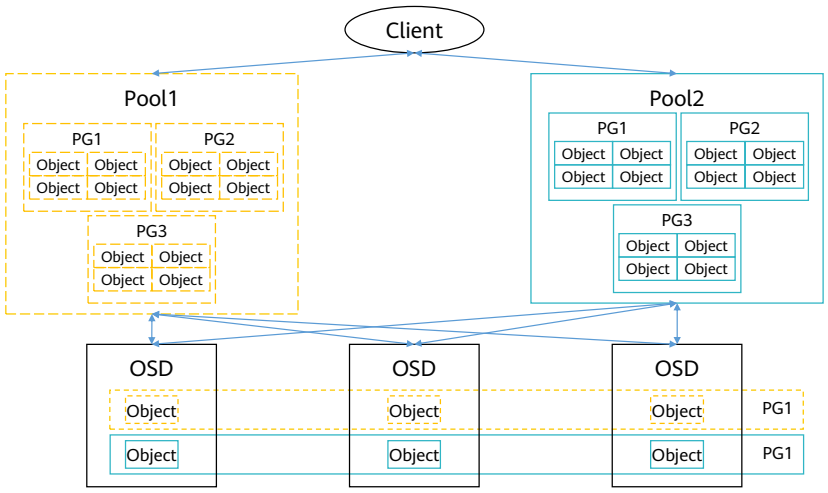
HUAWEI

Contents

1. **Working Principles of the Ceph Resource Pool**
2. Ceph Resource Pool Configuration Management
3. Ceph Erasure-coded Pool Management

HUAWEI

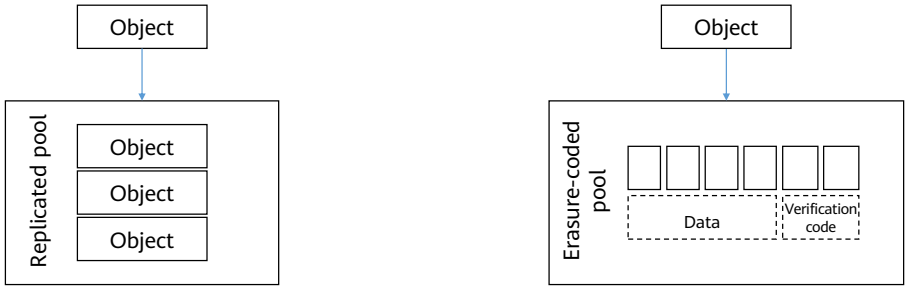
Ceph Data Storage Mechanism



HUAWEI

Pool Types

- Based on different data protection modes, pools can be classified into replicated pools and erasure-coded pools.



Mapping Between Pools, OSDs, and PGs

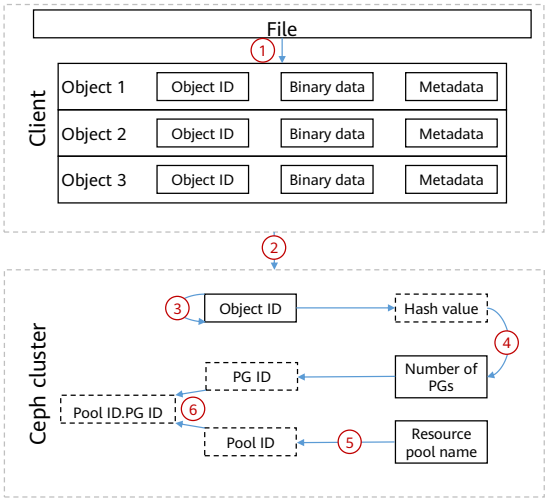
- A pool is a logical concept of Ceph and is used to store objects. Pools are isolated from each other.
- The placement group (PG) is a subset of a pool and a set of objects.
- RADOS objects can be distributed only in one PG, and one PG can belong to only one pool. Objects in PGs are mapped to different OSDs.
- When the client saves an object, Ceph calculates the hash value of the object name and uses the CRUSH algorithm to map the object to the PG.
- After the mapping between objects and PGs is determined, Ceph calculates the mapping between all object replicas and OSDs based on the number of cluster replicas, PGs, and CRUSH rules.
- When an OSD is added to or deleted from a cluster, the PG automatically re-balances objects to other OSDs.

HUAWEI

- The number of PGs affects the Ceph performance.
  - With too many PGs, each PG stores a small amount of data during data movement. As a result, Ceph has high CPU and memory overhead that affect the normal use of clients in the cluster.
  - Too few PGs require each PG to store more data. Moving a PG uses high bandwidth, affecting the use of cluster clients as well.
- In earlier Ceph versions, the PG calculation formula is as follows:
  - Total number of PGs in the Ceph cluster = (Number of OSDs x 100)/Maximum number of copies
  - Total number of PGs in one resource pool = (Number of OSDs x 100)/Maximum number of copies/Number of pools
- The calculated results must be rounded to the value closest to 2 to the power of N.
- Generally, if the number of OSDs is less than 5, set the number of PGs to 128. If the number of OSDs is between 5 and 10, set the number of PGs to 512. If the number of OSDs is between 10 and 50, set the number of PGs to 4096. If the number of OSDs is greater than 50, use a tool to calculate the number of OSDs. The official tool link is https://old.ceph.com/pgcalc/.

Process of Computing the PG ID

- When the client is reading or writing data, it only needs to provide the name and object ID of the resource pool to Ceph.
- A complete object consists of three parts:
  - Object ID
  - Binary data
  - Object metadata
- Process of computing the PG ID:
  - 1. The client divides the file into multiple unified-sized objects that can be processed by RADOS and allocates an ID to each object.
  - 2. The client inputs the resource pool name and object ID to the Ceph cluster.
  - 3. Ceph calculates the hash after obtaining the object ID.
  - 4. Ceph uses the calculated hash value to perform modulo hash on the number of PGs to obtain the PG ID.
  - 5. Ceph obtains the ID based on the resource pool name input by the client.
  - 6. Ceph places the pool ID before the PG ID to form the PG ID of the object.

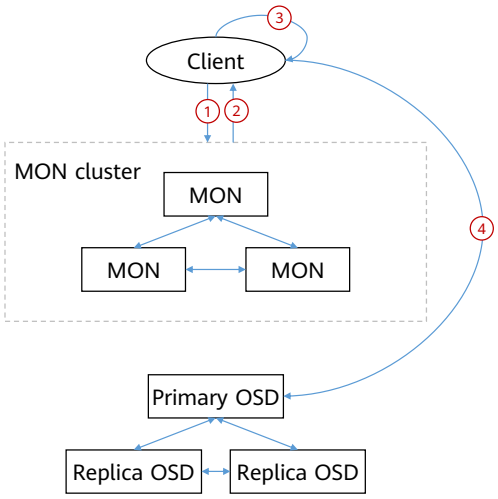


HUAWEI

- The object ID is unique in the entire Ceph cluster.
- The three parts of an object are automatically generated by the client based on the type of the saved data.
- Ceph uses the following algorithm to calculate the PG ID: PG = hash (*object_id*) % *pg_num*
- After determining the PG ID, Ceph saves the data to the corresponding OSD based on the mapping of PGs and OSDs, and the data protection rules of the resource pool.

Process of Accessing Ceph from a Client

- The process includes:
  - 1. The client sends a connection request to the MON cluster.
  - 2. After the client is connected to the MON, it retrieves the latest version of the cluster map to obtain MON, OSD, and MDS information, excluding the storage location of the object.
  - 3. The client calculates the PG and OSD of the object based on the CRUSH algorithm.
  - 4. The client communicates with the primary OSD, whose location is obtained in last step, to read and write the object.

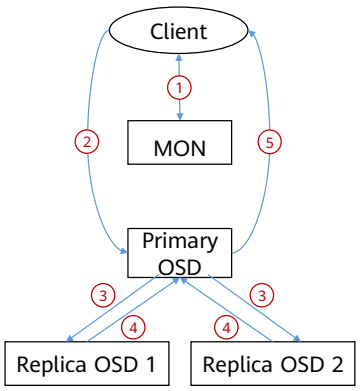


HUAWEI

- In Ceph, calculating the object storage location on the computing node is much faster than querying the location by interacting with the Ceph component. This is why, for data read and write, the client calculates the object storage location using CRUSH.

Process of Writing Data in Ceph

- Ceph applies strong data consistency to ensure data synchronization.
- Data write process:
  - 1. The client obtains the cluster map through the MON.
  - 2. The client obtains the information about the primary OSD node through the cluster map and sends a write request to the node.
  - 3. After receiving the write request, the primary OSD writes data and sends a data write instruction to two replica OSDs.
  - 4. The two replica OSDs write the data and return a confirmation message to the primary OSD.
  - 5. After receiving confirmation from all replica OSDs, the primary OSD returns a write completion message to the client.

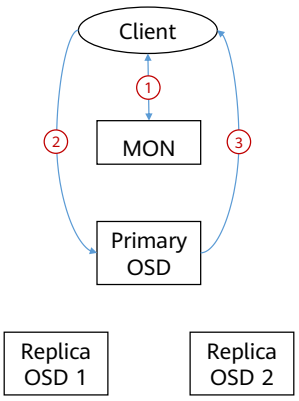


HUAWEI

- Strong consistency causes latency to increase. Therefore, Ceph optimizes data write in two stages.
  - In the first stage, after data is written to the OSD node cache, an acknowledgment is sent to the client, which determines the write operation to be complete and proceeds with subsequent operations.
  - In the second stage, after data is written from the cache to the disk, Ceph sends an acknowledgment to the client, which considers the data is written and deletes the corresponding local data.

Process of Reading Data in Ceph

- Data read process:
  - 1. The client obtains the cluster map through the MON.
  - 2. The client sends a read request to the primary OSD node whose information is obtained from the cluster map.
  - 3. The primary OSD returns the data requested to the client.



Contents

1. Working Principles of the Ceph Resource Pool
2. **Ceph Resource Pool Configuration Management**
3. Ceph Erasure-coded Pool Management

HUAWEI

Creating the Resource Pool

- Run the following command to create a resource pool:

  osd pool create *<pool> [<pg_num:int>] [<pgp_num:int>] [replicated|erasure] [<erasure_code_profile>]*

  - *<pool>*: resource pool name. This parameter is mandatory.
  - *<pg_num:int>*: number of PGs.
  - *<pgp_num:int>*: number of PGs for placement purpose (PGPs).
  - **replicated|erasure**: resource pool type. **replicated** means a replicated pool, and **erasure** means an erasure-coded pool.
  - *<erasure_code_profile>*: erasure-coded pool configuration template. It is used to set the number of data blocks and erasure codes. This parameter is mandatory when the resource pool is an erasure-coded one.
  - If only the resource pool name is set, Ceph automatically completes other parameters. By default, the created resource pool is a replicated pool.

```
[ceph: root@ceph01 /]# ceph osd pool create test
pool 'test' created
```


HUAWEI

- If only the resource pool name is specified, the created pool is a replicated pool and the number of replicas is three.

Setting the Application of a Resource Pool

- The created resource pool can be used for block storage, file storage, or object storage by setting the application type to **rbd**, **cephfs**, or **rgw**, respectively. The commands are **ceph osd pool application enable** *<pool> <app>*.
- To disable the application of a resource pool, run **ceph osd pool application disable** *<pool> <app>* **[--yes-i-really-mean-it]**.

```
[ceph: root@ceph01 /]# ceph osd pool application enable test rbd
enabled application 'rbd' on pool 'test'
[ceph: root@ceph01 /]# ceph osd pool application disable test rbd --yes-i-really-mean-it
enabled application 'rbd' on pool 'test'
[ceph: root@ceph01 /]# ceph osd pool application enable test cephfs
enabled application 'rbd' on pool 'test'
[ceph: root@ceph01 /]# ceph osd pool application enable test rgw
enabled application 'rgw' on pool 'test'
[ceph: root@ceph01 /]# ceph osd pool application disable test rgw --yes-i-really-mean-it
disable application 'rgw' on pool 'test'
```


HUAWEI

- You are not advised to set the application type of the resource pool to **rbd**, **cephfs**, and **rgw** at the same time.

Testing the Resource Pool Function

- After a resource pool is created, you can run the **rados** command set to check whether the resource pool is normal. The common commands are as follows:
  - **rados -p** *<pool_name>* **put** *<obj-name>* *<infile>*: uploads an object to a specified resource pool.
  - **rados -p** *<pool_name>* **get** *<obj-name>* *<infile>*: downloads an object from a specified resource pool.
  - **rados -p** *<pool_name>* **rm** *<obj-name>*: deletes an object from a specified resource pool.
  - **rados -p** *<pool_name>* **ls**: lists objects in a specified resource pool.

```
[ceph: root@ceph01 /]# rados -p test put test-obj /etc/hosts
[ceph: root@ceph01 /]# rados -p test ls
test-obj
[ceph: root@ceph01 /]# rados -p test get test-obj /hosts
[ceph: root@ceph01 /]# ls | grep hosts
hosts
[ceph: root@ceph01 /]# rados -p test rm test-obj
[ceph: root@ceph01 /]# rados -p test ls
```


HUAWEI

Checking Resource Pool Information

- Run **ceph osd pool ls** to list the resource pools in the current Ceph cluster.

```
[ceph: root@ceph01 /]# ceph osd pool ls
device_health_metrics
test
```

- Run **ceph osd pool ls detail** to list detailed information about resource pools in the current Ceph cluster.

```
[ceph: root@ceph01 /]# ceph osd pool ls detail
pool 1 'device_health_metrics' replicated size 3 min_size 2 crush_rule 0 object_hash rjenkins pg_num 1 pgp_num 1 autoscale_mode on last_change 25
flags hashpspool stripe_width 0 pg_num_max 32 pg_num_min 1 application mgr_devicehealth
pool 2 'test' replicated size 3 min_size 2 crush_rule 0 object_hash rjenkins pg_num 32 pgp_num 32 autoscale_mode on last_change 66 flags hashpspool
stripe_width 0 application rgw
```

- Run **ceph osd pool stats** *[<pool_name>]* to view the status of a resource pool or all resource pools.

```
[ceph: root@ceph01 /]# ceph osd pool stats
pool device_health_metrics id 1
  nothing is going on

pool test id 2
  nothing is going on
```


HUAWEI

- **device_health_metrics** is the default resource pool of the system and is used to store system information.
- The **ceph osd pool ls detail** command output shows the number of replicas (**size 3**) of test, the minimum number of available OSDs (**min_size 2**) of the resource pool, and the crush rule (**crush_rule 0**).

Checking Resource Pool Information

- Run **ceph df** to obtain the remaining space information of the cluster where the resource pool resides.
- Run **ceph osd df** to check the OSD usage of the cluster to which the resource pool belongs.

```
[ceph: root@ceph01 /]# ceph df
--- RAW STORAGE ---
CLASS    SIZE   AVAIL    USED  RAW USED  %RAW USED
hdd    90 GiB  90 GiB  56 MiB    56 MiB       0.06
TOTAL  90 GiB  90 GiB  56 MiB    56 MiB       0.06

--- POOLS ---
POOL                   ID  PGS  STORED  OBJECTS  USED  %USED  MAX AVAIL
device_health_metrics   1    1     0 B        0   0 B      0     28 GiB
test                    2   32     0 B        0   0 B      0     28 GiB
[ceph: root@ceph01 /]# ceph osd df
ID  CLASS  WEIGHT   REWEIGHT  SIZE    RAW USE  DATA     OMAP     META     AVAIL   %USE  VAR   PGS  STATUS
 0    hdd  0.00980   1.00000  10 GiB  6.4 MiB  700 KiB      0 B  5.8 MiB  10 GiB  0.06  1.04    9      up
 1    hdd  0.00980   1.00000  10 GiB  6.5 MiB  700 KiB      0 B  5.8 MiB  10 GiB  0.06  1.05   11      up
 2    hdd  0.00980   1.00000  10 GiB  6.4 MiB  700 KiB      0 B  5.8 MiB  10 GiB  0.06  1.04   13      up
 3    hdd  0.00980   1.00000  10 GiB  6.1 MiB  700 KiB      0 B  5.4 MiB  10 GiB  0.06  0.99   16      up
 4    hdd  0.00980   1.00000  10 GiB  6.1 MiB  700 KiB      0 B  5.4 MiB  10 GiB  0.06  0.98    8      up
 5    hdd  0.00980   1.00000  10 GiB  6.0 MiB  700 KiB      0 B  5.3 MiB  10 GiB  0.06  0.97    9      up
 6    hdd  0.00980   1.00000  10 GiB  6.1 MiB  700 KiB      0 B  5.4 MiB  10 GiB  0.06  0.99    8      up
 7    hdd  0.00980   1.00000  10 GiB  6.1 MiB  700 KiB      0 B  5.4 MiB  10 GiB  0.06  0.99   12      up
 8    hdd  0.00980   1.00000  10 GiB  6.1 MiB  700 KiB      0 B  5.4 MiB  10 GiB  0.06  0.98   13      up
                       TOTAL  90 GiB   56 MiB  6.2 MiB  5.7 KiB   50 MiB  90 GiB  0.06
MIN/MAX VAR: 0.97/1.05  STDDEV: 0.00
```


HUAWEI

Setting Storage Pool Quotas

- Ceph supports the configuration of resource pool quotas. The configuration command is **ceph osd pool set-quota** *<pool>* **max_objects|max_bytes** *<val>*.
- After the setting is complete, run **ceph osd pool get-quota** *<pool>* to query the resource pool quota.
- Resource pool quotas can be set by limiting the number of objects or limiting the resource pool capacity.

```
[ceph: root@ceph01 /]# ceph osd pool set-quota test max_objects 2
set-quota max_objects = 2 for pool test
[ceph: root@ceph01 /]# ceph osd pool set-quota test max_bytes 5M
set-quota max_bytes = 5242880 for pool test
[ceph: root@ceph01 /]# ceph osd pool get-quota test
quotas for pool 'test':
  max objects: 2 objects  (current num objects: 0 objects)
  max bytes  : 5 MiB  (current num bytes: 0 bytes)
```


HUAWEI

Managing Storage Pool Snapshot

- Ceph supports snapshots for storage pools. To create a snapshot, run **ceph osd pool mksnap** *<pool>* *<snap>* or **rados -p** *<pool>* **mksnap** *<snap>*.
- To delete snapshots from a storage pool, run **ceph osd pool rmsnap** *<pool>* *<snap>* or **rados -p** *<pool>* **rmsnap** *<snap>*.
- After a resource pool snapshot is created, run **rados -p** *<pool>* **lssnap** *<snap>* to query the snapshot information.

```
[ceph: root@ceph01 /]# ceph osd pool mksnap test test-snap1
created pool test snap test-snap1
[ceph: root@ceph01 /]# rados -p test mksnap test-snap2
created pool test snap test-snap2
[ceph: root@ceph01 /]# rados -p test lssnap
1       test-snap1      2023.09.06 08:56:28
2       test-snap2      2023.09.06 08:58:13
2 snaps
```

- Ceph snapshots of storage pools are implemented similar to version management. To view the object information of the corresponding snapshot, run **rados -p** *<pool>* **-s** *<snap>* **ls**. When restoring an object, you need to download the object from the corresponding snapshot and upload it to the snapshot version to be restored. For example, to restore object **test-obj** in **test-snap1** to the resource pool, perform the following operations:

```
[ceph: root@ceph01 /]# rados -p test -s test-snap1 get test-obj test-obj
selected snap 1 'test-snap1'
[ceph: root@ceph01 /]# rados -p test put test-obj test-obj
```


HUAWEI

- The snapshot function of Ceph storage pools is similar to version management.

Adjusting the Number of PGs in a Resource Pool

- Ceph supports to change the number of PGs, specifically, an existing PG can be divided into smaller PGs or smaller PGs can be combined into larger PGs.
- The pg_autoscaler function of Ceph enables a cluster to automatically adjust the number of PGs based on the amount of data stored in the resource pool.

```
[ceph: root@ceph01 /]# ceph osd pool get test all
size: 3
min_size: 2
pg_num: 32
pgp_num: 32
crush_rule: replicated_rule
hashpspool: true
nodelete: false
nopgchange: false
nosizechange: false
write_fadvise_dontneed: false
noscrub: false
nodeep-scrub: false
use_gmt_hitset: 1
fast_read: 0
pg_autoscale_mode: on
bulk: false
```

```
[ceph: root@ceph01 /]# ceph osd pool set test pg_num 16
set pool 2 pg_num to 16
[ceph: root@ceph01 /]# ceph osd pool get test all
size: 3
min_size: 2
pg_num: 16
pgp_num: 16
crush_rule: replicated_rule
hashpspool: true
nodelete: false
nopgchange: false
nosizechange: false
write_fadvise_dontneed: false
noscrub: false
nodeep-scrub: false
use_gmt_hitset: 1
fast_read: 0
pg_autoscale_mode: on
bulk: false
```


HUAWEI

- Adjusting the number of PGs takes some time, and the number of PGPs is adjusted accordingly. Adjust **pgp_num** first and then **pg_num**.
- Adjusting the number of PGs will cause object migration. Adjusting PGPs will not, but it causes PG distribution change on OSDs.

Deleting the Storage Pool

- Before deleting a storage pool, enable **mon_allow_pool_delete**.
- When deleting a storage pool, enter the storage pool name twice and add the **--yes-i-really-really-mean-it** parameter.

```
[ceph: root@ceph01 /]# ceph osd pool rm test test --yes-i-really-really-mean-it
Error EPERM: pool deletion is disabled; you must first set the mon_allow_pool_delete config option to true before you can destroy a pool
[ceph: root@ceph01 /]# ceph config set mon mon_allow_pool_delete true
[ceph: root@ceph01 /]# ceph config get mon mon_allow_pool_delete
True
[ceph: root@ceph01 /]# ceph osd pool rm test test --yes-i-really-really-mean-it
pool 'test' removed
[ceph: root@ceph01 /]# ceph osd pool ls
device_health_metrics
```


HUAWEI

Configuring Prevention of Resource Pool Mis-deletion

- Set **nodelete** to **true** to prevent users from deleting resource pools.

```
[ceph: root@ceph01 /]# ceph osd pool set test nodelete true
set pool 3 nodelete to true
[ceph: root@ceph01 /]# ceph osd pool get test all
size: 3
min_size: 2
pg_num: 32
pgp_num: 32
crush_rule: replicated_rule
hashpspool: true
nodelete: true
nopgchange: false
nosizechange: false
write_fadvise_dontneed: false
noscrub: false
nodeep-scrub: false
use_gmt_hitset: 1
fast_read: 0
pg_autoscale_mode: on
bulk: false
[ceph: root@ceph01 /]# ceph osd pool rm test test --yes-i-really-really-mean-it
Error EPERM: pool deletion is disabled; you must unset nodelete flag for the pool first
```


HUAWEI

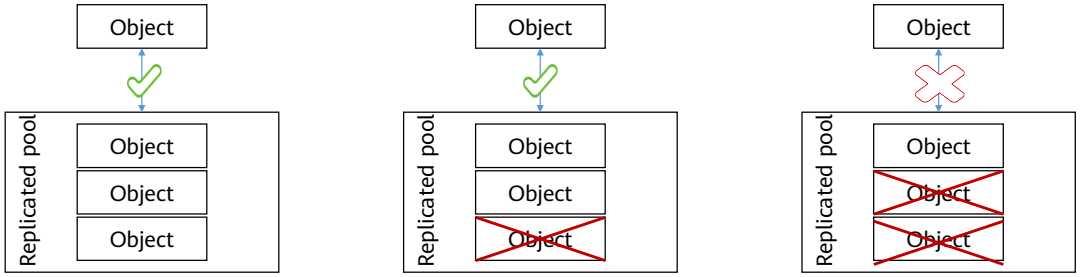
Contents

1. Working Principles of the Ceph Resource Pool
2. Ceph Resource Pool Configuration Management
3. Ceph Erasure-coded Pool Management

HUAWEI

Number and the Minimum Number of Replicas in a Replicated Pool

- When storing an object, Ceph creates and stores multiple replicas for data reliability. If any data is corrupted, the remaining replicas can prevent data loss.
- The number of replicas corresponds to the number of copies of the same object. The minimum number of replicas relates to the minimum required replicas for normal I/O read and write.
- In the following figure, the number of replicas is three, and the minimum number of replicas is two.



Data Chunks and Coding Chunks of the Erasure-coded Pool

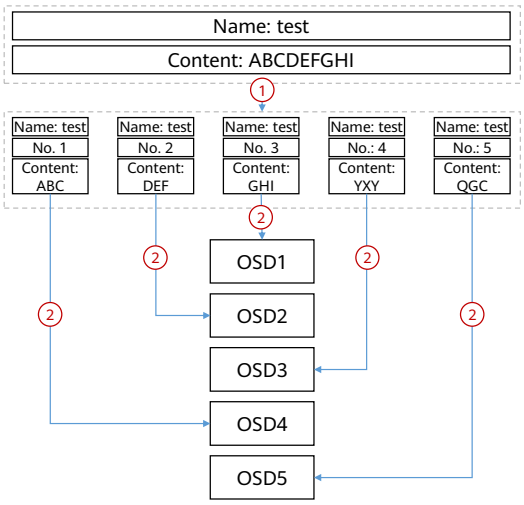
- In an erasure-coded pool, data is split into data chunks and parity chunks. Once a data chunk is unavailable, Ceph restores it based on the parity chunks to prevent data loss. An erasure-coded pool has higher storage utilization than a replicated pool, but requires more computing resources.
- In Ceph, *K* is used to represent the quantity of data chunks. *M* is used to represent the quantity of parity chunks, which are referred to as "coding chunks". *N* indicates the total number, that is, $N = M + K$.
- In the erasure-coded pool, any data chunk or coding chunk can restore data. The pool can tolerate up to $K - 1$ data chunks and $M - 1$ coding chunks damage at the same time.
- The recommended combinations of *M* and *K* are 3 + 2, 4 + 2, 8 + 3, and 8 + 4.

HUAWEI

- Data here refers to the objects after the actual client data is split.
- The data chunk size is the same as that of the coding chunk.
- In the erasure-coded pool, *M* determines the amount of redundant data.

Working Principles of the Erasure-coded Pool

- Before data is written to the erasure-coded pool, the file is divided into *K* data chunks and *M* coding chunks based on settings.
- The content of the test file is ABCDEFGHI. Save the file in Ceph in the 3 + 2 format, where 3 indicates the number of data chunks and 2 indicates the number of coding chunks. The process is as follows:
  - 1. Split the file into 3 + 2 chunks based on the settings.
  - 2. Save the split objects to the corresponding OSDs.



HUAWEI

Creating an Erasure-coded Pool

- To create an erasure-coded pool, run **ceph osd pool create** *<pool> [<pg_num:int>] [<pgp_num:int>]* **erasure** *[<erasure_code_profile>] [<rule>]*.
- *<erasure_code_profile>* specifies the name of the rule used by the erasure-coded pool.
- By default, the number of data chunks and the number of coding chunks in the erasure-coded pool are both two.

```
[ceph: root@ceph01 /]# ceph osd pool create test erasure default
pool 'test' created
[ceph: root@ceph01 /]# ceph osd pool get test all
...
crush_rule: erasure-code
hashpspool: true
allow_ec_overwrites: false
nodelete: false
nopgchange: false
nosizechange: false
write_fadvise_dontneed: false
noscrub: false
...
```


HUAWEI

Configuring the Erasure-coded Pool Rule

- To configure the erasure-coded pool rule, run **ceph osd erasure-code-profile set** *<name> [<profile>...]*.
- After the rule is assigned to an erasure-coded pool, it cannot be modified or deleted.
- The following parameters can be configured for the erasure-coded pool rule:
  - **k**: number of data chunks
  - **m**: number of coding chunks
  - **plugin**: erasure coding algorithm plug-in. The default value is **jerasure**.
  - **crush-failure-domain**: failure domain. The default value is **host**.
  - **crush-device-class**: device type
  - **technique**: algorithm provided by the plug-in.

```
[ceph: root@ceph01 /]# ceph osd erasure-code-profile set test k=4 m=2 crush-failure-domain=osd
[ceph: root@ceph01 /]# ceph osd erasure-code-profile get test
crush-device-class=
crush-failure-domain=osd
crush-root=default
jerasure-per-chunk-alignment=false
k=4
m=2
plugin=jerasure
technique=reed_sol_van
w=8
```


HUAWEI

Quiz

1. (True or False) An erasure-coded pool has three data chunks and two coding chunks. A maximum of three storage units can be damaged in the pool.
   A. T
   B. F

HUAWEI

- B

Summary

- This course introduces the working principles and management of resource pools, including creating, managing, and deleting resource pools, to compound knowledge of Ceph at the backend storage.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling	Description
PG	Placement group	Subset of the Ceph resource pool
RBD	RADOS block device	N/A
RGW	RADOS gateway	N/A
MDS	Ceph metadata server	RADOS metadata server

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

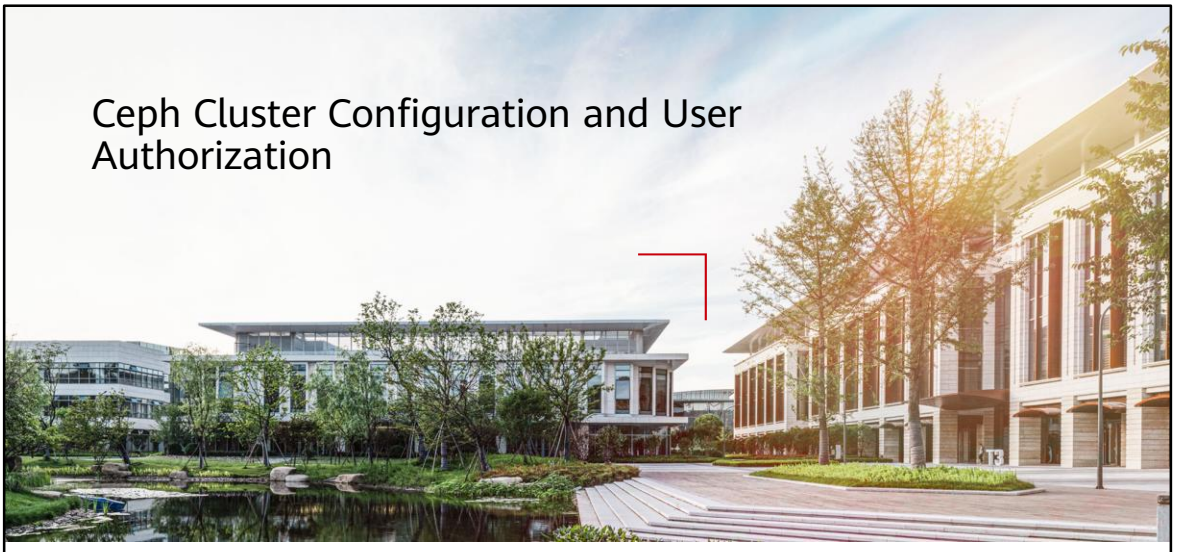
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Ceph Cluster Configuration and User Authorization



Foreword

- In the previous two courses, we looked at Ceph installation and resource pool management, including have learned how to configure the Ceph cluster and determine related attributes. This course will introduce how to configure the Ceph cluster.
- Ceph authentication and authorization are used to specify user roles to manage and use the cluster. This is an important part of using Ceph, which will be introduced in this course as well.

HUAWEI

Objectives

- On completion of this course, you will understand:
  - Ceph cluster configuration
  - CephX knowledge

Contents

1. **Ceph Cluster Configuration**
2. CephX Details

HUAWEI

Ceph Cluster Configuration File Overview

- After Ceph is installed using Cephadm, the Ceph configuration file **ceph.conf** is automatically generated in the **/etc/ceph** directory.
- The Ceph configuration file is in the INI file format. Use the number sign (#) or semicolon (;) to comment out the configuration.
- Ceph cluster configurations come from:
  - Default values set during compilation
  - Centralized configuration database of the MON cluster
  - Configuration files saved on the local host
  - Environment variables
  - Command input
  - Configurations that take effect immediately after being configured by the administrator

HUAWEI

- INI is a common configuration file format, where:
- [name]
- key1 = value 1
- key 2 = value 2
- *key* is the parameter name, and *value* is the corresponding parameter value.

Configuration Sections of the ceph.conf File

- To facilitate cluster configuration, the configuration file is divided into multiple configuration sections. Each section is configured for a specific Ceph daemon, including:
  - **[global]**: global configurations, which are valid for all daemons and clients. They have the lowest priority.
  - **[mon]**: ceph-mon daemon configurations
  - **[mgr]**: ceph-mgr daemon configurations
  - **[osd]**: ceph-osd daemon configurations
  - **[mds]**: ceph-mds daemon configurations
  - **[client]**: Ceph client configurations
- The configurations in **[global]** has the lowest priority. The configurations in other sub-sections can overwrite those in **[global]**.

HUAWEI

Metavariables in Ceph Cluster Configuration

- Metavariables streamline cluster configurations. After the values of metavariables are set, Ceph applies them to all configuration files. Ceph supports the following metavariables:
  - **$***cluster*: cluster name, used when multiple clusters run on the same hardware.
  - **$***type*: service or process type, such as **mds**, **osd**, and **mon**.
  - **$***id*: service or client ID. For example, in **osd.0**, **osd** is the value of **$***type*, and **0** is the value of **$***id*.
  - **$***host*: name of the host where the service is located.
  - **$***name*: combination of **$***type* and **$***id*.
  - **$***pid*: process ID of the service.

HUAWEI

Paths for Storing the Cluster Configuration File

- Ceph searches for cluster-related configuration files in the following directories:
  - **$CEPH_CONF**: path contained in the **$***CEPH_CONF* environment variable
  - **-c path/path**: path specified by the parameter **-c**
  - **/etc/ceph/$cluster.conf**
  - **~/.ceph/$cluster.conf**
  - **./$cluster.conf**

HUAWEI

- In this slide, **$***cluster* indicates the cluster name. The default value is **ceph**.

Modifying Ceph Cluster Configurations

- To modify Ceph cluster configurations when services are running, run **ceph config set** *<who>* *<name>* *<value>* **[--force]**.
- To make temporary configuration changes, run **ceph tell** *<name>* **config set** *<option>* *<value>*.

```
[ceph: root@ceph01 /]# ceph tell osd.0 config set debug_osd 20
{
    "success": ""
}
[ceph: root@ceph01 /]# ceph config show osd.0 debug_osd
20/20
```


HUAWEI

- The **ceph config set** command changes the settings to the centralized database. If the local file has the corresponding configuration, however, these changes are ignored due to their low priority.
- When you run **ceph config show** to query the temporary changes, the value of **source** is **override**.
- The temporary changes are restored after the MON is restarted.
- Run **ceph config ls** to view all configuration items of the Ceph cluster.

Querying the Ceph Cluster Configuration

- Run **ceph config show** *<who> [<key>]* to query the current configuration of a service in the cluster when Ceph is running.

```
[ceph: root@ceph01 /]# ceph config show osd.0
NAME                         VALUE                                                
```

Querying the Ceph Cluster Configuration

- To query all runtime configurations of a specified service by invoking the socket of the local host, run **ceph daemon** *<who>* **config show**.

```
[ceph: root@ceph01 /]# ceph daemon osd.0 config show | more
{
    "name": "osd.0",
    "cluster": "ceph",
    "admin_socket": "/var/run/ceph/ceph-osd.0.asok",
    "admin_socket_mode": "",
    "auth_allow_insecure_global_id_reclaim": "true",
    "auth_client_required": "cephx, none",
    "auth_cluster_required": "cephx",
    "auth_debug": "false",
    "auth_expose_insecure_global_id_reclaim": "true",
    "auth_mon_ticket_ttl": "259200.000000",
    "auth_service_required": "cephx",
    "auth_service_ticket_ttl": "3600.000000",
    "auth_supported": "",
    "bdev_aio": "true",
    ...
```


HUAWEI

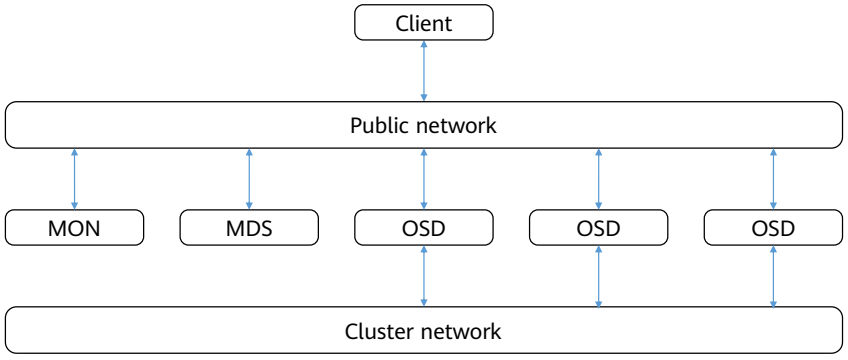
Ceph Centralized Configuration Management Database

- By default, the Ceph centralized configuration management database is stored in and maintained by each MON node under **/var/lib/ceph/$fsid/mon.$host/store.db**.
- The database capacity increases over time. To enable the MON to automatically compress the database during each startup, run **ceph config set mon mon_compact_on_start true**.
- When the database capacity exceeds 15 GB or the available capacity is less than or equal to 30% of its full capacity, the system changes the cluster running status to **HEALTH_WARN**.
- When the available capacity of the database is less than or equal to 5% of the full capacity, the system changes the cluster running status to **HEALTH_ERR**.

HUAWEI

Case: Configuring Ceph Cluster Network

- Plan two network planes to carry Ceph, namely the public network and cluster network.
- The public network is a front-end network used by clients to access storage resources provided by Ceph.
- The cluster network is a back-end network used for interaction and data replication between Ceph components.



HUAWEI

- The two network planes can be carried by two physical planes or the aggregation network in AA mode.

Case Requirement Description and Configuration

- Set **cluster_network** of the existing cluster to **172.16.0.0/24** and **public_network** to **192.168.1.0/24**.
- Procedure:
  - On two NICs for the hosts in the cluster, set their IP addresses to **192.168.1.0/24** and **172.16.0.0/24**, respectively.
  - Run **ceph config set** *<who> <name> <value>* to modify the network settings.

```
[ceph: root@ceph01 /]# ceph config get mon public_network
192.168.1.0/24
##### The current public_network meets the requirements and does not need to be modified.
[ceph: root@ceph01 /]# ceph config get mon cluster_network

##### cluster_network is empty.
[ceph: root@ceph01 /]# ceph config set mon cluster_network 172.16.0.0/24
[ceph: root@ceph01 /]# ceph config get mon cluster_network
172.16.0.0/24
[ceph: root@ceph01 /]# ceph -s
  cluster:
    id:     81ef2624-4c67-11ee-b2c8-000c296af047
    health: HEALTH_WARN
            clock skew detected on mon.ceph02, mon.ceph03
...
```


HUAWEI

- Question: What causes the warning in the cluster, and how can you solve it?
- Answer: The 172.16.0.0/24 network segment is not bypassed when the clock server is configured, causing the clock-related warning to be reported. Modify the configuration file of the chronyd server and restart the service.

Contents

1. Ceph Cluster Configuration
2. **CephX Details**

HUAWEI

Background of Ceph Authentication and Authorization

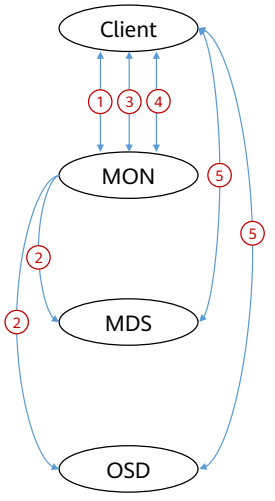
- Ceph authentication and authorization are used in the following scenarios:
  - Clients connect to the Ceph resource pool.
  - Ceph cluster components interact with each other.
- By default, Ceph uses the CephX protocol for authentication and authorization. When accessing Ceph, you need to specify the following parameters:
  - User
  - Keyring

HUAWEI

- If no user is specified during Ceph access, the user name **admin** and the corresponding keyring are used. By default, the keyring is stored in **/etc/ceph/ceph.client.admin.keyring**.

Ceph Authentication Process

- By default, Ceph uses the CephX protocol to authenticate users and services.
- CephX uses shared secret keys for authentication, where the client and the MON cluster have a copy of the client's secret key.
- CephX protocol authentication process:
  - 1. The client sends a request to the MON to create a new user. The MON generates a secret key and returns the key to the client.
  - 2. The MON shares the secret key with the MDS and OSD.
  - 3. The client sends an authentication request to the MON. Once authenticated, the MON generates a session key, encrypts it using the secret key, and returns the session key to the client.
  - 4. After receiving the session key, the client decrypts the key and requests a ticket representing this session from the MON. The MON generates a ticket and returns it to the client.
  - 5. The client uses the received ticket to interact with the MDS and OSD. The client, MDS, and OSD share the secret key generated by the MON. Therefore, they can verify the validity of the ticket.



Users in CephX

- **$*type*.$*id*** must be used for user format, as required by the CephX protocol.
- The CephX protocol classifies users into two categories:
  - Client: used for client access. **$*type*** is permanently set to **client**, for example, **client.admin** and **client.user1**.
  - Daemon: used for internal component access. **$*type*** indicates the component name, for example, **osd.0** and **mgr.ceph01.oxyyfb**.

```
[ceph: root@ceph01 /]# ceph auth ls
osd.0
        key: AQD1+PdkDAt6NBAAKlH3ta6JNO/ueUEpPLH2qw==
        caps: [mgr] allow profile osd
        caps: [mon] allow profile osd
...
client.bootstrap-mds
        key: AQAj9vdkZ3DNChAA001KbFsOLAWmogmkbejnAA==osd.2
...
mgr.ceph01.oxyyfb
        key: AQDr9fdkHC1vFhAAvziPddCJexFIpS50Ipp9IA==
        caps: [mds] allow *
...
```


HUAWEI

Specifying a Ceph User

- When running commands in Ceph, the user and the corresponding keyring need to be specified. If not, **admin** is used by default. The command format is *<command>* **--id** *<user>* **--keyring** *<keyring>*.

```
[ceph: root@ceph01 /]# ceph -s --id admin --keyring /etc/ceph/ceph.keyring
  cluster:
    id:     81ef2624-4c67-11ee-b2c8-000c296af047
    health: HEALTH_OK

  services:
    mon: 3 daemons, quorum ceph01,ceph02,ceph03 (age 84m)
    mgr: ceph01.oxyyfb(active, since 83m), standbys: ceph02.xlxvrg
    osd: 9 osds: 9 up (since 84m), 9 in (since 2d)

  data:
    pools:   1 pools, 1 pgs
    objects: 0 objects, 0 B
    usage:   76 MiB used, 90 GiB / 90 GiB avail
    pgs:     1 active+clean
```


HUAWEI

Creating a Ceph User

- There are three ways to create a user in Ceph:
  - **ceph auth add** *<entity> [<caps>...]*
    - If there is no user, create one and add permissions.
    - If the user exists and the specified permission is the same as the current permission, no modification is made and no output is displayed.
    - If the user exists but the specified permission is different from the current permission, a corresponding message is displayed.
  - **ceph auth get-or-create** *<entity> [<caps>...]*
    - If there is no user, create one and add permissions. The user name and key will be returned.
    - If the user exists and the specified permission is the same as the current permission, no modification is made and the user name and key are returned.
    - If the user exists but the specified permission is different from the current permission, the permission remains unchanged and an error message is returned.
  - **ceph auth get-or-create-key** *<entity> [<caps>...]*
    - If there is no user, create one and add permissions. The key will be returned.
    - If the user exists and the specified permission is the same as the current permission, no modification is made and the key is returned.
    - If the user exists but the specified permission is different from the current permission, the permission remains unchanged and an error message is returned.

HUAWEI

Authorizing a Ceph User

- Ceph uses capabilities (**caps**) to indicate the authorization of authenticated users.
- Authorized users can use MON, OSD, and MDS features, while access to pooled data can be restricted based on application labels.
- User authorization is set by the administrator when creating a user or updating user configurations. The authorization command is *{daemon-type}* '*{cap-spec}[, {cap-spec}...]*'.
- For example, to grant the test user the read, write, and execute permissions on the MON, run the following commands:

```
[ceph: root@ceph01 /]# ceph auth add client.test mon 'allow rwx'
added key for client.test
[ceph: root@ceph01 /]# ceph auth get client.test
[client.test]
        key = AQDifP5kXEmqMBAA0R5kjkmw3km7N5LCYZZT3g==
        caps mon = "allow rwx"
exported keyring for client.test
```


HUAWEI

Common Authorization for Ceph Users

Permission	Description
allow	Sets permissions for daemons.
r	Grants the read permission to users.
w	Sets object access permissions for users.
x	Enables users to invoke class methods (read and write) and perform authentication operations on the MON.
*	Grants all permissions.
profile osd (MON only)	Allows users to connect to other OSDs or MONs as an OSD for OSD heartbeat and status reporting.
profile bootstrap-osd (MON only)	Allows users to bootstrap the OSD for the Cephadm or ceph-volume tools.
profile mds (MON only)	Allows users to connect to other MDSs or MONs as an MDS.
profile bootstrap-mds (MON only)	Allows users to bootstrap MDSs.
profile rbd (Manager, Monitor, and OSD)	Allows users to use block devices.
profile rbd-read-only	Grants the read-only permission on block devices to users.

HUAWEI

Common Commands for Ceph Users

- View all users.
  - **ceph auth ls**
- View details about a specified user.
  - **ceph auth get** *<entity>*
- Obtain the key of a specified user.
  - **ceph auth print-key** *<entity>*
- Delete a specified user.
  - **ceph auth del** *<entity>*
- Back up for the user.
  - **ceph auth get** *<entity>* **-o** *<keyring>*
- Import the user.
  - **ceph auth import -i** *<keyring>*

HUAWEI

Case: Configuring User test as Required

- Requirement: Create a user named test and grant it the resource pool read/write permissions, in addition to view the cluster status. Once authorized, back up the user to the **/tmp/usr** directory.

```
[ceph: root@ceph01 /]# ceph auth add client.test
added key for client.test
[ceph: root@ceph01 /]# ceph auth caps client.test mon 'allow r'
updated caps for client.test
[ceph: root@ceph01 /]# ceph auth caps client.test osd 'allow rw pool=test'
updated caps for client.test
[ceph: root@ceph01 /]# ceph auth get client.test
[client.test]
        key = AQCagP5kCL0bLxAA+yzPZzSW0yGZzZwLHMURWg==
        caps osd = "allow rw pool=test"
exported keyring for client.test
[ceph: root@ceph01 /]#  ceph auth get client.test -o /tmp/usr/client.test.keyring
exported keyring for client.test
[ceph: root@ceph01 /]# cat /tmp/usr/client.test.keyring
[client.test]
        key = AQCagP5kCL0bLxAA+yzPZzSW0yGZzZwLHMURWg==
        caps osd = "allow rw pool=test"
```


HUAWEI

- To show the implementation process, the operations in this slide are performed in multiple steps, but in real deployments, these steps can be combined.

Quiz

1. (Multiple-answer question) Which of the following options may store the Ceph configuration file?
   A. /etc/ceph
   B. ~/.ceph
   C. /etc/config
   D. ./

HUAWEI

- ABCD

Summary

- This course introduces Ceph cluster configuration and user authorization management, and also discusses their relationship with resource pools.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling	Description
PG	Placement group	Subset of the Ceph resource pool
RBD	RADOS block device	N/A
RGW	RADOS gateway	N/A
MDS	Ceph metadata server	RADOS metadata server

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Ceph Object Storage Gateway RGW



Foreword

- Object storage is the biggest advantage of Ceph. This course introduces the working principles of the Ceph RGW, including the installation and deployment at one and multiple sites.

HUAWEI

Objectives

- On completion of this course, you will understand:
  - RGW configuration
  - RGW multisite configuration

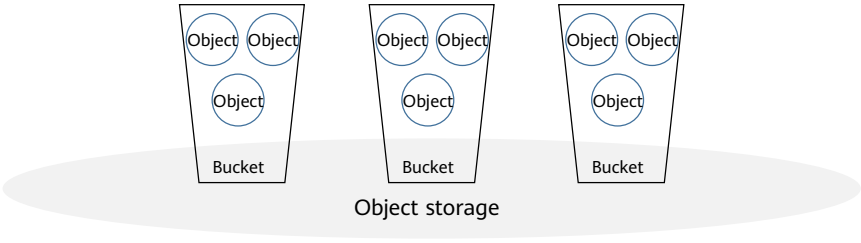
Contents

1. **Object Storage**
2. Single Site RGW Deployment
3. Multisite RGW Deployment

HUAWEI

Introduction to Object Storage

- Object storage provides secure, highly reliable, and low-cost data storage for massive object-based data.
- Object is the basic storage unit in OBS and consists of the file data and file attributes. It is composed of three parts: data, key, and metadata.
  - Key: object name. An object key is a string ranging from 1 to 1,024 characters in UTF-8 format. Each object in a bucket must have a unique key.
  - Metadata: object description. It contains system and user metadata. The metadata is uploaded to object storage as key-value pairs.
  - Data: object content.



HUAWEI

- System metadata is automatically generated by OBS and is used during object data processing, such as **Date**, **Content-length**, **Last-modify**, and **ETag**.
- User metadata is specified to describe objects when users upload the objects.

Features of Object Storage

- Flexible deployment
  - It is compatible with multiple types of clients, such as object storage interfaces, RESTful interfaces, and SDKs.
- Simple access protocol
  - HTTP or HTTPS is used for access.
- Unrestricted network access
  - It can be accessed through the Internet or LAN.
- Flattened structure
  - Objects are directly stored in buckets and have no directory structure.

HUAWEI

- These features are obtained by comparing with file storage and block storage.

Buckets in Object Storage

- A bucket is a container used to store objects in OBS. Object storage offers a flattened structure based on buckets and objects. This structure enables all objects to be stored at the same logical layer, instead of hierarchically.
- Each bucket has its own attributes, such as the storage class, access control, and region. Users can create buckets specified for different regions, and configure many other advanced attributes to meet storage requirements in different scenarios.

HUAWEI

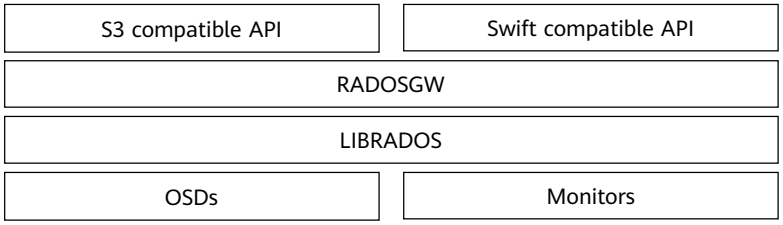
Contents

1. Object Storage
2. **Single Site RGW Deployment**
3. Multisite RGW Deployment

HUAWEI

RGW Overview

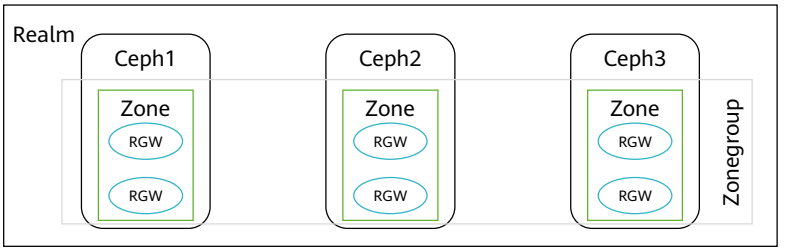
- Applications can access the Ceph object storage service by using the Reliable Autonomic Distributed Object Store (RADOS) or the RADOS gateway (RGW).
- RGW is an object storage interface on librados. It provides a RESTful gateway for applications to access Ceph storage clusters. Currently, RGW is compatible with two types of interfaces:
  - S3
  - Swift



HUAWEI

Concepts Related to Ceph RGW

- Realm
  - A globally unique namespace comprising zonegroups. Each realm has a corresponding period. During configuration modification, the period needs to be updated and committed to make the configurations take effect.
- Zone
  - A logical concept consisting of one or more RGW instances. The zone configuration is saved in the period. Multiple zones can form a zonegroup for multisite services.



HUAWEI

Storage Pools Involved in RGW

- After RGW is created, the system automatically generates the following resource pools:
  - .rgw.root
  - <zonename>.rgw.log
  - <zonename>. rgw.control
  - <zonename>.rgw.meta
  - <zonename>. rgw.buckets.index
  - <zonename>. rgw.buckets.data
  - default.rgw.log
  - default.rgw.control
  - default.rgw.meta

HUAWEI

- If RGW is deleted, the corresponding resource pools need to be manually deleted.

Deploying RGW – Creating a Realm and a Zonegroup

```
[ceph: root@ceph01 /]# radosgw-admin realm create --rgw-realm=test --default
{
    "id": "4d7879c3-cdbf-42c7-8a14-9dd53a183ab3",
    "name": "test",
    "current_period": "86a851d3-d768-41b5-9f98-50bb0cabb496",
    "epoch": 1
}
```

Create a realm

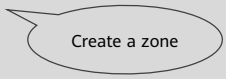
```
[ceph: root@ceph01 /]# radosgw-admin zonegroup create --rgw-realm=test --rgw-zonegroup=testzg --master --default
{
    "id": "e79c6586-151d-47f4-b3a0-6538d8012b7a",
    "name": "testzg",
    "api_name": "testzg",
    "is_master": "true",
    "endpoints": [],
    "hostnames": [],
    "hostnames_s3website": [],
    "master_zone": "",
    "zones": [],
    "placement_targets": [],
    "default_placement": "",
    "realm_id": "4d7879c3-cdbf-42c7-8a14-9dd53a183ab3",
    "sync_policy": {
        "groups": []
    }
}
```

Create a zonegroup

HUAWEI

Deploying RGW – Creating a Zone

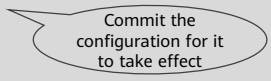
```
[ceph: root@ceph01 /]# radosgw-admin zone create --rgw-realm=test --rgw-zonegroup=testzg --rgw-zone=testzone --master --default
{
    "id": "e3a47b71-3d85-4ccb-93bd-288360c1f775",
    "name": "testzone",
    "domain_root": "testzone.rgw.meta:root",
    "control_pool": "testzone.rgw.control",
    "gc_pool": "testzone.rgw.log:gc",
    "lc_pool": "testzone.rgw.log:lc",
    "log_pool": "testzone.rgw.log",
    "intent_log_pool": "testzone.rgw.log:intent",
    "usage_log_pool": "testzone.rgw.log:usage",
    "roles_pool": "testzone.rgw.meta:roles",
    "reshard_pool": "testzone.rgw.log:reshard",
    "user_keys_pool": "testzone.rgw.meta:users.keys",
.........
"system_key": {
        "access_key": "",
        "secret_key": ""
    },
"realm_id": "4d7879c3-cdbf-42c7-8a14-9dd53a183ab3",
    "notif_pool": "testzone.rgw.log:notif"
}
```



HUAWEI

Deploying RGW – Committing the Configuration

```
[ceph: root@ceph01 /]# radosgw-admin period update --rgw-realm=test --commit
{
    "id": "05406871-a136-4dd1-81e9-5db088674f0b",
    "epoch": 1,
    "predecessor_uuid": "86a851d3-d768-41b5-9f98-50bb0cabb496",
    "sync_status": [],
    "period_map": {
        "id": "05406871-a136-4dd1-81e9-5db088674f0b",
        "zonegroups": [
            {
                "id": "e79c6586-151d-47f4-b3a0-6538d8012b7a",
                "name": "testzg",
                "api_name": "testzg",
                "is_master": "true",
                "endpoints": [],
                "hostnames": [],
                "hostnames_s3website": [],
                "master_zone": "e3a47b71-3d85-4ccb-93bd-288360c1f775",
.....
```



HUAWEI

Deploying RGW – Creating the RGW

```
[ceph: root@ceph01 /]# ceph orch apply rgw test-rgw --realm=test --zone=testzone --placement="2" --port=8080
Scheduled rgw.test-rgw update...
[ceph: root@ceph01 /]# ceph -s
  cluster:
    id:     81ef2624-4c67-11ee-b2c8-000c296af047
    health: HEALTH_OK

  services:
    mon: 3 daemons, quorum ceph01,ceph02,ceph03 (age 51m)
    mgr: ceph01.oxyyfb(active, since 50m), standbys: ceph02.xlxvrg
    osd: 9 osds: 9 up (since 51m), 9 in (since 3d)
    rgw: 2 daemons active (2 hosts, 1 zones)

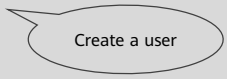
  data:
    pools:   6 pools, 161 pgs
    objects: 237 objects, 7.7 KiB
    usage:   124 MiB used, 90 GiB / 90 GiB avail
    pgs:     161 active+clean
```

Create the RGW

HUAWEI

Deploying RGW – Creating a User

```
[ceph: root@ceph01 /]# radosgw-admin user create --uid=test --access-key=Huawei@123 --secret=Huawei12#$ --email=test@test.com --display-name=test
{
    "user_id": "test",
    "display_name": "test",
    "email": "test@test.com",
    "suspended": 0,
    "max_buckets": 1000,
    "subusers": [],
    "keys": [
        {
            "user": "test",
            "access_key": "Huawei@123",
            "secret_key": "Huawei12#$"
        }
    ],
    "swift_keys": [],
    "caps": [],
    "op_mask": "read, write, delete",
    "default_placement": "",
    "default_storage_class": "",
    "placement_tags": [],
...
```



HUAWEI

RGW User Management – Querying the RGW User Information

```
[ceph: root@ceph01 /]# radosgw-admin user info --uid=test
{
    "user_id": "test",
    "display_name": "test",
    "email": "test@test.com",
    "suspended": 0,
    "max_buckets": 1000,
    "subusers": [],
    "keys": [
        {
            "user": "test",
            "access_key": "Huawei@123",
            "secret_key": "Huawei12#$"
        }
    ],
...
```


HUAWEI

RGW User Management

- To create a user key, run **radosgw-admin user create --uid=***<id>* **--access-key=***<key>* **--secret/--secret-key=***<key>*.
- To delete the user key, run **radosgw-admin user rm --uid=***<id>* **--access-key=***<key>*.
- To enable AK and SK automatic generation, run **radosgw-admin user create --uid=***<id>* **--gen-access-key –gen-secret**.
- The **--key-type=***<type>* parameter specifies the user key type. The current version of Ceph supports Swift and S3.
- Quota limits can be set for users. The commands are as follows:
  - To enable the quota setting, run **radosgw-admin quota enable --quota-scope=***[user|bucket]*.
  - To set the quota, run **radosgw-admin quota set --quota-scope=***[user|bucket]* **--uid=***<id>* **--max-objects=***<num>* **--max-size=***<num>*.

HUAWEI

- Replace **create** with **modify** in the command to modify the user key.

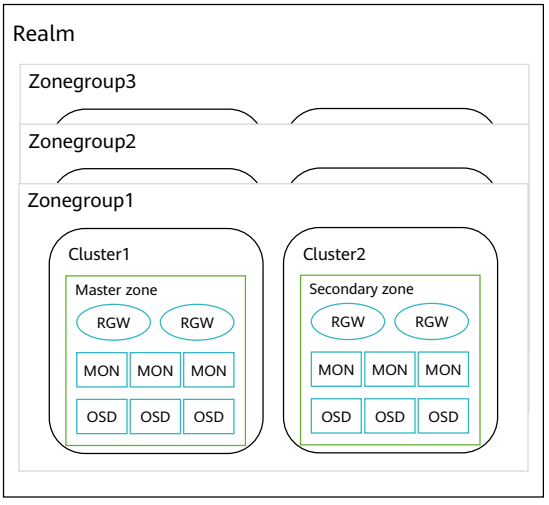
Contents

1. Object Storage
2. Single Site RGW Deployment
3. **Multisite RGW Deployment**

HUAWEI

Multisite RGW Deployment Overview

- Ceph disaster recovery is implemented by deploying RGW at multiple sites.
- In the multisite RGW deployment solution,
  - Multiple RGWs can be deployed in one zone. Multiple zones can be deployed in one Ceph cluster, but not across multiple clusters.
  - A zonegroup consists of one or more zones, where one zone is the master zone and the others are secondary zones. Data is synchronized between zones in the zonegroup.
  - A realm consists of one or more zonegroups, where one zonegroup is the master zonegroup and the others are secondary zonegroups. All zonegroups share the realm global configurations. All RGWs in the realm obtain configurations from the RGWs in the master zonegroup and master zone.



HUAWEI

Multisite RGW Deployment Architecture

- Architectures for the multisite RGW deployments:
  - Single-zone: A realm contains one zonegroup and one zone, and multiple RGWs are deployed in one zone.
  - Multi-zone: A realm contains one zonegroup that contains multiple zones.
  - Multi-zonegroup: A realm contains multiple zonegroups, and each zonegroup contains one or more zones.
  - Multi-realm: Multiple realms are deployed.

HUAWEI

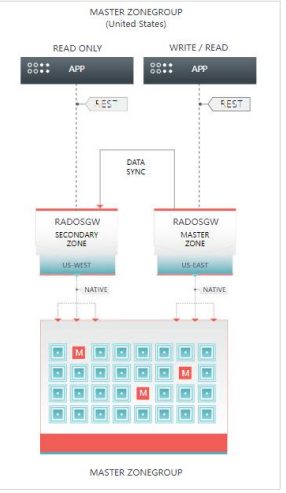
Version Management of Periods and Epochs

- Period records and traces the configuration status of zones, zonegroups, and realms. Epoch is used to record and trace the version number of zone period configuration changes.
- Each zone is associated with a period, and each period is associated with an epoch.
- Each period has a unique ID, including the current zone configurations and the period ID of the previous version.
- When the zone configuration is updated, the current period of the zone needs to be updated as well.

HUAWEI

Multi-Zone RGW Synchronization

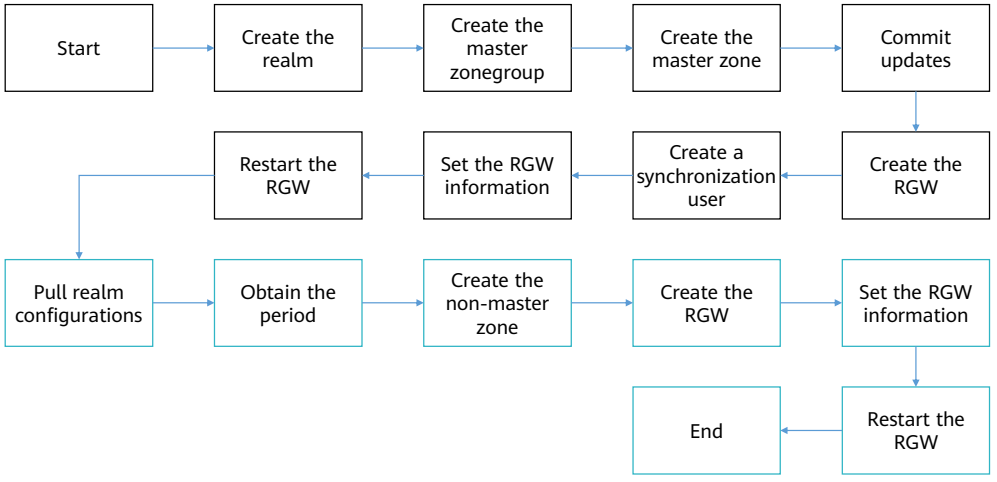
- RGW synchronizes metadata and data among all zonegroups.
- With multi-zone setups, RGW performs a full synchronization between the master zone and the non-master zone, and then subsequent incremental synchronization.
- When a user writes data to a zone in a zonegroup, data is synchronized to other zones in the same zonegroup.
- When data is synchronized between RGWs, all gateways in the active state update the data logs and notify other gateways.
- RGW synchronizes metadata, where the active gateway updates metadata logs and notifies other RGWs.



HUAWEI

- Image source: https://docs.ceph.com/en/quincy/radosgw/multisite/#delete-default-zonegroup-and-zone

Multisite Architecture Configuration

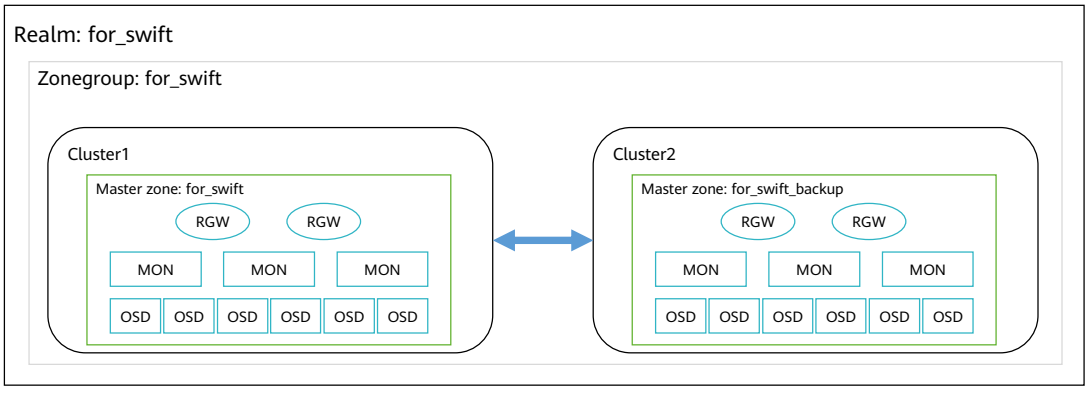


HUAWEI

- Synchronization user: a type of system user, who synchronizes information between zones.

Case: Practicing Setting the Multi-Zone Architecture of Multisite Object Storage

- Requirement: Set up the RGW in the multi-zone architecture according to the following figure.



HUAWEI

- Ceph installation and deployment are not described here.

Master Zone Configuration: Creating a Realm and a Master Zonegroup.

[ceph: root@ceph01 /]# **radosgw-admin realm create --rgw-realm=for_swift –default**

```
[ceph: root@ceph01 /]# radosgw-admin realm create --rgw-realm=for_swift --default
{
    "id": "441c10e3-2da3-4f21-bed1-5c37a8bba35a",
    "name": "for_swift",
    "current_period": "af8e331d-0d16-4812-9c0a-8a10b84616aa",
    "epoch": 1
}
```

Create a realm

[ceph: root@ceph01 /]# **radosgw-admin zonegroup create --rgw-realm=for_swift --rgw-zonegroup=for_swift --master –default**

```
[ceph: root@ceph01 /]# radosgw-admin zonegroup create --rgw-realm=for_swift --rgw-zonegroup=for_swift --master --default
{
    "id": "38aeeade-30f2-4e65-bc5f-682667b6ca46",
    "name": "for_swift",
    "api_name": "for_swift",
    "is_master": "true",
    "endpoints": [],
    "hostnames": [],
    "hostnames_s3website": [],
    "master_zone": "",
    "zones": [],
    "placement_targets": [],
    "default_placement": "",
    "realm_id": "441c10e3-2da3-4f21-bed1-5c37a8bba35a",
    "sync_policy": {
        "groups": []
    }
}
```

Create a master zonegroup

HUAWEI

Master Zone Configuration: Creating a Master Zone

[ceph: root@ceph01 /]# **radosgw-admin zone create --rgw-realm=for_swift --rgw-zonegroup=for_swift --rgw-zone=for_swift --master --default**

```
[ceph: root@ceph01 /]# radosgw-admin zone create --rgw-realm=for_swift --rgw-zonegroup=for_swift --rgw-zone=for_swift --master --default
{
    "id": "eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2",
    "name": "for_swift",
    "domain_root": "for_swift.rgw.meta:root",
    "control_pool": "for_swift.rgw.control",
    "gc_pool": "for_swift.rgw.log:gc",
    "lc_pool": "for_swift.rgw.log:lc",
    "log_pool": "for_swift.rgw.log",
    "intent_log_pool": "for_swift.rgw.log:intent",
    "usage_log_pool": "for_swift.rgw.log:usage",
    "roles_pool": "for_swift.rgw.meta:roles",
    "reshard_pool": "for_swift.rgw.log:reshard",
    "user_keys_pool": "for_swift.rgw.meta:users.keys",
    "user_email_pool": "for_swift.rgw.meta:users.email",
    "user_swift_pool": "for_swift.rgw.meta:users.swift",
    "user_uid_pool": "for_swift.rgw.meta:users.uid",
    "otp_pool": "for_swift.rgw.otp",
    "system_key": {
        "access_key": "",
        "secret_key": ""
    },
    "placement_pools": [
        {
            "key": "default-placement",
            "val": {
                "index_pool": "for_swift.rgw.buckets.index",
                "storage_classes": {
                    "STANDARD": {
                        "data_pool": "for_swift.rgw.buckets.data"
                    }
                },
                "data_extra_pool": "for_swift.rgw.buckets.non-ec",
                "index_type": 0
            }
        }
    ],
    "realm_id": "441c10e3-2da3-4f21-bed1-5c37a8bba35a",
    "notif_pool": "for_swift.rgw.log:notif"
}
```

Create a master zone

HUAWEI

Master Zone Configuration: Committing Updates

[ceph: root@ceph01 /]# **radosgw-admin period update --rgw-realm=for_swift --commit**

```
[ceph: root@ceph01 /]# radosgw-admin period update --rgw-realm=for_swift --commit
{
    "id": "49d3e015-46e5-4a96-bd26-f8f8014157b4",
    "epoch": 1,
    "predecessor_uuid": "af8e331d-0d16-4812-9c0a-8a10b84616aa",
    "sync_status": [],
    "period_map": {
        "id": "49d3e015-46e5-4a96-bd26-f8f8014157b4",
        "zonegroups": [
            {
                "id": "38aeeade-38f2-4e65-bc5f-682667b6ca46",
                "name": "for_swift",
                "api_name": "for_swift",
                "is_master": "true",
                "endpoints": [],
                "hostnames": [],
                "hostnames_s3website": [],
                "master_zone": "eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2",
                "zones": [
                    {
                        "id": "eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2",
                        "name": "for_swift",
                        "endpoints": [],
                        "log_meta": "false",
```

Commit updates

HUAWEI

Master Zone Configuration: Creating the RGW

```
[ceph: root@ceph01 /]# ceph orch apply rgw for_swift --realm=for_swift --zone=for_swift --placement="2" --port=8080
```

Create the RGW

```
[ceph: root@ceph01 /]# ceph -s
  cluster:
    id:     d5faf9ba-56c0-11ee-9ffe-fa163edb3348
    health: HEALTH_OK

  services:
    mon: 3 daemons, quorum ceph01.novalocal,ceph03,ceph02 (age 35m)
    mgr: ceph03.uessau(active, since 35m), standbys: ceph02.puzvmw, ceph01.novalocal.udalan
    osd: 9 osds: 9 up (since 35m), 9 in (since 7d)
    rgw: 2 daemons active (2 hosts, 1 zones)

  data:
    pools:   11 pools, 321 pgs
    objects: 413 objects, 6.0 MiB
    usage:   231 MiB used, 90 GiB / 90 GiB avail
    pgs:     321 active+clean
```


HUAWEI

Master Zone Configuration: Creating a System User

[ceph: root@ceph01 /]# **radosgw-admin user create --uid=system-user --display-name=system-user --access-key=Huawei@123 --secret=Huawei12#$ --system**

```
[ceph: root@ceph01 /]# radosgw-admin user create --uid=system-user --display-name=system-user --access-key=Huawei@123 --secret=Huawei12#$ --system
{
    "user_id": "system-user",
    "display_name": "system-user",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "subusers": [],
    "keys": [
        {
            "user": "system-user",
            "access_key": "Huawei@123",
            "secret_key": "Huawei12#$"
        }
    ],
    "swift_keys": [],
    "caps": [],
    "op_mask": "read, write, delete",
    "system": "true",
    "default_placement": "",
    "default_storage_class": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
```

Create a system user

HUAWEI

Master Zone Configuration: Creating a System User and Binding It to the Master Zone

[ceph: root@ceph01 /]# **radosgw-admin user create --uid=system-user --display-name=system-user --access-key=Huawei@123 --secret=Huawei12#$ --system**

```
[ceph: root@ceph01 /]# radosgw-admin user create --uid=system-user --display-name=system-user --access-key=Huawei@123 --secret=Huawei12#$ --system
{
    "user_id": "system-user",
    "display_name": "system-user",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "subusers": [],
    "keys": [
        {
            "user": "system-user",
            "access_key": "Huawei@123",
            "secret_key": "Huawei12#$"
        }
    ],
    "swift_keys": [],
    "caps": [],
    "op_mask": "read, write, delete",
    "system": "true",
    "default_placement": "",
    "default_storage_class": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
```

Create a system user

[ceph: root@ceph01 /]# **radosgw-admin zone modify --rgw-realm=for_swift --rgw-zonegroup=for_swift --rgw-zone=for_swift --endpoints 'http://ceph01:8080,http://ceph02:8080' --access-key=Huawei@123 --secret=Huawei12#$ --master --default**

Bind the user to the master zone

HUAWEI

Master Zone Configuration: Setting the RGW Information

[ceph: root@ceph01 /]#**ceph orch ps | grep rgw**

```
[ceph: root@ceph01 /]# ceph orch ps | grep rgw
rgw.for_swift.ceph01.cjavrh  ceph01.novalocal  *:8080
rgw.for_swift.ceph02.gfipgo  ceph02.novalocal  *:8080
```

Query the RGW Information

```
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph01.cjavrh rgw_realm for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph01.cjavrh rgw_zonegroup for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph01.cjavrh rgw_zone for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph02.gfipgo rgw_realm for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph02.gfipgo rgw_zonegroup for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph02.gfipgo rgw_zone for_swift
```

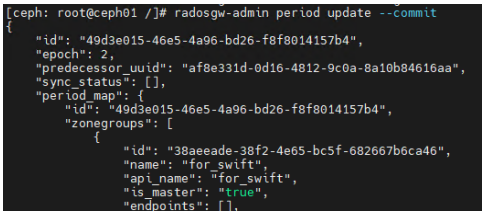
Set the RGW Information


```
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph01.cjavrh rgw_realm for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph01.cjavrh rgw_zonegroup for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph01.cjavrh rgw_zone for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph02.gfipgo rgw_realm for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph02.gfipgo rgw_zonegroup for_swift
[ceph: root@ceph01 /]# ceph config set client.rgw.for_swift.ceph02.gfipgo rgw_zone for_swift
```


HUAWEI

Master Zone Configuration: Committing Configurations and Restarting the RGW Daemon

[ceph: root@ceph01 /]# **radosgw-admin period update --commit**



Commit configurations

[ceph: root@ceph01 /]# **ceph orch restart rgw.for_swift**

Restart the RGW

HUAWEI

- You can run **ceph orch ls** to query the service name.

Secondary Zone Configuration: Pulling the Realm Configurations of the Master Zone

[ceph: root@ceph01 /]# **radosgw-admin realm pull --url=http://192.168.0.21:8080 --access-key=Huawei@123 --secret=Huawei12#$**

Pull realm configurations

```
[ceph: root@ceph11 /]# radosgw-admin realm pull --url=http://192.168.0.21:8080 --access-key=Huawei@123 --secret=Huawei12#$
2023-09-27T08:30:15.514+0000 7f8fcfb5c4c0  1 error read_lastest_epoch .rgw.root:periods.49d3e015-46e5-4a96-bd26-f8f8014157b4.latest_epoch
2023-09-27T08:30:15.530+0000 7f8fcfb5c4c0  1 Set the period's master zonegroup 38aeeade-38f2-4e65-bc5f-682667b6ca46 as the default
{
    "id": "441c10e3-2da3-4f21-bed1-5c37a8bba35a",
    "name": "for_swift",
    "current_period": "49d3e015-46e5-4a96-bd26-f8f8014157b4",
    "epoch": 2
}
```

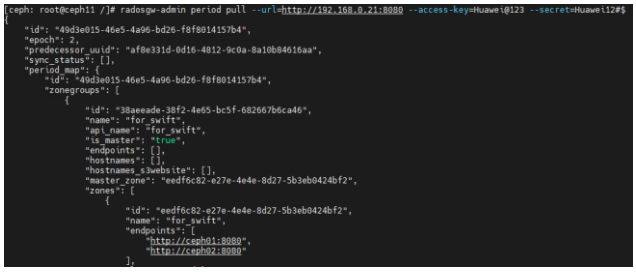
```
[ceph: root@ceph11 /]# radosgw-admin realm list
{
    "default_info": "441c10e3-2da3-4f21-bed1-5c37a8bba35a",
    "realms": [
        "for_swift"
    ]
}
[ceph: root@ceph11 /]# radosgw-admin zonegroup list
{
    "default_info": "38aeeade-38f2-4e65-bc5f-682667b6ca46",
    "zonegroups": [
        "for_swift"
    ]
}
```


HUAWEI

Secondary Zone Configuration: Pulling Period

```
[ceph: root@ceph01 /]# radosgw-admin period pull --url=http://192.168.0.21:8080 --access-key=Huawei@123 --secret=Huawei12#$
```

Pull the period




```
[ceph: root@ceph11 /]# radosgw-admin period list
{
    "periods": [
        "49d3e015-46e5-4a96-bd26-f8f8014157b4"
    ]
}
```


HUAWEI

Secondary Zone Configuration: Creating a Non-Master Zone

[ceph: root@ceph01 /]# **radosgw-admin zone create --rgw-zonegroup=for_swift --rgw-zone=for_swift_backup --endpoints "http://ceph11:8080, http://ceph12:8080" --access-key=Huawei@123 --secret=Huawei12#$**

Create a zone

```
[ceph: root@ceph11 /]# radosgw-admin zone create --rgw-zonegroup
2023-09-27T09:23:28.782+0000 7f469d25b4c0  0 failed reading obj
2023-09-27T09:23:28.782+0000 7f469d25b4c0  0 WARNING: could not
{
    "id": "ac2f559d-70c1-4d25-92fe-42c96f53aab0",
    "name": "for_swift_backup",
    "domain_root": "for_swift_backup.rgw.meta:root",
    "control_pool": "for_swift_backup.rgw.control",
    "gc_pool": "for_swift_backup.rgw.log:gc",
    "lc_pool": "for_swift_backup.rgw.log:lc",
    "log_pool": "for_swift_backup.rgw.log",
    "intent_log_pool": "for_swift_backup.rgw.log:intent",
    "usage_log_pool": "for_swift_backup.rgw.log:usage",
    "roles_pool": "for_swift_backup.rgw.meta:roles",
    "reshard_pool": "for_swift_backup.rgw.log:reshard",
    "user_keys_pool": "for_swift_backup.rgw.meta:users.keys",
    "user_email_pool": "for_swift_backup.rgw.meta:users.email",
    "user_swift_pool": "for_swift_backup.rgw.meta:users.swift",
    "user_uid_pool": "for_swift_backup.rgw.meta:users.uid",
    "otp_pool": "for_swift_backup.rgw.otp",
    "system_key": {
        "access_key": "Huawei@123",
        "secret_key": "Huawei12#$"
    },
    "placement_pools": [
```


HUAWEI

Secondary Zone Configuration: Creating the RGW and Configuring RGW Information.

[ceph: root@ceph01 /]# **ceph orch apply rgw for_swift --realm=for_swift --zone=for_swift_backup --placement="2" --port=8080**

```
[ceph: root@ceph11 /]# ceph -s
  cluster:
    id:     9b0381fc-5d0e-11ee-b606-fa163edb3352
    health: HEALTH_WARN
            Degraded data redundancy: 184/552 objects degraded (33

  services:
    mon: 2 daemons, quorum ceph11,ceph12 (age 6m)
    mgr: ceph11.zjjbgc(active, since 6m), standbys: ceph12.mtqqip
    osd: 6 osds: 6 up (since 6m), 6 in (since 106m)
    rgw: 2 daemons active (2 hosts, 1 zones)

  data:
    pools:   5 pools, 129 pgs
    objects: 184 objects, 6.7 KiB
    usage:   41 MiB used, 60 GiB / 60 GiB avail
    pgs:     184/552 objects degraded (33.333%)
             78 active+undersized
             51 active+undersized+degraded
```

Create the RGW

Configure the RGW

[ceph: root@ceph11 /]# **ceph config set client. rgw.for_swift.ceph11.ykrfbf rgw_realm for_swift**
[ceph: root@ceph11 **/]# ceph config set client. rgw.for_swift.ceph11.ykrfbf rgw_zonegroup for_swift**
[ceph: root@ceph11 /]# **ceph config set client. rgw.for_swift.ceph11.ykrfbf rgw_zone for_swift_backup**
[ceph: root@ceph11 /]# **ceph config set client.rgw.for_swift.ceph12.rhraxx rgw_realm for_swift**
[ceph: root@ceph11 /]# **ceph config set client.rgw.for_swift.ceph12.rhraxx rgw_zonegroup for_swift**
[ceph: root@ceph11 /]# **ceph config set client.rgw.for_swift.ceph12.rhraxx rgw_zone for_swift_backup**

HUAWEI

Secondary Zone Configuration: Updating the Realm and Restarting the RGW

[ceph: root@ceph01 /]# **radosgw-admin period update --rgw-realm=for_swift --commit**

Update the realm

```
[ceph: root@ceph11 /]# radosgw-admin period update --rgw-realm=for_swift --commit
2023-09-27T10:31:47.586+0000 7f9eae7c84c0  1 Cannot find zone id=edbc1161-e26b-4214-8184-5a66e3fb0f62 (name=for_swift_backup), switching to local zonegroup configuration
Sending period to new master zone eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2
{
    "id": "49d3e015-46e5-4a96-bd26-f8f8014157b4",
    "epoch": 3,
    "predecessor_uuid": "af8e331d-0d16-4812-9c0a-8a10b84616aa",
    "sync_status": [],
    "period_map": {
        "id": "49d3e015-46e5-4a96-bd26-f8f8014157b4",
        "zonegroups": [
            {
                "id": "38aeeade-38f2-4e65-bc5f-682667b6ca46",
                "name": "for_swift",
                "api_name": "for_swift",
                "is_master": "true",
                "endpoints": [],
                "hostnames": [],
                "hostnames_s3website": [],
                "master_zone": "eedf6c82-e27e-4e4e-8d27-5b3eb0424bf2",
                "zones": [
                    {
```

```
[ceph: root@ceph11 /]# ceph orch restart rgw.for_swift
Scheduled to restart rgw.for_swift.ceph11.ykrfbf on host 'ceph11'
Scheduled to restart rgw.for_swift.ceph12.rhraxx on host 'ceph12'
```

Restart the RGW

HUAWEI

Quiz

1. (Multiple-answer question) In the Ceph multisite architecture, which of the following operations in a non-master zone will be synchronized to the master zone?

   A. Create a user.

   B. Upload an object.

   C. Create a bucket.

   D. Download an object.

HUAWEI

- BC

Summary

- This course introduces how to deploy, install, and use the Ceph object storage gateway at the single site and multiple sites.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling	Description
PG	Placement group	Subset of the Ceph resource pool
RBD	RADOS block device	N/A
RGW	RADOS gateway	N/A
MDS	Ceph metadata server	RADOS metadata server

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

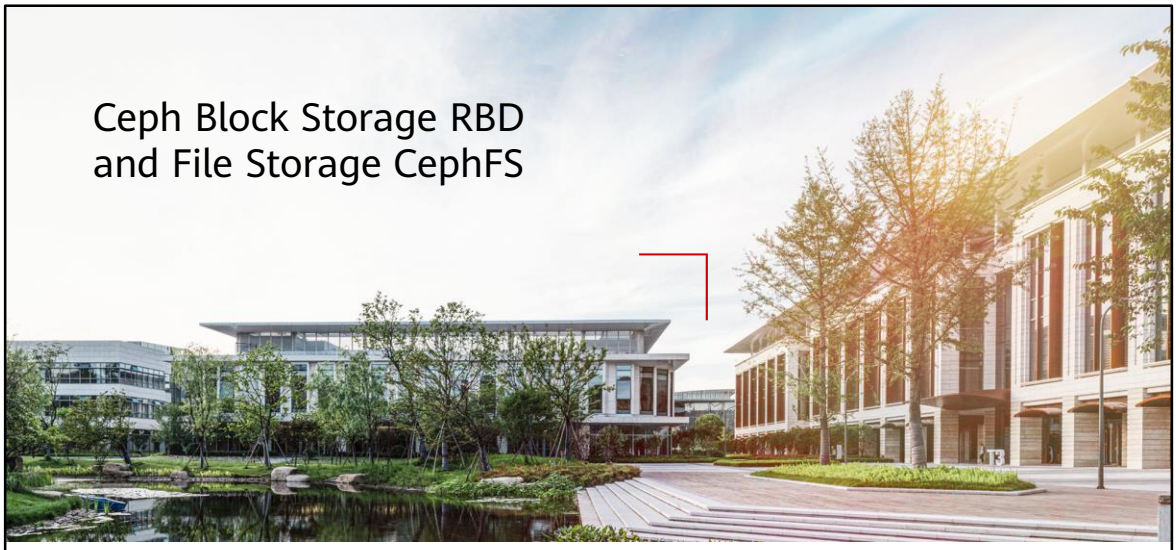
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Ceph Block Storage RBD and File Storage CephFS



Foreword

- Ceph also supports block and file storage besides object storage. This course introduces the implementation, configuration, and usage of Ceph block storage and file storage.

Objectives

- On completion of this course, you will understand:
  - RBD block storage
  - CephFS file storage

HUAWEI

Contents

1. **Basic Usage of Ceph Block Storage**
2. Advanced Features of Ceph Block Storage
3. Introduction to CephFS

HUAWEI

Ceph Block Storage

- The Ceph block storage function is built based on RADOS and stores data striped over multiple OSDs after thin provisioning and size adjustment. Ceph block storage implements strong consistency between snapshots, replication, and storage.
- Access to block storage:
  - Librbd user-mode interface: The **librados.so** library acts as a client to connect an application to the RADOS cluster.
  - Krbd kernel-mode interface: Run the **rbd** command to map a block device to a host device, which can be mounted to a specified directory.

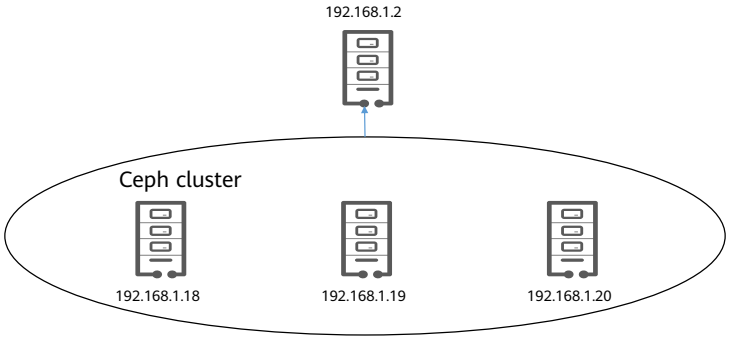


HUAWEI

- Krbd is used to directly mount Ceph block devices in the Linux operating system.
- Librbd is mainly used in virtualization and cloud platform scenarios.

Case 1: Creating and Mounting Two Ceph RBDs to the Client

- Requirement: Create a block storage resource pool in the Ceph cluster, create two block storage devices (RADOS block devices, or RBDs) with 3 GB and 5 GB, respectively, then mount them to the client and implement automatic mount upon system startup.
- The following figure shows the address planning and topology architecture.



Step 1: Creating Resource Pool test-rbd and Setting the Application Type to rbd

- Create resource pool test-rbd in the existing cluster.

```
[ceph: root@ceph01 /]# ceph osd pool create test-rbd
pool 'test-rbd' created
[ceph: root@ceph01 /]# ceph osd pool ls
device_health_metrics
test
test-rbd
```

- Set the application type of the test-rbd resource pool to **rbd**.

```
[ceph: root@ceph01 /]# ceph osd pool application enable test-rbd rbd
enabled application 'rbd' on pool 'test-rbd'
[ceph: root@ceph01 /]# ceph osd pool ls detail | grep test-rbd
pool 13 'test-rbd' replicated size 3 min_size 2 crush_rule 0 object_hash rjenkins pg_num 32 pgp_num 32 autoscale_mode on last_change
289 flags hashpspool stripe_width 0 application rbd
```


HUAWEI

Step 2: Creating a User and the Required RBD

- Create a user named test and grant it the read, write, and execute permissions on resource pool test-rbd.

```
[ceph: root@ceph01 /]# ceph auth add client.test mon 'allow r' osd 'allow rwx pool=test-rbd'
added key for client.test
[ceph: root@ceph01 /]# ceph auth get client.test
[client.test]
        key = AQCpIgBlLvFaEBAADuPgtRAnD+Ok3IV/AvHJLg==
        caps mon = "allow r"
caps osd = "allow rwx pool=test-rbd"
```

- Create RBDs test1 (3G) and test2 (5G).

```
[ceph: root@ceph01 /]# rbd create --size 3G test-rbd/test1
[ceph: root@ceph01 /]# rbd create --size 5G test-rbd/test2
[ceph: root@ceph01 /]# rbd info test-rbd/test1
rbd image 'test1':
        size 3 GiB in 768 objects
...
[ceph: root@ceph01 /]# rbd info test-rbd/test2
rbd image 'test2':
        size 5 GiB in 1280 objects
...
```


HUAWEI

- You can run **ceph auth add client.test osd 'profile rbd pool=test-rbd** to create the user.

Step 3: Importing the User Keyring to the Client and Mapping the RBDs on the Client

- Install ceph-common on the client.
- Import the user keyring to the client on the controller.

```
[ceph: root@ceph01 /]# ceph auth get client.test -o ceph.client.test.keyring
[ceph: root@ceph01 /]# scp ceph.client.test.keyring root@192.168.1.2:/etc/ceph/ceph.client.test.keyring
ceph.client.test.keyring                                      100%  100   209.4KB/s   00:00
[ceph: root@ceph01 /]# scp /etc/ceph/ceph.conf root@192.168.1.60:/etc/ceph/ceph.conf
ceph.conf                                                     100%  271   676.5KB/s   00:00
```

- On the client, use the test user to map the RBDs.

```
[root@client ~]# rbd map test-rbd/test1 --name client.test
/dev/rbd0
[root@client ~]# rbd map test-rbd/test2 --name client.test
/dev/rbd1
[root@client ~]# lsblk
NAME               MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
...........
rbd0               252:0    0    3G  0 disk
rbd1               252:16   0    5G  0 disk
```


HUAWEI

- Before performing this operation, run **yum install -y ceph-common** to install ceph-common on the client.

Step 4: Formatting RBDs and Mounting Them to the Specified Directory

- Format rbd0 and rbd1 and mount them to **/mnt/rbd0** and **/mnt/rbd1**.

```
[root@client ~]# mkfs.xfs /dev/rbd0
.........
[root@client ~]# mkfs.xfs /dev/rbd1
......
[root@client ~]# mount /dev/rbd0 /mnt/rbd0
[root@client ~]# mount /dev/rbd1 /mnt/rbd1
[root@client ~]# df -h
Filesystem                Size  Used Avail Use% Mounted on
.......
/dev/rbd0                 3.0G   54M  3.0G   2% /mnt/rbd0
/dev/rbd1                 5.0G   69M  5.0G   2% /mnt/rbd1
```


HUAWEI

Step 5: Adding Configurations to Enable the RBD Automatic Mounting upon System Startup

- Add corresponding configurations to the **/etc/fstab** file. The content after the modification is as follows:

```
.......
/dev/rbd/test-rbd/test1          /mnt/rbd0       xfs     defaults,_netdev 0 0
/dev/rbd/test-rbd/test2          /mnt/rbd1       xfs     defaults,_netdev 0 0
```

- Compile rbdmap and modify it as follows:

```
# RbdDevice             Parameters
#poolname/imagename     id=client,keyring=/etc/ceph/ceph.client.keyring
test-rbd/test1          id=test,keyring=/etc/ceph/ceph.client.test.keyring
test-rbd/test2          id=test,keyring=/etc/ceph/ceph.client.test.keyring
```

- Set the test-rbd daemon to start upon system startup.

```
[root@client ~]# systemctl enable rbdmap
Created symlink /etc/systemd/system/multi-user.target.wants/rbdmap.service → /usr/lib/systemd/system/rbdmap.service.
```


HUAWEI

Other Commands Related to Ceph Block Storage

- List all RBDs in the resource pool: **rbd [--pool** *pool-name***] ls**
- Query detailed information about a specified RBD: **rbd info** *[pool-name/] image-name*
- Query the status of a specified RBD: **rbd status** *[pool-name/]image-name*
- Query the capacity of a specified RBD: **rbd du** *[pool-name/]image-name*
- Check whether the specified RBD is locked: **rbd lock ls** *[pool-name/]image-name*
- Delete the lock of a specified RBD: **rbd [--pool** *pool-name***] lock rm** *image-name lock-id locker*

HUAWEI

Case 2: Viewing all RBDs in test-rbd and Adjusting Them as Required

- Requirements:
  - 1. In the resource pool test-rbd, expand the RBD capacity from 3 GB to 6 GB and adjust the client accordingly to make the expansion take effect.
  - 2. Rename the expanded RBD rbd-6G.
  - 3. Unmap and delete the 5 GB RBD from the resource pool test-rbd.

HUAWEI

Step 1: Querying the RBDs in the test-rbd Resource Pool

- Query the RBDs in the resource pool.

```
[ceph: root@ceph01 /]# rbd --pool test-rbd ls
test1
test2
```

- Query the capacity of the two RBDs.

```
NAME   PROVISIONED  USED
test1        3 GiB  36 MiB
[ceph: root@ceph01 /]# rbd --pool test-rbd du test2
NAME   PROVISIONED  USED
test2        5 GiB  36 MiB
```


HUAWEI

- You can also run **rbd --pool test-rbd info test1** to view the capacity.

Step 2: Expanding the Capacity of the test1 RBD

- Expand the capacity of test1 to 6 GB.

```
[ceph: root@ceph01 /]# rbd --pool test-rbd resize test1 --size 6G
Resizing image: 100% complete...done.
[ceph: root@ceph01 /]# rbd --pool test-rbd du test1
NAME   PROVISIONED  USED
test1        6 GiB  36 MiB
```

- Expand the capacity of the partition on the client for the configurations to take effect.

```
[root@client ~]# xfs_growfs -d /mnt/rbd0
meta-data=/dev/rbd0              isize=512    agcount=8, agsize=98304 blks
         =                       sectsz=512   attr=2, projid32bit=1
.........
data blocks changed from 786432 to 1572864
[root@client ~]# df -h
Filesystem                Size  Used Avail Use% Mounted on
......
/dev/rbd0                 6.0G   76M  6.0G   2% /mnt/rbd0
/dev/rbd1                 5.0G   69M  5.0G   2% /mnt/rbd1
tmpfs                     342M     0  342M   0% /run/user/0
```


HUAWEI

Step 3: Renaming test1 to rbd-6G

- Rename the test1 RBD to rbd-6G.

```
[ceph: root@ceph01 /]# rbd mv test-rbd/test1 test-rbd/rbd-6G
[ceph: root@ceph01 /]# rbd --pool test-rbd ls
rbd-6G
test2
```

- Update the client configurations. The modified rbdmap is as follows:

```
# RbdDevice             Parameters
#poolname/imagename     id=client,keyring=/etc/ceph/ceph.client.keyring
test-rbd/rbd-6G         id=test,keyring=/etc/ceph/ceph.client.test.keyring
test-rbd/test2          id=test,keyring=/etc/ceph/ceph.client.test.keyring
```

- After the modification, the **/etc/fstab** configurations are as follows:

```
......
/dev/rbd/test-rbd/rbd-6G            /mnt/rbd0       xfs     defaults,_netdev 0 0
/dev/rbd/test-rbd/test2             /mnt/rbd1       xfs     defaults,_netdev 0 0
```


HUAWEI

- After the configurations are modified, restart the system to check whether the configurations take effect.

Step 4: Deleting the test2 RBD

- Remove the mapping of test2 on the client.

```
[root@client ~]# umount /mnt/rbd1
[root@client ~]# rbd unmap test-rbd/test2
[root@client ~]# lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
.......
rbd0                252:0    0    6G  0 disk /mnt/rbd0
```

- Delete the configurations related to test2 from rbdmap and fstab on the client.
- Delete the RBD from the controller.

```
[ceph: root@ceph01 /]# rbd trash mv test-rbd/test2
[ceph: root@ceph01 /]# rbd --pool test-rbd ls
rbd-6G
```

- Permanently delete the RBD.

```
[ceph: root@ceph01 /]# rbd trash ls test-rbd
17270444f1efb test2
[ceph: root@ceph01 /]# rbd --pool test-rbd trash rm 17270444f1efb
Removing image: 100% complete...done.
```


HUAWEI

Contents

1. Basic Usage of Ceph Block Storage
2. **Advanced Features of Ceph Block Storage**
3. Introduction to CephFS

HUAWEI

RBD Features

- When creating an RBD, enable the corresponding feature with the **--image-feature** parameter. The RBD supports the following features:
  - **layering**: refers to the creation of copy-on-write clones of block devices.
  - **striping**: stripes that can improve performance, version 2
  - **exclusive-lock**: exclusive lock
  - **object-map**: object mapping. Available with **exclusive-lock**.
  - **fast-diff**: fast difference calculation. Available with **object-map**.
  - **deep-flatten**: snapshot flattening
  - **journaling**: log function. Available with **exclusive-lock**.
  - **data-pool**: erasure-coded pool support

HUAWEI

Configuring RBD Features

- Run **rbd info** to query the features enabled on the RBD.

```
[ceph: root@ceph01 /]# rbd info test-rbd/rbd-6G
rbd image 'rbd-6G':
.......
features: layering, exclusive-lock, object-map, fast-diff, deep-flatten
```

- To enable or disable a feature after an RBD is created, run **rbd feature enable/disable** *[pool_name/]* *<image_name>* *<feature_name>*.

```
[ceph: root@ceph01 /]# rbd feature enable test-rbd/rbd-6G journaling
[ceph: root@ceph01 /]# rbd info test-rbd/rbd-6G
rbd image 'rbd-6G':
        size 6 GiB in 1536 objects
        order 22 (4 MiB objects)
        snapshot_count: 0
        id: 172505c22d28b
        block_name_prefix: rbd_data.172505c22d28b
        format: 2
        features: layering, exclusive-lock, object-map, fast-diff, deep-flatten, journaling
        op_features:
.......
```


HUAWEI

RBD Client Cache Mechanism

- When a client uses librbd to invoke Ceph block storage, application data needs to be written to the RBD mounted to the OS to interact with RADOS over librbd, and written to OSDs. If the OSD has a delay, there is a delay on the application waiting for the data write, causing the application to freeze. The cache mechanism of the RBD client resolves this problem.
- Cache mechanisms of the RBD client:
  - Direct write mode
  - Writeback mode
- The cache memory used by the RBD client is provided by the client host.

HUAWEI

- The RBD client cache is mainly used when librbd invokes Ceph block storage.
- Direct write is also called writethrough. In this mode, data is directly written to OSD disks.
- In writeback mode, librbd writes data to the local cache of the server and then periodically flushes the data to OSD disks.
- The writeback mode improves the read and write performance of Ceph block devices, but can cause data loss.

RBD Client Cache Configuration Parameters

Parameter	Description	Default Value
rbd_cache	Switch of the cache function. The value **true** means that the cache function is enabled, and **false** indicates disabled.	True
rbd_cache_policy	Cache mechanism, which can be writearound, writethrough, or writeback.	writearound
rbd_cache_size	Size of the cache space allocated to each RBD, used in direct write and writeback modes.	30M
rbd_cache_max_dirty	Maximum dirty data size of each RBD. The value must be less than that of **rbd_cache_size**. This parameter is used in direct write mode.	24M
rbd_cache_target_dirty	Cache size of each RBD starting to clear dirty data. The value must be less than that of **rbd_cache_max_dirty**.	16M
rbd_cache_max_dirty_age	Period for clearing dirty data in the cache, in seconds.	1

HUAWEI

RBD Snapshots

- A snapshot of an RBD is a read-only copy created at a point in time during the RBD lifecycle.
- The RBD snapshot supports the layering feature. A new RBD can be quickly cloned based on the snapshot.
- Snapshots can be implemented by running the **rbd** command or by invoking interfaces of upper-layer applications, such as QEMU, libvirt, and OpenStack.
- When creating a snapshot, you are advised to suspend or stop I/Os on the RBD.



HUAWEI

Basic Operations of RBD Snapshots

- Basic RBD snapshot operations include creating, viewing, rolling back, and deleting snapshots. The commands are as follows:

```
[ceph: root@ceph01 /]# rbd snap create test-rbd/rbd-6G@snap1          ##### Create snapshots.
Creating snap: 100% complete...done.
[ceph: root@ceph01 /]# rbd snap create test-rbd/rbd-6G@snap2
Creating snap: 100% complete...done.
[ceph: root@ceph01 /]# rbd snap ls test-rbd/rbd-6G                   ##### List all snapshots.
SNAPID  NAME   SIZE   PROTECTED  TIMESTAMP
     4  snap1  6 GiB             Wed Sep 13 08:07:53 2023
     5  snap2  6 GiB             Wed Sep 13 08:07:59 2023
[ceph: root@ceph01 /]# rbd snap rollback test-rbd/rbd-6G@snap1       ##### Roll back to snapshots.
Rolling back to snapshot: 100% complete...done.
[ceph: root@ceph01 /]# rbd snap rm test-rbd/rbd-6G@snap1             ##### Remove snapshots.
Removing snap: 100% complete...done.
[ceph: root@ceph01 /]# rbd snap ls test-rbd/rbd-6G
SNAPID  NAME   SIZE   PROTECTED  TIMESTAMP
     5  snap2  6 GiB             Wed Sep 13 08:07:59 2023
[ceph: root@ceph01 /]# rbd snap purge test-rbd/rbd-6G                ##### Remove all snapshots.
Removing all snapshots: 100% complete...done.
[ceph: root@ceph01 /]# rbd snap ls test-rbd/rbd-6G
[ceph: root@ceph01 /]#
```


HUAWEI

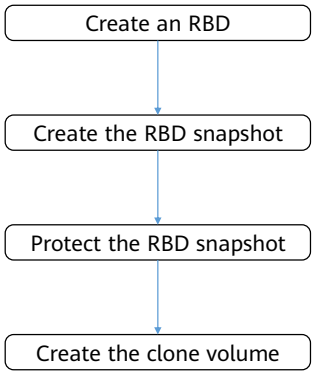
Layering and Cloning of RBD Snapshots

- Ceph supports to create multiple copy-on-write (COW) clone volumes based on RBD snapshots.
- The snapshot layering feature enables a client to quickly clone multiple volumes.
- RBD snapshots are read-only, and clone volumes created are readable and writable.
- Clone volumes depend on RBD snapshots. Therefore, snapshots must be protected before clone volumes are created.



Creating a Snapshot Clone Volume

- Create a snapshot volume as follows:
  - 1. Create an RBD.
  - 2. Create the RBD snapshot.
  - 3. Protect the RBD snapshot to ensure the availability of clone volumes.
  - 4. Create the clone volume based on the RBD snapshot.



Creating the Clone Volume of the RBD Snapshots

- To create the clone volume of the RBD snapshots, run the following commands:

```
[ceph: root@ceph01 /]# rbd snap create test-rbd/rbd-6G@snap1                ##### Create a snapshot.
Creating snap: 100% complete...done.
[ceph: root@ceph01 /]# rbd snap protect  test-rbd/rbd-6G@snap1              ##### Protect the snapshot.
[ceph: root@ceph01 /]# rbd snap ls test-rbd/rbd-6G
SNAPID  NAME   SIZE   PROTECTED  TIMESTAMP
     8       snap1  6 GiB    yes        Wed Sep 13 08:55:00 2023            ###### Change the value of PROTEECTED to yes.
[ceph: root@ceph01 /]# rbd clone test-rbd/rbd-6G@snap1 test-rbd/clone1   ##### Create a clone volume.
[ceph: root@ceph01 /]# rbd children  test-rbd/rbd-6G@snap1                  ##### View the clone volume of the specified snapshot.
test-rbd/clone1
[ceph: root@ceph01 /]# rbd flatten test-rbd/clone1                              ##### Convert the clone volume to an
independent RBD.
Image flatten: 100% complete...done.
[ceph: root@ceph01 /]# rbd snap unprotect  test-rbd/rbd-6G@snap1          ##### Remove the snapshot protection.
[ceph: root@ceph01 /]# rbd snap ls test-rbd/rbd-6G
SNAPID  NAME   SIZE   PROTECTED  TIMESTAMP
     8  snap1  6 GiB             Wed Sep 13 08:55:00 2023
[ceph: root@ceph01 /]# rbd snap purge test-rbd/rbd-6G                      ##### The snapshot can be removed after the protection
is canceled.
Removing all snapshots: 100% complete...done.
[ceph: root@ceph01 /]# rbd snap ls test-rbd/rbd-6G
[ceph: root@ceph01 /]#
```


HUAWEI

Importing and Exporting RBDs

- RBDs support full import and export. The detailed operations are as follows:

```
[ceph: root@ceph01 /]# ceph osd pool create target                          ##### Create the resource pool target.
pool 'target' created
[ceph: root@ceph01 /]# ceph osd pool application enable target rbd
enabled application 'rbd' on pool 'target'
[ceph: root@ceph01 /]# rbd -p test-rbd export rbd-6G /tmp/rbd-export        ##### Export the RBD as the rbd-export file.
Exporting image: 100% complete...done.
[ceph: root@ceph01 /]# rbd -p target import /tmp/rbd-export rbd-export      ##### Use the exported file to import the RBD to the
resource pool target.
Importing image: 100% complete...done.
[ceph: root@ceph01 /]# rbd -p target list
rbd-export
```

- RBDs can also use snapshots to perform incremental import and export. The procedure is as follows:

```
[ceph: root@ceph01 /]# rbd snap create test-rbd/rbd-6G@snap1
Creating snap: 100% complete...done.
[ceph: root@ceph01 /]# rbd export-diff test-rbd/rbd-6G@snap1 /tmp/snap1      ##### Export incremental data as file /tmp/snap1
based on the snapshot.
Exporting image: 100% complete...done.
[ceph: root@ceph01 /]# rbd import-diff /tmp/snap1 target/rbd-export           ##### Import incremental data to rbd-export based
on the exported file.
Importing image diff: 100% complete...done.
```


HUAWEI

Contents

1. Basic Usage of Ceph Block Storage
2. Advanced Features of Ceph Block Storage
3. **Introduction to CephFS**

HUAWEI

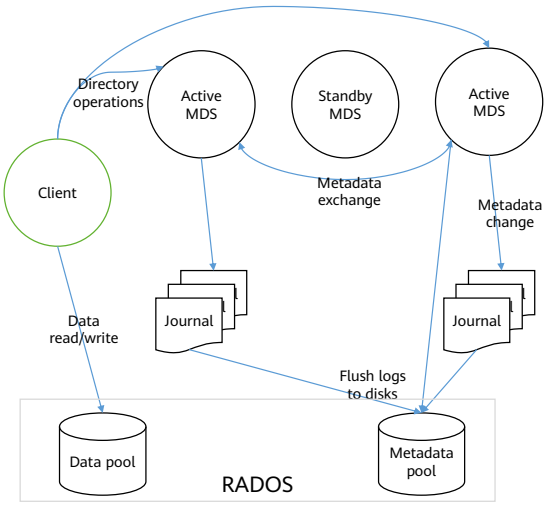
Ceph File System

- The Ceph File System (CephFS) is a POSIX-compatible file system built on RADOS, the Ceph's distributed store.
- CephFS provides multi-user, high-availability, and high-performance file storage.
- It is used for shared home directory, temporary storage space for HPC, and shared storage for workflows.
- CephFS must run the metadata server (MDS) daemon in the cluster. The MDS daemon process is ceph-mds, which is used to manage metadata of files stored in CephFS and coordinate cluster access.
- ceph-mds separates metadata from data, reducing complexity and improving reliability.

HUAWEI

CephFS Working Principles

- MDS manages the metadata of CephFS clients. ceph-mds provides related information for client access, such as the specific location of a file in the file system structure.
- MDS also manages the directory structure and file metadata, such as the owner, timestamp, and permission.
- MDS responds to cache access and management to maintain cache consistency.
- MDS has two states:
  - Active: Manages CephFS metadata.
  - Standby: When the active MDS does not respond, the standby MDS changes to the active state.
- During data access, CephFS passes the MON verification and obtains the cluster mapping information. Then, CephFS requests related metadata from MDS and communicates with OSDs based on the requested metadata.

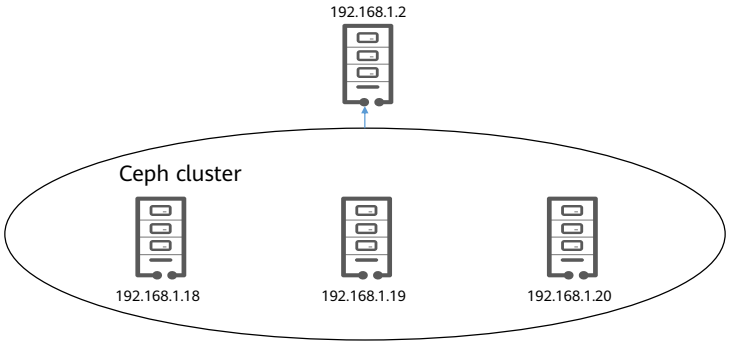


HUAWEI

- Ceph supports MDS multi-active deployment. However, this is a beta function and is not recommended for production systems.

Practice: Using Ceph to Create and Mount a File System to a Client

- Requirement: Manually create two resource pools data_cephfs and metadata_cephfs for the CephFS data pool and metadata pool, respectively. Then, mount the created CephFS to the client and perform tests and verification.
- The following figure shows the address planning and topology architecture.



HUAWEI

- The CephFS resource pool can be deployed automatically. The deployed resource pool is a replicated pool by default. The command is **ceph fs volume create xxx --placement= "2"**.
- At least two MDSs need to be deployed; otherwise, the system will report an error.

Step 1: Creating Resource Pools and Formatting the File System

- Create two replicated pools: data_cephfs and metadata_cephfs. data_cephfs is the data pool of CephFS, and metadata_cephfs is the metadata pool of CephFS. The detailed operations are as follows:

```
[ceph: root@ceph01 /]# ceph osd pool create metadata_cephfs
pool 'metadata_cephfs' created
[ceph: root@ceph01 /]# ceph osd pool create data_cephfs
pool 'data_cephfs' created
[ceph: root@ceph01 /]#
```

- Format the file system in the created resource pools.

```
[ceph: root@ceph01 /]# ceph fs new fs01 metadata_cephfs data_cephfs
new fs with metadata pool 18 and data pool 17
[ceph: root@ceph01 /]# ceph fs ls
name: fs01, metadata pool: metadata_cephfs, data pools: [data_cephfs ]
```

- Deploy the MDS.

```
[ceph: root@ceph01 /]# ceph orch apply mds fs01 --placement="2"
Scheduled mds.fs01 update...
```


HUAWEI

Step 2: Creating CephFS Users and Mounting Them to the File System

- Create users user01 and user02. user01 has all permissions, and user02 has the read permission.

```
[ceph: root@ceph01 /]# ceph fs authorize fs01 client.user01 / rwps -o ceph.client.user01.keyring
[ceph: root@ceph01 /]# ceph fs authorize fs01 client.user02 / r -o ceph.client.user02.keyring
[ceph: root@ceph01 /]# ceph auth ls | grep -A 4 user
installed auth entries:

client.user01
        key: AQDIfgJlsLGSMxAA9fP9luu5ZFqfl2+20JOezw==
        caps: [mds] allow rwps fsname=fs01
        caps: [mon] allow r fsname=fs01
        caps: [osd] allow rw tag cephfs data=fs01
client.user02
        key: AQDWfgJlSQIfJxAAvVTWC1Hml12DBAEXw1yQ3A==
        caps: [mds] allow r fsname=fs01
        caps: [mon] allow r fsname=fs01
        caps: [osd] allow r tag cephfs data=fs01
```


HUAWEI

- CephFS permissions include:
  - **r**: read-only. If no other restriction is specified, the **r** permission is granted to CephFS subdirectories.
  - **w**: write. If no other restriction is specified, the **w** permission is granted to CephFS subdirectories.
  - **p**: permission to use quotas
  - **s**: permission to create snapshots

Step 3: Mounting the Client

- Create directories **/mnt/user01** and **//mnt/user02**, and mount the created file system to the corresponding directories as users, user01 and user02. The detailed operations are as follows:

```
[root@client ~]# mount -t ceph 192.168.1.18:6789,192.168.1.19:6789:/ /mnt/user01 -o
name=user01,secret="AQDIfgJlsLGSMxAA9fP9luu5ZFqfl2+20JOezw=="
[root@client ~]# mount -t ceph 192.168.1.18:6789,192.168.1.19:6789:/ /mnt/user02 -o
name=user02,secret="AQDWfgJlSQIfJxAAvVTWC1Hml12DBAEXw1yQ3A=="
[root@client ~]# cd /mnt/user01
[root@client user01]# touch test
[root@client user01]# cd ../user02
[root@client user02]# rm -f test
rm: cannot remove 'test': Permission denied
```

- To implement permanent mounting, you need to modify **/etc/fstab** and save the authentication information to a specified file. The content is as follows:

```
:/        /mnt/user02     ceph     defaults,_netdev,name=user02,secretfile=/etc/ceph/user02.txt 0 0
```


HUAWEI

- In this slide, the Linux kernel is used for mounting. Ceph also supports mounting using FUSE, but its performance is not as good as that of mounting the kernel. You are advised to use the kernel module for mounting.
- After the **ceph.conf** file is retained on the client, you can omit the MON information. Change the commands to **mount -t ceph :/ /mnt/user02 -o name=user02,secret="AQDWfgJlSQIfJxAAvVTWC1Hml12DBAEXw1yQ3A==**" during mounting.
- When using a key file for mounting, you only need to save the key in the corresponding file. For example, the content of **user02.txt** in the body is **AQDWfgJlSQIfJxAAvVTWC1Hml12DBAEXw1yQ3A==**.
- The permission on the key file must be set to **600**.

Quiz

1. (Multiple-answer question) Which of the following scenarios will use the RBD snapshot function?
   A. Data backup
   B. Destructive testing of applications
   C. Ceph version upgrade
   D. Resource pool capacity expansion

HUAWEI

- AB

Summary

- This course introduces Ceph block storage and file storage, and also discusses their relationship with resource pools.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling	Description
PG	Placement group	Subset of the Ceph resource pool
RBD	RADOS block device	N/A
RGW	RADOS gateway	N/A
MDS	Ceph metadata server	RADOS metadata server

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



CRUSH Map Overview and Configurations



Foreword

- The core algorithm of Ceph is CRUSH, which is used to save objects in corresponding OSDs. This course focuses on the working principles of CRUSH.

HUAWEI

Objectives

- On completion of this course, you will understand:
  - Ceph architecture
  - Ceph components
  - Ceph working principles

HUAWEI

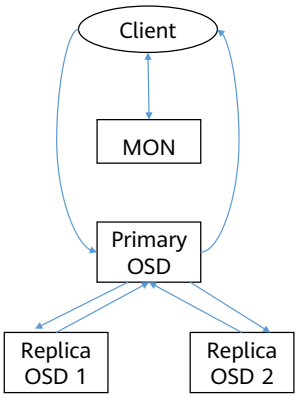
Contents

1. **Introduction to CRUSH Map**
2. CRUSH Map Configurations

HUAWEI

Introduction to CRUSH and CRUSH Map

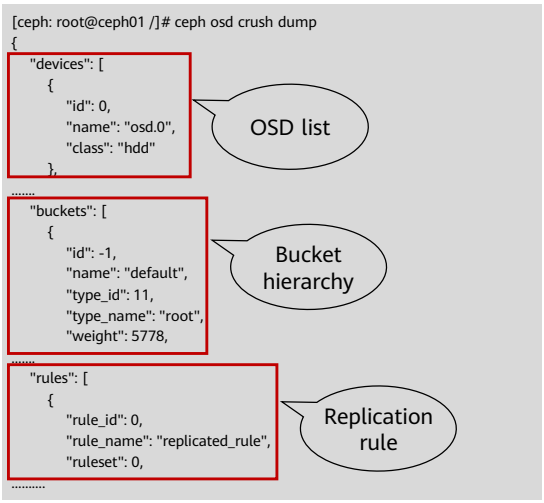
- CRUSH is the core of Ceph. It is used to calculate the data storage location to store and retrieve data.
- The biggest advantage of CRUSH is that it empowers Ceph clients to directly communicate with OSDs rather than through a centralized server, which avoids performance bottlenecks.
- The CRUSH map is the key mechanism of CRUSH. It maps data to OSDs based on replication and failure domain policies.



HUAWEI

CRUSH Map Components

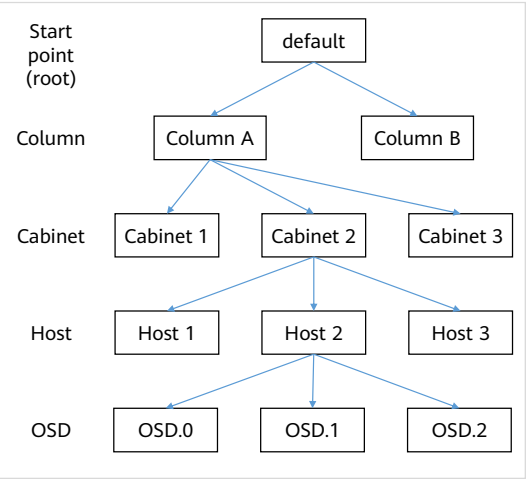
- A CRUSH map consists of three parts:
  - OSD list
  - Bucket hierarchy
  - Replication rule



HUAWEI

OSD List – CRUSH Location of the OSD

- The location of the OSD in CRUSH is specified by a series of key-value pairs. For example, OSD1 is located on the second server in the second cabinet in column A. Generally, the OSD location starts from "default".
  - The key sequence is not the most important indicator.
  - The key name must be an existing CRUSH type. By default, the key can be **root**, **datacenter**, **room**, **row**, **pod**, **pdu**, **rack**, **chassis**, and **host**.
  - When determining the OSD location, it is unnecessary to set values for all CRUSH types.



HUAWEI

- The default CRUSH type is sufficient for most clusters. If the CRUSH type is insufficient, modify the CRUSH map to customize it.
- Run **ceph osd tree** to view the OSD location.

CRUSH Map Hierarchy

- A CRUSH map shows the hierarchical topology of a Ceph cluster. It uses the OSD, which stores data, as the leaf and the CRUSH types – devices with the same physical features – as the internal nodes.
- Each node in the CRUSH map hierarchy has a weight that indicates the proportion of the total data that the node should store. The weight is set in the OSD and automatically sent to the tree structure.

```
[ceph: root@ceph01 /]# ceph osd tree
ID  CLASS  WEIGHT   TYPE NAME        STATUS  REWEIGHT  PRI-AFF
-1         0.08817  root default
-3         0.02939      host ceph01
 0    hdd  0.00980          osd.0        up   1.00000  1.00000
 1    hdd  0.00980          osd.1        up   1.00000  1.00000
 2    hdd  0.00980          osd.2        up   1.00000  1.00000
-5         0.02939      host ceph02
 3    hdd  0.00980          osd.3        up   1.00000  1.00000
 4    hdd  0.00980          osd.4        up   1.00000  1.00000
 5    hdd  0.00980          osd.5        up   1.00000  1.00000
-7         0.02939      host ceph03
 6    hdd  0.00980          osd.6        up   1.00000  1.00000
 7    hdd  0.00980          osd.7        up   1.00000  1.00000
 8    hdd  0.00980          osd.8        up   1.00000  1.00000
```


HUAWEI

CRUSH Map Replication Rules

- The CRUSH map replication rules define how data is distributed on OSDs in the hierarchy.
- The rules specify the resource pool type, failure domain, and device type when a resource pool is created.

```
[ceph: root@ceph01 /]# ceph osd crush rule ls
replicated_rule
erasure-code
data_cephfs
metadata_cephfs
[ceph: root@ceph01 /]# ceph osd crush rule dump replicated_rule
{
    "rule_id": 0,
    "rule_name": "replicated_rule",
    "ruleset": 0,
    "type": 1,
    "min_size": 1,
    "max_size": 10,
    "steps": [
        {
            "op": "take",
            "item": -1,
            "item_name": "default"
        },
        {
            "op": "chooseleaf_firstn",
.....
```

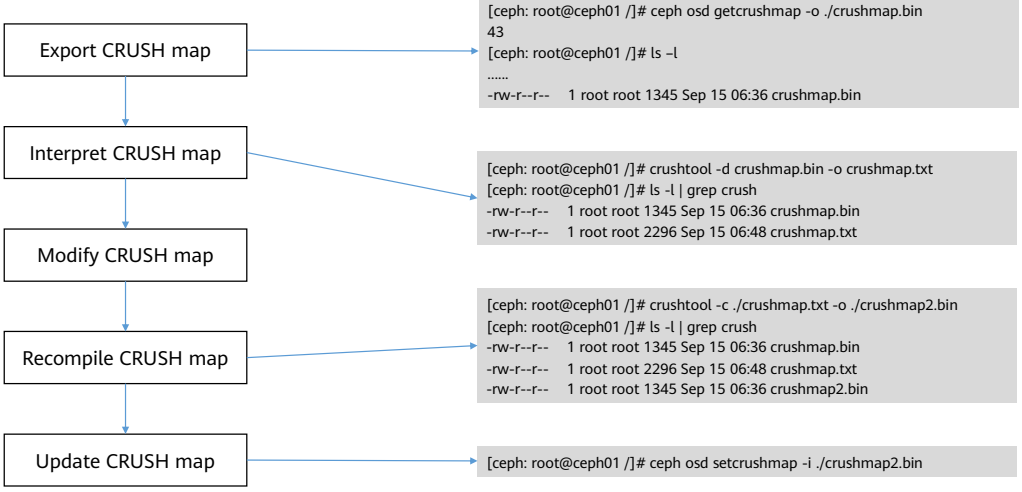

HUAWEI

Contents

1. Introduction to CRUSH Map
2. **CRUSH Map Configurations**

HUAWEI

CRUSH Map Configuration Process



HUAWEI

Composition of the CRUSH Map Configuration File

- The CRUSH map configuration file consists of five parts:
  - Algorithm attribute
    - Code can be found after # begin crush map.
  - OSD device list
    - Code can be found after # devices.
  - CRUSH types
    - Code can be found after "types".
  - Bucket hierarchy
    - Code can be found after "buckets".
  - Replication rules
    - Code can be found after "rules".

```
# begin crush map
tunable choose_local_tries 0
tunable choose_local_fallback_tries 0
tunable choose_total_tries 50
tunable chooseleaf_descend_once 1
.......
# devices
device 0 osd.0 class hdd
.....
# types
type 0 osd
type 1 host
type 2 chassis
.....
# buckets
host ceph01 {
        id -3           # do not change unnecessarily
        id -4 class hdd         # do not change unnecessarily
......
# rules
rule replicated_rule {
        id 0
.......
```


HUAWEI

- Algorithm

Algorithm Attributes of CRUSH Map

- The algorithm attribute parameters of the CRUSH map come from the predefined profiles. You are advised not to modify the parameters.
- The predefined profiles are as follows:
  - Legacy
  - Argonaut
  - Bobtail
  - Firefly
  - Hammer
  - Jewel
  - Optimal
  - Default

```
[ceph: root@ceph01 /]# ceph osd crush show-tunables
{
    "choose_local_tries": 0,
    "choose_local_fallback_tries": 0,
    "choose_total_tries": 50,
    "chooseleaf_descend_once": 1,
    "chooseleaf_vary_r": 1,
    "chooseleaf_stable": 1,
    "straw_calc_version": 1,
    "allowed_bucket_algs": 54,
    "profile": "jewel",
    "optimal_tunables": 1,
    "legacy_tunables": 0,
    "minimum_required_version": "jewel",
    "require_feature_tunables": 1,
    "require_feature_tunables2": 1,
    "has_v2_rules": 1,
    "require_feature_tunables3": 1,
    "has_v3_rules": 0,
    "has_v4_buckets": 1,
    "require_feature_tunables5": 1,
    "has_v5_rules": 0
}
```


HUAWEI

- Profile optimal refers to the optimal parameter configuration for the current version.

OSD Device List Configuration

- The OSD device list in the CRUSH map consists of four parts:
  - Device ID
  - Device name
  - Weight. Optional.
  - Device class. Possible options are:
    - HDD
    - SSD
    - NVMe SSD

```
# devices
device 0 osd.0 class hdd
device 1 osd.1 class hdd
device 2 osd.2 class hdd
device 3 osd.3 class hdd
device 4 osd.4 class hdd
device 5 osd.5 class hdd
device 6 osd.6 class hdd
device 7 osd.7 class hdd
device 8 osd.8 class hdd
```


HUAWEI

- By default, the OSD type is automatically identified based on the actual device medium. This may be inaccurate, in which case, you can manually change the OSD type.
- You can run **ceph osd crush set-device-class** *<class> <osd-name> [...]* to change the class of the OSD device.
- The OSD weight is set based on the OSD capacity and the data stored, but there may be imbalance between OSDs due to the Ceph placement mechanism. You can change the weight to manually balance OSDs. Ceph supports two types of weight sets:
  - compat weight set
  - per-pool weight set

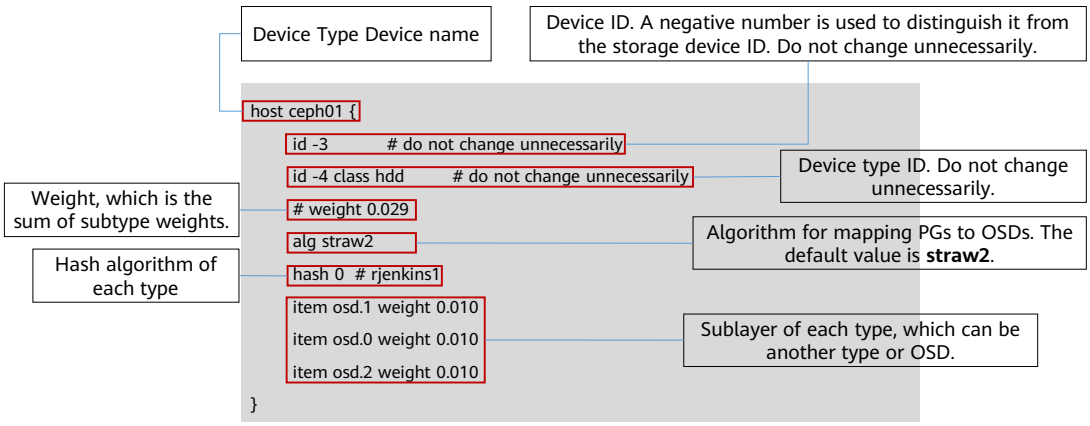
CRUSH Type Configurations

- CRUSH types are used in the hierarchical structure. New types can be customized. The default types are as follows:
  - osd (or device)
  - host
  - chassis
  - rack
  - row
  - pdu
  - pod
  - room
  - datacenter
  - zone
  - region
  - root

```
# types
type 0 osd
type 1 host
type 2 chassis
type 3 rack
type 4 row
type 5 pdu
type 6 pod
type 7 room
type 8 datacenter
type 9 zone
type 10 region
type 11 root
```

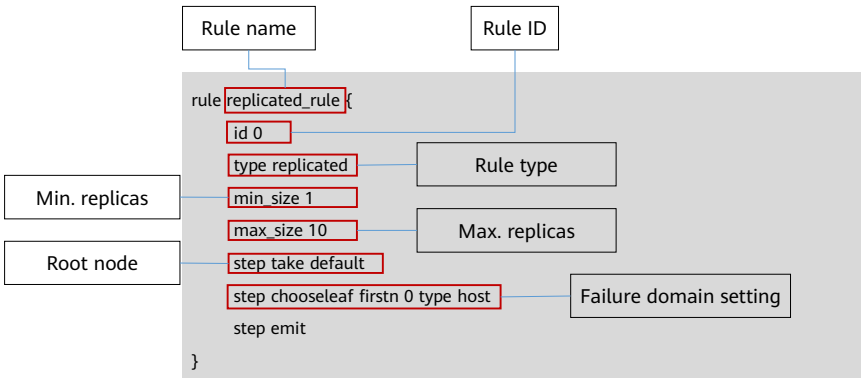

HUAWEI

CRUSH Map Hierarchy Configurations



HUAWEI

CRUSH Map Rules



HUAWEI

Quiz

1. (Multiple-answer question) Which of the following are the types of disks in the CRUSH map?
   A. ATA
   B. SSD
   C. NVMe SSD
   D. HDD

HUAWEI

- BCD

Summary

- This course introduces the Ceph CRUSH map and the corresponding configuration modification to enhance your understanding of the Ceph distribution algorithm.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling	Description
PG	Placement group	Subset of the Ceph resource pool
RBD	RADOS block device	N/A
RGW	RADOS gateway	N/A
MDS	Ceph metadata server	RADOS metadata server
CRUSH	Controlled Replication Under Scalable Hashing	Algorithm used by Ceph to calculate the object storage location

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

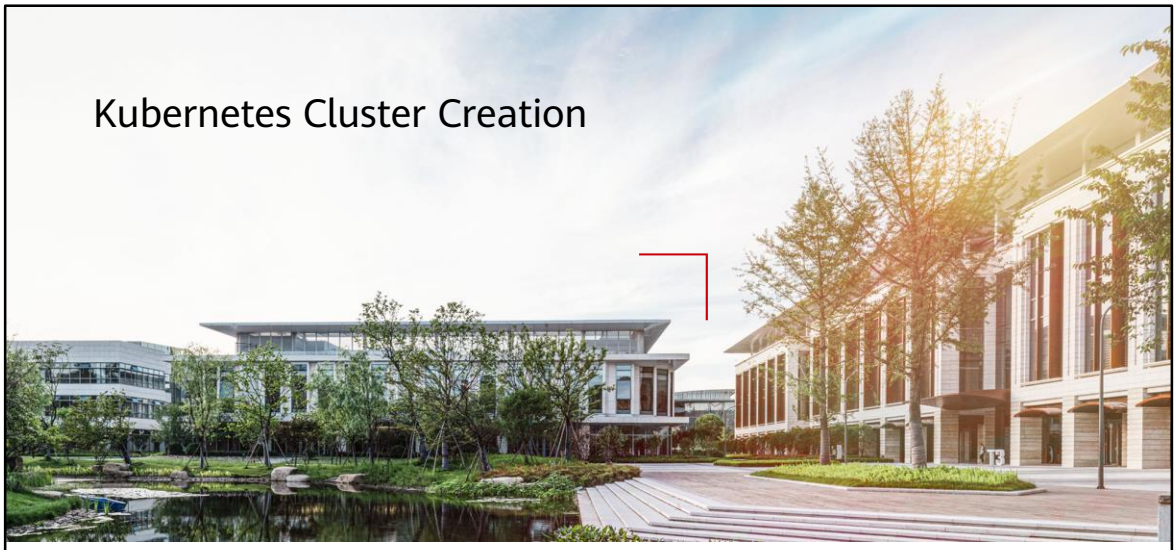
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Kubernetes Cluster Creation



Foreword

- This course describes how to create a high-availability Kubernetes cluster and how to use kubeadm to automatically create a cluster.

Objectives

- On completion of this course, you will understand:
  - Kubernetes cluster architecture.
  - Kubernetes cluster networking.
  - Kubernetes high availability.
  - Automated deployments of Kubernetes clusters.

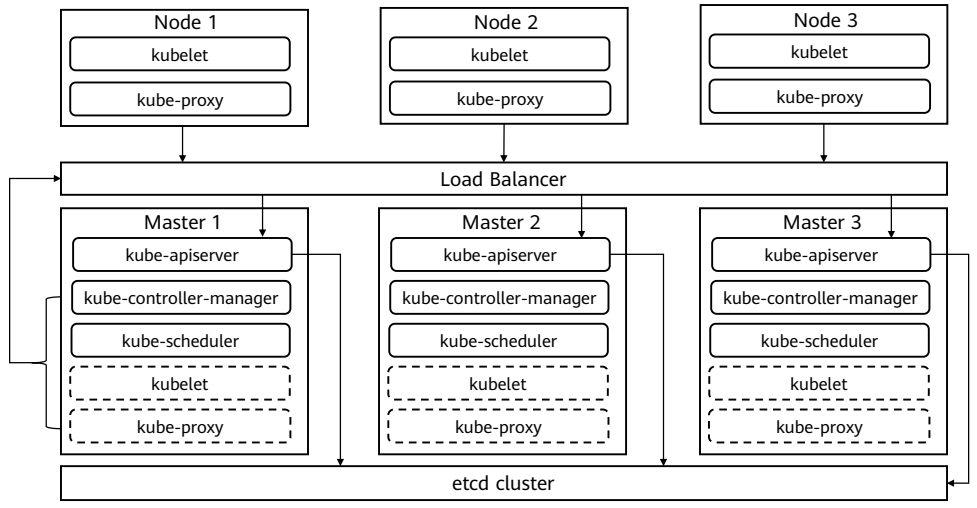
HUAWEI

Contents

1. **Introduction to Kubernetes Clusters**
2. Environment Setup
3. High Availability
4. Automated Cluster Creation
5. Cluster Status

HUAWEI

Kubernetes Cluster Architecture



HUAWEI

- Three master nodes are configured with NGINX and Keepalived for load balancing and high availability.
- Three worker nodes
- etcd is deployed on the three master nodes.

Kubernetes Cluster Networking

Node Name	IP Address		
k8smaster1	192.168.1.11/24	192.168.0.11/24	VIP 192.168.1.10/24
k8smaster2	192.168.1.12/24	192.168.0.12/24	
k8smaster3	192.168.1.13/24	192.168.0.13/24	
k8snode1	192.168.1.15/24	192.168.0.14/24	
k8snode2	192.168.1.16/24	192.168.0.15/24	
k8snode3	192.168.1.17/24	192.168.0.16/24	

Service	Planned Addresses
Kubernetes node	192.168.1.10/24
Storage node	192.168.0.10/24
Service network segment	172.16.0.0/16
Pod network segment	10.0.0.0/8

HUAWEI

How to Create Kubernetes Clusters

- Automated creation
  - kubeadm is a tool that provides **kubeadm init** and **kubeadm join** as fast paths for creating Kubernetes clusters. It streamlines cluster creation but masks certain details, making it difficult to locate faults.
- Manual creation
  - Using binaries
  - Using source code

HUAWEI

- Kubernetes comprises a group of executable programs. You can download the compiled binaries GitHub, or download and compile the source code.
- Download the binary of a distribution from GitHub and manually deploy each component to form a Kubernetes cluster.
- You are advised to use binaries. Although manual creation of Kubernetes clusters is difficult, the working principles are easy to learn and can facilitate subsequent maintenance.

Contents

1. Introduction to Kubernetes Clusters
2. **Environment Setup**
3. High Availability
4. Automated Cluster Creation
5. Cluster Status

HUAWEI

Environment Setup

- Set up a lab environment and perform the following configurations on all Kubernetes nodes:
  - Change the hostname.
  - Configure the firewall, SELinux, and swap partition.
  - Enable IPVS.
  - Configure time synchronization.
  - Configure the Yum repository.
  - Install Docker.

HUAWEI

- To prevent time asynchronization caused by Internet clock latency, you are advised to set one host as the clock source.

Configuring Docker

- Change the cgroup driver of Docker.
  - openEuler uses systemd for initialization, during which the root cgroup is generated and serves as the cgroup manager. Docker uses cgroupfs by default. To ensure stable runtime, change the cgroup driver to systemd.

```
cat <<EOF > /etc/docker/daemon.json
{
  "exec-opts": ["native.cgroupdriver=systemd"]
}
EOF
systemctl daemon-reload
systemctl restart docker
```


HUAWEI

- Add an image accelerator to the Docker configuration file according to the method described in the preceding course.

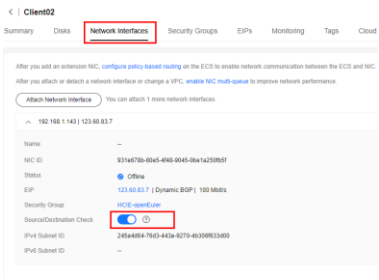
Contents

1. Introduction to Kubernetes Clusters
2. Environment Setup
3. **High Availability**
4. Automated Cluster Creation
5. Cluster Status

HUAWEI

High Availability

- Deploy three master nodes in HA mode.
  - Install NGINX and Keepalived.
  - Modify the NGINX and Keepalived configuration files.
  - Start NGINX and Keepalived.
  - Test whether the virtual IP addresses of the master nodes are reachable.
- Note: If Huawei Cloud ECSs are used, disable the source/destination check for the NICs of the ECSs.



HUAWEI

NGINX

- Modify the **/etc/nginx/nginx.conf** configuration file on the three master nodes.

```
stream {
    log_format main '$remote_addr $upstream_addr - [$time_local] $status $upstream_bytes_sent';
    access_log /var/log/nginx/k8s-access.log main;
    upstream k8s-apiserver {
        server 192.168.1.11:6443;
        server 192.168.1.12:6443;
        server 192.168.1.13:6443;
    }
    server {
        listen 16443;
        proxy_pass k8s-apiserver;
    }
}
```


HUAWEI

- Load balancing is achieved using the master modes. The default port number of **k8s-apiserver** is **6443**. Kubernetes and NGINX are co-deployed, where NGINX listens to port 16443 not port 6443. You are advised not to deploy NGINX on the master node in practice.

Keepalived

- Modify the **/etc/keepalived/keepalived.conf** configuration file on the three master nodes.

```
vrrp_instance Nginx {
    state MASTER
    interface ens3
    virtual_router_id 51
    priority 200
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass Huawei@1
    }
    virtual_ipaddress {
        192.168.1.10/24
    }
}
```

```
vrrp_instance Nginx {
    state BACKUP
    interface ens3
    virtual_router_id 51
    priority 150
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass Huawei@1
    }
    virtual_ipaddress {
        192.168.1.10/24
    }
}
```

```
vrrp_instance Nginx {
    state BACKUP
    interface ens3
    virtual_router_id 51
    priority 100
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass Huawei@1
    }
    virtual_ipaddress {
        192.168.1.10/24
    }
}
```


HUAWEI

- LVS does not need to be enabled.

Contents

1. Introduction to Kubernetes Clusters
2. Environment Setup
3. High Availability
4. **Automated Cluster Creation**
5. Cluster Status

HUAWEI

Automated Kubernetes Cluster Creation

- Install kubectl, kubeadm, and kubelet on the master and node nodes.
- Download the Kubernetes image to all master nodes.
- Initialize Kubernetes on Master 1.
- Copy the certificate generated on Master 1 to other nodes.
- Add the master nodes to the cluster.
- Add all other nodes to the cluster.
- Install the Calico network plug-in.

HUAWEI

- Manually select the node to be initialized, that is, the master Keepalived node.

Installing Kubernetes Software Packages

- Install kubelet, kubeadm, and kubectl on all nodes.
  - **yum install -y kubelet kubeadm kubectl --disableexcludes=kubernetes**
- Start kubelet.
  - **systemctl enable --now kubelet**

HUAWEI

Creating a kubeadm Initialization File

- Set **kubernetesVersion** to the Kubernetes version to be downloaded, **imageRepository** to the image repository, **controlplaneEndpoint** to the virtual IP address of the master node, **podSubnet** to the pod network segment, and **serviceSubnet** to the service network segment.

```
apiVersion: kubeadm.k8s.io/v1beta2
kind: ClusterConfiguration
kubernetesVersion: v1.23.1
imageRepository: swr.cn-east-3.myhuaweicloud.com/hcie_openeuler
controlPlaneEndpoint: 192.168.1.10:16443
dns:
  type: CoreDNS
apiServer:
 certSANs:
 - 192.168.1.10
networking:
 podSubnet: 10.0.0.0/8
 serviceSubnet: 172.16.0.0/16
---
apiVersion: kubeproxy.config.k8s.io/v1alpha1
kind: KubeProxyConfiguration
mode: ipvs
```


HUAWEI

- **swr.cn-east-3.myhuaweicloud.com/hcie_openeuler** is used in this lab practice.
- **controlPlaneEndpoint** is the forwarding address or DNS of the kube-apiserver load balancer. The load balancer listens to all api-server ports.

Downloading Images on All Master Nodes

- Download the image on all master nodes. The image contains kube-apiserver, kube-controller-manager and kube-scheduler required by the master nodes.
  - kubeadm config images pull --config kubeadm-init.yaml

```
[root@Master01 ~]# kubeadm config images pull --config kubeadm-init.yaml
unable to read config from "kubeadm-init.yaml" : open kubeadm-init.yaml: no such file or directory
To see the stack trace of this error execute with --v=5 or higher
[root@Master01 ~]# kubeadm config images pull --config kubeadm.yaml
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/kube-apiserver:v1.23.1
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/kube-controller-manager:v1.23.1
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/kube-scheduler:v1.23.1
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/kube-proxy:v1.23.1
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/pause:3.6
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/etcd:3.5.1-0
[config/images] Pulled swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/coredns:v1.8.6
```


HUAWEI

- **kubeadm-init.yaml** is the initialization file created in the previous step. The file name is created by the user.

Initializing the k8smaster1 Node

- Run the **kubeadm init** command to initialize kubeadm.
  - **kubeadm init --config kubeadm-init.yaml**
- Copy the **config** file as prompted.
- Copy the certificate generated on the **k8smaster1** node to other master nodes.

HUAWEI

Adding Nodes to a Cluster

- Two **join** commands are generated when the k8smaster1 node is initialized. To add the k8smaster2 and k8smaster3 nodes to the cluster, run the command containing **--control-plane**. To add other nodes to the cluster, run the other command.

```
To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

You can now join any number of control-plane nodes by copying certificate authorities
and service account keys on each node and then running the following as root:

  kubeadm join 192.168.1.10:16443 --token l82lty.m6nwxlivvfenpehy \
        --discovery-token-ca-cert-hash sha256:d3ee411569273e0fd89b8ec46343f9f196a2a49e7c58e92707d8df7e7af39b63 \
        --control-plane

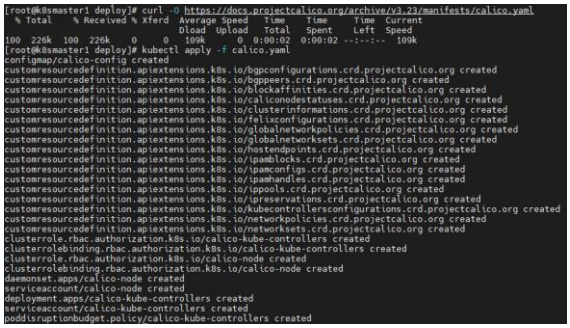
Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.1.10:16443 --token l82lty.m6nwxlivvfenpehy \
        --discovery-token-ca-cert-hash sha256:d3ee411569273e0fd89b8ec46343f9f196a2a49e7c58e92707d8df7e7af39b63
[root@k8smaster1 deploy]#
```


HUAWEI

Installing the Calico Network Plug-in

- Install the Calico network plug-in.
  - **curl -O https://docs.projectcalico.org/archive/v3.23/manifests/calico.yaml**
  - **kubectl apply -f calico.yaml**



Rectifying Initialization Errors

- If an error is reported when you run the **kubeadm init** command, you can reset the initialization and then perform the initialization again. If an error is reported when you run the **kubeadm join** command, you can reset the join operations, but this will delete the certificate, requiring you to transfer the certificate from the k8smaster1 node again.
  - **kubeadm reset -f**

HUAWEI

Rectifying Token Expiration

- The token generated after the k8smaster1 node is initialized is valid for 24 hours. If you want to run the **kubeadm join** command to add the master or other nodes after the token expires, run the **kubeadm token create** command to generate a new token.
  - **kubeadm token create --print-join-command**

HUAWEI

Contents

1. Introduction to Kubernetes Clusters
2. Environment Setup
3. High Availability
4. Automated Cluster Creation
5. **Cluster Status**

HUAWEI

Viewing Cluster Statuses

- **kubectl get nodes**

```
[root@k8smaster1 deploy]# kubectl get nodes
NAME         STATUS   ROLES                  AGE     VERSION
k8smaster1   Ready    control-plane,master   16m     v1.23.1
k8smaster2   Ready    control-plane,master   7m8s    v1.23.1
k8smaster3   Ready    control-plane,master   7m8s    v1.23.1
k8snode1     Ready    <none>                 5m57s   v1.23.1
k8snode2     Ready    <none>                 5m57s   v1.23.1
k8snode3     Ready    <none>                 5m45s   v1.23.1
```

- **kubectl get pods -n kube-system -o wide**

```
[root@k8smaster1 deploy]# kubectl get pods -n kube-system -o wide
NAME                                       READY   STATUS    RESTARTS      AGE   IP               NODE         NOMINATED NODE   READINESS GATES
calico-kube-controllers-54756b744f-lhdbt   1/1     Running   0             10m   10.38.249.2      k8snode1     <none>           <none>
calico-node-4z7p6                          1/1     Running   0             10m   192.168.1.13     k8smaster3   <none>           <none>
calico-node-6qj8b                          1/1     Running   0             10m   192.168.1.16     k8snode3     <none>           <none>
calico-node-ft7dj                          1/1     Running   0             10m   192.168.1.15     k8snode2     <none>           <none>
calico-node-jhwcv                          1/1     Running   0             10m   192.168.1.12     k8smaster2   <none>           <none>
calico-node-ljqxq                          1/1     Running   0             10m   192.168.1.14     k8snode1     <none>           <none>
calico-node-t69zf                          1/1     Running   0             10m   192.168.1.11     k8smaster1   <none>           <none>
coredns-665d974787-5wcnk                   1/1     Running   0             24m   10.38.249.1      k8snode1     <none>           <none>
coredns-665d974787-jt95v                   1/1     Running   0             24m   10.38.249.3      k8snode1     <none>           <none>
etcd-k8smaster1                            1/1     Running   0             24m   192.168.1.11     k8smaster1   <none>           <none>
etcd-k8smaster2                            1/1     Running   0             15m   192.168.1.12     k8smaster2   <none>           <none>
etcd-k8smaster3                            1/1     Running   0             14m   192.168.1.13     k8smaster3   <none>           <none>
kube-apiserver-k8smaster1                  1/1     Running   0             24m   192.168.1.11     k8smaster1   <none>           <none>
kube-apiserver-k8smaster2                  1/1     Running   0             15m   192.168.1.12     k8smaster2   <none>           <none>
kube-apiserver-k8smaster3                  1/1     Running   1 (14m ago)   15m   192.168.1.13     k8smaster3   <none>           <none>
kube-controller-manager-k8smaster1         1/1     Running   1 (14m ago)   24m   192.168.1.11     k8smaster1   <none>           <none>
kube-controller-manager-k8smaster2         1/1     Running   0             15m   192.168.1.12     k8smaster2   <none>           <none>
kube-controller-manager-k8smaster3         1/1     Running   0             15m   192.168.1.13     k8smaster3   <none>           <none>
kube-proxy-8dfbs                           1/1     Running   0             14m   192.168.1.15     k8snode2     <none>           <none>
kube-proxy-ckvgv                           1/1     Running   0             14m   192.168.1.14     k8snode1     <none>           <none>
kube-proxy-qm8dp                           1/1     Running   0             15m   192.168.1.13     k8smaster3   <none>           <none>
kube-proxy-spsll                           1/1     Running   0             24m   192.168.1.11     k8smaster1   <none>           <none>
kube-proxy-v7vdb                           1/1     Running   0             13m   192.168.1.16     k8snode3     <none>           <none>
kube-proxy-wvwjp                           1/1     Running   0             15m   192.168.1.12     k8smaster2   <none>           <none>
kube-scheduler-k8smaster1                  1/1     Running   1 (14m ago)   24m   192.168.1.11     k8smaster1   <none>           <none>
kube-scheduler-k8smaster2                  1/1     Running   0             15m   192.168.1.12     k8smaster2   <none>           <none>
kube-scheduler-k8smaster3                  1/1     Running   0             15m   192.168.1.13     k8smaster3   <none>           <none>
```


HUAWEI

Creating and Accessing a Deployment

- Create a deployment in the Kubernetes cluster and confirm it is running properly.

```
kubectl create deployment nginx --image=nginx
kubectl expose deployment nginx --port=80 --type=NodePort
kubectl get pod,svc
kubectl get pods -o wide
curl 172.16.137.173
```

```
[root@k8smaster1 deploy]# kubectl get pod,svc
NAME                         READY   STATUS    RESTARTS   AGE
pod/nginx-85b98978db-nccqh   1/1     Running   0          23s

NAME                 TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
service/kubernetes   ClusterIP   172.16.0.1       <none>        443/TCP        28m
service/nginx        NodePort    172.16.137.173   <none>        80:30143/TCP   23s
```

```
[root@k8smaster1 deploy]# curl 172.16.137.173
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```


HUAWEI

Quiz

1. (Multiple-answer question) Which of the following components must be deployed on the master nodes of Kubernetes?
   A. kube-proxy
   B. kube-apiserver
   C. kubelet
   D. kube-scheduler
2. (True or False) If a Kubernetes cluster contains multiple master and other nodes, you need to configure HA for all the nodes.

HUAWEI

- BD
- False

Summary

- This course describes how to set up a Kubernetes cluster, including how to deploy an HA cluster with multiple master nodes and how to use kubeadm to automatically create a Kubernetes cluster.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling
ECS	Elastic Cloud Server

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Kubernetes Network Components



Foreword

- This course describes the networking solutions for Kubernetes clusters, including the implementation of mainstream solutions Flannel and Calico, and Kubernetes network policies.

HUAWEI

Objectives

- On completion of this course, you will understand:
  - Kubernetes network model.
  - Principles of the Flannel and Calico networking solutions.
  - Kubernetes network policies.

HUAWEI

Contents

1. **Kubernetes Network Model**
2. Flannel Networking Solution
3. Calico Networking Solution
4. Kubernetes Network Policies

HUAWEI

Kubernetes Network Model

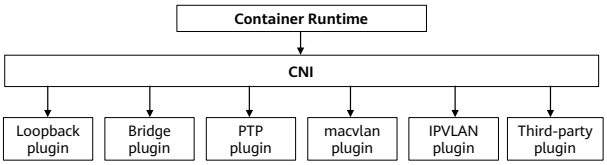
- Basic conditions
  - All pods directly communicate with each other without using NAT.
  - Nodes directly communicate with all pods without using NAT.
  - The IP address visible to a pod is used by other pods to communicate with the pod, negating the need for explicit conversion.
- Objectives
  - Communication between containers in the same pod (Container to Container)
  - Communication between pods (Pod to Pod)
  - Communication between a service and a pod (Service to Pod)
  - Communication between external components of a cluster and services (External to Service)

HUAWEI

- A basic principle of the Kubernetes network model is that each pod has an independent IP address, and all pods are in a flat network space that can be directly connected.
- They must be accessible using any IP address, regardless of whether they run on the same node (host machine).
- Users do not need to consider how to establish connections between pods or map container ports to host ports.
- IP addresses are allocated by pod. All containers in a pod share a network stack (equivalent to a network namespace, whose IP addresses, network devices, and configurations are shared).

Container Network Model Specification CNI

- A network model is designed in Kubernetes but implemented by the network plugin.
- For standardization and compatibility purposes and to define the container network model specification, CoreOS and Google jointly released the Container Network Interface (CNI), which connects the container management system and network plugins.
- CNI principles: Before creating a container, the container runtime creates a network stack and invokes the CNI plugin to configure the network for the stack, and then starts the processes in the container.



HUAWEI

- CNI supports various container runtimes.
- The CNI plugin model supports third-party plugins developed by different organizations and companies, offering flexibility to O&M personnel when determining a networking solution.

CNI Plugins

- In Kubernetes, CNI uses the following types of plugins:
  - Main plugins
    - Binary files used to create specific network devices, such as bridge, ipvlan, and vlan.
  - IP Address Management (IPAM) plugins
    - Assign and manage IP addresses for pods.
    - Serve as a part of the CNI plugin and work with the CNI plugin.
  - Meta plugins
    - Internal plugins maintained by the CNI community. Common plugins are as follows:
      - tuning
      - Bandwidth
      - Flannel
      - portmap

HUAWEI

- Main plugins depend on cri-tools.
- **/opt/cni/bin** describes the functions of meta plugins.

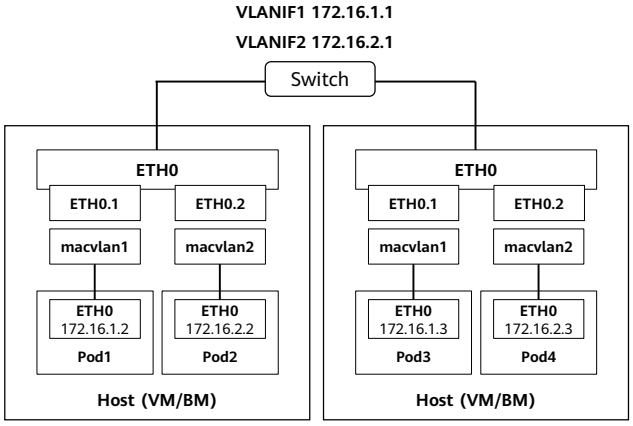
CNI Configuration File

```
{
  "cniVersion": "1.1.0",                                  #### Version
  "cniVersions": [".3.1", "0.4.0", "1.0.0", "1.1.0"],     #### All supported CNI versions
  "name": "dbnet"                                         #### Network name
  "plugins": [                                            #### CNI plugins and corresponding configurations
    {
      "type": "bridge",                                   #### Main plugin name, which must correspond to the binary file name
      .........
      "ipam": {                                           #### IPAM plugin configuration
        "type": "host-local",                             #### Name of the configuration file to be executed by the IPAM plugin
        .........
      },
    {
      "type": "tuning",
.............
    },
    {
        "type": "portmap",
.............
    }
  ]
}
```


HUAWEI

- A network name must be unique on the same host.
- The specified configuration file name cannot contain a path identifier, such as a forward slash (/) or a backslash (\).

Typical Kubernetes Networking: Layer 2 Bridging



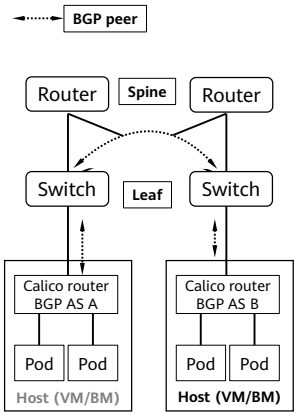
- Layer 2 bridging solution: macvlan.
- The configured container network and host network belong to the same LAN (NAT is not required for cross-node container access).
- You can use macvlan to configure multiple virtual network interfaces which have independent MAC addresses and IP addresses on a network interface or VLAN sub-interface of a host for containers.
- macvlan requires that physical NICs must support the promiscuous mode. The container gateway is deployed on the physical switch, and the container IP address needs to be manually configured and managed.

HUAWEI

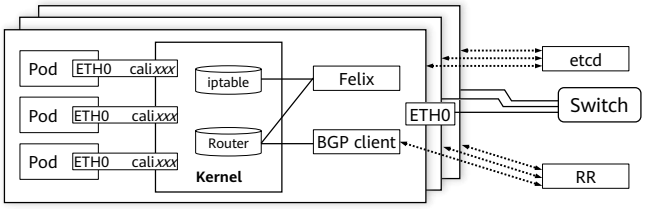
- macvlan is one of the Linux kernel features and also a native networking solution supported by Docker.

Typical Kubernetes Networking: Layer 3 Routing

Calico



Calico Layer 3 Routing Solution



Calico comprises the following components:

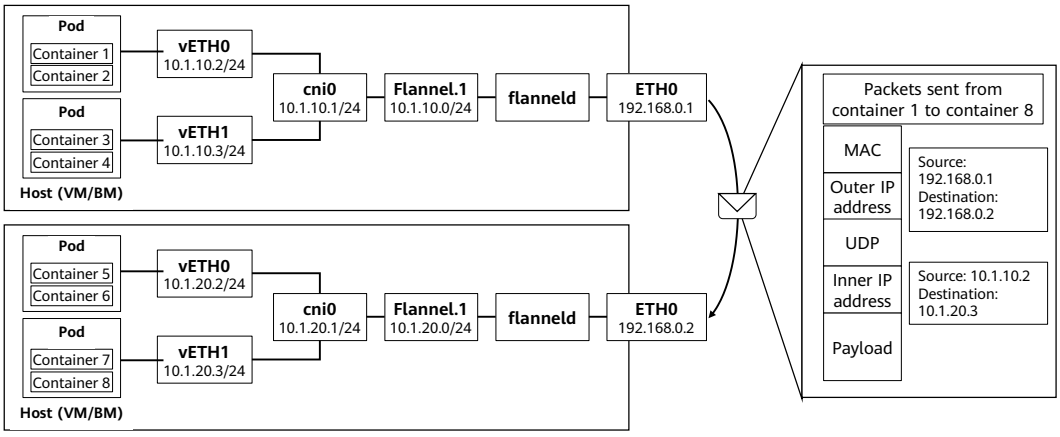
- etcd: distributed key-value store, which holds Calico metadata and acts as the communication hub of Calico components.
- BGP client: implements BGP, establishes BGP peer relationships with external route reflectors (RRs), advertises routes of the local pod through RRs, and learns pod routes of other nodes.
- Felix: runs on each Kubernetes worker node, and writes pod routes into the host routing table and ACL rules into iptables.
- RR: used to synchronize pod routes on the entire network in large-scale networking.

HUAWEI

- In Calico, each physical server/host is considered as its own vRouter, and BGP is run between routers.
- The forwarding plane of Calico is as follows:
  - The ETH0 and cali*xxx* form a virtual ethernet (veth) pair. The two ends of the veth pair are in the network namespaces of the container and host machine, respectively.
  - Only one default route in the pod points to the local link address 169.254.1.1. The MAC addresses of all cali*xxx* interfaces are **ee:ee:ee:ee:ee:ee**.
  - A pod initiates an access (ARP) request to obtain the MAC address of 169.254.1.1.
  - The host machine does not have the address 169.254.1.1. Instead, the proxy ARP is enabled. The ARP reply of the host machine returns the MAC address (**ee:ee:ee:ee:ee:ee**) of the other end of the veth pair to the pod.
  - The pod encapsulates the outer MAC address into an ARP reply and sends the reply to the host machine. The host machine searches the routing table for layer 3 forwarding.

Typical Kubernetes Networking: Host Overlay

Host overlay is represented by Flannel and is a software-only solution, meaning it does not depend on hardware network devices.



HUAWEI

- The Flannel host overlay solution was developed in the early stages of container technology development.
- The host overlay solution is a software-only solution that does not depend on hardware network devices, enabling agile development and quick launch of technical prototypes. It does, however, rely on the CPU for forwarding, meaning there is a performance bottleneck, especially for data forwarding.
- East-west traffic forwarding between pods:
  - After the flanneld process of a Kubernetes node is started, it applies for subnets of containers from etcd, and writes the IP address of the host machine NIC and the MAC address of the VXLAN device to etcd. In the figure, the two subnets are 10.1.10.0/24 and 10.1.20.0/24.
  - The Kubernetes node periodically queries network information from etcd. If a subnet is added, the node updates the subnet and IP address information in the local routing table.
  - When container 1 needs to access container 8, Flannel.1 performs VXLAN encapsulation in Linux kernel mode based on the local routing table. If the backend driver of Flannel is UDP, Flannel.0 performs UDP encapsulation and forwarding in user mode.
  - After the destination Kubernetes node receives the VXLAN- or UDP-encapsulated packet, it decapsulates the packet and forwards it to the

destination pod based on the local routing table.

- The Flannel host overlay solution implements the Kubernetes network model and enables pods to communicate across Kubernetes worker nodes. One bottleneck is peaks in data transmissions, like high-concurrency clusters, where CPU forwarding capabilities can hold back the overall performance.

Solution Comparison

Item	Layer 2 Bridging	Layer 3 Routing	Host Overlay
Network performance	High-performance forwarding	High-performance forwarding	vSwitches are used for VTEP-based forwarding, and the performance is relatively low.
Networking scale	Small-scale networking, limited by the broadcast domain	Large-scale networking	Large-scale networking
Requirements for the underlying network	Layer 2 interworking	Network interconnection	Network interconnection
Typical solutions	macvlan, Contiv	Calico, Contiv	Flannel, Open vSwitch, Contiv, Weave
Device dependencies	Physical switches must support a large number of MAC address entries and ARP entries.	Physical switches must support a large number of routes.	vSwitch forwarding consumes server hardware resources and has high requirements on server configurations.

- Layer 2 bridging is easy to deploy and has high forwarding performance, but has a limited deployment scale.
- Layer 3 routing suits large-scale clusters thanks to high forwarding performance and scalability.
- In the host overlay solution, vSwitches are used for VTEP-based forwarding. This results in low performance, making the solution suitable for independent PaaS platforms.

HUAWEI

Mainstream CNI Plugin Projects

- **Flannel**: a network plugin that provides overlay networks for Kubernetes. Based on Linux TUN/TAP, Flannel uses UDP to encapsulate IP packets to create overlay networks and uses etcd to maintain the networks. It supports only network configuration.
- **Calico**: a BGP-based layer 3 network plugin that supports network policies to control network access. It runs a vRouter on each server, uses the Linux kernel to forward network data packets, and uses iptables to implement functions such as firewalls.
- **Canal**: integrates the network layer functions provided by Flannel with the network policy function of Calico.

HUAWEI

Contents

1. Kubernetes Network Model
2. **Flannel Networking Solution**
3. Calico Networking Solution
4. Kubernetes Network Policies

HUAWEI

Flannel Networking Solution Overview

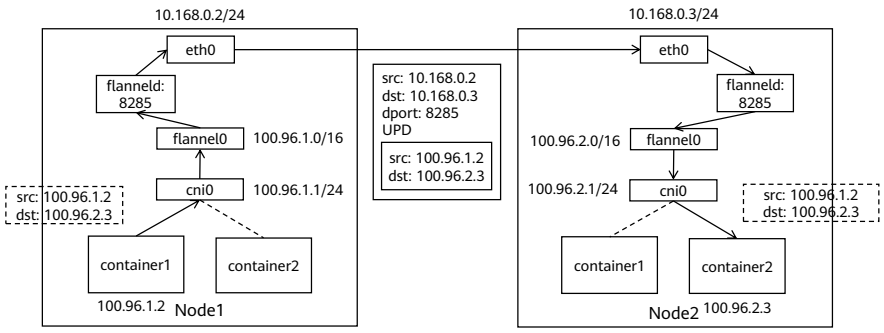
- Flannel runs an agent called flanneld on each host and allocates a subnet lease to each host out of a specified address space.
- It uses either the Kubernetes API or etcd to store the network configuration, allocated subnets, and auxiliary data.
- Packets are forwarded using one of several backend mechanisms as follows:
  - VXLAN
  - host-gw
  - UDP
- Flannel does not control how containers are networked to a host or pod, but only how traffic is forwarded between hosts.
- It focuses on network forwarding but not network policies.

HUAWEI

- On the control plane, Flannel uses etcd to configure the routing table on each node.
- On the data plane, it uses the tunnel mode (such as VXLAN and UDP) for encapsulation.
- A daemon process flanneld is run on each host. When a host is added to a cluster, flanneld applies for an IP subnet (**/24** by default) from etcd and obtains IP subnet information of other hosts.
- Because each host uses a different IP subnet, a container's IP address will be changed if the container is migrated to another host.
- Flannel allocates IP addresses to pods and subnets to nodes (hosts/VMs). All pods on the same node belong to the same IP subnet, and the subnet cannot work across nodes.

Flannel Forwarding Mechanism: UDP

- The Flannel UDP is a layer 3 overlay network. It encapsulates source IP packets using UDP, decapsulates them at the destination end to obtain the original IP packets, and then forwards the IP packets to the target container. This establishes a tunnel between two containers on different host machines so that the containers can directly use IP addresses for communication, regardless of container and host machine distribution.

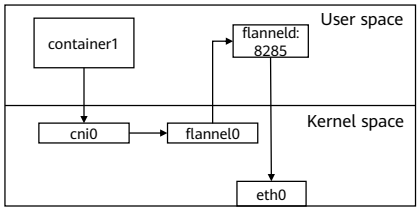


HUAWEI

- Flannel UDP is the earliest mode for cross-master communication.
- flannel0 is a TUN device (tunnel device) that sends IP packets between the kernel space and user space.
- The IP packet enters the kernel mode through the cni0 bridge. Then it enters flannel0 and returns to the user-mode flanneld process, which encapsulates the IP packet into a UDP packet. Finally, the UDP packet enters the kernel mode again and is then sent through the eth0 of the host machine.
- UDP encapsulation is completed in user mode. The cost is high for context switching and user-mode operations in Linux, and soon Flannel UDP became obsoleted due to the performance issue.

Issues in Flannel UDP

- Compared with the direct communication between host machines, the Flannel UDP has an extra step, where container communication is processed by flanneld. Because the TUN device flannel0 is used, data needs to be copied three times across the user and kernel modes when an IP packet is sent.
  - The first data copy is when the IP packet (sent by the user-mode container process) enters the kernel mode through the cni0 bridge.
  - The second data copy is when the IP packet enters the TUN (flannel0) device according to the routing table, and returns to the user-mode flanneld process.
  - The third data copy is when flanneld encapsulates the IP packet in a UDP packet, forwards the encapsulated packet to the kernel mode, and sends the encapsulated packet through eth0 of the host machine.

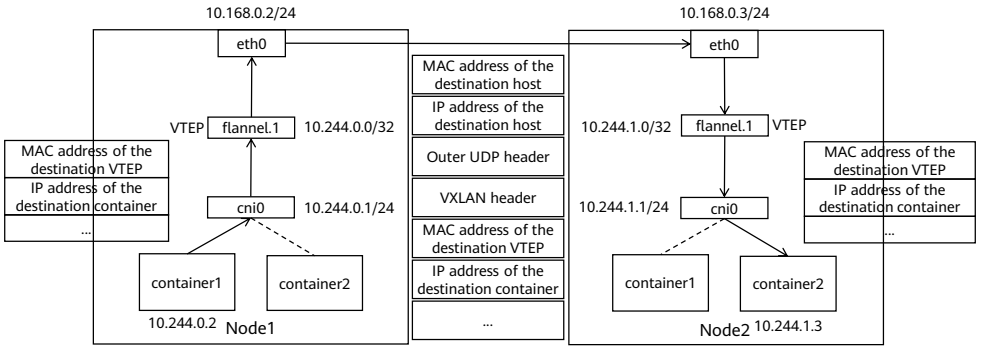


HUAWEI

- Flannel performs UDP encapsulation and decapsulation in user mode, but again has issues with cost and performance.

Flannel Forwarding Mechanism: VXLAN

- VXLAN is a Linux network virtualization technology that encapsulates and decapsulates data frames in kernel mode, so as to construct an overlay network through a tunnel mechanism.

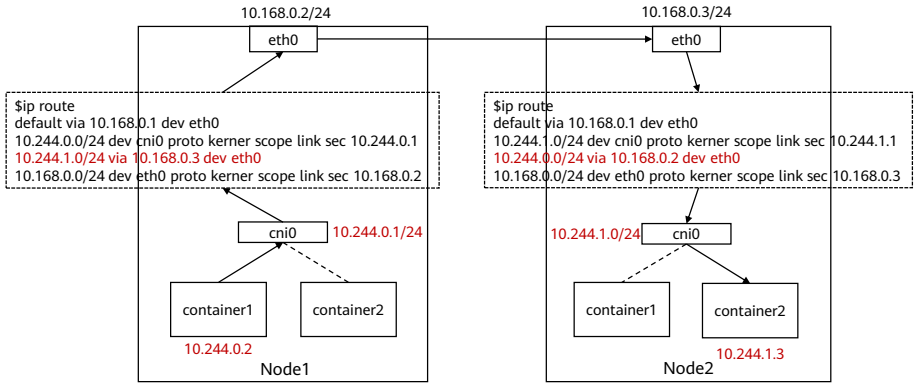


HUAWEI

- The VXLAN overlay network is designed to cover a virtual layer 2 network that is maintained by the kernel VXLAN module over the existing layer 3 network. In this way, hosts (VMs or containers) connected to the VXLAN layer 2 network can communicate with each other as though on the same LAN. These hosts may be distributed on different host machines or even in different physical equipment rooms.
- To establish a tunnel on the layer 2 network, VXLAN sets a special network device called VTEP on the host machine as the two ends of the tunnel.
- The VTEP device works in the same way as the flanneld process, but differs in encapsulation and decapsulation, where VTEP uses a layer 2 data frame (Ethernet frame). In addition, the execution process is completed in the kernel space because VXLAN is a module in the Linux kernel.

Flannel Forwarding Mechanism: host-gw

- host-gw sets the next hop of each Flannel subnet (for example, 10.244.1.0/24) to the IP address of the host machine corresponding to the subnet. This mode requires that the host machines in the cluster be connected at layer 2.



HUAWEI

Contents

1. Kubernetes Network Model
2. Flannel Networking Solution
3. **Calico Networking Solution**
4. Kubernetes Network Policies

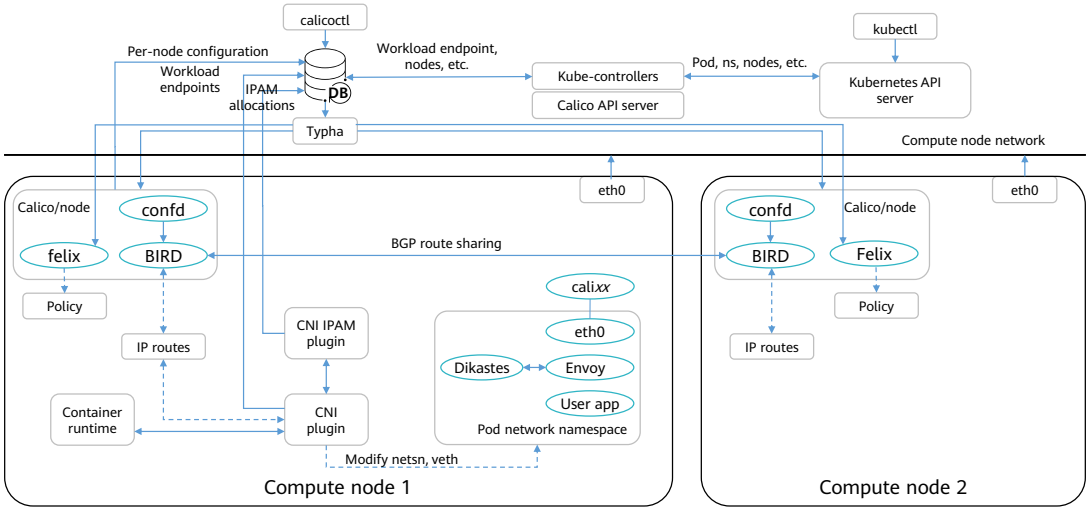
HUAWEI

Calico Overview

- Calico enables seamless and secure network communication between Kubernetes and non-Kubernetes workloads.
- It consists of two parts:
  - Calico CNI
    - A control plane that programs several data planes. It is a layer 3/layer 4 networking solution that secures containers, Kubernetes clusters, VMs, and workloads.
  - Calico network policy suite
    - An interface to the Calico CNI that contains rules for the data plane to execute.

HUAWEI

Logical Architecture of Calico



HUAWEI

Main Functional Components of Calico

- Calico API server: manages Calico resources using **kubectl** commands.
- Felix: runs on each node and programs forwarding routes, ACLs, and anything else required on the host to provide desired connectivity for the endpoints on that host.
- BIRD: runs on each node to obtain routes from Felix and forward the routes to BGP peers on the network.
- confd: monitors Calico datastore for changes to BGP configuration and global defaults.
- Dikastes: enforces network policies for service mesh.
- CNI plugin: provides Calico networking for Kubernetes clusters.
- Datastore plugin: increases scale by reducing each node's impact on the datastore.
- IPAM plugin: uses Calico's IP pool resource (IPPool) to control how IP addresses are allocated to pods within the cluster.
- kube-controllers: monitors the Kubernetes API and performs actions based on the cluster state.
- Typha: increases scale by reducing each node's impact on the datastore, just like the Datastore plugin.

HUAWEI

- Felix is used for:
  - Interface management
  - Route programming
  - ACL programming
  - Network state reporting

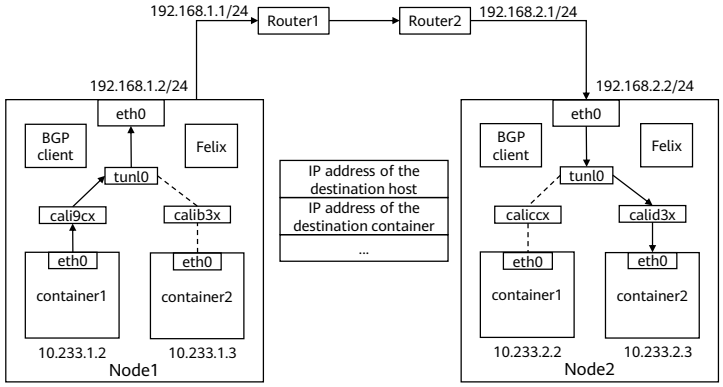
Calico Networking Architecture

- Calico realizes flexible networks by integrating the following architecture components:
  - Calico CNI network plugin
    - Connects pods to the host network namespace's layer 3 routing using a pair of virtual ethernet devices (veth pair).
  - Calico CNI IPAM plugin
    - Allocates IP addresses for pods out of one or more configurable IP address ranges, dynamically allocating small blocks of IPs per node as required.
  - Overlay network modes
    - Provides both VXLAN or IP-in-IP (or IPIP) overlay networks.
  - Non-overlay network modes
    - Provides non-overlay networks running on top of any underlying layer 2 network, or a layer 3 network that is either a public cloud network with appropriate cloud provider integration, or a BGP-capable network.
  - Network policy enforcement
    - Implements the full range of Kubernetes NetworkPolicy features, plus the extended features of Calico NetworkPolicy.

HUAWEI

Calico's Overlay Network Mode: IP-in-IP

- When the original IP packet enters the tunl0 device, the IPIP driver of the Linux kernel takes over the IP packet and encapsulates it into an IP packet of the host network. The destination address is the IP address of Node2.

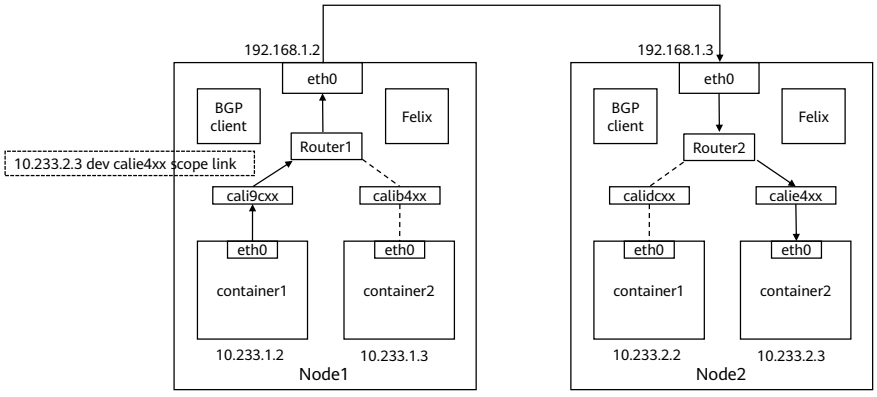


HUAWEI

- The IP-in-IP mode can also be abbreviated as the IPIP mode.

Calico's Non-overlay Network Mode: BGP

- There is no bridge, and instead it uses a veth pair for containers with routing rules. BGP maintains these rules and requires the cluster host machines be connected at layer 2. The default routing mode is node-to-node mesh..

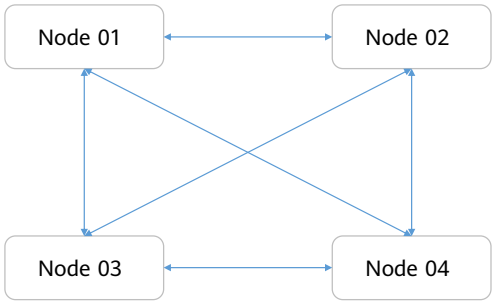


HUAWEI

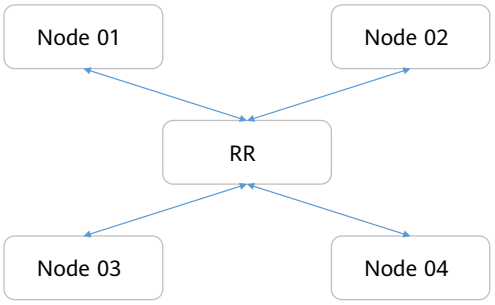
- BGP is a centerless routing protocol used to maintain routes between autonomous systems in large-scale data centers.
- Felix is the Calico agent running on each node, used for configuring routes and ACLs on the node.
- BGP client distributes routes configured on Felix to the Calico network to ensure layer 3 communication between nodes.

Calico's Route Learning Modes

- Method 1: Nodes exchange UPDATE packets to advertise BGP routing tables to BGP peers.



- Mode 2: Each node transfers its BGP routing table to the RR and synchronizes the routing tables of other nodes from the RR.



HUAWEI

- Method 2 is recommended for clusters with 100 or so nodes.

Comparison Between Different Calico Network Modes

Mode	Data Packet Encapsulation	Advantage	Disadvantage
VXLAN	On the VXLAN device, the source and destination MAC addresses of the data packets sent from the pod are replaced with the MAC addresses of the VXLAN NICs on the local and peer nodes. The outer destination IP address is obtained from the forwarding database (FDB) table based on the route and the MAC address of the peer VXLAN.	Nodes operate across network segments assuming they communicate with each other at layer 3. There is no special requirement for host gateway routes. Nodes communicate with each other at layer 2 through VXLAN devices.	Packet encapsulation and decapsulation have certain performance loss.
IPIP	On the tunl0 device, the MAC layer of the data packet sent by the pod is removed and the IP layer is left encapsulated. The outer destination IP address is obtained based on the route.	Nodes can work across network segments as long as they can communicate with each other at layer 3. There is no special requirement for host gateway routes.	Packet encapsulation and decapsulation have certain performance loss.
BGP	Data packets do not need to be encapsulated.	Packets do not need to be encapsulated or decapsulated. The BGP can be used to implement layer 3 connectivity between hosts on the pod network. When nodes are not in different network segments, the pod network is similar to the host-gw of Flannel. Cross-network segment communication is supported to meet the requirements of complex network architecture.	In cross-network segment scenarios, the host gateway router must function as a BGP speaker to learn and advertise pod subnet routes.

HUAWEI

- At layer 2, the source and destination MAC addresses of VXLAN packets are the MAC addresses of the local and peer VXLAN devices.
- At layer 3, VXLAN packets are encapsulated in UDP packets, which must be reachable at layer 3 between nodes.

Calico Resource Object Configuration File: IP Pool

```
apiVersion: crd.projectcalico.org/v1
kind: IPPool
metadata:
  annotations:
    projectcalico.org/metadata: '{"uid":"ca33f0d2-54d2-499e-b957-c5d4bc706bee","creationTimestamp":"2024-01-22T08:43:39Z"}'
  creationTimestamp: "2024-01-22T08:43:39Z"
  generation: 1
  name: default-ipv4-ippool
  resourceVersion: "5117"
  uid: 5c49a4ba-4f04-46c9-bfad-ba4424944ac0
spec:
  allowedUses:
  - Workload
  - Tunnel
  blockSize: 26                                    #### Subnet size allocated to the address pool. This value will be used in route
calculation.
  cidr: 10.0.0.0/8                                 #### IP address range used by the address pool.
  ipipMode: Always                                 #### When the IPIP mode is used, the value is set to Always.
  natOutgoing: true                                #### If this parameter is set to true, addresses in the local address pool will be
translated when they are sent to addresses outside the address pool.
  nodeSelector: all()
  vxlanMode: Never                                 #### Similar to ipipMode.
```


HUAWEI

- **ipipMode** and **vxlanMode** can be set to three values. The following takes **ipipMode** as an example:
  - **Always**: IPIP is used to route packets when the destination IP address is in an IP Pool that has IPIP enabled.
  - **CrossSubnet**: If the IP address of the destination node is in a different subnet, Calico will only route using IPIP. Generally speaking, if this parameter is used, the network mode changes to BGP.
  - **Never**: IPIP is not used.
- The value of **apiVersion** changes based on other configurations. If **calicoctl** is used, set **apiVersion** to **projectcalico.org/v3**. If **kubectl** is used, set **apiVersion** to **projectcalico.org/v1**.

Calico Resource Object Configuration File: RR

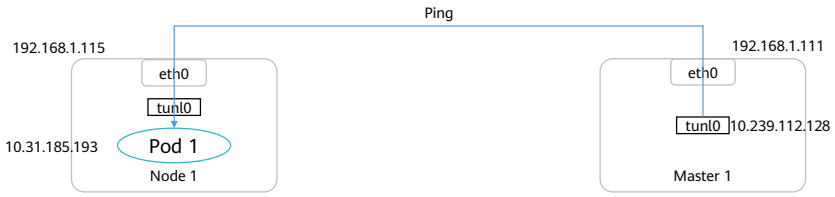
```
apiVersion: projectcalico.org/v3
kind: Node
metadata:
  labels:
    route-reflector: true                          #### Mandatory. Other nodes will find the RR based on this label.
  name: master01                                    #### Node name, which must be obtained by running the calicoctl get nodes command.
spec:
  bgp:
    ipv4Address: 192.168.1.111/24                 #### IP address of the specified node.
    ipv4IPIPTunnelAddr: 10.239.112.128          #### IP address of tunl0 on the specified node.
    routeReflectorClusterID: 1.1.1.1              #### Custom. It uniquely identifies a BGP network.
```

```
kind: BGPPeer
apiVersion: projectcalico.org/v3
metadata:
  name: bgppeer
spec:
  nodeSelector: all()                                 #### Specify all nodes.
  peerSelector: route-reflector=="true"           #### RR label, indicating that all backend nodes synchronize routes with the RR.
```


HUAWEI

- Deploying only one RR makes your system susceptible to a single point of failure. You are advised to deploy at least two RRs.

Packet Forwarding in IPIP Mode

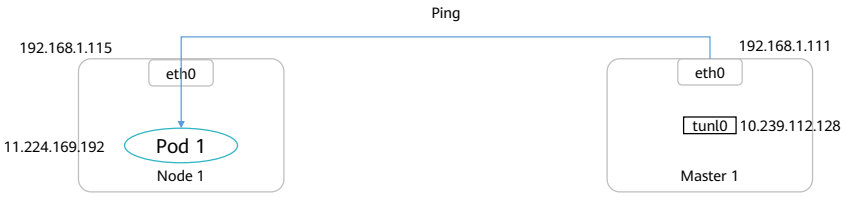



```
#### Capture packets on the eth0 of node 1. The details are as follows:
[root@node1 ~]# tcpdump -i eth0 -nn ip host 192.168.1.115 and host 192.168.1.111
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens192, link-type EN10MB (Ethernet), snapshot length 262144 bytes
09:54:36.138722 IP 192.168.1.111 > 192.168.1.115: IP 10.239.112.128 > 10.31.185.193: ICMP echo request, id 5902, seq 5, length 64
09:54:36.138811 IP 192.168.1.115 > 192.168.1.111: IP 10.31.185.193 > 10.239.112.128: ICMP echo reply, id 5902, seq 5, length 64
09:54:37.162851 IP 192.168.1.111 > 192.168.1.115: IP 10.239.112.128 > 10.31.185.193: ICMP echo request, id 5902, seq 6, length 64
09:54:37.162951 IP 192.168.1.115 > 192.168.1.111: IP 10.31.185.193 > 10.239.112.128: ICMP echo reply, id 5902, seq 6, length 64
09:54:38.186733 IP 192.168.1.111 > 192.168.1.115: IP 10.239.112.128 > 10.31.185.193: ICMP echo request, id 5902, seq 7, length 64
09:54:38.186826 IP 192.168.1.115 > 192.168.1.111: IP 10.31.185.193 > 10.239.112.128: ICMP echo reply, id 5902, seq 7, length 64
```


HUAWEI

- According to the captured packets, the inner source and destination addresses of the packet are 10.239.112.128 and 10.31.185.193, and the outer source and destination addresses are 192.168.1.111 and 192.168.1.115. The complete forwarding path of the packet is 10.239.112.128 --> 192.168.1.111 --> 192.168.1.115 --> 10.31.185.193.

Packet Forwarding in BGP Mode




```
#### Capture packets on the eth0 of node 1. The details are as follows:
[root@node01 ~]# tcpdump -i ens192 -nn ip host 11.224.169.192
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens192, link-type EN10MB (Ethernet), snapshot length 262144 bytes
14:23:45.578527 IP 192.168.1.111 > 11.224.169.192: ICMP echo request, id 5031, seq 10, length 64
14:23:45.578595 IP 11.224.169.192 > 192.168.1.111: ICMP echo reply, id 5031, seq 10, length 64
14:23:46.602534 IP 192.168.1.111 > 11.224.169.192: ICMP echo request, id 5031, seq 11, length 64
14:23:46.602608 IP 11.224.169.192 > 192.168.1.111: ICMP echo reply, id 5031, seq 11, length 64
14:23:47.626564 IP 192.168.1.111 > 11.224.169.192: ICMP echo request, id 5031, seq 12, length 64
14:23:47.626639 IP 11.224.169.192 > 192.168.1.111: ICMP echo reply, id 5031, seq 12, length 64
```


HUAWEI

Contents

1. Kubernetes Network Model
2. Flannel Networking Solution
3. Calico Networking Solution
4. **Kubernetes Network Policies**

HUAWEI

NetworkPolicy Overview

- NetworkPolicy defines Kubernetes network isolation and is described by dedicated API objects.
- It also provides network control policies to isolate applications and reduce attack surfaces. It uses label selectors to simulate segmented networks and control internal and external traffic.
- Before using NetworkPolicy, pay attention to the following:
  - Enable **extensions/v1bata1/networkpolicies** for **kube-apiserver**.
  - Network plugins, such as Calico, Romana, Weave, and kube-router, must support NetworkPolicy.
- The Kubernetes network plugin uses the NetworkPolicy definition to generate iptables rules on the host machine.
- NetworkPolicy policies can be controlled at multiple levels, including the cluster, namespace, pod, subnet, and port levels.

HUAWEI

- NetworkPolicy cannot control access between containers in a pod.

YAML File of a NetworkPolicy Definition

```
apiVersion: networking.k8s.io/v1                    ##### Object resource API and version number
kind: NetworkPolicy                                 ##### Name of the object resource type
metadata:
  name <string>                                         ##### Object resource name
  namespace <string>                                    ##### NetworkPolicy is a namespace-level object resource. Therefore, you need to
specify the namespace.
spec:
  podSelector <Object>                              ##### Destination pod. This parameter is mandatory. If this parameter is left empty, all pods
in the namespace are used.
  policyTypes <[]string>                            ##### Ingress indicates an inbound control, and Egress indicates an outbound control. To
implement all-round control, they must be set at the same time.
  ingress <[]Object>                                ##### Inbound traffic policies. The value is an allowlist. An empty value indicates all traffic
policies.
    - from <[]Object>                               ##### Inbound traffic source. If this parameter is left blank, all valid sources are used.
      - ipBlock  <Object>                           ##### Source address, which cannot be used together with namespaceSelector and
podSelector.
      - namespaceSelector <Object>                  ##### Source namespace.
        podSelector <Object>                                ##### Source pod.
      ports <[]Object>                              ##### Source port number.
  egress <[]Object>                                 ##### Destination of outbound traffic. If this parameter is left blank, it indicates all
destinations. The syntax is the same as that of ingress.from.
  - to <[]Object>
    ports <[]Object>
```


HUAWEI

AND and OR of Policies

- NetworkPolicy implements AND and OR of policies in the form of lists and dictionaries.
- **namespaceSelector** and **podSelector** indicate OR in a list, as shown in the YAML file in the lower left corner. The **from** field defines two cases, which means NetworkPolicy takes effect when the namespace or pod meets the conditions.
- **namespaceSelector** and **podSelector** indicate AND in a list, as shown in the YAML file in the lower right corner. The **from** field defines only one situation, where NetworkPolicy takes effect only when both the namespace and pod meet the conditions.

```
...
 ingress:
 - from:
   - namespaceSelector:
       matchLabels:
         user: alice
   - podSelector:
       matchLabels:
         role: client
...
```

OR

```
...
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          user: alice
      podSelector:
        matchLabels:
          role: client
...
```

AND

HUAWEI

NetworkPolicy Example

- This isolation rule is valid only for pods with the **role=db** label in the default namespace. The restricted request types include **Ingress** and **Egress**.
- Kubernetes rejects any request to access the isolated pod unless the request comes from an allowlisted object and accesses port 6379 of the isolated pod. The allowlisted objects include:
  - Pods with the **role=fronted** labels in the default namespace.
  - Any pod in the namespace that carries the **project=myproject** label.
  - Requests whose source addresses belong to 172.17.0.0/16 but do not belong to 172.17.1.0/24.
- Kubernetes rejects any request sent by the isolated pod unless the destination address of the request belongs to the 10.0.0.0/24 network segment and port 5978 of the network segment is accessed.

```
apiVersion:
networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: network-policy
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - ipBlock:
        cidr: 172.17.0.0/16
        except:
        - 172.17.1.0/24
    - namespaceSelector:
        matchLabels:
          project: myproject
    - podSelector:
        matchLabels:
          role: frontend
    ports:
    - protocol: TCP
      port: 6379
  egress:
  - to:
    - ipBlock:
        cidr: 10.0.0.0/24
    ports:
    - protocol: TCP
      port: 5978
```


HUAWEI

Time When NetworkPolicy Takes Effect

- 1. When a new NetworkPolicy object is created, the network plugin may take some time to process the new object.
  - Example: A pod is created before the network plugin completes the NetworkPolicy processing, and may be in the unprotected state initially and isolated by the isolation rule once the task is processed.
- 2. Each NetworkPolicy is applied to all selected pods. Network plugins may implement NetworkPolicy on a distributed scale. When a pod is created for the first time or when policies of a pod change, the pod may see a slightly inconsistent NetworkPolicy view.
  - Example: A new pod may be able to access pod A on node 1 immediately, but takes several seconds to access pod B on node 2.

HUAWEI

Quiz

1. (Multiple-answer question) Which of the following working modes are supported by the Calico network plugin?
   A. UDP
   B. VXLAN
   C. IPIP
   D. BGP

HUAWEI

- BCD

Summary

- This course describes the networking solutions for Kubernetes clusters, including the implementation of mainstream solutions Flannel and Calico, and Kubernetes network policies.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling
CNI	Container Network Interface
BGP	Border Gateway Protocol

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Kubernetes Persistent Storage



Foreword

- This course describes how to realize persistent storage by connecting Kubernetes to Ceph, including RADOS Block Devices (RBD) and Ceph File System (CephFS).

HUAWEI

Objectives

- On completion of this course, you will understand:
  - Kubernetes storage.
  - How to connect Kubernetes to RBD.
  - How to connect Kubernetes to CephFS.

HUAWEI

Contents

1. **Kubernetes Persistent Storage**
2. RBD
3. CephFS

HUAWEI

Kubernetes Persistent Storage

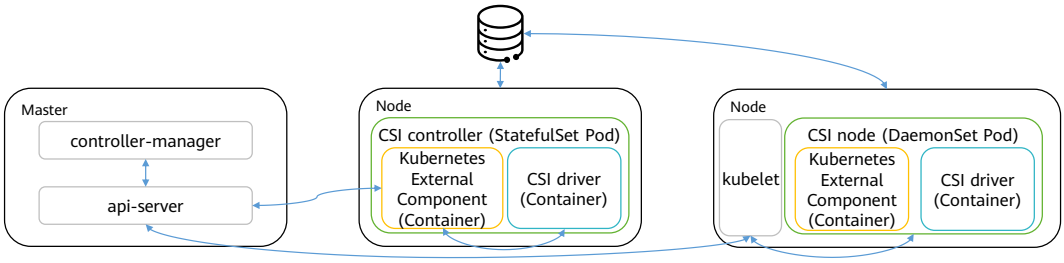
- Kubernetes uses volumes for persistent storage.
- Kubernetes volumes support multiple types of backend storage:
  - CephFS
  - RBD
  - Network File System (NFS)
  - Internet Small Computer System Interface Protocol (iSCSI)
  - GlusterFS

HUAWEI

- This slide lists only some of the supported backend storage devices.
- RBD and CephFS are no longer officially recommended for backend storage. Instead, third-party CSI drivers are recommended.

Kubernetes CSI

- Kubernetes Container Storage Interface (CSI) consists of three parts:
  - Kubernetes Core: a component of Kubernetes. api-server maintains resource objects such as PersistentVolume (PV), PersistentVolumeClaim (PVC), and StorageClass, and kubelet maintains volumes.
  - Kubernetes External Component: CSI extension provided by the Kubernetes community. Generally, sidecars and components provided by storage vendors form a pod.
  - External Component: CSI driver provided by the storage vendor, comprising the controller and node.



HUAWEI

- In addition to the controller and node, an external component also includes the identity used for authentication.
- The Kubernetes External Component container running in the CSI controller provides plug-ins such as external-attacher and external-provisioner, whereas that in the CSI node provides plug-ins such as node-driver-registrar.
  - external-attacher: monitors changes of VolumeAttachment objects and triggers ControllerPublish and ControllerUnpublish operations for CSl endpoints.
  - external-provisioner: monitors changes of PersistentVolumeClaim objects and triggers CreateVolume and DeleteVolume operations for CSl endpoints.
  - node-driver-registrar: registers storage drivers with kubelet.
- The CSI controller is deployed as a StatefulSet or Deployment.

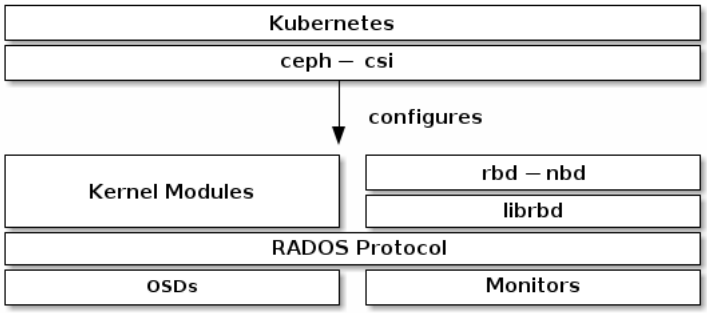
Kubernetes CSI Driver

- After the Kubernetes storage is connected to the backend storage through the standard CSI, the code of the storage product is decoupled from that of Kubernetes. The backend storage only provides a CSI driver to be invoked by Kubernetes.
- The CSI driver is deployed as a resource object to capture information about the CSI volume driver in the cluster.
- The Kubernetes controller uses the CSI driver to mount and unmount storage.
- kubelet uses the CSI driver to determine whether to transfer pod information during storage mounting.
- The CSI driver is not isolated by namespaces.

HUAWEI

Connecting Kubernetes to Ceph

- Kubernetes uses RBD and CephFS as permanent storage through ceph-csi.



HUAWEI

- Source of image 2: https://docs.ceph.com/en/latest/rbd/rbd-kubernetes/?highlight=kubernetes

Contents

1. Kubernetes Backend Storage
2. **ceph-csi Deployment**

HUAWEI

Deployment Process of ceph-csi

- 1. Create a resource pool in Ceph.
- 2. Create a user for the resource pool.
- 3. Use **ConfigMap** to transfer the Ceph cluster definition to Kubernetes.
- 4. Use **ConfigMap** to define the Key Management Service (KMS) server.
- 5. Use **ConfigMap** to define Ceph configurations.
- 6. Use the secret to create a CephX credential for authentication between Kubernetes and Ceph.
- 7. Deploy the ceph-csi plug-in.
- 8. Invoke the backend storage.
  - Static assignment
  - Dynamic assignment

HUAWEI

- The ceph-csi official website provides a YAML template. To find the RBD template, go to https://github.com/ceph/ceph-csi/tree/devel/deploy/rbd/kubernetes. For the CephFS template, go to https://github.com/ceph/ceph-csi/tree/devel/deploy/cephfs/kubernetes. Modify your configurations to fit the actual situation.
- If there is no KMS service, define a null value.
- Ceph configurations are those in **ceph.conf**.

Key Steps in ceph-csi Deployment – ConfigMap Definition

```
##### Query clusterID and monitors in Ceph.
---
apiVersion: v1
kind: ConfigMap
data:
  config.json: |-
    [
      {
        "clusterID": "b9127830-b0cc-4e34-aa47-
9d1a2e9949a8",
        "monitors": [
          "192.168.1.1:6789",
          "192.168.1.2:6789",
          "192.168.1.3:6789"
        ]
      }
    ]
metadata:
  name: ceph-csi-config
```

```
##### If there is no KMS, leave it empty.
---
apiVersion: v1
kind: ConfigMap
data:
  config.json: |-
    {}
metadata:
  name: ceph-csi-encryption-kms-config
```

```
##### keyring is mandatory and its value is left empty.
---
apiVersion: v1
kind: ConfigMap
data:
  ceph.conf: |
    [global]
    auth_cluster_required = cephx
    auth_service_required = cephx
    auth_client_required = cephx
  keyring: |
metadata:
  name: ceph-config
```


HUAWEI

- The **ConfigMap** on the left defines the Ceph cluster. In the **ConfigMap** on the right, retain the template when there are no special requirements.
- The created **ConfigMap** will be invoked by ceph-csi. If no namespace is specified in the YAML file, add namespace information to ensure the pods required by **ConfigMap** and ceph-csi are in the same namespace. Multiple namespaces may invoke the same **ConfigMap**. You are advised to specify the **ConfigMap** when deploying the namespace, instead of writing it in the YAML file.

Key Steps in ceph-csi Deployment – Secret Definition

```
---
apiVersion: v1
kind: Secret
metadata:
  name: csi-rbd-secret
  namespace: csi
stringData:
  # Key values correspond to a user name and its key, as defined in the
  # ceph cluster. User ID should have required access to the 'pool'
  # specified in the storage class
  userID: kubernetes
  userKey: AQAtBbNl/mm7AhAAXeVQDRVQ+oK2JIv95ZbHHQ==

  # Encryption passphrase
  encryptionPassphrase: test_passphrase
```

```
[ceph: root@ceph01 /]# ceph auth get client.kubernetes
[client.kubernetes]
        key = AQAtBbNl/mm7AhAAXeVQDRVQ+oK2JIv95ZbHHQ==
        caps mgr = "profile rbd pool=kubernetes"
        caps mon = "profile rbd"
        caps osd = "profile rbd pool=kubernetes"
exported keyring for client.kubernetes
```


HUAWEI

- In the downloaded ceph-csi, the **example** directory provides templates of required files. For example, the **secret.yaml** file.

Key Steps in ceph-csi Deployment – ceph-csi Plug-in

- To deploy the ceph-csi plug-in, select the YAML file based on the type of the connected Ceph resource pool.
  - RBD resource pool
    - csi-provisioner-rbac.yaml
    - csi-nodeplugin-rbac.yaml
    - csi-rbdplugin-provisioner.yaml
    - csi-rbdplugin.yaml
  - CephFS resource pool
    - csi-provisioner-rbac.yaml
    - csi-nodeplugin-rbac.yaml
    - csi-cephfsplugin-provisioner.yaml
    - csi-cephfsplugin.yaml

HUAWEI

- Note: The rbac files of RBD and CephFS are in the same name, but their contents are different. The files must be deployed separately and cannot replace each other.
- **csi-provisioner-rbac.yaml** and **csi-nodeplugin-rbac.yaml** must be in the same namespace as other resource objects.
- **csi-rbdplugin-provisioner** is deployed as a deployment. It has three replicas by default. You can change the number of replicas as required.

Key Steps in ceph-csi Deployment – Mounting

```
#### Compile the StroageClass file, in which secret and
namespace must correspond to each other.
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
   name: csi-rbd-sc
   namespace: csi
provisioner: rbd.csi.ceph.com
parameters:
   clusterID: 88ab034c-a622-11ee-9a8c-000c2905539b
   pool: kubernetes
   imageFeatures: layering
   csi.storage.k8s.io/provisioner-secret-name: csi-rbd-secret
   csi.storage.k8s.io/provisioner-secret-namespace: csi
   csi.storage.k8s.io/controller-expand-secret-name: csi-rbd-secret
   csi.storage.k8s.io/controller-expand-secret-namespace: csi
   csi.storage.k8s.io/node-stage-secret-name: csi-rbd-secret
   csi.storage.k8s.io/node-stage-secret-namespace: csi
reclaimPolicy: Delete
allowVolumeExpansion: true
mountOptions:
   - discard
```

```
#### Compile the PVC file, in which storageClassName
must correspond to StorageClass.
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: raw-block-pvc
  namespace: csi
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Block
  resources:
    requests:
      storage: 1Gi
  storageClassName: csi-rbd-sc
```


HUAWEI

- If an RBD resource pool is used, **volumeMode** of PVC can be **Block** or **Filesystem**. The default value is **Filesystem**. If **volumeMode** is **Filesystem**, **accessMode** can be **ReadWriteOnce** or **ReadOnlyMany**. If **volumeMode** is **Block**, **accessMode** can be **ReadWriteOnce**, **ReadWriteMany** or **ReadOnlyMany**.
- If a CephFS resource pool is used, PV can be created statically or dynamically, where users must be specified in the secret file.

Quiz

1. (Single-answer question) In Kubernetes storage, which of the following is used to define a directory for persistent storage on the host machine?
   A. PV
   B. PVC
   C. StorageClass
   D. CSI

HUAWEI

- A

Summary

- This course describes how to realize persistent storage by connecting Kubernetes to Ceph RBD or CephFS.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling
PV	Persistent Volume
PVC	Persistent Volume Claim
RBD	RADOS Block Devices

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

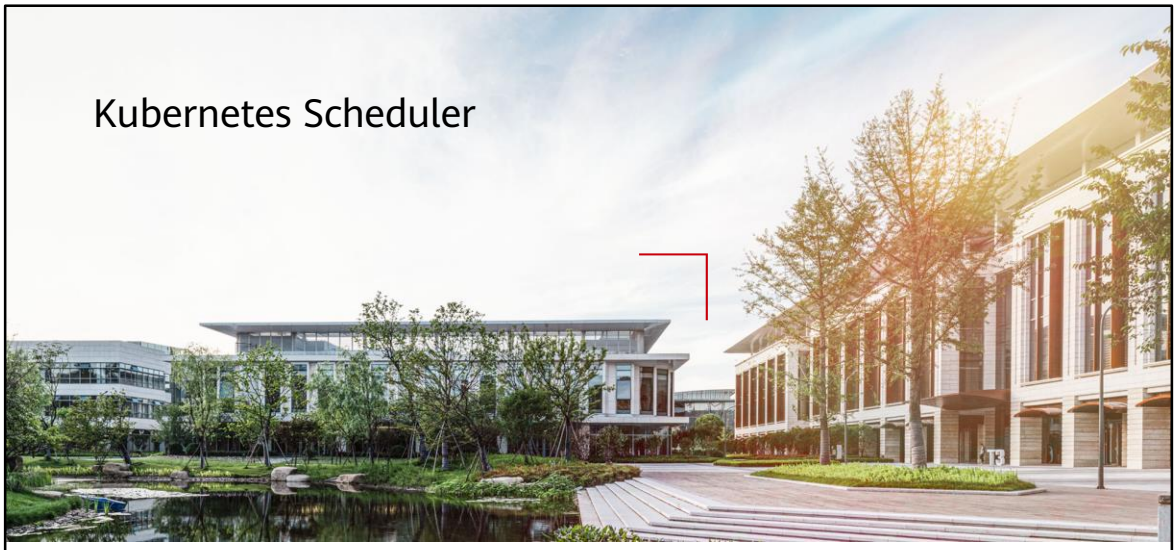
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Kubernetes Scheduler



Foreword

- This course describes how to schedule resources and pods in Kubernetes. In this process, kube-scheduler selects a node to run pods created by kube-controller-manager, and kubelet takes over the subsequent work on the node.

Objectives

- On completion of this course, you will understand:
  - Kubernetes resources and Quality of Service (QoS) classes for pods.
  - Principles of the default Kubernetes scheduler.
  - Pod scheduling.
  - Pod priority and preemption.
  - Scheduler performance tuning and eviction.

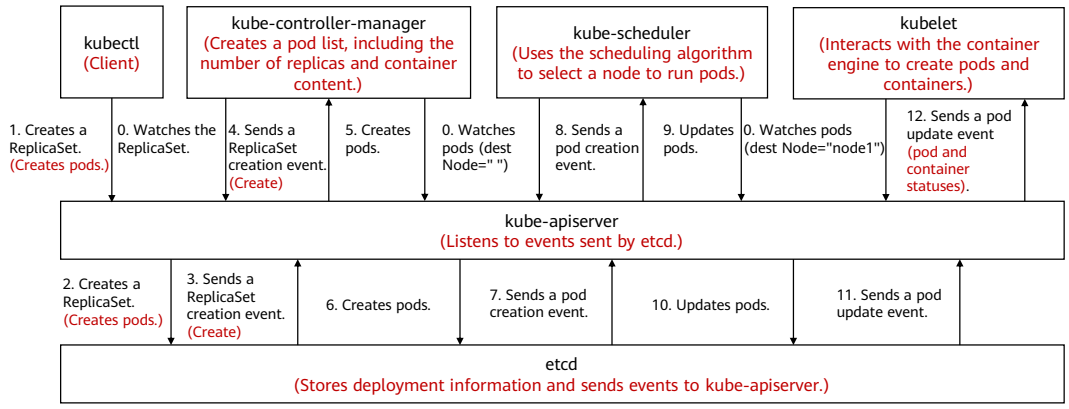
HUAWEI

Contents

1. **Kubernetes Scheduling Overview**
2. Default Scheduler
3. Assigning Pods to Nodes
4. Affinity and Anti-Affinity
5. Taints and Tolerations
6. Eviction
7. Pod Priority and Preemption

HUAWEI

How to Create and Initialize Pods



HUAWEI

- 0. There are three List-Watches: kube-controller-manager and kube-scheduler (running on the master), and kubelet (running on the worker node). They watch the events sent by kube-apiserver when processes are started.
- 1. You can use kubectl or other API clients to submit requests to kube-apiserver to create a pod replica.
- 2. kube-apiserver attempts to save the metadata of the pod object to etcd. After the write operation is complete, kube-apiserver returns a confirmation message to the client.
- 3. After receiving the pod creation information, etcd sends a Create event to kube-apiserver.
- 4. kube-controller-manager keeps watching events in kube-apiserver through HTTPS port 6443. After receiving the Create event, kube-apiserver sends it to kube-controller-manager.
- 5. After receiving the Create event, kube-controller-manager calls ReplicationController to ensure the number of replicas to be created on the node. Once the number of replicas is less than that defined in ReplicationController, ReplicationController automatically creates replicas.
- 6. After a pod replica is created on kube-controller-manager, kube-apiserver records detailed information about the pod in etcd, for example, the number of replicas in the pod and the content of the container.

Kubernetes Scheduling Process

- The scheduling process is as follows:
  - 1. Filters out nodes that do not meet certain conditions (also known as the predicates process).
  - 2. Sorts and scores the nodes by priority (also known as the priorities process).
  - 3. Selects the top ranked node. If an error occurs, an error is returned.
- If there is no appropriate node during the predicates process, the pods remain pending and retries scheduling until a node is selected. If multiple nodes meet the conditions, the priorities process sorts the nodes according to priorities.
- The algorithm scores all priority classes and weights to obtain the optimal node.

HUAWEI

Contents

1. Kubernetes Scheduling Overview
2. **Default Scheduler**
3. Assigning Pods to Nodes
4. Affinity and Anti-Affinity
5. Taints and Tolerations
6. Eviction
7. Pod Priority and Preemption

HUAWEI

Default Scheduler

- kube-scheduler is the default scheduler of Kubernetes. It selects the optimal node to run newly created pods.
  - Filtering: Finds a set of feasible nodes to schedule the pods.
  - Scoring: Ranks the remaining nodes to choose the most suitable pod placement.
- In a scheduling process, kube-scheduler first calls a group of scheduling algorithms called predicates to check each node, and then a group of scheduling algorithms called priorities to score each feasible node. Finally, it selects the node with the highest score.
- Factors that need to be considered during scheduling include independent and overall resource requirements, hardware, software, and policy restrictions, affinity and anti-affinity requirements, data locality, and interference between workloads.

HUAWEI

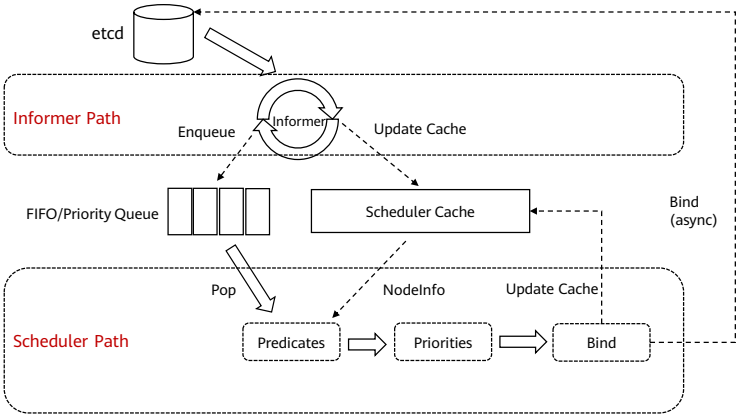
Principles

- kube-scheduler assigns defined pods to nodes in the cluster according to the following principles:
  - Fairness: Resources can be allocated to each node.
  - Efficient resource utilization: All cluster resources are used to the maximum extent.
  - Efficiency: The scheduling performance must be good so that a large number of pods can be scheduled as soon as possible.
  - Flexible: Scheduling logic is customized by users.
- kube-scheduler runs as an independent program, where it constantly watches kube-apiserver, obtains pods whose **spec.nodeName** is empty, and creates a binding for each pod to indicate the node where the pod is assigned.

HUAWEI

Scheduling Mechanism

- Two control loops: Informer Path and Scheduler Path



HUAWEI

- The first control loop is called Informer Path. It starts a series of Informers to watch the changes of scheduling-related API objects such as pods, nodes, and services in etcd. For example, after an unscheduled pod (whose **nodeName** is empty) is created, the scheduler adds the pod to the scheduling queue through the handler of PodInformer.
- The second control loop determines how the scheduler schedules pods, which is called Scheduler Path. The logic of Scheduler Path is to continuously dequeue pods from the scheduling queue, and then call predicates to find a set of nodes, that is, a list of host machines that can run the pods. The predicates obtain required node information from Scheduler Cache, which is one of the main methods for the scheduler to improve the algorithm execution efficiency.

Scheduler Performance Tuning

- In large clusters, you can tune the scheduler's behaviour to balance scheduling outcomes between latency (new pods are placed quickly) and accuracy (the scheduler rarely makes poor placement decisions).
- You can configure this tuning setting using **percentageOfNodesToScore** of kube-scheduler.
- This KubeSchedulerConfiguration setting determines a threshold for scheduling nodes in your cluster.
- To improve scheduling performance, kube-scheduler can stop looking for feasible nodes once it has found enough of them. In large clusters, this saves time compared to a naive approach that would consider every node.
- You specify a threshold for how many nodes are enough, as a whole number percentage of all the nodes in your cluster. kube-scheduler converts this into an integer number of nodes. During scheduling, if kube-scheduler has identified enough feasible nodes to exceed the configured percentage, it stops searching for more feasible nodes and moves on to the scoring phase.

HUAWEI

Resource Bin Packing

- In the scheduling plugin **NodeResourcesFit** of kube-scheduler, there are two scoring strategies that support the bin packing of resources: **MostAllocated** and **RequestedToCapacityRatio**.
  - The **MostAllocated** strategy scores the nodes based on the utilization of resources, favoring the ones with higher allocation. For each resource type, you can set a weight to modify its influence in the node score.
  - The **RequestedToCapacityRatio** strategy allows users to specify resources along with weights for each resource to score nodes based on the request to capacity ratio. This allows users to bin pack extended resources by using appropriate parameters to improve the utilization of scarce resources in large clusters. It favors nodes according to a configured function of the allocated resources.

HUAWEI

Contents

1. Kubernetes Scheduling Overview
2. Default Scheduler
3. **Assigning Pods to Nodes**
4. Affinity and Anti-Affinity
5. Taints and Tolerations
6. Eviction
7. Pod Priority and Preemption

HUAWEI

nodeSelector

- **pod.spec.nodeSelector**: You can add the **nodeSelector** field to your pod specification and specify the node labels you want the target node to have. Kubernetes only schedules the pod onto nodes that have each of the labels you specify. This is a forcible matching rule.
- Add a label to a node.
  - **kubectl label nodes <your-node-name> class=a**
- List the nodes in your cluster, along with their labels.
  - **kubectl get nodes --show-labels**

```
apiVersion: v1
kind: Pod
metadata:
  name: nodeselector-pod
spec:
  containers:
  - name: nodeselector-pod
    image: nginx
    imagePullPolicy: IfNotPresent
  nodeSelector:
    class: a
```


HUAWEI

nodeName

- If the **pod.spec.nodeName** field is not empty, the scheduler ignores the pod and kubelet on the named node tries to place the pod on that node. This is a forcible matching rule.
- **nodeName** is a more direct form of node selection than affinity or **nodeSelector**. Using **nodeName** overrules using **nodeSelector** or affinity and anti-affinity rules.
- Limitations of using **nodeName**:
  - If the named node does not exist, the pod will not run, and in some cases may be automatically deleted.
  - If the named node does not have the resources to accommodate the pod, the pod will fail and its reason will indicate why, for example **OutOfmemory** or **OutOfcpu**.
  - Node names in cloud environments are not always predictable or stable.

```
apiVersion: v1
kind: Pod
metadata:
  name: nodename-pod
spec:
  containers:
  - name: nodename-pod
    image: nginx
    imagePullPolicy: IfNotPresent
  nodeName: k8snode2
```


HUAWEI

Pod Topology Spread Constraints

- You can use topology spread constraints to control how pods are spread across your cluster among failure domains such as nodes (**kubernetes.io/hostname**) and other custom topology domains.
- Pod topology spread constraints can be configured declaratively.

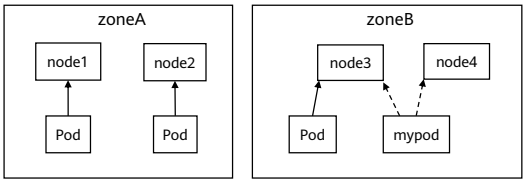
```
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
spec:
  # Configure a topology spread constraint.
  topologySpreadConstraints:
    - maxSkew: <integer>
      minDomains: <integer> # Optional, beta since v1.25
      topologyKey: <string>
      whenUnsatisfiable: <string>
      labelSelector: <object>
      matchLabelKeys: <list> # Optional, beta since v1.27
      nodeAffinityPolicy: [Honor|Ignore] # Optional, beta since v1.26
      nodeTaintsPolicy: [Honor|Ignore] # Optional, beta since v1.26
```


HUAWEI

- https://kubernetes.io/zh-cn/docs/concepts/scheduling-eviction/topology-spread-constraints/#example-conflicting-topologyspreadconstraints

One Topology Spread Constraint

- **topologyKey: zone** implies the even distribution will only be applied to nodes that are labeled **zone: <any value>**.
- If the scheduler placed **mypod** into zone **A**, the distribution of pods would become **[3, 1]**. That means the actual skew is then 2 (calculated as 3 - 1), which violates **maxSkew: 1**. To satisfy the constraints and context for this example, **mypod** can only be placed onto a node in zone **B**.
- Specify label **foo: bar**, which means only pods with the label can participate in calculation.



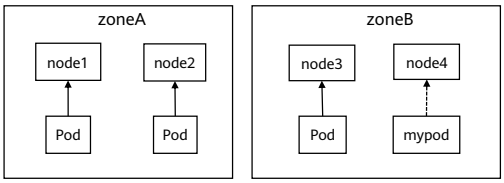

```
kind: Pod
apiVersion: v1
metadata:
  name: topo-pod1
  labels:
    foo: bar
spec:
  containers:
  - name: topo-pod
    image: nginx
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: zone
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        foo: bar
```


HUAWEI

- The field **whenUnsatisfiable: DoNotSchedule** tells the scheduler to let the incoming pod stay pending if the scheduler cannot find a way to satisfy the constraint.

Multiple Topology Spread Constraints

- To match the first constraint, **mypod** can only be placed onto nodes in zone **B**; while in terms of the second constraint, **mypod** can only be scheduled to **node4**. The scheduler only considers options that satisfy all defined constraints, so the only valid placement is onto **node4**.




```
kind: Pod
apiVersion: v1
metadata:
  name: topo-pod1
  labels:
    foo: bar
spec:
  containers:
  - name: topo-pod
    image: nginx
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: zone
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        foo: bar
  - maxSkew: 1
    topologyKey: kubernetes.io/hostname
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        foo: bar
```


HUAWEI

Other Topology Spread Constraints

- Conflicting topology spread constraints
  - Multiple constraints can lead to conflicts, making the pod stay in the pending state. To overcome this situation, you can either increase the value of **maxSkew** or modify one of the constraints to use **whenUnsatisfiable: ScheduleAnyway**.
- Topology spread constraints with node affinity
  - The scheduler will skip the non-matching nodes from the skew calculations if the incoming pod has **spec.nodeSelector** or **spec.affinity.nodeAffinity** defined.

HUAWEI

Contents

1. Kubernetes Scheduling Overview
2. Default Scheduler
3. Assigning Pods to Nodes
4. **Affinity and Anti-Affinity**
5. Taints and Tolerations
6. Eviction
7. Pod Priority and Preemption

HUAWEI

Affinity and Anti-Affinity

- Affinity enables the Kubernetes scheduler to place a pod on the node that meets the scheduling conditions, making scheduling more flexible.
- Affinity types:
  - **nodeAffinity** assigns a pod to a named node.
  - **podAffinity** assigns a pod to the same topology domain as a named pod.
  - **podAntiAffinity** assigns a pod to a different topology domain from a named pod.
- Affinity classification:
  - Hard affinity (**requiredDuringSchedulingIgnoredDuringExecution**) is a hard rule, where conditions must be satisfied for the pod to be scheduled; otherwise, the pod is always pending.
  - Soft affinity (**preferredDuringSchedulingIgnoredDuringExecution**) is a soft rule, where if a condition can be satisfied, it will be satisfied; but if not, it will be ignored.

HUAWEI

Node Affinity Configuration

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: zone
            operator: In
            values:
            - linux1
            - linux2
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: label-key
            operator: In
            values:
            - label-value
```

The **operator** field specifies a logical operator. The allowed values of **operator** are **In**, **NotIn**, **Exists**, **DoesNotExist**, **Gt**, and **Lt**, where **NotIn** and **DoesNotExist** allow you to define node anti-affinity behavior.

- The node must have a label whose key is **zone=linux1** or **zone=linux2**.
- The node preferably has a label whose key is **label-key=label-value**.

HUAWEI

- **In**: The label value is present in the supplied set of strings.
- **NotIn**: The label value is not contained in the supplied set of strings.
- **Gt**: The label value is greater than a certain value.
- **Lt**: The label value is less than a certain value.
- **Exists**: A label with this key exists on the object.
- **DoesNotExist**: No label with this key exists on the object.

Node Affinity Weight

```
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: label-1
            operator: In
            values:
            - key-1
      - weight: 50
        preference:
          matchExpressions:
          - key: label-2
            operator: In
            values:
            - key-2
```

You can specify a **weight** between 1 and 100 for each instance of the **preferredDuringSchedulingIgnoredDuringExecution** affinity type. The final sum of **weight** is added to the score of other priority functions for the node.

- If there are two possible nodes, one with the **label-1:key-1** label and another with the **label-2:key-2** label, the scheduler considers the **weight** of each node and adds the weight to the other scores for that node, and schedules the pod onto the node with the highest final score.
- The weight can be considered as a scheduling priority. A larger weight indicates a higher priority.

HUAWEI

Inter-Pod Affinity and Anti-Affinity

- Inter-pod affinity and anti-affinity allow you to constrain which nodes your pods can be scheduled on based on the labels of pods already running on that node, instead of the node labels.
  - For efficient communication, some pods sometimes need to be scheduled to similar or even the same domain (for example, the same node, equipment room, or zone), for example, the front-end pod and back-end pod of a service. The relationship between these pod objects is called affinity.
  - For security purposes, some pods are isolated from each other. Such relationship is called anti-affinity.
- The scheduler places the first pod in any domain, and then dynamically orchestrates the domains of pods that have affinity or anti-affinity relationships with the pod. Similar to node affinity, inter-pod affinity can be classified into hard affinity and soft affinity.
- Inter-pod affinity requires that related pod objects run in the same domain, while anti-affinity requires that they cannot run in the same domain. The domain here actually depends on the topology of the node. Whether pods are in the same domain varies according to the topology.

HUAWEI

- Inter-pod affinity and anti-affinity require substantial amounts of processing which can slow down scheduling in large clusters significantly. You are advised not to use them in clusters larger than several hundred nodes.
- Pod anti-affinity requires nodes to be consistently labeled. In other words, every node in the cluster must have an appropriate label matching **topologyKey**. If some or all nodes are missing the specified **topologyKey** label, it can lead to unintended behavior.
- If **topologyKey** is **kubernetes.io/hostname**, the scheduler determines whether the nodes are in the same domain based on their **hostname**.

Inter-pod Affinity and Anti-Affinity Configuration

```
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
          - key: security
            operator: In
            values:
            - S1
        topologyKey: topology.kubernetes.io/zone
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchExpressions:
            - key: security
              operator: In
              values:
              - S2
          topologyKey: topology.kubernetes.io/zone
```

The scheduler is allowed to place the example pod on a node only if that node belongs to a specific zone (specified by **topology.kubernetes.io/zone**) where other pods have been labeled with **security=S1**.

The scheduler should try to avoid scheduling the pod onto a node if that node belongs to a specific zone (specified by **topology.kubernetes.io/zone**) where other pods have been labeled with **security=S2**.

- In principle, **topologyKey** can be any allowed label key with the following exceptions for performance and security reasons:
  - For pod affinity, an empty **topologyKey** field is not allowed.
  - For **requiredDuringSchedulingIgnoredDuringExecution** pod anti-affinity rules, the admission controller **LimitPodHardAntiAffinityTopology** limits **topologyKey** to **kubernetes.io/hostname**, which is suited the situation where each node has only one pod replica.

HUAWEI

Contents

1. Kubernetes Scheduling Overview
2. Default Scheduler
3. Assigning Pods to Nodes
4. Affinity and Anti-Affinity
5. **Taints and Tolerations**
6. Eviction
7. Pod Priority and Preemption

HUAWEI

Taints and Tolerations

- Taints and tolerations
  - Taints allow a node to repel a set of pods.
  - Tolerations allow the scheduler to schedule pods with matching taints. They allow scheduling but do not guarantee scheduling.
- Scenarios
  - Taints and tolerations work together to ensure that pods are not scheduled onto inappropriate nodes. One or more taints are applied to a node. This marks that the node should not accept any pods that do not tolerate the taints.
  - If you want to dedicate a set of nodes for exclusive use by a particular set of applications, you can add a taint to those nodes and then add a corresponding toleration to their pods.

HUAWEI

Example Use Cases

- Add a taint to **node1**. The taint has key **key1**, value **value1**, and effect **NoSchedule**. This means that no pod will be able to schedule onto **node1** unless it has a matching toleration.
  - Add a taint to a node: **kubectl taint nodes node1 key1=value1:NoSchedule**
  - Remove a taint from a node: **kubectl taint nodes node1 key1=value1:NoSchedule-**
- Specify a toleration for a pod in the **PodSpec**.
- Both of the following tolerations match the taint created by the **kubectl taint** command above, and thus a pod with either toleration would be able to schedule onto **node1**.
  - **operator** is **Exists** (in which case no **value** should be specified).
  - **operator** is **Equal** and the values should be equal.

```
tolerations:
- key: "key1"
  operator: "Equal"
  value: "value1"
  effect: "NoSchedule"
```

```
tolerations:
- key: "key1"
  operator: "Exists"
  effect: "NoSchedule"
```


HUAWEI

Toleration Parameters

- The allowed values for the **effect** field are:
  - **NoExecute**: This affects pods that are already running on the node.
    - Pods that do not tolerate the taint are evicted immediately.
    - Pods that tolerate the taint without specifying **tolerationSeconds** in their toleration specification remain bound forever.
    - Pods that tolerate the taint with a specified **tolerationSeconds** remain bound for the specified amount of time. After that time elapses, the node lifecycle controller evicts the pods from the node.
  - **NoSchedule**: No new pods will be scheduled onto the tainted node unless they have a matching toleration. Pods currently running on the node are not evicted.
  - **PreferNoSchedule**: a "preference" or "soft" version of **NoSchedule**. The control plane will try to avoid placing a pod that does not tolerate the taint on the node, but it is not guaranteed.

```
tolerations:
- key: "node.kubernetes.io/unreachable"
  operator: "Exists"
  effect: "NoExecute"
  tolerationSeconds: 6000
```


HUAWEI

Multiple Taints and Tolerations

- You can put multiple taints on the same node and multiple tolerations on the same pod. The pod will be scheduled only when the tolerations match the taints.
  - **kubectl taint nodes node1 key1=value1:NoSchedule**
  - **kubectl taint nodes node1 key1=value1:NoExecute**
  - **kubectl taint nodes node1 key2=value2:NoSchedule**

```
tolerations:
- key: "key1"
  operator: "Equal"
  value: "value1"
  effect: "NoSchedule"
- key: "key1"
  operator: "Equal"
  value: "value1"
  effect: "NoExecute"
```

- In this case, the pod will not be able to schedule onto the node, because there is no toleration matching the third taint. But it will be able to continue running if it is already running on the node when the taint is added, because the third taint is the only one of the three that is not tolerated by the pod.

HUAWEI

Managing Nodes

- **cordon** marks a node as unschedulable. This changes the node status to **SchedulingDisabled**, so that new pods will not be scheduled onto the node. The existing pods are not affected and can still provide services for external systems.
  - **kubectl cordon <node-name>**
- **drain** evicts pods from a node and changes the node status to **SchedulingDisabled**.
  - **kubectl drain <node-name> --ignore-daemonsets --force**
- **delete** deletes pods from a node, creates the pods on another node, and deletes the node from the master node. In this way, the master node loses control over the node. To restore scheduling for the node, log in to the node and restart the kubelet service.
  - **kubectl delete node <node-name>**
- **uncordon** marks a node as schedulable.

```
Taints:             node.kubernetes.io/unschedulable:NoSchedule
Unschedulable:      true
```

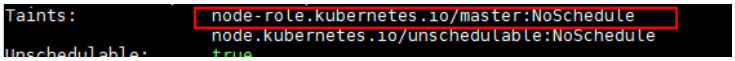
  - **kubectl uncordon <node-name>**

HUAWEI

- **cordon** and **uncordon** are actually adding and deleting taints.
- Results of pod eviction:
  - If the replica is controlled by the deployment, the pod is re-created on another node.
  - If an independent pod is not regenerative, it is deleted after being evicted and will not be scheduled onto other nodes.

Marking the Master Node as Schedulable

- Mark the master node as a schedulable node.
  - **kubectl taint node <master-node> node-role.kubernetes.io/master-**
- Restore the master node to the **MasterOnly** state.
  - **kubectl taint node <master-node> node-role.kubernetes.io/master="":NoSchedule**
- By default, Kubernetes adds a taint to the master node. **taint** is used to delete and add a taint.



Built-in Taints

- The node controller automatically taints a node when certain conditions are true. The following taints are built-in:
  - **node.kubernetes.io/not-ready**: The node is not ready. This corresponds to the NodeCondition **Ready** being **False**.
  - **node.kubernetes.io/unreachable**: The node is unreachable from the node controller. This corresponds to the NodeCondition **Ready** being **Unknown**.
  - **node.kubernetes.io/memory-pressure**: The node has memory pressure.
  - **node.kubernetes.io/disk-pressure**: The node has drive pressure.
  - **node.kubernetes.io/pid-pressure**: The node has PID pressure.
  - **node.kubernetes.io/network-unavailable**: The node network is unavailable.
  - **node.kubernetes.io/unschedulable**: The node is unschedulable.
  - **node.cloudprovider.kubernetes.io/uninitialized**: When kubelet is started with an external cloud provider, this taint is set on a node to mark it as unusable. After a controller from cloud-controller-manager initializes this node, kubelet removes this taint.

```
Taints:             node.kubernetes.io/unreachable:NoExecute
                    node.kubernetes.io/unreachable:NoSchedule
```

- If a node or the Docker service on a node is faulty and a certain condition is met, a taint is added to the node.

HUAWEI

Contents

1. Kubernetes Scheduling Overview
2. Default Scheduler
3. Assigning Pods to Nodes
4. Affinity and Anti-Affinity
5. Taints and Tolerations
6. **Eviction**
7. Pod Priority and Preemption

HUAWEI

Node-pressure Eviction

- Node-pressure eviction is the process by which kubelet proactively terminates pods to reclaim resources on nodes.
- kubelet monitors resources like memory, drive space, and filesystem inodes on your cluster's nodes. When one or more of these resources reach specific consumption levels, kubelet can proactively fail one or more pods on the node to reclaim resources and prevent starvation.
- During a node-pressure eviction, kubelet sets the phase for the selected pods to **Failed**, and terminates the pod.

HUAWEI

Eviction Signals and Thresholds

- Eviction signals
  - Current state of a particular resource at a specific point in time. kubelet uses eviction signals to make eviction decisions by comparing the signals to eviction thresholds, which are the minimum amount of the resource that should be available on the node.
- Eviction thresholds
  - You can specify custom eviction thresholds for kubelet to use when it makes eviction decisions. You can configure soft and hard eviction thresholds.
  - A soft eviction threshold pairs an eviction threshold with a required administrator-specified grace period. kubelet does not evict pods until the grace period is exceeded. kubelet returns an error on startup if you do not specify a grace period.
  - A hard eviction threshold has no grace period. When a hard eviction threshold is met, kubelet terminates pods immediately without graceful termination to reclaim the starved resource.
- Monitoring interval
  - kubelet evaluates eviction thresholds based on its configured **housekeeping-interval**, which defaults to **10s**.

HUAWEI

API-initiated Eviction

- API-initiated eviction is the process by which you use the Eviction API to create an Eviction object that triggers graceful pod termination.
- If the API server allows eviction, the pod is deleted as follows:
  - The **Pod** resource in the API server is updated with a deletion timestamp, after which the API server considers the **Pod** resource to be terminated. The **Pod** resource is also marked with the configured grace period.
  - kubelet on the node where the local pod is running notices that the pod resource is marked for termination and starts to gracefully shut down the local pod.
  - While kubelet is shutting the pod down, the control plane removes the pod from the **Endpoint** and **EndpointSlice** objects.
  - After the grace period for the pod expires, kubelet forcefully terminates the local pod.
  - kubelet tells the API server to remove the **Pod** resource.
  - The API server deletes the **Pod** resource.

HUAWEI

- You can request eviction by calling the Eviction API directly, or programmatically using a client of the API server, like the **kubectl drain** command. This creates an **Eviction** object, which causes the API server to terminate the pod.
- API-initiated evictions respect your configured **PodDisruptionBudgets** and **terminationGracePeriodSeconds**.

Self-healing Behavior

- kubelet attempts to reclaim node-level resources before it terminates end-user pods. For example, it removes unused container images when drive resources are starved.
- If the pods are managed by a workload management object (such as StatefulSet or Deployment) that replaces failed pods, the control plane (kube-controller-manager) creates new pods in place of the evicted pods.
- Self-healing of static pods
  - If you are running a static pod on a node that is under resource pressure, kubelet may evict that static pod. kubelet then tries to create a replacement, because static pods always represent an intent to run a pod on that node.
  - kubelet takes the priority of the static pod into account when creating a replacement. If the static pod manifest specifies a low priority, and there are higher-priority pods defined within the cluster's control plane, and the node is under resource pressure, kubelet may not be able to make room for that static pod. kubelet continues to attempt to run all static pods even when there is resource pressure on a node.

HUAWEI

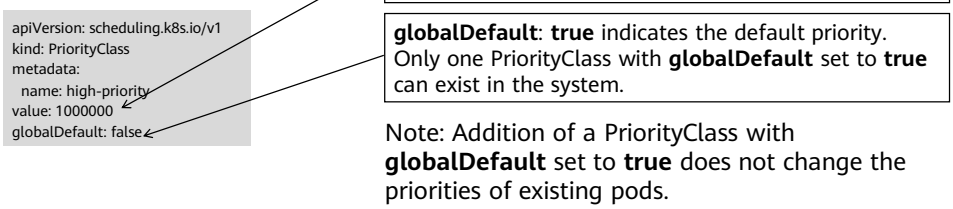
Contents

1. Kubernetes Scheduling Overview
2. Default Scheduler
3. Assigning Pods to Nodes
4. Affinity and Anti-Affinity
5. Taints and Tolerations
6. Eviction
7. **Pod Priority and Preemption**

HUAWEI

Pod Priority and Preemption

- Priority and preemption solve pod scheduling failures. In normal cases, a pod is temporarily pending and then rescheduled when it is updated or the cluster status changes.
- However, if a high-priority pod fails to be scheduled, it will not be pending. Instead, the scheduler tries to preempt low-priority pods to make scheduling of the pending pod possible.



Note: Addition of a PriorityClass with **globalDefault** set to **true** does not change the priorities of existing pods.

Pod Priority and Preemption

- Priority
  - The priority also affects the scheduling sequence. The scheduler sorts pending pods by priority. A pod with a higher priority may be scheduled earlier than that with a lower priority.
- Preemption
  - A preempted pod is a victim pod that gracefully exits.
- Question: If other nodes become schedulable within the graceful exit period of victims, will the preemptor pod continue waiting for graceful exit or be scheduled to a new schedulable node?

```
spec:
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
priorityClassName: high-priority
```


HUAWEI

Quiz

1. (Multiple-answer question) Which of the following can be used to prevent pods from being assigned to inappropriate nodes?
   A. Taints
   B. Tolerations
   C. Affinity
   D. Anti-affinity

HUAWEI

- AD

Summary

- This course describes scheduling in a Kubernetes cluster, including how to use taints and tolerations to assign pods to nodes and how to perform preemption and eviction during scheduling.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling
GT	Grater than
LT	Little than

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

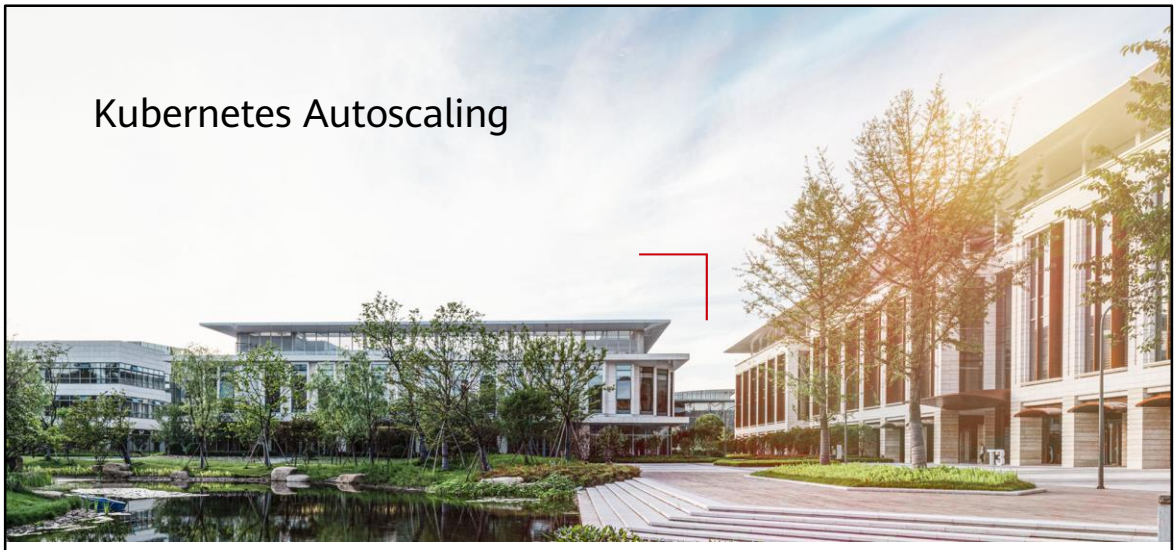
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Kubernetes Autoscaling



Foreword

- This course describes how to use a HorizontalPodAutoscaler (HPA) to perform autoscaling. Autoscaling refers to periodically adjusting the desired scale of its target in Kubernetes to match observed metrics such as average CPU utilization, average memory utilization, or any other custom metric you specify.

HUAWEI

Objectives

- On completion of this course, you will understand:
  - How HPA works.
  - How to use HPA.

HUAWEI

Contents

1. **HPA Working Principles**
2. HPA on CPU Utilization
3. HPA on Memory Utilization
4. HPA on Custom Metrics

HUAWEI

Kubernetes Autoscaling

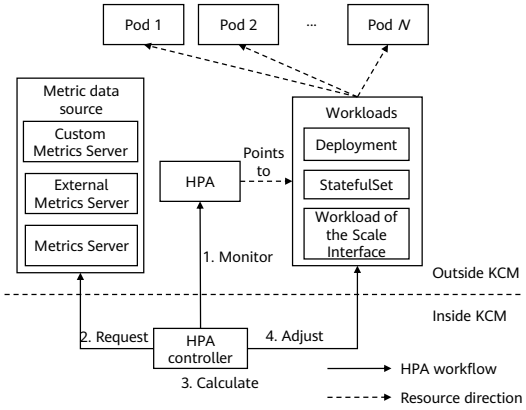
- In Kubernetes, an HPA automatically updates a workload resource (Deployment or StatefulSet), with the aim of automatically scaling the workload to match demand.
- Horizontal scaling means that the response to increased load is to deploy more pods. This is different from vertical scaling, which for Kubernetes would mean assigning more resources (for example, memory or CPUs) to the pods that are already running for the workload.
- If the load decreases, and the number of pods is above the configured minimum, the HPA instructs the workload resource to scale back down.
- The resource determines the behavior of the controller. The HPA controller, running within the Kubernetes control plane, periodically adjusts the desired scale of its target to match observed metrics such as average CPU utilization, average memory utilization, or any other custom metric you specify.

HUAWEI

- Horizontal pod autoscaling does not apply to objects that cannot be scaled (for example, a DaemonSet).
- The HPA is implemented as a Kubernetes API resource and a controller.

How HPA Works

- The HPA controller performs autoscaling in four steps:
  - Monitors HPA resources. Once HPA resources are generated or HPA configurations are modified, the HPA controller can detect and adjust HPA resources in a timely manner.
  - Obtains the metric data from the Metrics API.
  - Calculates the expected number of instances for each metric. The maximum value is used as the expected number of instances for the current workload.
  - Adjusts the workload.
- Steps 2 to 4 are performed every 15 seconds or based on the custom value in **--horizontal-pod-autoscaler-sync-period** of kube-controller-manager.



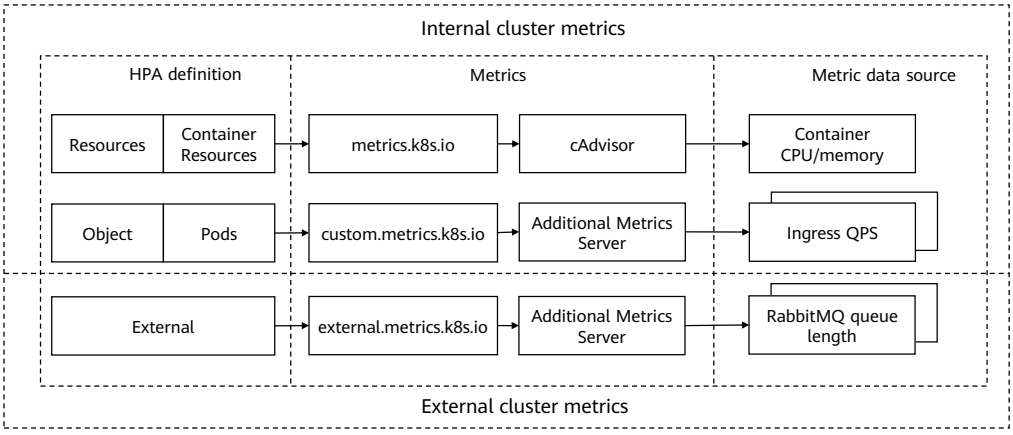
- Metrics Servers can be further classified into:
  - Kubernetes Metrics Server that provides container-level CPU/memory utilization.
  - Custom Metrics Server that provides metrics of custom resources in a Kubernetes cluster.
  - External Metrics Server that provides metrics from outside a Kubernetes cluster.

HPA Metrics

- Resource: provides pod-level CPU/memory utilization.
  - Uses metrics-server to fetch metrics and invokes metrics.k8s.io.
- ContainerResource: provides container-level CPU/memory utilization.
  - Uses metrics-server to fetch metrics and invokes metrics.k8s.io.
- Object: fetches metrics related to any resources in a Kubernetes cluster.
  - Uses prometheus-adapter to fetch metrics and invokes custom.metrics.k8s.io.
- Pods: fetches metrics related to any pods in a Kubernetes cluster.
  - Uses prometheus-adapter to fetch metrics and invokes custom.metrics.k8s.io.
- External: fetches metrics from outside a Kubernetes cluster.

HUAWEI

HPA Data Source

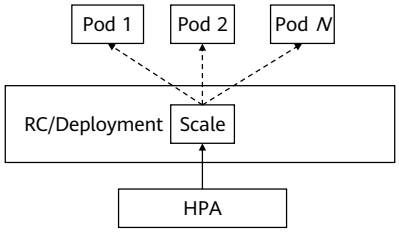


HUAWEI

- Resource provides pod-level CPU/memory utilization.
- ContainerResource provides container-level CPU/memory utilization.
- Object fetches metrics related to any resources in a Kubernetes cluster.
- Pods fetches metrics related to any pods in a Kubernetes cluster.
- External fetches metrics from outside a Kubernetes cluster.

How HPA Works

- During each period, the controller manager queries resource utilization against the metrics specified in each HPA definition. The controller manager finds the target resource defined by **scaleTargetRef**, selects the pods based on the **.spec.selector** labels of the target resource, and obtains the metrics from either the resource metrics API (for per-pod resource metrics) or the custom metrics API (for all other metrics).



HUAWEI

- For per-pod resource metrics, the controller fetches the metrics from the resource metrics API for each pod targeted by the HPA. Then, if a target utilization value is set, the controller calculates the utilization value as a percentage of the equivalent resource request on the containers in each pod. If a target raw value is set, the raw metric values are used directly. The controller then takes the average of the utilization or the raw value (depending on the type of target specified) across all targeted pods, and produces a ratio used to scale the number of desired replicas. If some of containers in a pod do not have the relevant resource request set, CPU utilization for the pod will not be defined and the HPA will not take any action for that metric.
- For per-pod custom metrics, the controller functions similarly to per-pod resource metrics, except that it works with raw values, not utilization values.
- For object metrics and external metrics, a single metric is fetched, which describes the object in question. This metric is compared to the target value, to produce a ratio as above. In the autoscaling/v2 API version, this value can optionally be divided by the number of pods before the comparison is made.

HPA Algorithm

- The HPA controller operates on the ratio between desired metric value and current metric value:
  - desiredReplicas = ceil[currentReplicas x (currentMetricValue/desiredMetricValue)]
  - For example, if the current metric value is **200m**, and the desired value is **100m**, the number of replicas is doubled, since **200.0/100.0 == 2.0**. If the current value is instead **50m**, the number of replicas is halved, since **50.0/100.0 == 0.5**. The control plane skips any scaling action if the ratio is sufficiently close to 1.0 (within a globally-configurable tolerance, **0.1** by default).
- When **targetAverageValue** or **targetAverageUtilization** is specified, **currentMetricValue** is computed by taking the average of the given metric across all pods in the HPA's scale target.

HUAWEI

Configurable Autoscaling

- Stabilization window: restricts the flapping of the replica count when the metrics used for scaling keep fluctuating. The autoscaling algorithm uses this window to infer a previous desired state and avoid unwanted changes to workload scale.
- Scaling policy: Specify one or more scaling policies in **behavior** of an HPA. When multiple policies are specified, the one that allows the maximum change of scale is the default policy.
- Multiple policies for selection:
  - Add four pods every 15 seconds.
  - Add 100% of the currently running replicas at most every 15 seconds.
  - If **selectPolicy** is set to **Max**, the policy that scales the target up to the maximum allowed pods is executed until the HPA reaches its steady state.

```
behavior:
  scaleDown:
    stabilizationWindowSeconds: 300
    policies:
    - type: Percent
      value: 100
      periodSeconds: 15
  scaleUp:
    stabilizationWindowSeconds: 0
    policies:
    - type: Percent
      value: 100
      periodSeconds: 15
    - type: Pods
      value: 4
      periodSeconds: 15
    selectPolicy: Max
```


HUAWEI

- For scaling down, the default stabilization window is 300 seconds (or the value of **stabilizationWindowSeconds**). There is only a single policy for scaling down, which allows a 100% of the currently running replicas to be removed, meaning the scaling target can be scaled down to the minimum allowed replicas.
- For scaling up, there is no stabilization window. When the metrics indicate that the target should be scaled up, the target is scaled up immediately.

HPA Precautions

- The scaling behavior always lags behind because an HPA can only passively respond to metrics.
- An HPA calculates utilization based on requests. It is normal that the actual utilization divided by the expected utilization exceeds 100%. To configure a high expected utilization, you need to properly plan cluster resources and assess risks.
- HPA tolerance helps alleviate system flapping caused by metric fluctuation, but there is always a scaling dead zone that needs to be avoided by O&M personnel.
- An HPA allows the extension of various types of metrics using a Metrics Server. For example, EDAS provides microservice RT and QPS metrics.
- An HPA has a default scaling behavior, even if parameters are not configured. The stabilization window of the scaling behavior is **0** by default. For applications that are often scaled due to noise data, a short stabilization window can avoid sharp noise.
- An HPA supports scaling on multiple metrics. Do not configure multiple HPAs for an application. Otherwise, application flapping may occur.

HUAWEI

Contents

1. HPA Working Principles
2. **HPA on CPU Utilization**
3. HPA on Memory Utilization
4. HPA on Custom Metrics

HUAWEI

Creating a Workload

- To create a workload, specify the **requests** field in **resources** of the Deployment.

```
template:
  metadata:
    labels:
      app: nginx
  spec:
    containers:
    - name: nginx-hpa
      image: nginx
      ports:
      - containerPort: 80
      resources:
        requests:
          cpu: 100m
```


HUAWEI

Creating an HPA

- Use kubectl to create an HPA.
  - **kubectl autoscale deployment xxx --cpu-percent=10 --min=1 --max=10**
- The HPA controller increases and decreases the number of replicas (by updating the Deployment) to keep the average CPU utilization of all pods at 10%. The Deployment then updates the ReplicaSet, which then adds or deletes pods based on changes in its **.spec** file.
- Check the status of the HPA.

```
[root@k8smaster1 ~]# kubectl get hpa
NAME       REFERENCE           TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
nginx-hpa  Deployment/nginx-hpa  13%/10%  1         10        2          119s
```


HUAWEI

Viewing the Load

- Query the load: **kubectl get hpa** *xxx* **--watch**

```
[root@k8smaster1 ~]# kubectl get hpa nginx-hpa --watch
NAME        REFERENCE              TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
nginx-hpa   Deployment/nginx-hpa   0%/10%    1         10        1          5h8m
nginx-hpa   Deployment/nginx-hpa   5%/10%    1         10        1          5h9m
nginx-hpa   Deployment/nginx-hpa   28%/10%   1         10        1          5h9m
```

- Query the number of replicas: **kubectl get deployment** *xxx*

```
[root@k8smaster1 ~]# kubectl get deployment nginx-hpa
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
nginx-hpa   3/3     3            3           5h10m
```


HUAWEI

Checking the HPA Status

```
[root@k8smaster1 ~]# kubectl describe hpa nginx-hpa
Name:                                                  nginx-hpa
Namespace:                                             default
Labels:                                                <none>
Annotations:                                           <none>
CreationTimestamp:                                     Mon, 16 Oct 2023 03:00:04 +0000
Reference:                                             Deployment/nginx-hpa
Metrics:                                               ( current / target )
  resource cpu on pods  (as a percentage of request):  13% (13m) / 10%
Min replicas:                                          1
Max replicas:                                          10
Deployment pods:                                       2 current / 3 desired
Conditions:
  Type            Status  Reason              Message
  ----            ------  ------              -------
  AbleToScale     True    SucceededRescale    the HPA controller was able to update the target scale to 3
  ScalingActive   True    ValidMetricFound    the HPA was able to successfully calculate a replica count from cpu resource utilization (percentage of request)
  ScalingLimited  False   DesiredWithinRange  the desired count is within the acceptable range
Events:
  Type    Reason             Age   From                       Message
  ----    ------             ----  ----                       -------
  Normal  SuccessfulRescale  10s   horizontal-pod-autoscaler  New size: 3; reason: cpu resource utilization (percentage of request) above target
```


HUAWEI

- **AbleToScale** indicates whether the HPA can fetch and update scales and whether any backoff-related conditions would prevent scaling.
- **ScalingActive** indicates whether the HPA is enabled (that is, the replica count of the target is not zero) and can calculate desired scales. When the value is **False**, it generally indicates problems with fetching metrics.
- **ScalingLimited** indicates that the desired scale was capped by the maximum or minimum of the HPA (that is, the maximum or minimum value has been reached). This is an indication that you may wish to raise or lower the minimum or maximum replica count constraints on your HPA.

Creating an HPA Declaratively

- Instead of using the **kubectl autoscale** command to create an HPA imperatively, you can use the following manifest to create it declaratively:

```
apiVersion: autoscaling/v2beta2
kind: HorizontalPodAutoscaler
metadata:
  name: hpa-test
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 10
```


HUAWEI

Metrics of Direct Values

- Specify resource metrics in terms of direct values, instead of as percentages of the requested value:
  - Use **target.type** of **AverageValue** instead of **Utilization**.
  - Set **target.averageValue** instead of **target.averageUtilization**.

```
[root@k8smaster1 ~]# kubectl get hpa
NAME       REFERENCE              TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
hpa-test   Deployment/nginx-hpa   15m/10    1         10        2          3m40s
```

```
apiVersion: autoscaling/v2beta2
kind: HorizontalPodAutoscaler
metadata:
  name: hpa-test
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: AverageValue
        averageValue: 10
```


HUAWEI

Contents

1. HPA Working Principles
2. HPA on CPU Utilization
3. **HPA on Memory Utilization**
4. HPA on Custom Metrics

HUAWEI

Creating a Workload

- Create a ConfigMap.
- Create a Deployment.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: increase-mem-config
data:
  increase-mem.sh: |
    #!/bin/bash
    mkdir /tmp/memory
    mount -t tmpfs -o size=40M tmpfs
/tmp/memory
    dd if=/dev/zero of=/tmp/memory/block
    sleep 60
    rm /tmp/memory/block
    umount /tmp/memory
    rmdir /tmp/memory
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hpa-mem-demo
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      volumes:
      - name: increase-mem-script
        configMap:
          name: increase-mem-config
```

```
    containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
        volumeMounts:
        - name: increase-mem-script
          mountPath: /etc/script
        resources:
          requests:
            memory: 50Mi
            cpu: 50m
        securityContext:
          privileged: true
```


HUAWEI

Creating an HPA

- Create an HPA using a YAML file.
- Use a script to increase the memory.

```
apiVersion: autoscaling/v2beta1
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: hpa-mem-demo
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: memory
      targetAverageUtilization: 60
```

```
[root@k8smaster1 ~]# kubectl exec -it hpa-mem-demo-74675cc6c9-7gd96 /bin/bash
kubectl exec [POD] [COMMAND] is DEPRECATED and will be removed in a future version.
Use kubectl exec [POD] -- [COMMAND] instead.
root@hpa-mem-demo-74675cc6c9-7gd96:/# ls /etc/script/
increase-mem.sh
root@hpa-mem-demo-74675cc6c9-7gd96:/# source /etc/script/increase-mem.sh
dd: writing to '/tmp/memory/block': No space left on device
81921+0 records in
81920+0 records out
41943040 bytes (42 MB, 40 MiB) copied, 0.0486124 s, 863 MB/s
```


HUAWEI

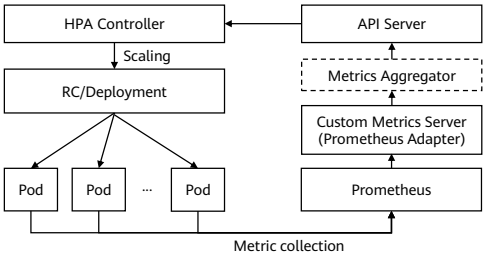
Contents

1. HPA Working Principles
2. HPA on CPU Utilization
3. HPA on Memory Utilization
4. **HPA on Custom Metrics**

HUAWEI

Custom Metrics

- Kubernetes uses the Aggregator API server extension to implement custom metrics.
- Custom Metrics Server is an API service (Prometheus adapter) that queries metrics. Once this service is started, Kubernetes exposes an API called **custom.metrics.k8s.io**. When this URL is requested, Custom Metrics Server queries the metrics in Prometheus and returns the query results in a specified format.



HUAWEI

- The data formats of multiple metrics provided by Prometheus are incompatible with Kubernetes. The k8s-prometheus-adapter component converts data into a format that can be identified by Kubernetes, and Metrics Aggregator then registers the custom metrics API with the API server.

Custom Metrics

- Pod metrics: These metrics describe pods, and are averaged together across pods and compared with a target value to determine the replica count. They work much like resource metrics, except that they only support a **target** type of **AverageValue**.
- Object metrics: These metrics describe a different object in the same namespace, instead of describing pods. The metrics are not necessarily fetched from the object; they only describe it. Object metrics support **target** types of both **Value** and **AverageValue**. With **Value**, the target is compared directly to the returned metric from the API. With **AverageValue**, the value returned from the custom metrics API is divided by the number of pods before being compared to the target.

```
type: Pods
pods:
  metric:
    name: packets-per-second
  target:
    type: AverageValue
    averageValue: 1k
```

```
type: Object
object:
  metric:
    name: requests-per-second
  describedObject:
    apiVersion: networking.k8s.io/v1
    kind: Ingress
    name: main-route
  target:
    type: Value
    value: 2k
```


HUAWEI

More Specific Metrics

- Many metrics pipelines allow you to describe metrics either by name or by a set of additional descriptors called labels. For all non-resource metric types (pod, object, and external), you can specify an additional label selector which is passed to your metric pipeline. For instance, if you collect a metric **http_requests** with the **verb** label, you can specify the following metric block to scale only on GET requests:

```
type: Object
object:
  metric:
    name: http_requests
    selector: {matchLabels: {verb: GET}}
```

- This selector uses the same syntax as the full Kubernetes label selectors. The monitoring pipeline determines how to collapse multiple series into a single value, if the name and selector match multiple series. The selector is additive, and cannot select metrics that describe objects that are not the target object (the target pods in the case of the **Pods** type, and the described object in the case of the **Object** type).

HUAWEI

Metrics Not Related to Kubernetes Objects

- Using external metrics requires knowledge of your monitoring system; the setup is similar to that required when using custom metrics. External metrics allow you to autoscale your cluster based on any metric available in your monitoring system. Provide a **metric** block with a **name** and **selector**, and use the **External** metric type instead of **Object**. If multiple time series are matched by the **metricSelector**, the sum of their values is used by the HPA. External metrics support both the **Value** and **AverageValue** target types, which function the same as when you use the **Object** type.
- For example if your application processes tasks from a hosted queue service, you could add the following section to your HPA manifest to specify that you need one worker per 30 outstanding tasks.

```
- type: External
  external:
    metric:
      name: queue_messages_ready
      selector:
        matchLabels:
          queue: "worker_tasks"
    target:
      type: AverageValue
      averageValue: 30
```


HUAWEI

- Applications running on Kubernetes may need to autoscale based on metrics that do not have an obvious relationship to any object in the Kubernetes cluster, such as metrics describing a hosted service with no direct correlation to Kubernetes namespaces.

Quiz

1. (Single-answer question) Which of the following commands can be used to create an HPA?
   A. kubectl autoscale
   B. kubectl get hpa
   C. kubectl describe hpa
   D. kubectl get deployment

HUAWEI

- A

Summary

- This course describes how to use the HPA for autoscaling in Kubernetes.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling
HPA	HorizontalPodAutoscaler

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Kubernetes Container Upgrade



Foreword

- This course covers Kubernetes container upgrade, including container OS upgrade and kernel live upgrade.

Objectives

- On completion of this course, you will understand:
  - Principles of container OS upgrade
  - Principles of kernel live upgrade

HUAWEI

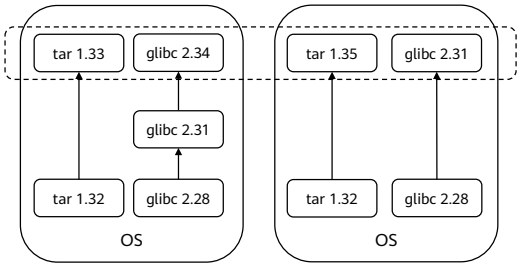
Contents

1. **Container OS Upgrade**
2. Kernel Live Upgrade

HUAWEI

OS Management Challenges in Cloud-Native Environments

- In native cloud environments where Kubernetes manages application containers, OS management often encounters the following challenges:
  - Independent management of Kubernetes and OS:
    - Overlapping functions
    - Difficult coordination between two scheduling systems
  - OS version management difficulties:
    - Version fragmentation
    - Tight coupling between OSs and applications

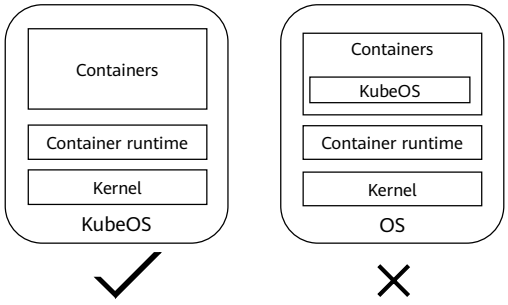


HUAWEI

- In cloud scenarios, the use of containers and Kubernetes is becoming increasingly prevalent. However, the current approach of managing containers and OSs independently often leads to functional redundancy and challenges in coordinating two separate scheduling systems. Furthermore, OS version management can be cumbersome. Identical OS versions, over time, might diverge due to individual installations, updates, and removals of software packages. This results in version fragmentation. Additionally, the OSs can be tightly coupled with specific applications, making major upgrades difficult. To solve the preceding problems, openEuler provides a container OS upgrade tool.

KubeOS

- KubeOS is a lightweight OS specifically built for running containers, containing only the essential components for that purpose. It is not designed to run within a container itself.

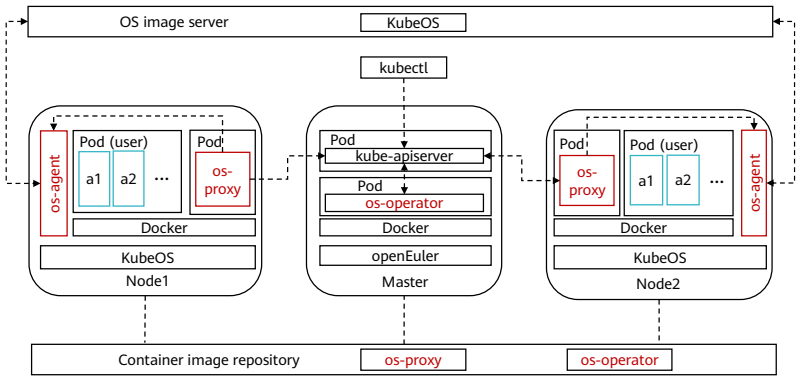


HUAWEI

- KubeOS is a lightweight OS designed specifically for running containerized services. By seamlessly integrating container OSs as components within Kubernetes, KubeOS elevates them to the same management level as services. This allows Kubernetes to orchestrate both containers and their underlying OSs in a unified system.
- Acting as a Kubernetes operator, KubeOS streamlines container OS upgrades. It orchestrates the entire process, ensuring seamless collaboration between OS management and service operations. Prior to upgrading container OSs, services are live-migrated to other nodes within the cluster. This minimizes service disruptions during the upgrade and configuration process. KubeOS employs an atomic upgrade approach for container OSs, maintaining synchronization and guaranteeing OS consistency across the cluster. This eliminates version fragmentation and ensures all nodes operate on the expected OS version.

KubeOS Architecture

- KubeOS primarily consists of three components: os-operator, os-proxy, and os-agent. os-operator and os-proxy run in containers and are deployed within a Kubernetes cluster. os-agent runs directly as a process on each node.

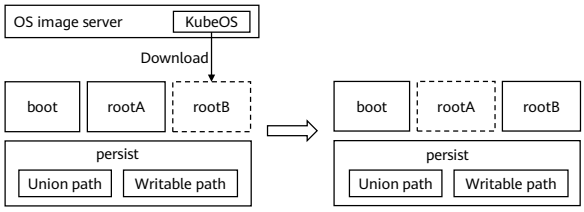


HUAWEI

- os-operator: global container OS manager that continuously monitors the container OS versions of all nodes. It controls the number of nodes upgraded concurrently based on user configurations and marks nodes ready for upgrade.
- os-proxy: node-level OS manager that constantly checks the container OS version of the current node. If the current node is marked for upgrade by os-operator, it locks the node, evicts pods, and forwards the upgrade information to os-agent.
- os-agent: receives information from the os-proxy, downloads the required container OS image for the upgrade from the OS image server, performs the upgrade, and restarts the node.

File Systems in KubeOS

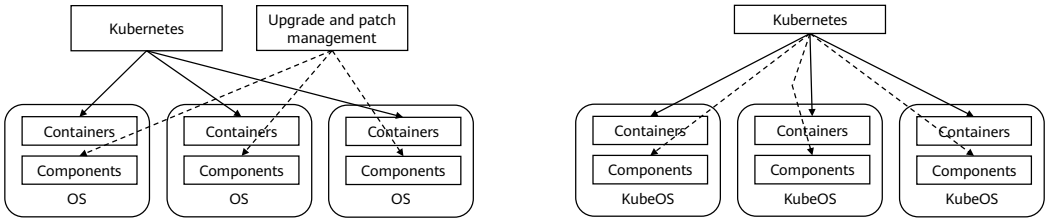
- KubeOS contains four partitions:
  - **boot**: stores GRUB2 files.
  - **persist**: stores persistent user data. Data in this partition is preserved even when KubeOS is upgraded.
    - **Union path**: uses overlay mounting to create an overlay layer on top of the image.
    - **Writable path**: uses bind mounting to directly add a writable layer to the image.
  - Two root partitions: KubeOS uses a dual-partition mode, dividing the **root** partition into **rootA** and **rootB**.



- Assume that the container OS initially runs on the **rootA** partition. During the upgrade, the new OS is downloaded to the **rootB** partition. GRUB will then present two boot options: A and B. KubeOS will set the default boot option to B and then restart the VM. After the restart, the container OS will run on the newly upgraded **rootB** partition.
- The KubeOS root partitions are read-only. Persistent user data is stored in the **persist** partition.

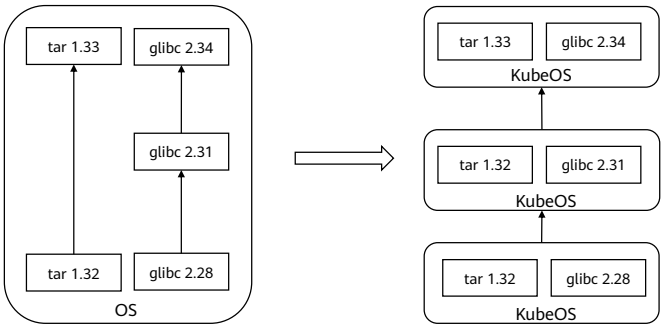
KubeOS Advantages

- Unified management: OSs of all nodes are integrated into the cluster as components, allowing Kubernetes to manage both the OSs and application containers uniformly.
- Collaborative scheduling: KubeOS assesses the cluster status before implementing OS changes, enabling collaborative scheduling of both application containers and OSs.
- API-driven operations: KubeOS leverages the native declarative API of Kubernetes for managing and operating OSs, standardizing operational channels.



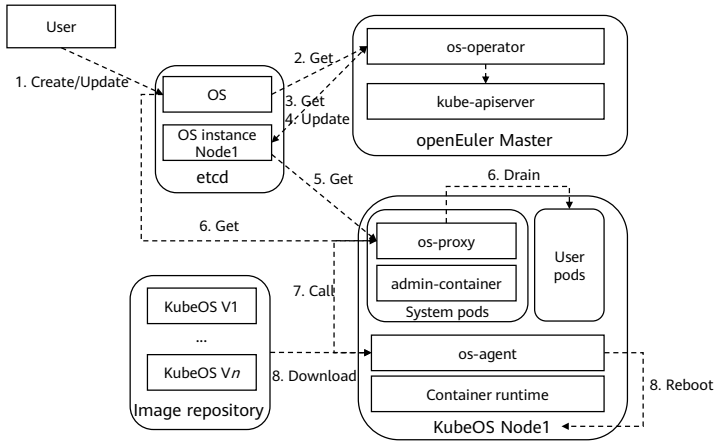
Atomic OS Management

- Atomic management: KubeOS does not provide a traditional package manager. Instead, it upgrades or rolls back entire OS images as atomic units.
- Lightweight and secure: KubeOS is designed to be lightweight, minimizing unnecessary components for a reduced attack surface. This also enables fast and efficient upgrades and replacements.



HUAWEI

Container OS Upgrade Using KubeOS



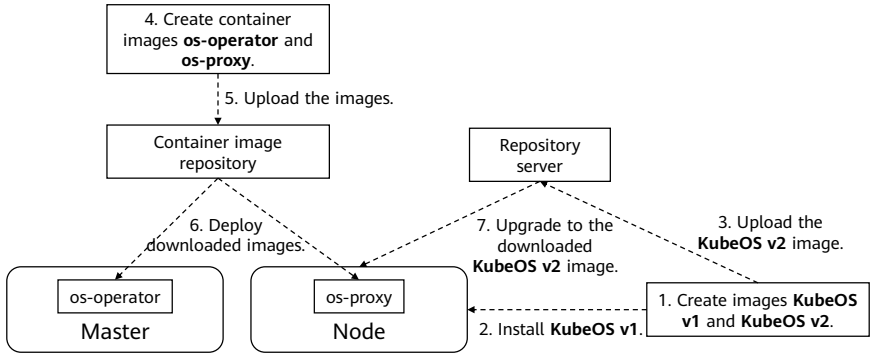
HUAWEI

KubeOS Installation

- Configure Yum repositories.
  - Add **openEuler22.03-LTS-SP2** and **openEuler22.03-LTS-SP2 EPOL**.
- Install KubeOS.
  - yum install KubeOS KubeOS-scripts -y
- KubeOS will be installed in the **/opt/kubeOS** directory, including the binary files for os-operator, os-proxy, and os-agent, container OS image build tool, and related configuration files.

HUAWEI

Container OS Upgrade Operations



HUAWEI

Container Upgrade Precautions

- KubeOS ensures OS version consistency by atomically upgrading all software packages simultaneously. Individual package upgrades are not supported.
- KubeOS supports container OSs with two partitions. Partitions more than two are not supported.
- Upgrade logs for a specific node are available in the **/var/log/messages** file on that node.
- Adhere to the provided upgrade and rollback procedures. Deviating from the specified sequence may result in upgrade or rollback failures.
- Docker image-based upgrades and mutual TLS (mTLS) authentication are supported only on openEuler 22.09 and later versions.
- Upgrade across major versions is not supported.

HUAWEI

Upgrade Guide

- Run the following command to deploy an instance in the cluster, which upgrades nodes according to the configured parameters:
  - kubectl apply -f upgrade_v1alpha1_os.yaml
- Check the OS version of the nodes to verify if the node upgrade is complete:
  - kubectl get nodes -o custom-columns='NAME:.metadata.name,OS:.status.nodeInfo.osImage'

```
apiVersion: upgrade.openeuler.org/v1alpha1
kind: OS
metadata:
  name: os-sample
spec:
  imagetype: disk
  opstype: upgrade
  osversion: KubeOS v2
  maxunavailable: edit.node.upgrade.number
  dockerimage: ""
  imageurl: ""
  checksum: ""
  flagSafe: false
  mtls: true
```


HUAWEI

Rollback Guide

- Scenarios:
  - If a node cannot be started, you can only manually roll back the container OS to the previous version that can start normally.
  - If a node can be started and enter the OS, you can manually or, preferably, use KubeOS to roll back the container OS, similar to the upgrade process.
- Manual rollback: Manually restart the node and select the second boot option to roll back the container OS to the version before the last upgrade.
- KubeOS-based rollback:
  - Rollback to any specific version: Update the image information in the YAML file to reflect the desired version.
  - Rollback to the previous version: In the YAML file, set **osversion** to the previous version and **opstype** to **rollback**.

HUAWEI

Container Image Creation

- The kbimg tool is designed to streamline the creation of images essential for KubeOS to deploy and upgrade container OSs. It enables you to generate images compatible with Docker, VMs, and physical machines.
  - bash kbimg.sh [--help | -h] create [COMMANDS] [OPTIONS]
- Upon container image creation with kbimg, the following files are generated within the **scripts** directory:
  - **system.img**: system image in raw format. It defaults to 20 GiB, with the **root** partition using less than 2,020 MiB and the **persist** partition using less than 16 GiB.
  - **system.qcow2**: system image in QCOW2 format.
  - **update.img**: partition image of the root file system designed for system upgrades.

HUAWEI

- Before creating a container OS image, ensure you have a secure Docker image, either pulled or created.
- Container OS images created for VMs are compatible with x86 and AArch64 architectures only.
- x86 VMs cannot boot the created container OSs in legacy mode.
- Creation of a Container OS image requires a minimum of 25 GB drive space with the default RPM list, potentially more with a custom RPM list.

Contents

1. Container OS Upgrade
2. **Kernel Live Upgrade**

HUAWEI

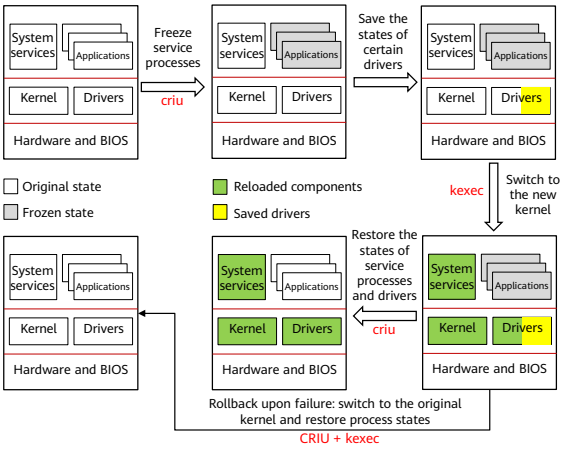
Kernel Live Upgrade

- openEuler's kernel live upgrade feature, implemented through rapid kernel restart and process live migration, is automated by the nvwa user-mode tool.
- The goal of kernel live upgrade is to save and restore the process running state within seconds.
- Kernel live upgrade is typically used in scenarios with the following characteristics:
  - The kernel needs to be restarted due to vulnerability fixes and version updates.
  - Services running on the kernel need to be quickly recovered after the kernel restarted.

HUAWEI

Principles of Kernel Live Upgrade

- kexec:
  - Enables rapid kernel upgrades by quickly booting a new kernel.
- Checkpoint/Restore In Userspace (CRIU):
  - Prevents data loss in critical applications during the kernel upgrade by backing up and restoring their states.
- Hardware state persistence:
  - Ensures uninterrupted operation by preserving the state of hardware and BIOS, allowing mechanisms like direct memory access (DMA) to continue functioning during the kernel swap.



HUAWEI

nvwa Installation

- Mount the ISO file of openEuler.
  - mount openEuler-22.03-LTS-x86_64-dvd.iso /mnt
- Configure the local Yum repository.
  - vi /etc/yum.repos.d/local.repo
- Import the GPG public key for RPM digital signatures.
  - rpm –import /mnt/RPM-GPG-KEY-openEuler
- Install nvwa.
  - yum install nvwa -y
- Check whether the installations are successful. The verification command and expected output are as follows:
  - rpm –qa | grep nvwa
  - nvwa -xxx

```
[local]
name=local
baseurl=file:///mnt
gpgcheck=1
enabled=1
```


HUAWEI

nvwa Deployment (1)

- The configuration files for nvwa are located in **/etc/nvwa**, including **nvwa-restore.yaml** and **nvwa-server.yaml**.
- **nvwa-restore.yaml** controls how nvwa saves and restores the system state during a kernel live upgrade:
  - **pids** specifies the processes to be saved and restored.
  - **services** specifies the services to be saved and restored.
  - **restore_net** indicates whether network configuration should be saved and restored.
  - **enable_quick_kexec** indicates whether to enable the quick kexec feature.
  - **enable_pin_memory** indicates whether to enable the pin memory feature.

```
pids:
  - 14109
services:
  - redis
restore_net: false
enable_quick_kexec: true
enable_pin_memory: true
```


HUAWEI

nvwa Deployment (2)

- **nvwa-server.yaml** stores configuration details required by nvwa.
  - **criu_dir** specifies the directory for storing information generated by nvwa when saving the system state.
  - **criu_exe** specifies the path to the CRIU executable file used by nvwa.
  - **kexec_exe** specifies the path to the kexec executable file, except for debugging kexec itself.
  - **systemd_etc** specifies the directory containing files used to overwrite systemd configurations.
  - **log_dir** specifies the intended directory for storing logs from nvwa (logging is currently not supported).

```
criu_dir: /var/nvwa/running/
criu_exe: /usr/sbin/criu
kexec_exe: /usr/sbin/kexec
systemd_etc: /etc/systemd/system/
log_dir: /etc/nvwa/log/
```


HUAWEI

Enabling nvwa

- nvwa requires configuration files to run. After modifying the configuration files, you need to restart nvwa.
- After successful installation, you can control nvwa using systemd commands.
- Enabling nvwa:
  - systemctl enable nvwa
- Starting nvwa:
  - systemctl start nvwa
- Checking nvwa status and information:
  - systemctl status nvwa
- For more details, see the usage of systemd.

nvwa Commands

- nvwa help
  - Prints help information.
- nvwa update
  - Performs a kernel live upgrade to a specific kernel version. nvwa searches for kernel images and ramfs in the **/boot** directory. The kernel file name should be **vmlinuz-***<kernel version>* and the rootfs file name should be **initramfs-***<kernel version>***.img**. Note that the upgrade process may fail. In case of failure, some dumped processes or services will stop running.
- nvwa init
  - Cleans up the state information generated by nvwa and configuration modifications made to systemd. This command is used before running nvwa or after a failed execution to clean up the environment.

HUAWEI

Constraints

- Services configured to be saved by nvwa must have standard output and error output defined.
- To use the acceleration features, you need to modify the command line and allocate sufficient memory.
- Although technically you only need to disable SELinux between running **nvwa update** and rebooting the system for nvwa to restore the state, you are advised to disable SELinux for the entire process to ensure stability.

```
[Unit]
Description=Redis persistent key-value database
After=network.target
[Service]
ExecStart=/usr/bin/redis-server /etc/redis.conf --supervised systemd
Type=notify
User=redis
Group=redis
RuntimeDirectory=redis
RuntimeDirectoryMode=0755
StandardOutput=file:/root/log1.log
StandardError=file:/root/log2.log
[Install]
WantedBy=multi-user.target
```


HUAWEI

nvwa Acceleration Features

- cpu park (faster kernel restart):
  - cpu park accelerates the kexec process by putting CPUs in a busy waiting state. This allows them to respond faster to interrupts from the active kernel, minimizing state transitions and speeding up the restart.
  - To enable cpu park, add **cpuparkmem=0x200000000** to the kernel command line. You can replace **0x200000000** with the starting address of any free 1 MB memory region.
  - For optimal compatibility, choose an address after the 4 GB mark (0x100000000), as the first 4 GB are often reserved by the system.
- quick kexec (faster kernel boot):
  - quick kexec optimizes the kernel image loading process of kexec, resulting in faster kernel boot.
  - To enable it, configure the relevant options in the configuration file.
- pin memory (faster state restoration):
  - pin memory accelerates the state restoration process for CRIU.
  - To enable it, configure the relevant options in the configuration file.

HUAWEI

Quiz

1. (Multiple-answer question) Which of the following are included in KubeOS?
   A. os-operator
   B. os-proxy
   C. os-agent
   D. nvwa

HUAWEI

- ABC

Summary

- This course covers the principles of KubeOS, for Kubernetes container OS upgrade, and nvwa, for kernel live upgrade.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling
OS	Operating system
BIOS	Basic input/output system

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Hybrid Deployment of Kubernetes Containers



Foreword

- This course covers the hybrid deployment of Kubernetes containers, including the Rubik deployment tool usage and hybrid deployment method.

HUAWEI

Objectives

- On completion of this course, you will understand:
  - Installation of the Rubik hybrid deployment tool
  - Hybrid deployment method

HUAWEI

Contents

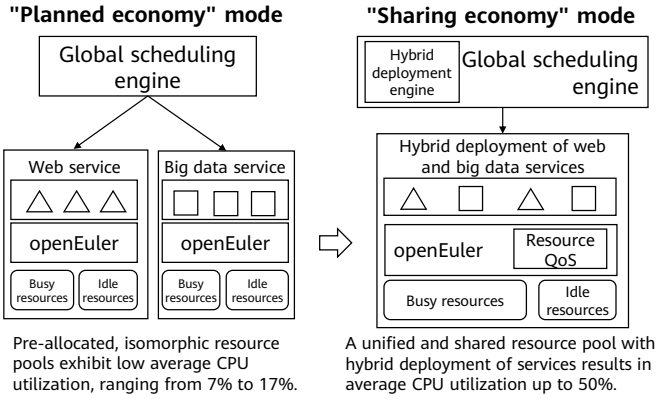
1. **Rubik Installation and Deployment**
2. Hybrid Deployment Example

HUAWEI

Hybrid Deployment of Online and Offline Services

- Low server resource utilization is a longstanding challenge in the industry. The advent of cloud-native technologies has enabled the hybrid deployment of online (high-priority) and offline (low-priority) services, proving to be an effective solution to optimize resource utilization.

- Resource QoS: System resources (CPU, memory bandwidth, network, I/O) are isolated to ensure the service level agreement (SLA) of both online and offline services during hybrid deployment.
- Hybrid deployment engine: The status of co-deployed services is detected in real time with millisecond-level scheduling response within nodes.



Pre-allocated, isomorphic resource pools exhibit low average CPU utilization, ranging from 7% to 17%.

A unified and shared resource pool with hybrid deployment of services results in average CPU utilization up to 50%.

HUAWEI

- As Internet market growth slows down, optimization efforts have shifted towards cost reduction and efficiency improvement. CPU and memory, being major cost contributors, often suffer from low utilization. For instance, CPU utilization typically hovers between a mere 7% and 17%.

Rubik

- Rubik enhances resource utilization in hybrid deployment scenarios by scheduling resources based on quality of service (QoS) levels.
- Running as a container within Kubernetes, Rubik isolates and limits resources for workloads of different priorities, ensuring unaffected online services. This results in improved overall resource utilization without sacrificing online service quality. While Rubik currently schedules CPU and memory resources, you need to write **1** to the **/proc/sys/vm/memcg_qos_enable** interface to enable the memory priority feature.

HUAWEI

Rubik Installation and Deployment

- Install Rubik.
  - sudo yum install –y rubik
- Deploy a Rubik DaemonSet.
  - Build a Rubik image by running **docker build -f /var/lib/rubik/Dockerfile --tag rubik:0.1.0**.
  - On the master node, change the Rubik image name in the **/var/lib/rubik/rubik-daemonset.yaml** file.
  - On the master node, use the **kubectl** command to deploy the Rubik DaemonSet. Rubik will be deployed on all Kubernetes nodes.
  - Run **kubectl get pods -A** to check whether Rubik has been deployed on each node.

HUAWEI

Common Rubik Prioritization Methods

- Automatic pod prioritization
  - If automatic pod prioritization is enabled by setting **autoConfig** to true in the Rubik configuration, you can simply specify the desired priorities for your pods through **annotations** within their deployment YAML files. Rubik will automatically detect pod creation and update, configuring their priorities accordingly.
- kubelet-based pod prioritization
  - Automatic prioritization relies on pod creation events from the API server, introducing a slight delay. This can lead to performance fluctuations as priorities may not be applied before the application starts. To avoid this, you can disable automatic prioritization and modify your kubelet source code. You can then call the Rubik HTTP interface to configure pod priorities after container cgroup creation but before the container process starts.
- Automatic pod priority check
  - Rubik can perform a consistency check on pod QoS prioritization during startup. This ensures alignment between the Kubernetes cluster configuration and Rubik's own pod priority settings.

HUAWEI

- Rubik prioritization can only change pods from online to offline, not the other way around.
- Automatic pod priority check is disabled by default. You can enable it using the **autoCheck** parameter. When enabled, the function will automatically validate and correct pod prioritization during Rubik startup or restart.

Example Hybrid Deployment Configuration

- After deploying Rubik, you can configure your hybrid deployment of online and offline services in the service YAML file, specifying the desired service type (see the example below for deploying an online service **nginx**). Rubik will then prioritize resource allocation based on your setting, ensuring resource isolation.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: qosexample
  annotations:
    volcano.sh/preemptable: "false"
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      limits:
        memory: "200Mi"
        cpu: "1"
      requests:
        memory: "200Mi"
        cpu: "1"
```


HUAWEI

- The value of **volcano.sh/preemptable** determines the online status of a service. **true** indicates an offline service, and **false** (default) indicates an online service.

Contents

1. Rubik Installation and Deployment
2. **Hybrid Deployment Example**

HUAWEI

Hybrid Deployment Example

- Online service (ClickHouse): utilizing the clickhouse-benchmark tool for performance testing, gather performance metrics such as QPS, P50, P90, and P99
- Offline service (stress): using the stress CPU test tool to launch multiple concurrent CPU-intensive tasks through the **--cpu** option
- Example hybrid deployment procedure:
  - Start a ClickHouse container (online service).
  - Enter the container and run the **clickhouse-benchmark** command with the following parameters: 10 concurrent threads, 10,000 query iterations, and a total query duration of 30 seconds.
  - Simultaneously, Start a stress container (offline service) to simulate load by running 10 CPU-intensive tasks.
  - Upon completion of the **clickhouse-benchmark** command, a performance test report will be generated.

HUAWEI

Quiz

1. (Multiple-answer question) Which of the following resources can be preempted in Rubik hybrid deployment?
   A. CPU
   B. Memory
   C. GPU
   D. Storage

HUAWEI

- AB

Summary

- This course covers how to use Rubik, a hybrid deployment tool for Kubernetes containers.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling
QoS	Quality of Service

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Kubernetes Optimization



Foreword

- This course describes how to optimize Kubernetes cluster and etcd performance through tuning of OS kernel and Kubernetes component parameters.

Objectives

- On completion of this course, you will understand:
  - Kernel parameter optimization
  - etcd performance optimization
  - Kubernetes component optimization
  - Pod optimization

HUAWEI

Contents

1. **Kernel Parameter Optimization**
2. etcd Performance Optimization
3. Image Pulling Optimization
4. Kubernetes Component Optimization
5. Pod Optimization

HUAWEI

Kernel Parameters in /etc/sysctl.conf

```
fs.file-max=1000000
# System-wide limit on open file descriptors.
# This is typically adjusted when errors such as "Too many open files" or "Socket/File: Can't open so many files" are reported.
 # ARP cache size
net.ipv4.neigh.default.gc_thresh1=1024
# Minimum entries in the ARP cache. The garbage collector will not run if there are fewer entries than this. The default value is 128.
net.ipv4.neigh.default.gc_thresh2=4096
# Soft limit on the maximum number of entries in the ARP cache.
# The garbage collector will allow the number of entries to exceed this number for 5 seconds before starting collection. The default value is 512.
net.ipv4.neigh.default.gc_thresh3=8192
# Hard limit on the maximum number of entries in the ARP cache.
# The garbage collector will run immediately if the number of entries in the cache exceeds this number. The default value is 1024.
# The above three parameters can be optimized when the kernel-maintained ARP table is too large.
net.netfilter.nf_conntrack_max=10485760
# Maximum number of connection tracking entries.
# This represents the number of "tasks" (connection tracking entries) that netfilter can handle simultaneously in kernel memory.
net.netfilter.nf_conntrack_tcp_timeout_established=300
net.netfilter.nf_conntrack_buckets=655360
# Hash table size (read-only). The value defaults to 65536 on 64-bit systems with 8 GB of RAM, doubles with 16 GB, and so on.
net.core.netdev_max_backlog=10000
# Maximum number of packets allowed to be queued when the packet receiving rate on a network interface exceeds the kernel processing rate
fs.inotify.max_user_instances=524288
# Maximum number of inotify instances that can be created per real user ID. The default value is 128.
fs.inotify.max_user_watches=524288
# Maximum number of watches that can be associated with each inotify instance. The default value is 8192.
```


HUAWEI

Network Parameter Optimization (1)

net.ipv4.tcp_keepalive_time=600 # Duration a TCP connection must remain idle before the first keepalive probe is sent, in seconds
net.ipv4.tcp_keepalive_intvl=30 # Interval between TCP keepalive probes
net.ipv4.tcp_keepalive_probes=10 # Number of TCP keepalive probes to send before declaring a connection dead
net.ipv6.conf.all.disable_ipv6=1 # Disables IPv6. Set to 0 to enable IPv6.
net.ipv6.conf.default.disable_ipv6=1 # Disables IPv6. Set to 0 to enable IPv6.
net.ipv6.conf.lo.disable_ipv6=1 # Disables IPv6. Set to 0 to enable IPv6.
net.ipv4.neigh.default.gc_stale_time=120 # ARP cache entry timeout
net.ipv4.conf.all.rp_filter=0 # Disabled (1) by default. When it is enabled, the system strictly checks the reverse path of packets, which may cause packet loss.
net.ipv4.conf.default.rp_filter=0 # Disables source address validation.
net.ipv4.conf.default.arp_announce=2 # Always uses the best local IP address corresponding to the destination IP address as the source IP address for ARP requests.
net.ipv4.conf.lo.arp_announce=2 # Always uses the best local IP address corresponding to the destination IP address as the source IP address for ARP requests.
net.ipv4.conf.all.arp_announce=2 # Always uses the best local IP address corresponding to the destination IP address as the source IP address for ARP requests.
net.ipv4.ip_local_port_range= 45001 65000 # Defines the minimum and maximum port range available for network connections to use as their source (local) port. This applies to both TCP and UDP connections.
net.ipv4.ip_forward=1 # A value of 0 means IP forwarding is disabled, and a value of 1 means IP forwarding is enabled.
net.ipv4.tcp_max_tw_buckets=6000 # Configures the maximum number of TIME_WAIT sockets allowed on the server.
net.ipv4.tcp_syncookies=1 # Set this to 1 to prevent SYN Flood attacks.
net.ipv4.tcp_synack_retries=2 # Number of retries (5 by default) for the second handshake packet (SYN+ACK) sent to the client IP if the third handshake packet (ACK) is not received
net.bridge.bridge-nf-call-ip6tables=1 # Whether to filter IPv6 packets in the ip6tables chain
net.bridge.bridge-nf-call-iptables=1 # Layer 2 bridges are also filtered by the FORWARD rules of iptables when forwarding packets, which sometimes causes the problem of L3 iptables rules filtering L2 frames.
net.netfilter.nf_conntrack_max=2310720 # Size of the connection tracking table. You are advised to calculate this value based on memory: CONNTRACK_MAX = RAMSIZE (in bytes)/16384/(x/32), and satisfy nf_conntrack_max=4 × nf_conntrack_buckets. The default value is 262144.

HUAWEI

Network Parameter Optimization (2)

```
net.ipv6.neigh.default.gc_thresh1=8192
net.ipv6.neigh.default.gc_thresh2=32768
net.ipv6.neigh.default.gc_thresh3=65536
# gc_thresh3 is the absolute limit on the table size.
# gc_thresh2 is set to a value equal to the maximum expected number of neighbor entries for the system.
# In this case, gc_thresh3 should be set to a value higher than gc_thresh2, for example, 25%-50% higher than gc_thresh2, treating it as surge capacity.
# gc_thresh1 is raised to a larger value. The effect of this setting is that if the table contains fewer entries than gc_thresh1, the kernel will never delete (time out) outdated entries.
net.core.netdev_max_backlog=16384 # Network device backlog queue length per CPU
net.core.rmem_max = 16777216 # Size of the read and write buffer for all protocol types
net.core.wmem_max = 16777216 # Maximum TCP data send window size
net.ipv4.tcp_max_syn_backlog = 8096 # Length of the first backlog queue
net.core.somaxconn = 32768 # Length of the second backlog queue
fs.inotify.max_user_instances=8192 # Upper limit on the number of inotify instances that can be created by each real user ID. The default value is 128.
fs.inotify.max_user_watches=524288 # Number of watches that the same user can add at the same time. The default value is 8192.
fs.file-max=52706963 # Maximum number of open file descriptors
fs.nr_open=52706963 # Maximum number of open file descriptors for each process
kernel.pid_max = 4194303 # Maximum number of processes
net.bridge.bridge-nf-call-arptables=1 # Whether to filter the ARP packets of the bridge in the FORWARD of arptables
vm.swappiness=0 # Allows the use of swap space only when the system is out of memory (OOM).
vm.overcommit_memory=1 # Do not check whether the physical memory is enough.
vm.panic_on_oom=0 # Enables the OOM killer.
vm.max_map_count = 262144
```


HUAWEI

Contents

1. Kernel Parameter Optimization
2. **etcd Performance Optimization**
3. Image Pulling Optimization
4. Kubernetes Component Optimization
5. Pod Optimization

HUAWEI

Key Factors Determining etcd Performance

- Latency: Time it takes to complete an operation
- Throughput: Total number of operations completed within a specific time period. When etcd receives concurrent client requests, average latency typically increases as overall throughput increases.

High-Availability etcd Cluster Setup

- The etcd operator, developed by CoreOS, simplifies the deployment and management of complex stateful applications. As a controller aware of application state, it extends the Kubernetes API to automate the creation, management, and configuration of application instances. The etcd operator offers the following features:
  - Automatically deploys and removes etcd clusters without requiring manual intervention or configuration.
  - Dynamically scales etcd clusters up or down.
  - Supports data backup and recovery for etcd clusters.
  - Enables seamless etcd cluster upgrades without interrupting services.
  - Leverages SSD storage to optimize etcd cluster performance.

HUAWEI

Storage Optimization

- Optimizing drives: etcd is highly sensitive to drive write latency. Therefore, for clusters with heavy loads, it is crucial to use local SSDs or high-performance cloud drives for etcd storage. You can run the **fio** command to measure the actual sequential IOPS of the drives.

```
fio -filename=/dev/sda1 -direct=1 -iodepth 1 -thread -rw=write -ioengine=psync
-bs=4k -size=60G -numjobs=64 -runtime=10 -group_reporting -name=file
```

- Increasing etcd storage limit: By default, etcd has a storage quota of 2 GiB. Once this limit is reached, etcd will stop writing data. You can increase this quota by starting etcd with the **--quota-backend-bytes** option. The maximum supported quota is 8 GiB.

```
| --quota-backend-bytes 8589934592 | header |
| -------------------------------- | ------ |
|                                  |        |
```


HUAWEI

etcd Priority Adjustment

- Process priority: etcd needs to persistently store data in drive log files. Drive activity from other processes can increase write latency, leading to etcd request timeouts and temporary leader loss. To mitigate this, you can assign a higher drive priority to the etcd process, ensuring its stable operation alongside other drive-intensive processes.

```
| ionice -c2 -n0 -p $(pgrep etcd) | header |
| ------------------------------- | ------ |
|                                 |        |
```

- Network priority: If the etcd leader handles a high volume of concurrent client requests, network congestion might delay processing of peer requests from followers. This can cause "send buffer full" errors on follower nodes. You can address these errors by prioritizing peer-to-peer network traffic of etcd. On Linux, you can use the **tc** command to prioritize peer traffic.

```
dropped MsgProp to 247ae21ff9436b2d since streamMsg's sending buffer is full
dropped MsgAppResp to 247ae21ff9436b2d since streamMsg's sending buffer is full
```

```
tc qdisc add dev eth0 root handle 1: prio bands 3
tc filter add dev eth0 parent 1: protocol ip prio 1 u32 match ip sport 2380 0xffff flowid 1:1
tc filter add dev eth0 parent 1: protocol ip prio 1 u32 match ip dport 2380 0xffff flowid 1:1
tc filter add dev eth0 parent 1: protocol ip prio 2 u32 match ip sport 2379 0xffff flowid 1:1
tc filter add dev eth0 parent 1: protocol ip prio 2 u32 match ip dport 2379 0xffff flowid 1:1
```


HUAWEI

Other Optimizations

- Memory: etcd has a default storage size limit of 2 GiB, which can be configured using the **--quota-backend-bytes** flag. You are advised to use a maximum size of 8 GiB in a production environment. If the configured value exceeds this, etcd will issue a warning during startup.
- Request body: etcd is designed for processing small key-value pairs of metadata. While it can handle larger requests, they may increase latency for other requests. By default, the maximum size for any request is 1.5 MiB. This limit can be configured using the **--max-request-bytes** flag on the etcd server.
- Key history compression: etcd stores multiple versions of data. As writes to a key increase, the number of historical versions will also increase. By default, etcd does not automatically clean up historical data. When the data reaches the quota value set by **--quota-backend-bytes**, no more data can be written. Historical data must be compressed and cleaned up before further writes are possible.

```
--auto-compaction-mode
--auto-compaction-retention
```


HUAWEI

etcd Backup

- All Kubernetes objects are stored in etcd. It is critical to regularly back up the etcd cluster data to recover the Kubernetes cluster under disaster scenarios, such as the loss of all master nodes. The snapshot file contains all Kubernetes status and critical information. The snapshot file can be encrypted to ensure the security of sensitive Kubernetes data. Backing up the etcd cluster can be done in two ways: etcd built-in snapshot and volume snapshot.
- Built-in snapshot: etcd supports built-in snapshots, making it easy to back up etcd clusters. Snapshots can be taken from an active member using the **etcdctl snapshot save** command, or by copying the **member/snap/db** file from the etcd data directory, which is not currently in use by the etcd process. Taking a snapshot generally does not impact member performance. The following is an example for taking a snapshot of the keyspace provided by **$ENDPOINT** to the **snapshotdb** file:

```
ETCDCTL_API=3 etcdctl --endpoints $ENDPOINT snapshot save snapshotdb
# exit 0
# verify the snapshot
ETCDCTL_API=3 etcdctl --write-out=table snapshot status snapshotdb
+----------+----------+------------+------------+
|   HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
+----------+----------+------------+------------+
| fe01cf57 |       10 |          7 | 2.1 MB     |
+----------+----------+------------+------------+
```

- Volume snapshot: If etcd is running on a storage volume that supports backups, such as Amazon Elastic Block Storage, you can back up the etcd data by taking a snapshot of the storage volume.

HUAWEI

etcd Recovery

- etcd supports restoring from snapshots taken by etcd processes of different patch versions, even if the major and minor versions differ. A restore operation is used to recover the data of a failed cluster. You must have a snapshot file before starting a restore operation. It can be a snapshot file from a previous backup operation or a snapshot file from the remaining data directory. If the restored cluster has a different access URL from the previous cluster, you must reconfigure the Kubernetes API server accordingly.
- Restart the Kubernetes API server using the **--etcd-servers=***$NEW_ETCD_CLUSTER* option instead of **--etcd-servers=***$OLD_ETCD_CLUSTER*. Replace *$NEW_ETCD_CLUSTER* and *$OLD_ETCD_CLUSTER* with the respective IP addresses. If you use load balancing in front of the etcd cluster, you might need to update the load balancer. An etcd cluster is considered failed if the majority of etcd members fail permanently. In this case, Kubernetes cannot make any changes to its current state. Although already scheduled pods might continue to run, new pods cannot be scheduled. In this case, restore the etcd cluster and potentially reconfigure the Kubernetes API server.

HUAWEI

Contents

1. Kernel Parameter Optimization
2. etcd Performance Optimization
3. **Image Pulling Optimization**
4. Kubernetes Component Optimization
5. Pod Optimization

HUAWEI

Docker Optimization

- Configure the Docker daemon for parallel image downloads to enhance image pulling efficiency. Add the following configuration to the **/etc/docker/daemon.json** file:
  - "max-concurrent-downloads":10
- Use a local SSD or a high-performance cloud drive as the persistent data directory for Docker containers. Add the following configuration to **/etc/docker/daemon.json**:
  - "data-root":"/ssd_mount_dir"
- The pause image needs to be pulled during each pod startup. Pre-load the pause image to minimize network bandwidth usage by running the following command on each node:
  - docker load –i /tmp/preloaded_pause_image.tar

HUAWEI

kubelet Optimization

- Increase concurrency by setting **--serialize-image-pulls=false**. This option determines whether to pull images serially. The default value is true. Setting it to false can increase concurrency. However, this option cannot be changed if the Docker daemon version is less than 1.9 and the advanced multi-layered unification filesystem (aufs) is used.
  - --serialize-image-pulls=false
- Set the image pulling timeout using **--image-pull-progress-deadline=30**. The default value is 1 (minute). For pulling large images, it is necessary to increase the timeout appropriately.
  - --image-pull-progress-deadline=30
- **--max-pods=110** defines the maximum number of Pods allowed to run on a single kubelet node. The default value is 110. Change it as required.
  - --max-pods=110

HUAWEI

Contents

1. Kernel Parameter Optimization
2. etcd Performance Optimization
3. Image Pulling Optimization
4. **Kubernetes Component Optimization**
5. Pod Optimization

HUAWEI

ApiServer Parameters

- **--max-mutating-requests-inflight**: Maximum number of mutating requests per unit of time. The default value is 200.
- **--max-requests-inflight**: Maximum number of non-mutating (read) requests per unit of time. The default value is 400.
- **--watch-cache-sizes**: Watch cache for various resources. The default value is 100. Increase the value when there are a large number of resources.

HUAWEI

High Availability

- Set **--apiserver-count** and **--endpoint-reconciler-type** to allow multiple kube-apiserver instances to join the endpoints of the Kubernetes service, achieving high availability.

HUAWEI

Node Quantity Optimization

- Set **--max-requests-inflight** and **--max-mutating-requests-inflight**, which default to 200 and 400, respectively.
- For node quantities between 1,000 and 3,000, recommended settings are:
  - --max-requests-inflight=1500
  - --max-mutating-requests-inflight=500
- For node quantities greater than or equal to 3,000, recommended settings are:
  - --max-requests-inflight=3000
  - --max-mutating-requests-inflight=1000

HUAWEI

kube-apiserver Memory Configuration

- Set **--target-ram-mb** to a reasonable value to configure the memory of kube-apiserver:
  - --target-ram-mb=*<number of nodes>**60

HUAWEI

kube-controller-manager Parameters

--node-cidr-mask-size # Number of mask bits for pod CIDR on a node. The default value is 24, which allows up to 253 usable addresses. Adjust the value based on address space and number of pods.
--node-monitor-period # Interval for checking the health status of the current node. The default value is 5 seconds.
--node-monitor-grace-period # If the current node remains unresponsive for longer than this period, it is considered unhealthy and enters the ConditionUnknown state. The default value is 40 seconds.
--pod-eviction-timeout # When the node enters the notReady state, pods on the node will start to be evicted after this specified time. The default value is 5 minutes.
--large-cluster-size-threshold # Threshold for determining whether the cluster is a large cluster. The default value is 50, meaning that a cluster with 50 or more nodes is considered a large cluster.
--unhealthy-zone-threshold # Percentage of nodes that must be faulty before the cluster is considered unhealthy. The default value is 55%.
--node-eviction-rate # Rate of node evictions. The default value is 0.1 per second, limiting evictions to a maximum of one pod per node every 10 seconds. This throttling avoids additional pressure on the master due to simultaneous evictions if the master experiences issues.
--secondary-node-eviction-rate # The default value is 0.001. If 55% of the nodes in a large cluster are unhealthy, the master is considered failed, and the eviction rate will be reduced from 0.1 to 0.001. If 55% of the nodes in a small cluster are unhealthy, the eviction will stop.

HUAWEI

kube-scheduler Parameters

- kube-scheduler has relatively few configuration options because the scheduling rules are already well-defined. However, you can customize filtering and scoring policies.
  - **--kube-api-qps**: Maximum queries per second (QPS) for requests to the API server. The default value is 50.
  - **--policy-config-file**: Optional JSON file to specify custom preemption and prioritization policies.

HUAWEI

QPS Limit for Communication with kube-apiserver

- Increase the value of **--kube-api-qps**: You can increase the value to 100 from the default 20.
- Increase the value of **--kube-api-burst**: You can increase the value to 150 from the default 30.
- Disable unnecessary controllers: Kubernetes v1.14 already has 35 controllers. The default startup parameter is **--controllers=***, which starts all controllers. You can disable unnecessary controllers.
- Adjust the resource synchronization period of controllers: This avoids excessive resource synchronization leading to cluster resource consumption. All parameters with the **--concurrent** prefix limit the QPS of communication with kube-apiserver.
- Add the following parameters:
  - --kube-api-qps=100
  - --kube-api-burst=150

HUAWEI

kube-proxy Optimization

- Switching to the IPVS proxy mode: In large clusters, iptables matching and rule updates can suffer from exponential latency increases, making rule additions and deletions time-consuming. IPVS, using a hash table, avoids this issue, as rule changes are not significantly impacted by the rule base size.
- Deploying kube-proxy independently: By default, kube-proxy is co-located with kubelet. You can deploy the kube-proxy component independently on a non-Kubernetes node to avoid generating a large number of iptables rules on all nodes.

HUAWEI

Contents

1. Kernel Parameter Optimization
2. etcd Performance Optimization
3. Image Pulling Optimization
4. Kubernetes Component Optimization
5. **Pod Optimization**

HUAWEI

Resource Requests and Limits of Containers

- Set resource requests and limits for containers, especially for essential add-on services.

```
spec.containers[].resources.limits.cpu
spec.containers[].resources.limits.memory
spec.containers[].resources.requests.cpu
spec.containers[].resources.requests.memory
spec.containers[].resources.limits.ephemeral-storage
spec.containers[].resources.requests.ephemeral-storage
```

- Kubernetes classifies pods into different QoS classes based on the resource requests and limits specified for their containers.

```
- Guaranteed
- Burstable
- BestEffort
```


HUAWEI

Protection Mechanisms

- Apply **nodeAffinity**, **podAffinity**, and **podAntiAffinity** to critical applications to ensure they are scheduled and distributed across different nodes. For example, set kube-dns configuration as follows:

```
affinity:
 podAntiAffinity:
   requiredDuringSchedulingIgnoredDuringExecution:
   - weight: 100
     labelSelector:
       matchExpressions:
       - key: k8s-app
         operator: In
         values:
         - kube-dns
     topologyKey: kubernetes.io/hostname
```


HUAWEI

Quiz

1. (Multiple-answer question) What are the key factors determining etcd performance?
   A. Throughput
   B. Latency

HUAWEI

- AB

Summary

- This course details methods to optimize kernel settings, etcd configuration, Kubernetes components, and pod parameters for improved Kubernetes performance.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling
SSD	Solid state drive

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Introduction to the CI/CD Tool Chain and GitLab



Foreword

- CI/CD, standing for continuous integration and continuous delivery/deployment, is associated with DevOps and has played an important role in the IT field. This course will briefly introduce the CI/CD tool chain and basic usage of Git and GitLab.

Objectives

- Upon completion of this course, you will:
  - Understand the CI/CD tool chain
  - Understand how Git works
  - Master the basic usage of Git and GitLab

HUAWEI

Contents

1. **Introduction to CI/CD**
2. Basic Usage of Git
3. Installation, Deployment, and Basic Usage of GitLab

HUAWEI

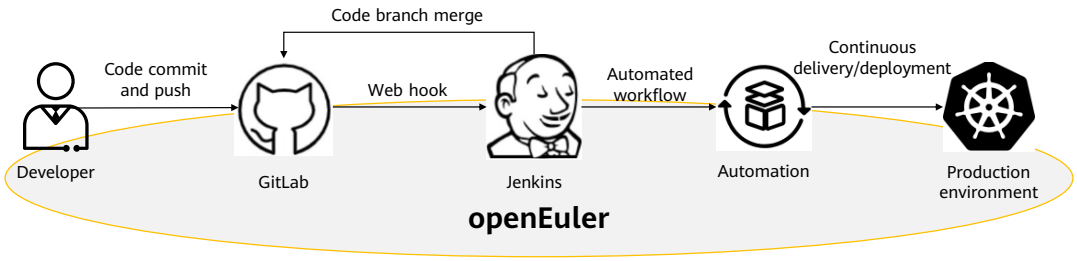
CI/CD and DevOps

- Software is initially developed by two teams: development team and O&M team.
- DevOps is a methodology that integrates software development and O&M to address the agile development requirements of the Internet era.
- A complete software development process includes PLAN, CODE, BUILD, TEST, DELIVERY/DEPLOY, OPERATE, MONITOR, and INTEGRATE.
- "I" and "D" in CI/CD correspond to INTEGRATE and DELIVERY/DEPLOY in the software process. CI/CD involves the following three processes:
  - Continuous integration: Software code can be continuously integrated into the mainline and automatically built and tested.
  - Continuous delivery: Code that has been continuously integrated can be manually deployed.
  - Continuous deployment: Code that is continuously delivered can be automatically deployed anytime and anywhere.

HUAWEI

- The development team is responsible for software design, construction, and version iteration. The O&M team is responsible for software test and deployment.
- The O&M team reports bugs to be fixed and tasks to be reworked to the development team. The development team can update the software version after receiving feedback from the O&M team. However, this process leads to prolonged software development cycles, which doesn't align well with the demands of the current Internet era. That's precisely why the DevOps approach has emerged.

CI/CD Tool Chain



HUAWEI

- Image source: icon library

Mainstream Tools in Each DevOps Phase

- To ensure the efficiency of software development, DevOps provides tools in each phase. This course will focus on tools in the following phases:
  - CODE: Git and GitHub
  - INTEGRATE: Jenkins
  - DELIVERY/DEPLOY: AWX and Kubernetes

HUAWEI

Tools in the CODE Phase

- In this phase, the development team needs to store code of different versions in a repository.
- GitLab is a popular Git-based code hosting platform that has housed many source code repositories. It provides a web-based interface that allows project code to be centrally stored in a cloud repository, facilitating sharing, collaboration, and discussions among developers. GitLab provides many functions related to code hosting, including problem tracking, pull requests (PRs), and team collaboration.
- Git is a distributed version control system. It can record historical file changes, create branches, and merge branches. Developers can use Git to effectively manage code versions within a project, ensuring smooth code synchronization and merging when multiple developers work on the same codebase.

HUAWEI

- Git is an open-source version control system developed by Linus Torvalds to manage Linux kernel.

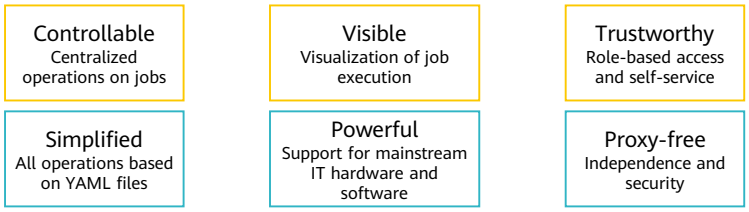
Tools in the INTEGRATE Phase

- Thera are several technologies for continuous integration and deployment. Jenkins is a well-liked open-source tool for continuous integration.
- Jenkins is able to build code and deliver compiled code to the test and production environments.
- As a commonly utilized tool, Jenkins is a key component of many Internet businesses' DevOps implementations, along with GitLab, Docker, and Kubernetes.
- Plugins are the most potent component of Jenkins. Jenkins has several plugin libraries available to automate simple tasks during CI/CD.

HUAWEI

Tools in the DELIVERY/DEPLOY Phase

- AWX, developed based on Ansible, is an open source engine for automatic management, providing a web-based user interface and REST API.



HUAWEI

- The lower part shows the features of Ansible, and the upper part shows the features of AWX.

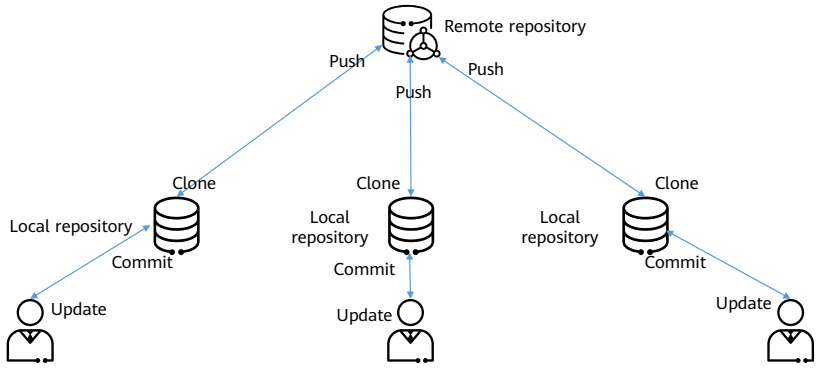
Contents

1. Introduction to CI/CD
2. **Basic Usage of Git**
3. Installation, Deployment, and Basic Usage of GitLab

HUAWEI

Introduction to Git

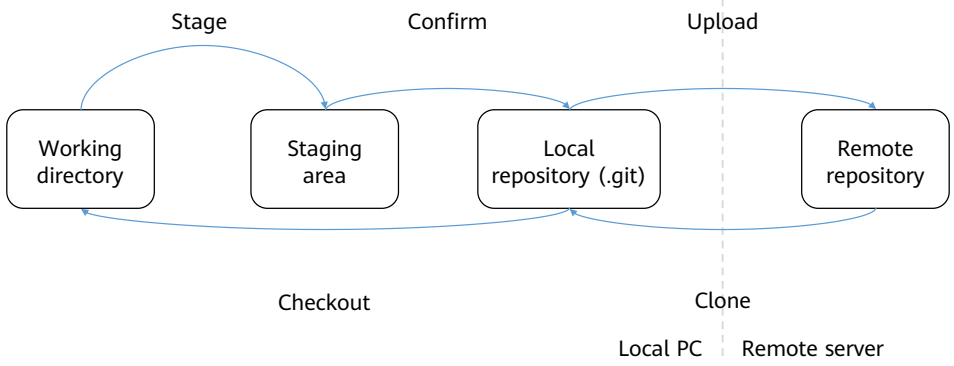
- Git is a distributed version management tool. It decouples code merging and commit so that each file user has a complete local data repository, which can be used offline.



HUAWEI

Basic Concepts of Git

- This slide describes basic concepts of Git, including the remote repository, local repository, working directory, and staging area.

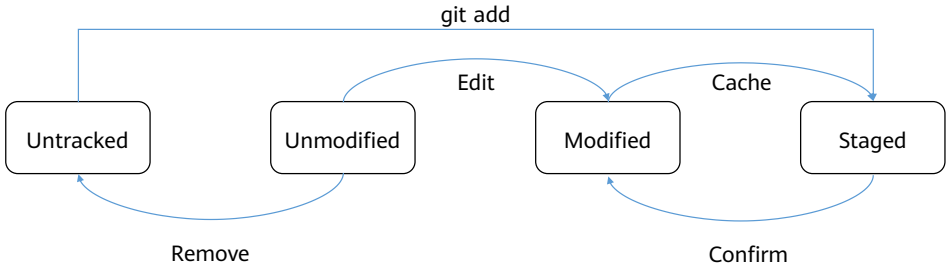


HUAWEI

- The staging area is an important part of Git. It enables users to submit specific files at a certain time point by allowing them to place the files to be submitted in this area.

Git File Status

- A Git file can be in any of the following status:
  - **Untracked**: Files in this status exist in a directory but are not added to a Git repository. They are not involved in version control.
  - **Unmodified**: Files in this status exist in a Git repository, and their contents are consistent with file snapshots in the repository.
  - **Modified**: Files in a directory are modified.
  - **Staged**: Files are staged after the **git add** command is executed.



Basic Usage of Git – Creating a Git Repository

- Run the **git init** command in a directory to generate a .git directory.
- The .git directory stores information about the created Git repository, which is used to track, manage, and store the version control history of a project.

```
[root@controller ~]# mkdir mygit
[root@controller ~]# cd yGit/
[root@controller mygit]# ls -a
.  ..
[root@controller mygit]# git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /root/mygit/.git/
[root@controller mygit]# ls -a
.  ..  .git
```


HUAWEI

Basic Usage of Git – Configuring Git User Information

- To enable Git to trace changes and map each change to a user during version management, the user information needs to be configured.

```
[root@controller mygit]# git config --global user.name "openeuler hcie"
[root@controller mygit]# git config --global user.email openeuler@hcie.com
[root@controller mygit]# git config --list
user.name=openeuler hcie
user.email=openeuler@hcie.com
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
```


HUAWEI

Basic Usage of Git – git status

- The **git status** command is an important Git command, which is used to check version status.
- In the left figure, **master** is the default version of a file. Sometimes, **main** is used to indicate the version.
- The right figure shows the status after a file is created. Content in **Untracked files** indicates that a file is in the **Untracked** status and the file name is **README.md**.

```
[root@controller mygit]# git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

```
[[root@controller mygit]# vim README.md
[root@controller mygit]# git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
```


HUAWEI

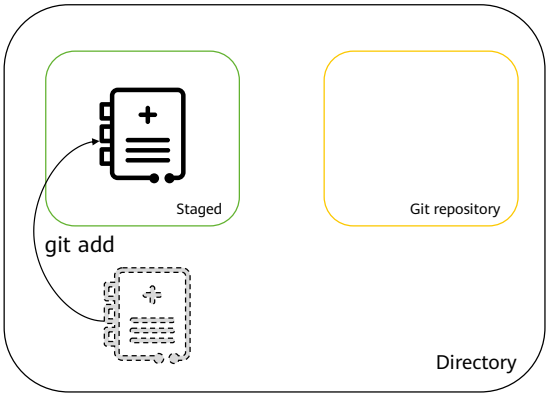
Basic Usage of Git – git add

- The **git add** command is used to add untracked files to the staging area before uploading the files to the repository.

```
[root@controller mygit]# git add README.md
[root@controller mygit]# git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
```



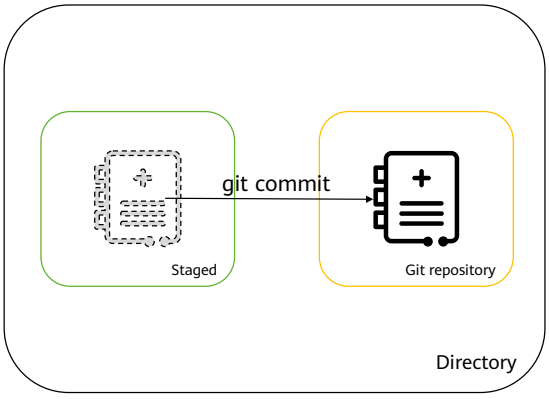
HUAWEI

- In the command output of **git status**, it can be seen that **git rm --cached <file>** is used to remove files from the staging area.

Basic Usage of Git – git commit

- The **git commit** command is used to submit files in the staging area to a Git repository.
- **git commit** can submit a single file, several files at once, or all files in the staging area in the current directory to a repository.
- **-m** indicates information to be submitted.

```
[root@controller mygit]# git commit -m "first version"
[master (root-commit) 2a8fbb8] first version
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
[root@controller mygit]# git status
On branch master
nothing to commit, working tree clean
```



HUAWEI

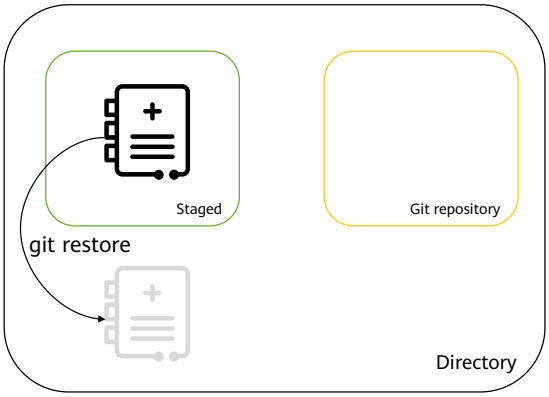
- "-m"

Basic Usage of Git – git restore

- The **git restore** command is used to cancel all modifications after files in the working directory are modified.

```
[root@controller mygit]# cat  README.md
this is a test repository for HCIE-openEuler
[root@controller mygit]# echo "this is a test" >> README.md
[root@controller mygit]# git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
[root@controller mygit]# git restore README.md
[root@controller mygit]# cat README.md
this is a test repository for HCIE-openEuler
[root@controller mygit]# git status
On branch master
nothing to commit, working tree clean
```



HUAWEI

Basic Usage of Git – git log

- The **git log** command is used to view the submission history of a file. You can add **--patch** to view the details.

```
[root@controller mygit]# git log
commit 2a8fbb8dae9b6f0be52ffc866d52effda26983a2 (HEAD -> master)
Author: openeuler hcie <openeuler@hcie.com>
Date:   Thu Nov 9 01:10:30 2023 +0000

    first version
[root@controller mygit]# git log --patch
commit 2a8fbb8dae9b6f0be52ffc866d52effda26983a2 (HEAD -> master)
Author: openeuler hcie <openeuler@hcie.com>
Date:   Thu Nov 9 01:10:30 2023 +0000

    first version

diff --git a/README.md b/README.md
new file mode 100644
index 0000000..b3bfcf7
--- /dev/null
+++ b/README.md
@@ -0,0 +1 @@
+this is a test repository for HCIE-openEuler
```


HUAWEI

Basic Usage of Git – Submitting a Directory

- Git can only manage and track versions of files. To track an empty directory or folder, add **.gitkeep** to the directory to be tracked.

```
[root@controller mygit]# git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

[root@controller mygit]# touch test/.gitkeep
[root@controller mygit]# git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test/
```


HUAWEI

- If you do not want to upload some directories to a repository, you can create a .gitignore file to record the directories or files that do not need to be uploaded.

Basic Usage of Git – Deleting a File

- A file that has been placed to the staging area will not be entirely erased if it is removed just from the directory. After the **git add** command is run again, it can be completely deleted from the staging area.

```
[root@controller mygit]# git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
        new file:   test/.gitkeep

[root@controller mygit]# rm -rf test
[root@controller mygit]# git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
        new file:   test/.gitkeep

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    test/.gitkeep
```

```
[root@controller mygit]# git add .
[root@controller mygit]# git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
```


HUAWEI

Basic Usage of Git – Managing a Branch

- Generally, **master** or **main** is the default branch. When an application is modified and the modification does need to be merged into the master branch, run the **git checkout** command to create a new branch.

```
[root@controller mygit]# git checkout -b test/version1.0
Switched to a new branch 'test/version1.0'
[root@controller mygit]# git status
On branch test/version1.0
nothing to commit, working tree clean
[root@controller mygit]# echo "this is a new branch" >> README.md
[root@controller mygit]# git add .
[root@controller mygit]# git commit -m "new branch"
[test/version1.0 81a0f01] new branch
 1 file changed, 1 insertion(+)
[root@controller mygit]# cat README.md
this is a test repository for HCIE-openEuler
this is a test
this is a new branch
[root@controller mygit]# git checkout master
Switched to branch 'master'
[root@controller mygit]# cat README.md
this is a test repository for HCIE-openEuler
```


HUAWEI

- **git branch -d xxx**: deletes a branch.
- **git merge xxx**: merges a branch to the master branch.

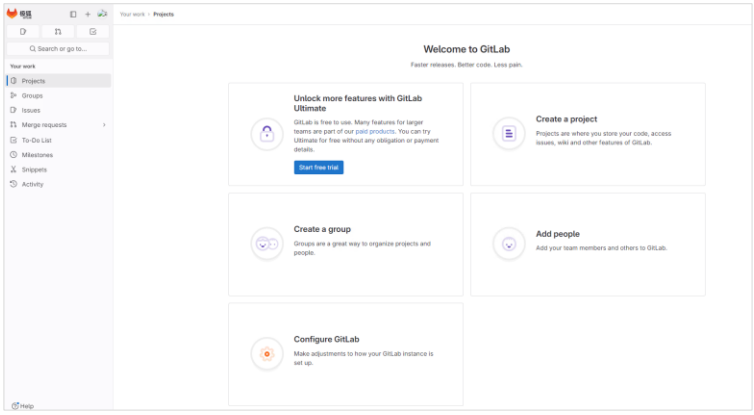
Contents

1. Introduction to CI/CD
2. Basic Usage of Git
3. **Installation, Deployment, and Basic Usage of GitLab**

HUAWEI

Introduction to GitLab

- GitLab is a Git-based remote repository, which is usually used to set up private repositories within enterprises.



HUAWEI

GitLab Installation

- Gitlab supports the following three installation modes:
  - Installation package
  - Cloud native mode
  - Container mode
- In openEuler 20.0. LTS, GitLab 15.5.4 and later versions can be installed using the installation package.
- In cloud native mode, Helm can be used for installation. However, you are not advised to deploy it in the production environment.

HUAWEI

- In the lab guide, GitLab is deployed in container mode.

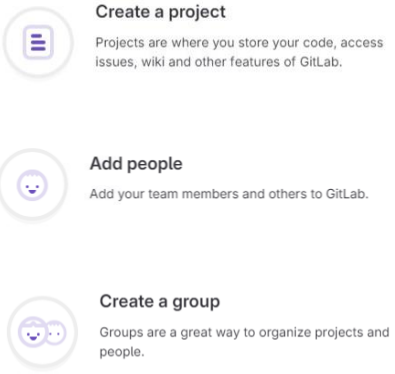
Basic Configurations of GitLab

- The GitLab configuration file **gitlab.rb** is stored in the **/etc/gitlab** directory of a container.
  - **external_url**: external access link, which can be set to the host IP address.
  - **gitlab_rails['gitlab_ssh_host']**: If the code is pulled or pushed through SSH, you need to specify a host by using the host name or IP address.
  - **gitlab_rails[' gitlab_shell_ssh_port ']**: port number used to pull or push code through SSH, which must be the same as that set during container creation.

```
.......
external_url 'http://gitlab.example.com'
......
gitlab_rails['gitlab_ssh_host'] = 'gitlab.example.com'
......
gitlab_rails['gitlab_shell_ssh_port'] = 202
```


HUAWEI

Basic Concepts of GitLab



Project: Most of the work in GitLab is done in a project. Files and code are saved in the project, and implement most functions of the project.

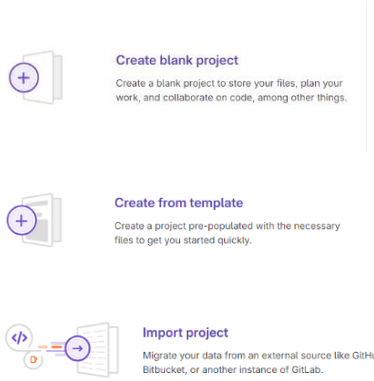
Member: Members are users and groups that have access to your project.
Each member has a role, which determines what they can do in the project.

Group: In GitLab, administrators can use groups to manage one or more projects at the same time.

HUAWEI

Creating a Project

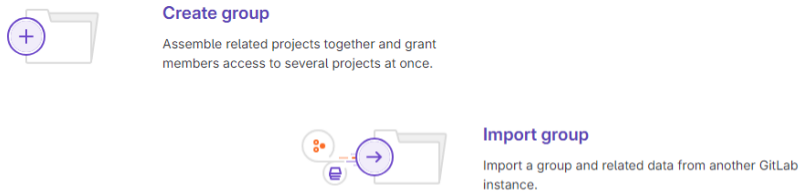
- A project can be created in any of the following ways:
  - Create a blank project.
  - Create a project from a built-in template.
  - Import a project from an external repository.



HUAWEI

Creating a Group

- A group in GitLab can be created in either of the following ways:
  - Create a group and associate it with related projects and users.
  - Import a group and related data from other GitLab instances.



HUAWEI

- Groups are classified into the following types:
  - Private: Only members within a project group can pull code and view the **/public** directory with the public permission.
  - Internal: All authenticated users can pull code and view the **/public** directory with the public permission.
  - Public: Everyone can pull the code.

Creating a User

- A user can be created in either of the following ways:
  - Created by email invitation
  - Directly created by the administrator
- You need to configure the email server for GitLab when inviting users by email.
- The created user cannot be directly used. The user needs to be activated and added to a group to obtain the corresponding permission.

Pushing Code

- There are three scenarios for pushing local code to a newly created project:
  - Create a repository.
  - Push the existing folder.
  - Push the existing Git repository.

```
git clone http://gitlab.example.com/hcie-openeuler/test1.git
cd test1
git switch --create main
touch README.md
git add README.md
git commit -m "add README"
git push --set-upstream origin main
```

```
cd existing_folder
git init --initial-branch=main
git remote add origin http://gitlab.example.com/hcie-openeuler/test1.git
git add .
git commit -m "Initial commit"
git push --set-upstream origin main
```

```
cd existing_repo
git remote rename origin old-origin
git remote add origin http://gitlab.example.com/hcie-openeuler/test1.git
git push --set-upstream origin --all
git push --set-upstream origin --tags
```


HUAWEI

Quiz

1. (Single-answer question) Which of the following commands is used to pull code from the code repository?
   A. git push
   B. git pull
   C. git commit
   D. git clone

HUAWEI

- D

Summary

- This course describes the CI/CD tool chain and mainstream tools in each phase, as well as basic operations of Git and GitLab, laying a foundation for subsequent courses.

HUAWEI

Acronyms and Abbreviations

CI	continuous integration	
CD	continuous delivery/continuous deployment	

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Advanced Automation Management



Foreword

- This course provides advanced information about automation management, which was covered in basic detail in the HCIP-openEuler course.

Objectives

- On completion of this course, you will have knowledge of the following:
  - Facts and magic variables
  - Conditionals and loops of a playbook
  - Usage of the jinja2 template
  - Usage of Ansible roles
  - Ansible privilege escalation configurations

Contents

1. **Ansible Facts and Magic Variables**
2. Ansible Playbook Conditionals
3. Ansible Playbook Loops
4. Introduction to Ansible Roles
5. Ansible Task Privilege Escalation and Execution Sequence
6. Introduction to the Jinja2 Template
7. AWX Description

HUAWEI

Introduction to Ansible Facts

- Ansible facts are data about the controlled end and include the operating system type, IP address, and drive information of the controlled end. You can obtain facts through the setup module of Ansible or by enabling **gather_facts** (enabled by default) in a playbook.
- You can use facts to filter out controlled hosts that meet or do not meet conditions so that tasks can be deployed on specified hosts.
- Facts can be manually set by adding the **fact** file to the **/etc/ansible/facts.d directory** on a controlled host.
- The setup module can filter out qualified hosts. However, it supports only the first level of **ansible_facts**.
- In a playbook, facts are often used for conditionals.

HUAWEI

Obtaining and Filtering Facts – setup

- The setup module can be used to obtain all facts corresponding to the information about controlled hosts. The values of facts can be obtained through **filter**.

```
[root@controller ~]# ansible 172.16.0.12 -m setup
172.16.0.12 | SUCCESS => {
    "ansible_facts": {
        "ansible_all_ipv4_addresses": [
            "172.16.0.12"
        ],
        "ansible_all_ipv6_addresses": [
            "fe80::f816:3eff:fe43:333f"
        ],
        "ansible_apparmor": {
            "status": "disabled"
        },
        "ansible_architecture": "x86_64",
        "ansible_bios_date": "04/01/2014",
        "ansible_bios_version": "rel-1.12.1-0-ga5cab58-20211128_033230-
build10b29b94b135",
        "ansible_cmdline": {
            "BOOT_IMAGE": "/vmlinuz-5.10.0-60.18.0.50.oe2203.x86_64",
            "console": "ttyS0",
            "crashkernel": "256M",
.........
```

```
[root@controller ~]# ansible 172.16.0.12 -m setup -a
'filter=ansible_python_version'    #### View the Python script.
172.16.0.12 | SUCCESS => {
    "ansible_facts": {
        "ansible_python_version": "3.9.9",
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false
}
.........
```


HUAWEI

- Common facts for filtering:
  - **ansible_python_version**: obtains the Python version.
  - **ansible_distribution**: obtains the OS distribution version.
  - **ansible_distribution_major_version**: obtains the major system version.
  - **ansible_distribution_version**: obtains the specific OS version.
  - **ansible_devices**: queries device information.
  - **ansible_lvm**: obtains logical volume information.
  - **ansible_fqdn**: obtains a fully qualified domain name.
  - **ansible_hostname**: obtains the host name.

Obtaining and Filtering Facts – gather_facts

- You can use **gather_facts** to obtain facts in a playbook. When referencing facts, you need to start from the lower level of **ansible_facts** and use periods (.) to split the corresponding variable layer by layer. For example, to obtain the capacity of partition vda2 of the host 172.16.0.12, the key of the fact is **ansible_devices.vda.partitions.vda2.size**.

```
- hosts: 172.16.0.12
  remote_user: root
  gather_facts: true

  tasks:
    - name: facts debug
      debug:
        msg: "{{ ansible_devices.vda.partitions.vda2.size }}"
```

```
[root@controller ~]# ansible-playbook test.yml
........
TASK [Gathering Facts]
*********************************************************************************
ok: [172.16.0.12]
TASK [facts debug]
*********************************************************************************
ok: [172.16.0.12] => {
    "msg": "38.00 GB"
}
```

```
[root@controller ~]# ansible 172.16.0.12 -m setup
172.16.0.12 | SUCCESS => {
    "ansible_facts": {
......
        "ansible_devices": {{
            "vda": {
.......
                    "vda2": {
......
                        "size": "38.00 GB",
                        "start": "4194304",
                        "uuid": "ae2cd454-a1aa-4683-8c9d-711a1d610f40"
                    }
........
```


HUAWEI

- When referencing facts, you do not need to start from **ansible_facts**. Instead, you need to start from its lower level because Ansible automatically registers the content in **ansible_facts** as a variable. Therefore, **ansible_facts** does not need to be defined in the variable.

Customizing Facts

- To customize facts, create a **/etc/ansible/facts.d** directory on the controlled host and create a file suffixed with .**fact** in the initialization file (INI) format. When referencing customized facts, start with **ansible_local**.
- In the following example, the controlled host (172.16.0.12) is specified as **HCIE-openeuler**.

```
[root@clinet-1 ~]# mkdir -p /etc/ansible/facts.d
[root@clinet-1 ~]# cd /etc/ansible/facts.d
[root@clinet-1 facts.d]# cat utility.fact
[utility]
name=HCIE-openeuler
```

```
[root@controller ~]# ansible 172.16.0.12 -m setup -a "filter=ansible_local"
172.16.0.12 | SUCCESS => {
    "ansible_facts": {
        "ansible_local": {
            "utility": {
                "utility": {
                    "name": "HCIE-openeuler"
                }
            }
        },
.....
```

```
- hosts: 172.16.0.12
  remote_user: root
  gather_facts: true

  tasks:
    - name: facts debug
      debug:
        msg: "{{ ansible_local.utility.utility.name }}"
```


HUAWEI

Calling Facts to Define Variables

- You can use the **set_fact** module to call facts to define new variables in a playbook.

```
- hosts: 172.16.0.12
  remote_user: root
  gather_facts: true

  tasks:
    - name: new vars
      set_fact:
        tag: "{{ ansible_local.utility.utility.name }}-{{ ansible_default_ipv4.address }}"
    - name: display vars
      debug:
        msg: "{{ tag }}"
```

```
[root@controller ~]# ansible-playbook test.yml
.....
TASK [display vars] ************************************************************************
ok: [172.16.0.12] => {
    "msg": "HCIE-openeuler-172.16.0.12"
}
.........
```


HUAWEI

Introduction to Magic Variables

- The built-in magic variables of Ansible are used to implement specific functions. Common magic variables are as follows:
  - **hostvars**: obtains variables of a specified controlled host.
  - **inventory_hostname**: obtains the name of the controlled host where tasks are running.
  - **groups**: obtains host group information from the inventory.
  - **play_hosts**: obtains the hosts running in plays.
  - **inventory_dir**: obtains the directory where the host inventory is located.
  - **inventory_file**: obtains the path of inventory files.

HUAWEI

- **hostvars** and **inventory_hostname** are used together in the format of "hostvars[inventory_hostname]".
- The variables obtained by **hostvars** are mainly the variables involved in **play**, including the variables configured in the host inventory, playbook, and configuration file. After **gather_facts** is enabled, facts can also be obtained.

Use Cases of Magic Variables – hostvars

- Use **hostvars** to obtain host variables in the host inventory.

```
- hosts: servera
  remote_user: root
  gather_facts: false

  tasks:
    - name: hostvars
      debug:
        msg: "{{ hostvars }}"
```


HUAWEI

- Note 1: In the listed playbook, **gather_facts** is set to **false**, indicating that facts cannot be obtained by using **hostvars**.
- Note 2: If **hosts** is set to **servera** in a playbook, **hostvars** still obtains variables of all hosts in the host inventory. To specify a host, use **{{hostvars['*host name*']}}**. The host name must be the same as that in the host inventory. and cannot be set to an IP address.

Use Cases of Magic Variables – inventory_hostname

- Use **inventory_hostname** to obtain the host name in the client host inventory.

```
- hosts: client
  remote_user: root
  gather_facts: false

  tasks:
    - name: hostvars
      debug:
        msg: "{{ inventory_hostname }}"
```

```
[root@controller ~]# ansible client -m debug -a "msg='{{inventory_hostname}}'"
172.16.0.12 | SUCCESS => {
    "msg": "172.16.0.12"
}
172.16.0.13 | SUCCESS => {
    "msg": "172.16.0.13"
}
```


HUAWEI

- Note: When this variable is used together with **hostvars**, do not use single quotation marks in the square brackets of **hostvars[inventory_hostname]**.
- There are two methods listed in this slide to meet the requirements.

Use Cases of Magic Variables – groups

- Use **groups** to list information about all hosts in the client host group.

```
[root@controller ~]# ansible client -m debug -a "msg='{{groups}}'"
172.16.0.12 | SUCCESS => {
    "msg": {
        "all": [
            "172.16.0.12",
            "172.16.0.13"
        ],
        "client": [
            "172.16.0.12",
            "172.16.0.13"
        ],
        "ungrouped": []
    }
}
172.16.0.13 | SUCCESS => {
    "msg": {
        "all": [
            "172.16.0.12",
            "172.16.0.13"
        ],
        "client": [
            "172.16.0.12",
            "172.16.0.13"
        ],
        "ungrouped": []
    }
}
```


HUAWEI

Defining an External Data Source as a Variable

- In addition to utilizing facts and magic variables, Ansible can read external data sources and define them as variables by using the lookup plugin. The common methods of this plugin are as follows:
  - file
  - password
  - pipe
  - env
  - url
- Syntax format of the lookup plugin: **lookup('method','object')**.

HUAWEI

- Run the **ansible-doc -t lookup -l** command to view all plugins supported by lookup.

Variable Definition Case of the Lookup Plugin – file

- Use **file** to read the content in the **username** file and set it as a variable.

```
[root@controller yaml]# cat username
zhangsan
lisi
wangwu
zhaoliu
```

```
- hosts: servera
  gather_facts: false

  tasks:
    - debug:
        msg: "username is {{ lookup('file','/root/yaml/username') }}"
```

```
[root@controller yaml]# ansible-playbook file.yml

PLAY [servera] *****************************************************************

TASK [debug] *******************************************************************
ok: [servera] => {
    "msg": "username is zhangsan\nlisi\nwangwu\nzhaoliu"
}

PLAY RECAP *********************************************************************
servera                    : ok=1    changed=0    unreachable=0    failed=0
```


HUAWEI

Variable Definition Case of the Lookup Plugin – password

- The password module of lookup is used to generate random passwords and export the generated passwords to a specified file. The password module supports multiple options. **chars** is used to set the character set used by a password. For example, **digit** indicates that a password consists of digits. **length** indicates the password length. The default value is **20**.
- Requirement: Create a test user, automatically generate a password with eight characters, and save the password in the **/root/yaml/password** file.

```
[root@controller yaml]# cat password.yml
- hosts: servera
  gather_facts: false

  tasks:
    - user:
        name: test
        password: "{{ lookup('password','/root/yaml/password length=8')}}"
[root@controller yaml]# cat password
hXQeKUm,
```


HUAWEI

Variable Definition Case of the Lookup Plugin – env

- The env module of lookup is used to query available environment variables on the controller.
- Example: check the value of the environment variable **HOME** on the controller.

```
[root@controller yaml]# cat env.yml
- hosts: servera
  gather_facts: false

  tasks:
    - debug:
        msg: "the value of $HOME is {{ lookup('env','HOME') }}"
[root@controller yaml]# echo $HOME
/root
```

```
[root@controller yaml]# ansible-playbook file.yml

PLAY [servera] *****************************************************

TASK [debug] *******************************************************
ok: [servera] => {
    "msg": "the value of $HOME is /root"
}

PLAY RECAP *********************************************************
servera                    : ok=1    changed=0    unreachable=0
```


HUAWEI

Contents

1. Ansible Facts and Magic Variables
2. **Ansible Playbook Conditionals**
3. Ansible Playbook Loops
4. Introduction to Ansible Roles
5. Ansible Task Privilege Escalation and Execution Sequence
6. Introduction to the Jinja2 Template
7. AWX Description

HUAWEI

Application Scenarios of Ansible Playbook Conditionals

- In a playbook, conditionals allows you to execute different tasks, depending on the value of a variable, the result, or the output of a previous task. You can do all of these things with conditionals.
- Example: set **hosts** in a playbook to **all**, but display "hello world" only on the host whose IP address is 172.16.0.12.

```
- hosts: all
  remote_user: root
  gather_facts: true

  tasks:
    - name: hello world
      debug:
        msg: "hello world"
      when:  ansible_default_ipv4.address == "172.16.0.12"
```

```
[root@controller ~]# vim test.yml
[root@controller ~]# ansible-playbook test.yml
.......
ok: [172.16.0.12] => {
    "msg": "hello world"
}
skipping: [172.16.0.13]
.......
```


HUAWEI

Comparison Operators for Conditionals

- The following lists comparison operators for Ansible playbook conditionals:
  - ==: compares whether two objects are equal. If they are equal, **true** is returned. It is used to compare strings and numbers.
  - !=: compares whether two objects are different. If they are different, **true** is returned.
  - >: compares the values of two objects. If the value on the left is greater than that on the right, **true** is returned.
  - <: compares the values of two objects. If the value on the left is less than that on the right, **true** is returned.
  - >=: compares the values of two objects. If the value on the left is greater than or equal to that on the right, **true** is returned.
  - <=: compares the values of two objects. If the value on the left is less than or equal to that on the right, **true** is returned.

HUAWEI

- During comparison, the data types on both sides of the symbol must be the same. Otherwise, the comparison will not be performed.
- You can use the type module of debug to view the type of the object to be compared.

Logical Operators for Conditionals

- The following lists logical operators for Ansible playbook conditionals:
  - And: logical AND
    - [Condition 1] and [Condition 2]: The value is true only when both conditions 1 and 2 are true.
  - Or: logical OR
    - [Condition 1] or [Condition 2]: The value is true if either condition 1 or condition 2 is true.
  - Not: logical NOT
    - Not [Condition 1]: The value is true when condition 1 is false, and the value is false when condition 1 is true.
- Several logical operations of Ansible playbook conditionals can be performed together for judgment. For example:
  - ([Condition 1] and [Condition 2]) or ([Condition 3] and [Condition 4])

HUAWEI

- Multiple conditions can be listed for logic judgment. This slide uses only two conditions just for description.
- When performing combined logic operations, use parentheses "()" to combine related conditions into a whole, and then compare it with another combined operation.

Path Check of Conditionals

- The path of a conditional of the Ansible playbook can be used to determine files on the control host.
  - **File**: checks whether the specified path is a file. If so, the value is **true**.
  - **Directory**: checks whether the specified path is a directory. If so, the value is **true**.
  - **Link**: checks whether the specified path is a soft link. If so, the value is **true**.
  - **Mount**: checks whether the specified path is a mount point. If so, the value is **true**.
  - **Exists**: checks whether the specified path exists. If so, the value is **true**.

HUAWEI

- Example of path determination:
  - To check whether **/home/test** exists: **when: '/home/test' is exists**

Variable Check of Conditionals

- The Ansible playbook conditionals can be used to determine whether a variable exists.
  - **Defined**: The variable is defined.
  - **Undefined**: The variable is not defined.
  - **None**: The variable value is null. (It is used to determine whether the variable value is null after a variable is defined.)

HUAWEI

Task Execution Results of Conditionals

- Task execution results of Ansible playbook conditionals:
  - **Ok**: The target status is the same as the expected one and does not change.
  - **Change** or **changed**: The target is changed and is the same as the expected value.
  - **Success** or **succeeded**: The target status is the same as the expected one, or the task is successfully executed.
  - **Failure** or **failed**: The task fails to be executed.
  - **Skip** or **skipped**: The task is skipped.

HUAWEI

- Before determining the task execution result, register it as a variable. For details, see the *Lab Guide*.

Version Comparison of Conditionals

- Syntax for comparing versions of Ansible playbook conditionals:

version("*version number*","*comparison operator*")

- Supported comparison operators:
  - Greater than: **>** or **gt**
  - Greater than or equal to: **>=** or **ge**
  - Less than: **<** or **lt**
  - Less than or equal to: **<=** or **le**
  - Equal to: **=**, **==**, or **eq**
  - Not equal to: **!** =, **<>**, or **ne**

HUAWEI

Other Operations of Conditionals

- Check whether a character string is case sensitive.
  - **upper**: uppercase
  - **lower**: lowercase
- Determine the parity of a number.
  - **Even**: **true** if the value is an even number
  - **Odd**: **true** if the value is an odd number
  - **divisibleby(num)**: **true** if *num* can be exactly divided
- Determine the relationship between sets.
  - **Subset**
  - **Superset**
- Determine whether an object is a character string or number.
  - **String**
  - **Number**
- Use **in** to determine whether a character string is in a specified list.

HUAWEI

Conditionals – block

- In a playbook, when multiple tasks use the same condition for judgment, all related tasks can be encapsulated into a single block.

```
- hosts: all
  remote_user: root
  gather_facts: true

  tasks:
    - block:
      - name: install httpd
        yum:
          name: httpd
          state: present
      - name: install mysql
        yum:
          name: mysql-server
          state: present
      when: ansible_default_ipv4.address == "172.16.0.12"
```


HUAWEI

Conditionals – rescue/always

- **Rescue** and **always** are associated with **block**. When a task in **block** fails to be executed, the task in **rescue** is triggered. A task in **always** is executed regardless of whether **block** fails to be executed.

```
- hosts: all
  remote_user: root
  gather_facts: true

  tasks:
    - block:
      - name: install http
        yum:
          name: httpd
          state: present
      rescue:
        - name: display result
          debug:
            msg: "http installation is fail"
      always:
        - name: install mysql
          yum:
            name: mysql-server
            state: present
      when: ansible_default_ipv4.address == "172.16.0.12"
```


HUAWEI

Conditionals – fail

- **fail** is used to end the running of a playbook. When this module is used together with **when**, a playbook ends running when a certain condition is met. **msg** of **fail** can be used to output the termination information.

```
- hosts: 172.16.0.12
  remote_user: root
  gather_facts: true

  tasks:
    - fail:
        msg: "mission is terminated"
      when: ansible_default_ipv4.address == "172.16.0.12"
    - name: install http
      yum:
        name: httpd
        state: present
```

```
[root@controller ~]# ansible-playbook test.yml

PLAY [172.16.0.12] ************************************************************************
TASK [Gathering Facts] ********************************************************************
ok: [172.16.0.12]
TASK [fail] *******************************************************************************
fatal: [172.16.0.12]: FAILED! => {"changed": false, "msg": "mission is terminated"}
PLAY RECAP ********************************************************************************
172.16.0.12                : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
```


HUAWEI

Contents

1. Ansible Facts and Magic Variables
2. Ansible Playbook Conditionals
3. **Ansible Playbook Loops**
4. Introduction to Ansible Roles
5. Ansible Task Privilege Escalation and Execution Sequence
6. Introduction to the Jinja2 Template
7. AWX Description

HUAWEI

Application Scenarios of Ansible Playbook Loops

- Daily tasks often involve **repeated operations**, such as installing software packages, creating multiple users, or performing actions on directories and files. A loop can be compiled once and executed multiple times, improving compilation efficiency of a playbook. Loops are primarily employed for iterating over lists and dictionaries
  - **with_items** for iterating over lists
  - **with_disc** for iterating over dictionaries
  - In Ansible 2.5 and later versions, both **with_items** and **with_disc** can be replaced by **loop**, which is recommended.

HUAWEI

- In Ansible of a later version, **with_item** and **with_disc** are still available but will be deprecated. Therefore, this course mainly focuses on the usage of **loop**.

Iterating Over Lists

- The list elements of a playbook loop can be defined through variables or directly defined in **loop**.

```
- hosts: 172.16.0.12
  remote_user: root
  gather_facts: no

  vars:
    content:
      - hcia
      - hcip
      - hcie

  tasks:
    - name: test
      debug:
        msg: "{{ item }}"
      loop: "{{ content }}"
```

```
- hosts: 172.16.0.12
  remote_user: root
  gather_facts: no

  tasks:
    - name: test
      debug:
        msg: "{{ item }}"
      loop:
        - hcia
        - hcip
        - hcie
```

```
[root@controller ~]# ansible-playbook loop.yml

PLAY [172.16.0.12] *************************************************************************************************************

TASK [test] ********************************************************************************************************************
ok: [172.16.0.12] => (item=hcia) => {
    "msg": "hcia"
}
ok: [172.16.0.12] => (item=hcip) => {
    "msg": "hcip"
}
ok: [172.16.0.12] => (item=hcie) => {
    "msg": "hcie"
}

PLAY RECAP *********************************************************************************************************************
172.16.0.12                : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```


HUAWEI

- You can define variables in a playbook or directly use a variable file.
- To define variables in **loop**:
  - loop: ["hcia", "hcip", "hcie"]

Iterating Over Dictionaries

- When you use **loop** to replace **with_dict**, use **dict2itemd** to convert a dictionary to a list.

```
- hosts: 172.16.0.12
  remote_user: root
  gather_facts: no

  vars:
    certification:
      hcia:
        version: 1.5
        public_data: 2019
      hcip:
        version: 1.0
        public_data: 2023
      hcie:
        version: 1.0
        public_data: 2024

  tasks:
    - name: test
      debug:
        msg: "{{ item.value.version }}"
      loop: "{{ certification|dict2items }}"
```

```
- hosts: 172.16.0.12
  remote_user: root
  gather_facts: no

  vars:
    certification:
      hcia:
        version: 1.5
        public_data: 2019
      hcip:
        version: 1.0
        public_data: 2023
      hcie:
        version: 1.0
        public_data: 2024

  tasks:
    - name: test
      debug:
        msg: "{{ item.value.version }}"
      with_dict: "{{ certification }}"
```


HUAWEI

- During directory loop, identify whether the loop object is a key or value. As shown in the code body, **hcia**, **hcip**, and **hcie** are dictionary keys, and their values are also of the dictionary type. The corresponding keys are **version** and **public_data**, which also have respective values. During the loop, you need to specify the key layer by layer to locate the target value.

Example of Ansible Playbook Loops

- Create users contained in the user file based on the username information.

```
[root@controller yaml]# cat username
user:
  - name: zhangsan
    department: sale
    uid: 1100
  - name: lisi
    department: sale
    uid: 1101
  - name: wangwu
    department: technology
    uid: 1102
```

```
user:
  - name: zhangsan
    department: sale
    uid: 1100
  - name: lisi
    department: sale
    uid: 1101
  - name: wangwu
    department: technology
    uid: 1102
[root@controller yaml]# cat user.yml
- hosts: all
  gather_facts: false
  vars_files:
    /root/yaml/username
  tasks:
    - name: create user
      user:
        name: "{{ item.name }}"
        comment: "{{ item.department }}"
        uid: "{{ item.uid }}"
        password: "{{ lookup('password','/root/yaml/password-{{ item.name }} chars=digits length=8') |
password_hash('sha512') }}"
      loop: "{{ user }}"
```


HUAWEI

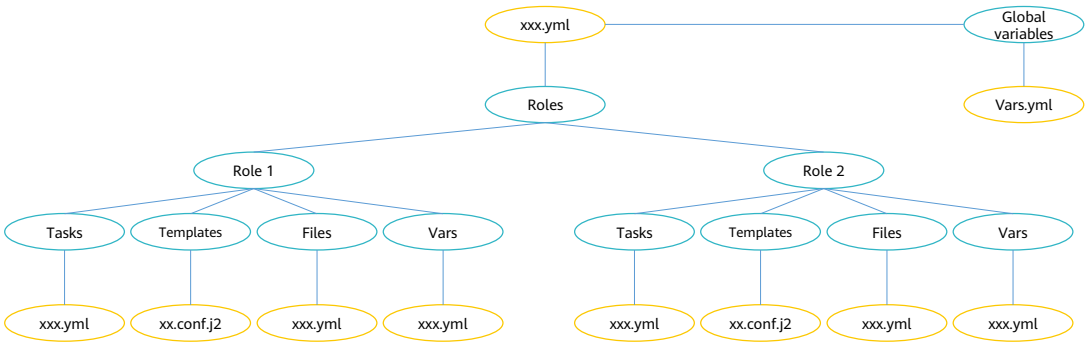
Contents

1. Ansible Facts and Magic Variables
2. Ansible Playbook Conditionals
3. Ansible Playbook Loops
4. **Introduction to Ansible Roles**
5. Ansible Task Privilege Escalation and Execution Sequence
6. Introduction to the Jinja2 Template
7. AWX Description

HUAWEI

Role Overview

- Roles are used for hierarchical and structured playbooks. Variable files, tasks, and handlers can be automatically loaded based on defined and hierarchical directories to complete complex O&M tasks in batches.
- Common directory structure:



HUAWEI

Role Directory Structure

- A role defines eight main standard directories. When creating a role, you need to create directories as required and ignore unnecessary directories. The eight directories and their functions are as follows:

```
roles/
   common/               # this hierarchy represents a "role"
      tasks/             #
         main.yml        #  <-- Task list
      handlers/          #
         main.yml        #  <-- Handler list
      templates/         #  <-- Template file used by a task
         ntp.conf.j2     #  <-------  The template must be suffixed with .j2.
      files/             #
         ntp.conf.j2     #  <-------  File that will be copied to the controlled end in a task
         foo.sh          #  <-- Shell script used in a task
      vars/              #
         main.yml        #  <--  File with defined variables that is used by a task
      defaults/          #
         main.yml        #  <-- Low-priority variable file
      meta/              #
         main.yml        #  <-- Dependency of role
      library/           # Self-developed module
      module_utils/      # Self-developed module unit
      lookup_plugins/    # Plugins, for example, lookup plugins.
```


HUAWEI

- Ansible reads the necessary material by default from the role directory's **main.yml** file. If the material is brief, **main.yml** should be enough. It is recommended that if the material is substantial, it be divided into many files, imported using **include_tasks**. And **main.yml** is only used as the entry.

Example – Installing WordPress

- The following lists files contained in a task list.

```
[root@controller tasks]# ls
configure_mariadb.yml    create_repo_rhel7.yml
install_packages_deb.yml    main.yml            start_services.yml
configure_wordpress.yml  download_wordpress.yml
install_packages_rhel7.yml  selinux_rhel7.yml  unarchive_wordpress.yml
```

- The right part shows the content of the **main.yml** file.

```
---
# tasks file for mariadb_apache_wordpress
- name: Include vars
  include_vars: "{{ ansible_os_family }}.yml"
#
- name: Create repository for MariaDB
  import_tasks: create_repo_rhel7.yml
  when: ansible_os_family == 'RedHat'
#
- name: Install {{ packages }}
  import_tasks: install_packages_rhel7.yml
  when: ansible_os_family == 'RedHat'
#
- name: Install {{ packages }}
  import_tasks: install_packages_deb.yml
  when: ansible_os_family == 'Debian'
#
- name: Start services
  import_tasks: start_services.yml
#
- name: Configure SELinux permissive mode
  import_tasks: selinux_rhel7.yml
  when: ansible_os_family == 'RedHat'
#
- name: Download WordPress
  import_tasks: download_wordpress.yml
#
- name: Unarchive WordPress
  import_tasks: unarchive_wordpress.yml
#
- name: Configure MariaDB
  import_tasks: configure_mariadb.yml
#
- name: Configure WordPress file
  import_tasks: configure_wordpress.yml
#
- name: Finish the installation at the link below
  debug:
    msg: "{{ ansible_default_ipv4.address }}/wordpress/wp-admin/install.php"
```


HUAWEI

Storing, Searching, and Using a Role

- By default, Ansible searches for a role in the following locations:
  - Role collections
  - Directory named **roles** in a playbook
  - Role path specified in the configuration file
    - Default path: **~/.ansible/roles:/usr/share/ansible/roles:/etc/ansible/roles**
  - Path specified in a playbook
- Ways to use a role:
  - In a play by calling **roles**.
  - In a task by calling **include_role**.
  - In a task by calling **import_role**.

HUAWEI

- Differences between **roles**, **include_role**, and **import_role**:
  - When you use **roles** in a play, roles are transferred to each task contained in the play through tags.
  - **include_role** is a dynamic reference. That is, the tasks to be executed in a playbook are known only in the execution phase.
    - The name of the referenced role can contain variables.
    - **include_role** itself is regarded as a task.
  - **import_role** is a static reference. That is, the tasks to be executed in a playbook are known in the pre-compilation phase.
    - The referenced role cannot use variables (except **vars** and **vars-file**).
    - **import_role** is not regarded as a task.

Introduction to ansible-galaxy

- Ansible Galaxy is an online repository used to share compiled roles. **ansible-galaxy** is a command line tool of Galaxy and provides the following options:
  - ansible-galaxy role init
  - ansible-galaxy role remove
  - ansible-galaxy role delete
  - ansible-galaxy role list
  - ansible-galaxy role search
  - ansible-galaxy role import
  - ansible-galaxy role setup
  - ansible-galaxy role info
  - ansible-galaxy role install

HUAWEI

Role Practice – Searching for and Pulling Roles from Galaxy

- Use Ansible roles and install WordPress in the current environment.

```
######## Search for roles related to WordPress.
[root@controller ~]# ansible-galaxy role search wordpress

Found 145 roles matching your search:

 Name                                          Description
 ----                                          -----------
 xxxxxxx_wordpress                             Install and Configure MariaDB, Apache and WordPress
.........
###### View the details of a project.
[root@controller ~]#ansible-galaxy role info xxxxxxxx_wordpress
Role: xxxxxxxxxx_wordpress
        description: Install and Configure MariaDB, Apache and WordPress
        commit: 09fda699ce110f975250c8aa315c87a0c6d56506
        commit_message: Update unarchive_wordpress.yml
.........
###### Download a proper role.
[root@controller ~]# ansible-galaxy role install xxxxxxxxxxx_wordpress
- downloading role 'mariadb_apache_wordpress', owned by xxxxxxx
- downloading role from https://github.com/xxxxxxxxxxx/master.tar.gz
- extracting acandid.mariadb_apache_wordpress to /root/.ansible/roles/xxxxxxxxxx_wordpress
- xxxxxxxxxx_wordpress (master) was installed successfully
```


HUAWEI

Role Practice – Customizing Roles

- Install the httpd service on server a and server b. After the installation is complete, the service listens to port 81 of the first NIC (default NIC) of the host. When the service accesses port 81 of the host, the message "this is xxx" is displayed, where *xxx* indicates the host name.

```
[root@controller roles]# tree httpd
httpd
├── defaults
│   └── main.yml
├── files
├── handlers
│   └── main.yml
├── meta
│   └── main.yml
├── README.md
├── tasks
│   ├── firewalld.yml
│   ├── httpd-config.yml
│   ├── httpd-install.yml
│   └── main.yml
├── templates
│   ├── index.html.j2
│   └── roles.conf.j2
├── tests
│   ├── inventory
│   └── test.yml
└── vars
    └── main.yml
```

```
[root@controller roles]# cat httpd/tasks/httpd-install.yml
- name: install httpd
  yum:
    name: httpd
    state: present
[root@controller roles]# cat httpd/tasks/httpd-config.yml
- name: config httpd
  template:
    src: roles.conf.j2
    dest: /etc/httpd/conf.d/roles.conf
  notify: restart httpd
- name: config main page
  template:
    src: index.html.j2
    dest: /var/www/html/index.html
- name: enable httpd
  service:
    name: httpd
    enabled: yes
    state: started
```


HUAWEI

Role Practice – Customizing Roles

- Install the httpd service on server a and server b. After the installation is complete, the service listens to port 81 of the first NIC (default NIC) of the host. When the service accesses port 81 of the host, the message "this is xxx" is displayed, where *xxx* indicates the host name.

```
[root@controller roles]# cat httpd/tasks/firewalld.yml
- name: config firewalld
  firewalld:
    port: "{{ PORT }}/tcp"
    permanent: yes
    immediate: yes
    state: enabled
[root@controller roles]# cat httpd/templates/index.html.j2
this is {{ ansible_hostname }}
[root@controller roles]# cat httpd/templates/roles.conf.j2
Listen {{ ansible_default_ipv4.address }}:{{PORT}}
[root@controller roles]# cat httpd/vars/main.yml
---
# vars file for httpd
PORT: 81    ##### Listening port of the httpd service
```

```
[root@controller roles]# cat httpd/handlers/main.yml
---
# handlers file for httpd
- name: restart httpd
  service:
    name: httpd
    state: restarted
```


HUAWEI

Contents

1. Ansible Facts and Magic Variables
2. Ansible Playbook Conditionals
3. Ansible Playbook Loops
4. Introduction to Ansible Roles
5. **Ansible Task Privilege Escalation and Execution Sequence**
6. Introduction to the Jinja2 Template
7. AWX Description

HUAWEI

Overview of Ansible Task Privilege Escalation

- In practice, tasks are not executed with root privileges unless absolutely necessary. Therefore, Ansible must raise a user's privilege in order to carry out a specified task on a designated host.
- Commands involved in privilege escalation:
  - **become**: **true** to enable privilege escalation; **false** to disable privilege escalation.
  - **become_user**: specifies the user for whom the privilege is escalated.
  - **become_method**: set the privilege escalation mode, such as **sudo**, **su**, and **pbrun**, depending on the mode supported by the target host.
  - **become_ask_pass**: specifies whether the password needs to be verified during privilege escalation.
  - **become_flags**: sets privilege escalation parameters.

HUAWEI

Location of Privilege Escalation Configuration – Global Configuration

- You can modify the Ansible configuration file to configure privilege escalation in global mode. After the configuration is complete, all tasks are executed by the user whose privilege is escalated. The configuration is stored in **[privilege_escalation]**.

```
[privilege_escalation]
become=True
become_method=sudo
become_user=root
become_ask_pass=False
```


HUAWEI

Location of Privilege Escalation Configuration – Playbook and Task

- If privilege escalation is required only in a specific playbook or task, escalate the privilege in the corresponding playbook or task.

```
- hosts: 172.16.0.12
  become: True
  become_method: sudo
  become_user: root
  remote_user: test
  gather_facts: no

 tasks:
    - name: test
.......
```

```
- hosts: 172.16.0.12
.........

 tasks:
    - name: test
      become: True
      become_method: sudo
      become_user: root
      debug:
        msg: "{{ item.value.version }}"
      with_dict: "{{ certification }}"
```


HUAWEI

Location of Privilege Escalation Configuration – Role

- You can configure privilege escalation in a role in either of the following ways:
  - In a playbook or task
  - In the reference of a role

```
- hosts: 172.16.0.12
  remote_user: test
  gather_facts: no

  roles:
    - role: wordpress
      become: True
      become_method: sudo
      become_user: root

.......
```


HUAWEI

Locations of Privilege Escalation Configuration – Block and Host

- If privilege escalation is configured in a block, all tasks under the block are executed by the user whose privilege is escalated.

```
- hosts: all
  remote_user: test
  gather_facts: true

  tasks:
    - block:
      - name: install httpd
        yum:
          name: httpd
          state: present
      - name: install mysql
        yum:
          name: mysql-server
          state: present
      become: true
      become_user: root
      become_method: sudo
      when: ansible_default_ipv4.address == "172.16.0.12"
```

- In the host inventory, you can configure privilege escalation for one or more hosts.

```
all:
  children:
    ungrouped:
      hosts:
        servera:
          ansible_become: true
          ansible_become_user: root
          ansible_become_method: sudo
```


HUAWEI

Execution Sequence of Playbook Tasks

- After a role is used, the Ansible playbook executes tasks in the following sequence:
  - **pre_tasks** defined in a play
  - Tasks triggered by **handler** in **pre_tasks**
  - Tasks in **roles** (executed in sequence)
  - Other tasks defined in a play
  - Tasks triggered by **handler** of other tasks in a play
  - **post_tasks** defined in a play
  - Tasks triggered by **handler** in **post_tasks**

Contents

1. Ansible Facts and Magic Variables
2. Ansible Playbook Conditionals
3. Ansible Playbook Loops
4. Introduction to Ansible Roles
5. Ansible Task Privilege Escalation and Execution Sequence
6. **Introduction to the Jinja2 Template**
7. AWX Description

HUAWEI

Introduction to the Jinja2 Template

- The template module of Ansible can render variables in a file to corresponding values. Such a file must be a jinja2 template.
- Example: Deploy the httpd service on server a, server b, and server c, and require the service to listen to port 80 of the NIC (ens3).
- Implementation:

```
- hosts: all
  remote_user: root
  gather_facts: true

  tasks:
    - yum:
        name: httpd
        state: present
    - template:
        src: /root/httpd.conf.j2
        dest: /etc/httpd/conf/httpd.conf
```

```
[root@controller ~]# cat httpd.conf.j2
.....
Listen {{ ansible_ens3.ipv4.address }}:80
......
```


HUAWEI

Conditionals of the Jinja2 Template

- In the Jinja2 template, a conditional starts with **if** and ends with **endif.** For a branch statement, **elif** and **else** are used. The format is as follows:
  - Single-branch structure

```
{% if conditional expression %}
   Execution statement
{%  endif  %}
```

  - Dual-branch structure

```
{% if conditional expression 1 %}
    Execution statement 1
{% else %}
    Execution Statement 2
{%  endif  %}
```

  - Multi-branch structure

```
 {% if conditional expression 1 %}
    Execution statement 1
{ % elif conditional expression 2 %}
   Execution Statement 2
{% else %}
    Execute statement n
{%  endif  %}
```


HUAWEI

Loops of the Jinja2 Template

- The Jinja2 template supports loops to perform repeated operations. The syntax is as follows:

```
{% for variable in  list_element %}
Content of a loop
{% endfor %}
```

- Example: Bind host IP addresses and host names in the host inventory one by one.
  - Compile the Jinja2 template.

```
{% for host in groups.all %}
{{hostvars[host].ansible_default_ipv4.address}} {{hostvars[host].ansible_fqdn}} {{hostvars[host].ansible_hostname}}
{% endfor %}
```

  - Use the template module in a playbook to call the compiled template.

```
- hosts: all
  remote_user: root
  gather_facts: true

  tasks:
    - template:
        src: /root/jinja.j2
        dest: /root/host.txt
```


HUAWEI

Filters of the Jinja2 Template

- A filter is used to process and convert variables in the Jinja template and is essentially a function.
- You can use the filter to format, filter, and modify variable values. Common filters are as follows:
  - Default
  - String filter
  - Number filter
  - List filter
  - File path filter
  - Encryption/Decryption filter
  - Network address operation filter
  - URL address operation filter
  - Find/Replace filter
  - Dictionary variable filter
  - Custom filter

HUAWEI

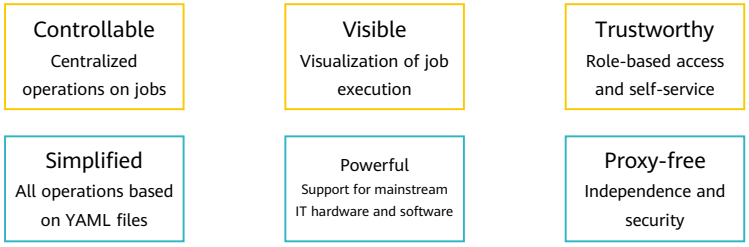
Contents

1. Ansible Facts and Magic Variables
2. Ansible Playbook Conditionals
3. Ansible Playbook Loops
4. Introduction to Ansible Roles
5. Ansible Task Privilege Escalation and Execution Sequence
6. Introduction to the Jinja2 Template
7. **AWX Description**

HUAWEI

Introduction to AWX

- AWX, built on top of Ansible, is an open source engine for automation management and provides a web-based user interface and REST API.



HUAWEI

- The lower part shows the features of Ansible, and the upper part shows the features of AWX.

Features of AWX

- Views the outputs of a playbook promptly
- Supports Web UI operations
- Enhances and simplifies role-based access control and auditing
- Supports automatic expansion
- Supports RESTful APIs
- Supports backup and restoration
- Integrates with Ansible Galaxy
- Calls OpenStack for dynamic host inventories
- Supports remote command execution
- Traces a system using facts
- Supports multiple message services

HUAWEI

AWX Installation Method

- The AWX can be installed via the following methods:
  - Using a container
  - Using the AWX Operator (officially recommended in versions later than 18.0)
  - Using the Ansible playbook

Installation and Deployment Procedure of the AWX Operator

- Resource preparations
  - The AWX database requires persistent storage. Therefore, you need to create corresponding resources in advance, such as the storage spaces provided by Ceph, GlusterFS, and NFS.
  - If AWX uses StorageClass to create PVs for persistent storage, prepare the corresponding configuration file in advance.
- AWX deployment
  - Deploy the operator-controller-manager.
  - Deploy components such as the database, AWX, and web service.
- Service release
  - Use Ingress to release AWX services.

HUAWEI

- The following content uses Ceph as an example.

Preparing Storage Resources (Using Ceph as an Example)

- Create a resource pool.

```
[ceph: root@ceph01 /]# ceph osd pool create metadata_for_awx
[ceph: root@ceph01 /]# ceph osd pool create data_for_awx
[ceph: root@ceph01 /]# ceph fs new pool_for_awx metadata_for_awx  data_for_awx
```

- Create a user and grant permissions to the user.

```
[ceph: root@ceph01 /]# ceph fs authorize pool_for_awx client.awx / rwps -o ceph.client.awx.keyring
```

```
[ceph: root@ceph01 /]# ceph fs authorize pool_for_awx client.awx / rwps -o ceph.client.awx.keyring
[ceph: root@ceph01 /]# ceph auth ls | grep -A 4 awx
installed auth entries:

client.awx
        key: AQAHjkllPax2CBAACfRE0qQUqmZzjQtOxzQHIg==
        caps: [mds] allow rwps fsname=pool_for_awx
        caps: [mon] allow r fsname=pool_for_awx
        caps: [osd] allow rw tag cephfs data=pool_for_awx
```


HUAWEI

Configuring a StorageClass File (Using Ceph as an Example)

- Configure the MON address in the **csi-config-map.yaml** file.
- Configure an authentication mode in the **ceph-conf-map.yaml** file.
- Deploy ConfigMaps and pods required for StorageClass.
- Compile the StorageClass file and the secret file required later.

Deploying AWX

- Deploy AWX Using kustomization.

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  # Find the latest tag here: https://github.com/ansible/awx-operator/releases
  - https://gitee.com/yftyxa/awx-operator/config/default?ref=2.7.1
  - awx-demo.yml
# et the image tags to match the git version from above
images:
  - name: swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/awx-operator
    newTag: latest

# Specify a custom namespace in which to install AWX
namespace: awx
```

- https://gitee.com/yftyxa/awx-operator/config/default?ref=2.7.1 and **awx-demo.yml** are resources required for AWX deployment.

HUAWEI

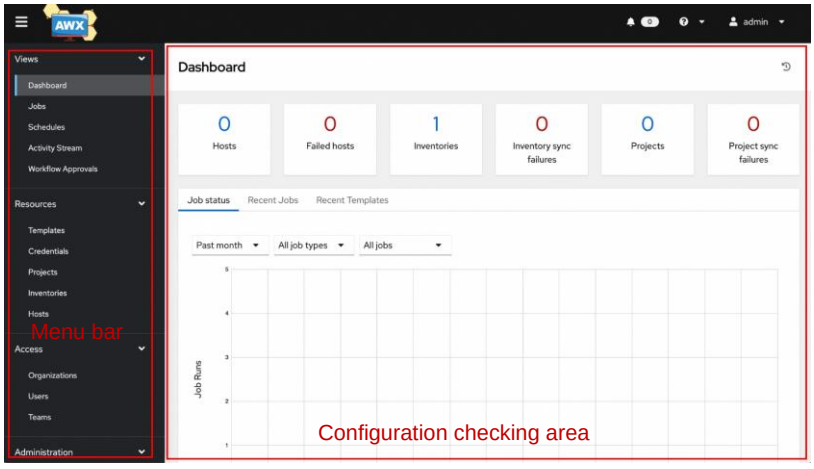
Releasing Services

- Release AWX services through Ingress.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: awx
  namespace: awx
  annotations:
kubernetes.io/ingress.class: "nginx"
nginx.ingress.kubernetes.io/rewrite-target: /$2
nginx.ingress.kubernetes.io/use-regex: "true"
spec:
 rules:
   - host: www.openeuler-hcie.com
     http:
       paths
         - backend:
             service:
               name: awx-openeuler-service
               port:
                 number: 80
           path: /awx(/|$)(.*)
           pathType: Prefix
```


HUAWEI

AWX Home Page

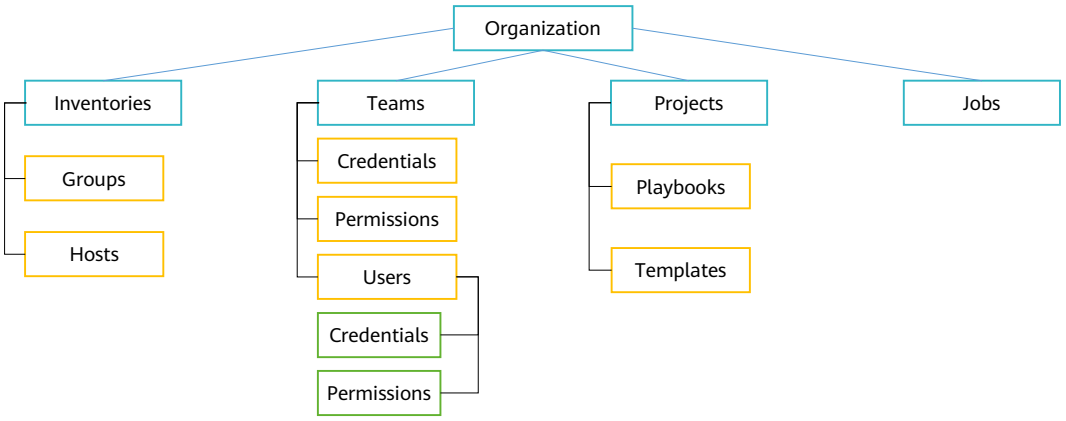


HUAWEI

- The menu bar contains all functions of AWX. Each module contains specific sub-modules.
- In the configuration checking area, you can configure and view configurations of a specific resource.

Basic Concepts of AWX – Organization

- An organization is a logical collection of users, teams, projects, and inventories. It is the highest level object in the AWX object hierarchy.

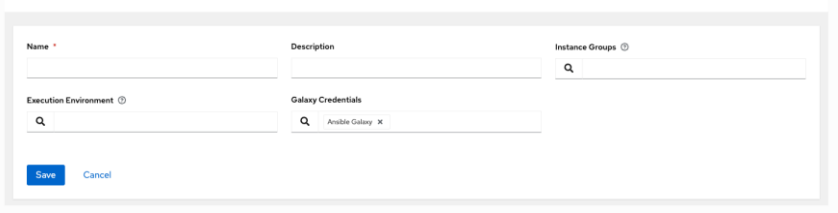


HUAWEI

AWX User Model

- AWX controls accesses based on roles.
- An organization of AWX is a collection of users, teams, projects, and inventories, and it needs to comply with the following requirements:
  - All users must belong to an organization.
  - A team is a collection of users.
  - Roles of users in a team are determined by the team role.
  - Roles determine the permissions of users and teams.

Creating an Organization



- **Name**: user-defined. It is recommended that you choose an identifiable name.
- **Description**: supplementary information about the purpose, attribute, and scope of your organization.
- **Instance Groups**: a group that runs your organization.
- **Execution Environment**: Ansible running environment. The default value is **AWX EE (latest)**.
- **Galaxy Credentials**: used to access Ansible Galaxy.

HUAWEI

- The execution environment is used to execute Ansible commands or playbooks. In AWX, the image **quay.io/ansible/awx-ee:latest** is used to create a pod, and then the Ansible controller runs in the pod to execute tasks on the controlled node.
- Instances and instance groups correspond to pods automatically generated during AWX installation. If there are a large number of tasks, you can create instances and instance groups to meet performance requirements.

Basic Concepts of AWX – Team

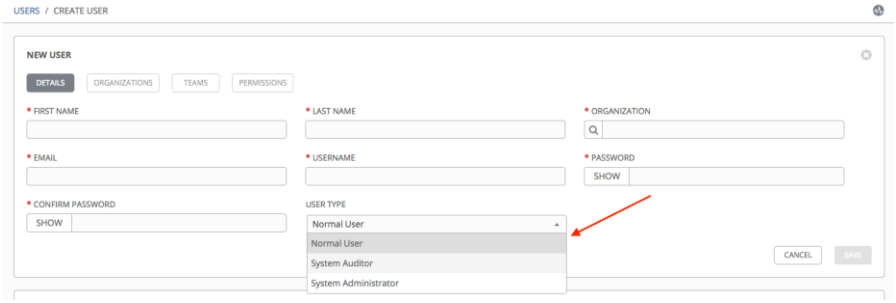
- A team is a subdivision of an organization with associated users, projects, credentials, and permissions.
- Teams provide the means to implement role-based access control schemes and delegates responsibilities across organizations.



Basic Concepts of AWX – User

- A user is someone who has access to AWX with associated permissions and credentials.
- Users are classified into the following types:
  - Normal user
  - System auditor
  - System administrator

Creating a User



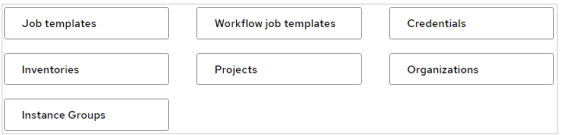
- Normal user: has permissions on specific resources. Its permissions are defined by its roles.
- System auditor: implicitly inherits the read-only capability of all objects within the AWX environment.
- System administrator: has full system administration privileges. The system administrator is responsible for managing all aspects of the AWX and delegating responsibilities for day-to-day work to various users.

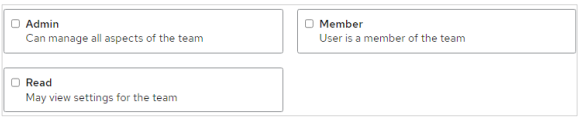
HUAWEI

AWX Permission Control Settings

- You can grant AWX permissions to teams or users in organizations, set roles for users in teams, or specify roles for resources in teams or users.
  - The figure on the left shows roles that can be set for teams or users in an organization.
  - The figures on the right show roles that can be set for users or teams.







HUAWEI

Basic Concepts of AWX – Credentials

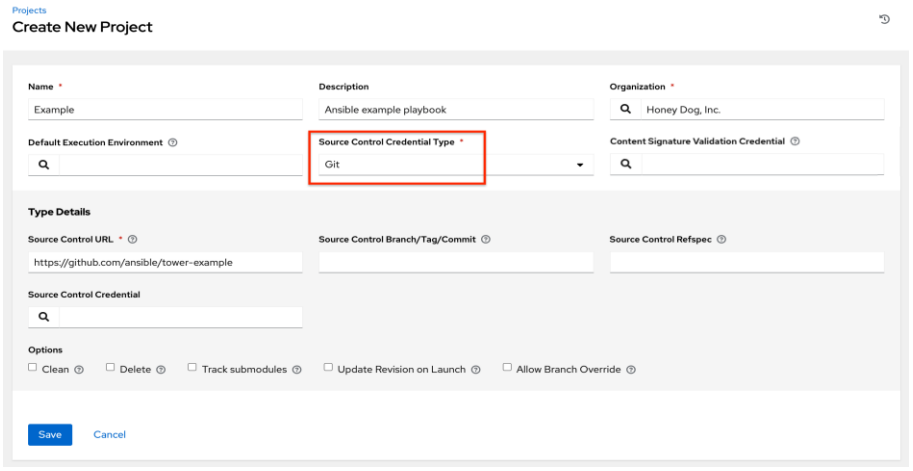
- Credentials of AWX are used for permission authentication and apply to the following scenarios:
  - Task launch on a specified host
  - Synchronization of inventory resources
  - Project content import from a version control system
- AWX credentials support multiple types.
  - Machine: enables Ansible to deliver commands to hosts or executes playbook tasks.
  - Source control: pulls or updates code in a project from a remote code repository.
  - OpenStack: enables synchronization of cloud inventory with OpenStack.

HUAWEI

Basic Concepts of AWX – Project

- A project in AWX is a logical collection of Ansible playbooks.
- Playbooks of a project can be stored in a local directory on the AWX server or in the source code management (SCM) system.
- AWX supports the following SCM systems:
  - Git
  - SVN
  - Mercurial

Creating a Project (Using Git as an Example)

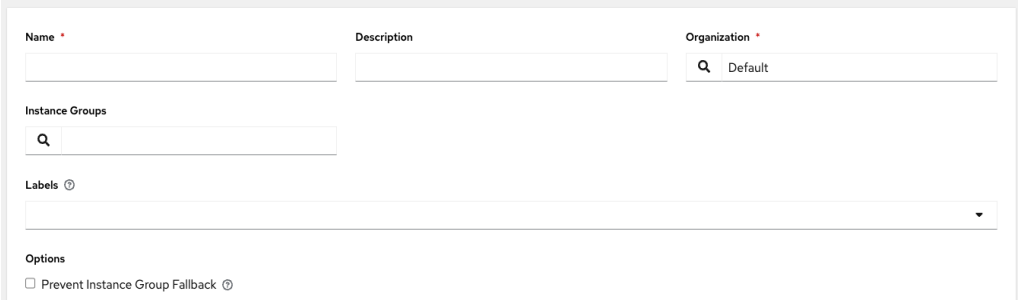


- **Clean**: removes any local modifications prior to performing an update.
- **Delete**: deletes the entire local repository before an update. Depending on the size of the repository, this may significantly increase the amount of time required to complete an update.
- **Trace submodules**: tracks the latest commit of the master branch or other branches specified in **.gitmodules**. If there is no latest commit, the submodule remains in the revision specified by the main project. This is equivalent to specifying the **--remote** flag in the **git submodule update** command.
- **Update Revisions on Launch**: updates the revisions of a project before starting a job each time you use this project to run a job.
- **Allow Branch Override**: allows you to change the SCM branch or revision in the job template that uses this project.

Basic Concepts of AWX – Inventory

- An inventory is a collection of hosts against which jobs may be launched and is the same as an Ansible inventory file.
- Inventories are divided into groups containing one or more actual hosts.
- There are two types of AWX inventories:
  - Static inventory
  - Dynamic inventory

Creating a Static Inventory



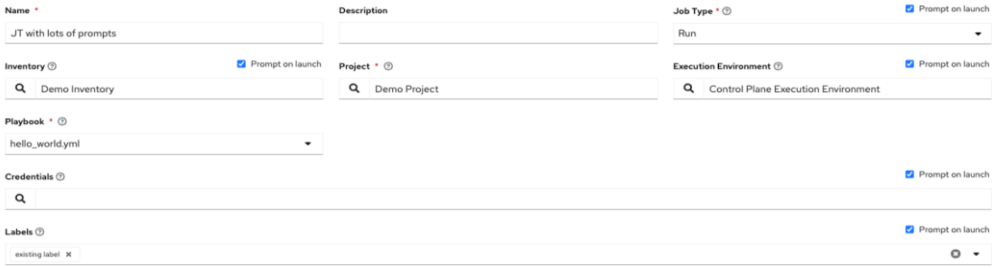
- **Labels**: Similar to group names in Ansible, it is used to group and filter inventories and completed jobs.
- **Prevent Instance Group Fallback**: If it is enabled, the inventory prevents any instance group of an organization from being added to the list of preferred instance groups that run the associated job template. Note: If this option is enabled and an empty list is provided, a global instance group is applied.

HUAWEI

Basic Concepts of AWX – Job Template

- A job template in AWX is a collection of tasks and their parameters. Job templates can be used to execute the same job many times.
- Although tasks can be directly executed through REST APIs, AWX requires job templates to be created before tasks are executed.

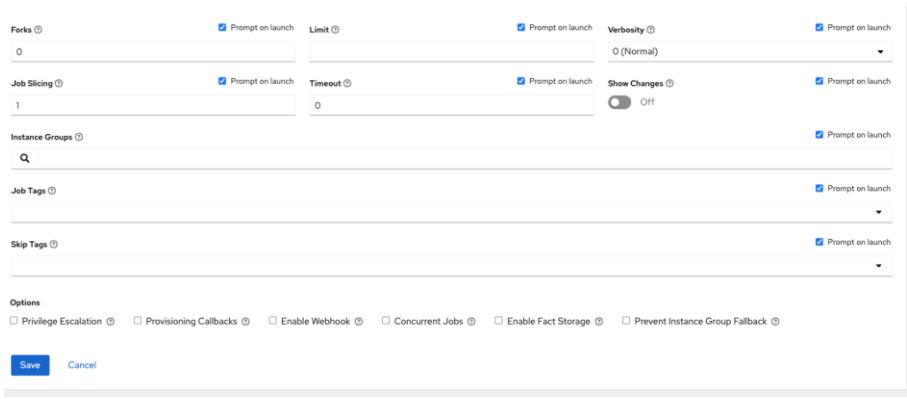
Creating a Job Template – Configuring Basic Information



- Input **Name** and **Description** based on the actual situation.
- **Job Type**: Select **Check** (checks syntax in a playbook and testing environment and reports issues) or **Run** (executes playbooks.)
- **Labels**: similar to the Ansible inventory. The created job template will be executed on the corresponding host.

HUAWEI

Creating a Job Template – Configuring Advanced Information



HUAWEI

- **Forks**: number of parallel or simultaneous processes to use while executing a playbook.
- **Verbosity**: level of detail of the playbook execution logs, which is equivalent to **-v**, **-vv**, and **-vvv**.
- **Job Tags**: similar to **tag** in a playbook.
- **Skip Tags**: skips tasks with specified tags.

Basic Concepts of AWX – Workflow Job Template

- A workflow job template is a collection of resources that can be used to complete complex tasks. The resources include:
  - Job templates
  - Workflow job templates
  - Project syncs
  - Inventory source syncs

HUAWEI

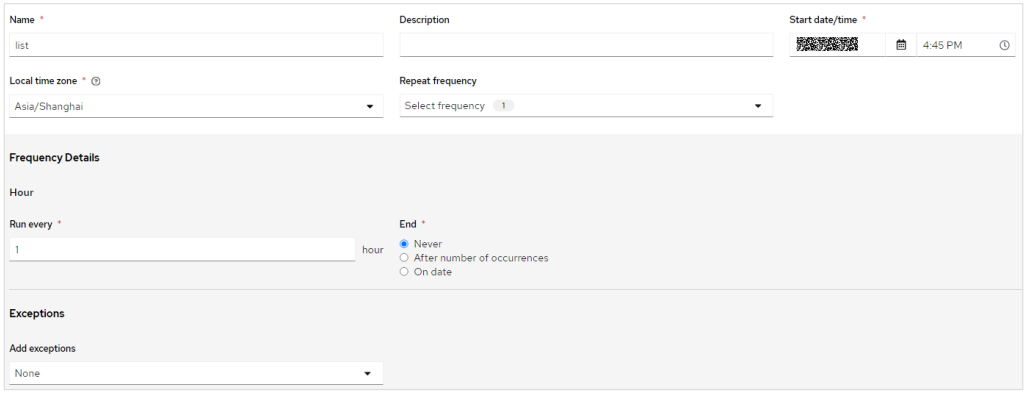
- The biggest difference between a workflow job template and a job template is that a workflow job template can run multiple playbook tasks at a time, but a job template can run only one playbook task.

Workflow Visualizer

- The Workflow Visualizer provides a graphical way of linking together job templates, workflows, project syncs, and inventory syncs to build a workflow template.
- The node types of workflow templates include **Approval**, **Inventory Source Sync**, **Job Template**, **Project Sync**, **Workflow Job Template**, and **Management Job**.
- The conditions between nodes can be classified into **On Success**, **On Failure**, and **Always**.

Basic Concepts of AWX – Schedule

- Schedule is used to periodically trigger jobs, specify the number of times when jobs are executed, and set the attributes of periodic jobs.
- A schedule , such as how often to execute templates or update projects or inventories, must be created in **resources**.



AWX API Scheduling

- AWX can perform operations on tasks and templates through APIs.
- You can view all AWX resources by entering **http://*AWX IP address*/api/V2**.
- You can use methods such as GET and POST to query or operate resources.
- When an API is queried, **json_pp** is used to convert the output content to JSON format, which is more readable.
- Example: run the following command to start a resource:

```
curl -X POST --user admin:7yv37sFLaj3EPQ00LgwuT3sOoVQLkCgg http://www.openeuler-hcie.com:32453/awx/api/v2/workflow_job_templates/14/launch/
```


HUAWEI

Quiz

1. (Multiple-answer question) Which of the following host information can be obtained through facts?
   A. Drive space
   B. IP address
   C. Host name
   D. Host user

HUAWEI

- ABC

Summary

- This course provides an introduction to the implementation of CI/CD by describing the advanced functionalities of the automation management tool, including magic variables and facts.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling	Description
AWX	-	An open-source automation management tool
SVN	software version number	-

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

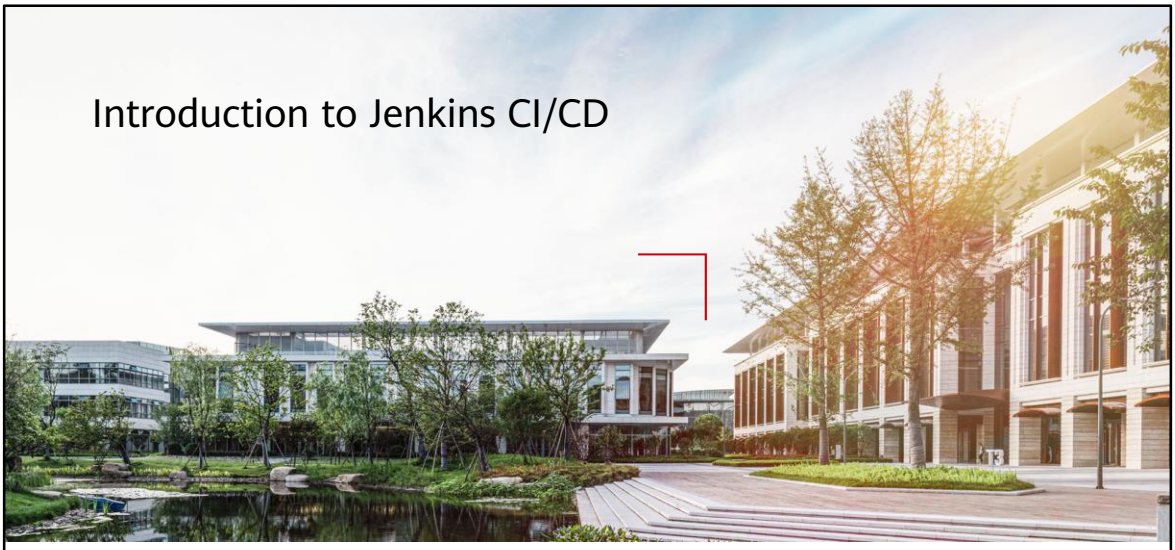
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Introduction to Jenkins CI/CD



Foreword

- This course covers the installation and usage of Jenkins, a popular CI/CD tool, as well as the automatic deployment of applications using the automation management and cloud native skills covered in prior courses.

Objectives

- Upon completion of this course, you will:
  - Master basic Jenkins operations.
  - Master how Jenkins and AWX cooperate to implement CI/CD of applications.
  - Master how Jenkins and Kubernetes are used to implement CI/CD of applications.

HUAWEI

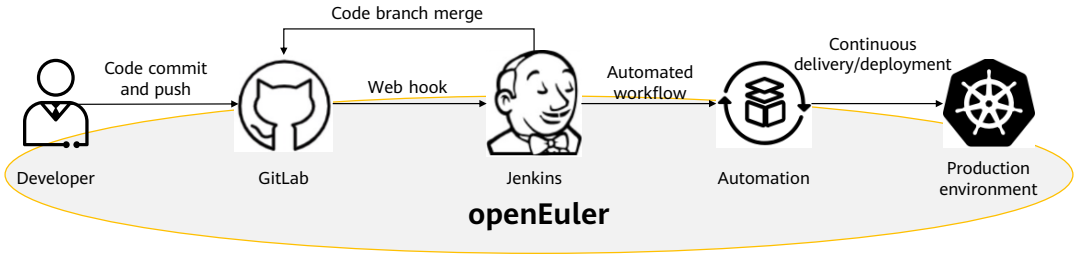
Contents

1. **Core Component of CI/CD – Jenkins**
2. Installing and Deploying Jenkins
3. Basic Jenkins Operations
4. Dynamic Slave Deployment of Jenkins

HUAWEI

Jenkins Introduction

- Jenkins is a popular, open source CI/CD tool that is used to automate various tasks of building, testing, delivering, or deploying applications.
- Jenkins can be installed using native system packages, Docker, or on any machine where the Java runtime environment (JRE) is installed.



HUAWEI

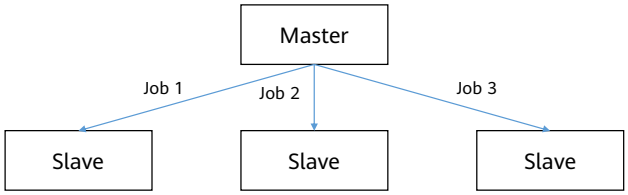
Features of Jenkins

- Jenkins has the following features:
  - Free open-source tool
  - Cross-platform deployment
  - Web-based visualized management
  - Easy installation and configuration
  - **Support for more than 1800 plugins**

HUAWEI

Jenkins Architecture

- Jenkins uses a master-slave architecture.
  - The master is the core of Jenkins. It periodically runs jobs, distributes jobs to slaves, and monitors the running status of jobs.
  - A slave listens to instructions from master and runs jobs.



HUAWEI

Overview of Jenkins Plugins

- Plugins are the most powerful component of Jenkins. Jenkins has several plugin libraries available to automate simple tasks during CI/CD.



HUAWEI

- Image source: https://plugins.jenkins.io/

Contents

1. Core Component of CI/CD – Jenkins
2. **Installing and Deploying Jenkins**
3. Basic Jenkins Operations
4. Dynamic Slave Deployment of Jenkins

HUAWEI

Installation Modes

- Jenkins supports the following installation modes:
  - Docker-based deployment
  - Kubernetes-based deployment
  - Directly installed in Linux and its distributions
  - Directly installed in macOS
  - Directly installed in Windows
  - Installed in other OSs
  - Other modes

Deploying Jenkins in Kubernetes

- To deploy Jenkins in Kubernetes, perform the following steps:
  - Create a dedicated namespace for Jenkins.
  - Create a service account required by Jenkins.
  - Create permanent storage.
  - Create a deployment running on Jenkins.
  - Deploy a service required for accessing Jenkins.
  - Use Ingress to release the Jenkins service.

HUAWEI

- In this course, Jenkins is installed in Kubernetes, and CephFS provided by Ceph is used for permanent storage.

Creating a Service Account Required by Jenkins

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole                    ##### Define a group of permissions that have all permissions on all API groups and resources.
metadata:
  name: jenkins-admin
rules:
  - apiGroups: [""]
    resources: ["*"]
    verbs: ["*"]
---
apiVersion: v1
kind: ServiceAccount                 ##### Specify an application or component that has a token to access Kubernetes APIs in a cluster.
metadata:
  name: jenkins-admin
  namespace: devops
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding         ##### ClusterRoleBinding binds the previously defined ServiceAccount named "jenkins-admin" to ClusterRole named "jenkins-admin" so that the ServiceAccount has all permissions to operate Kubernetes APIs. But the ServiceAccount can access resources only in the "devops" namespace.
metadata:
  name: jenkins-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: jenkins-admin
subjects:
- kind: ServiceAccount
  name: jenkins-admin
  namespace: devops
```


HUAWEI

Creating Permanent Storage

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: jenkins
  namespace: devops
spec:
  storageClassName: Jenkins      ##### Set this parameter based on the actual situation.
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi               #### ≥ 3 Gi
```


HUAWEI

Deploying a Service Required for Accessing Jenkins

```
apiVersion: v1
kind: Service
metadata:
  name: jenkins-service
  namespace: devops
  annotations:
      prometheus.io/scrape: 'true'
      prometheus.io/path:   /
      prometheus.io/port:   '8080'
spec:
  selector:
    app: jenkins-server
  type: NodePort
  ports:
    - name: http
      port: 8080
      targetPort: 8080
      nodePort: 32000
    - name: agent
      port: 50000
      targetPort: 50000
      nodePort: 50000
```

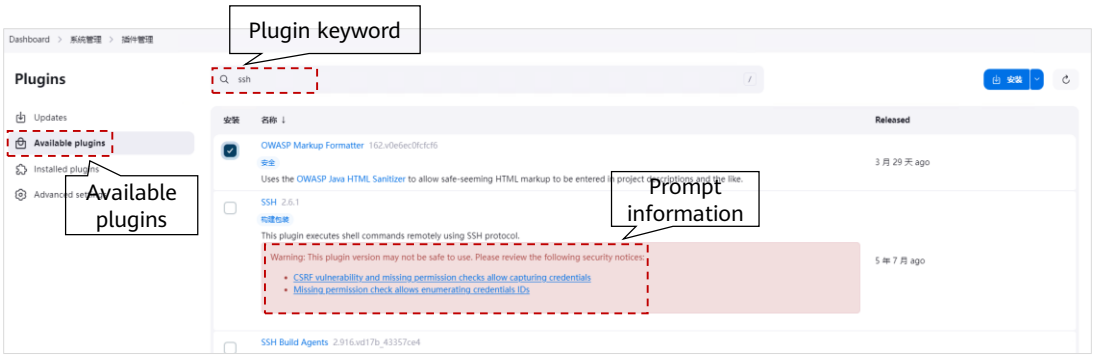

HUAWEI

Contents

1. Core Component of CI/CD – Jenkins
2. Installing and Deploying Jenkins
3. **Basic Jenkins Operations**
4. Dynamic Slave Deployment of Jenkins

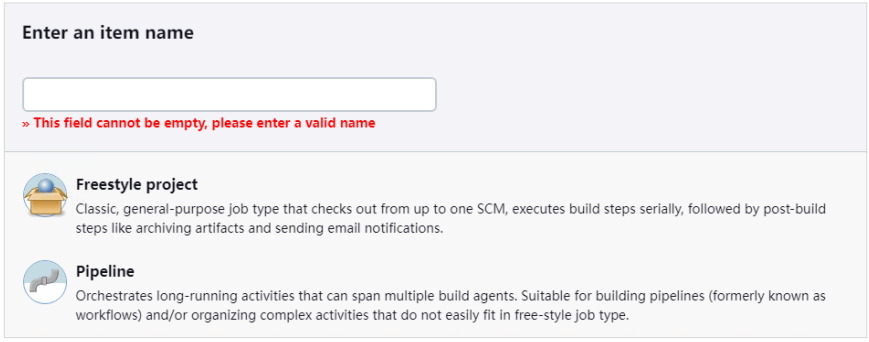
HUAWEI

Installing Jenkins Plugins



HUAWEI

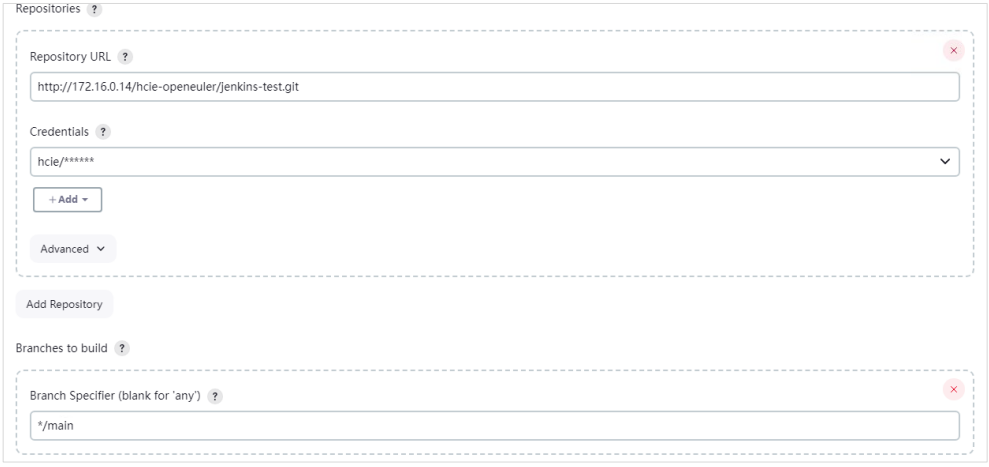
Creating a Project



HUAWEI

- Freestyle projects provide an open way to build projects to complete various types of tasks. Before pipelines appear, freestyle projects are considered the most flexible project types.
- Pipelines are one of the most important project types of Jenkins. They are used to integrate CI/CD pipelines into Jenkins so that Jenkins can automatically and continuously deliver specific software projects.

Project Settings – Source Code Management

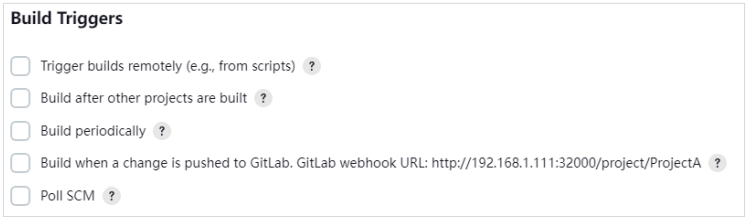


HUAWEI

- The source code in Jenkins can be obtained from Git, but Git plugins are required.
- Authentication is required when source code is pulled. The created authentication information is stored in credentials of Jenkins.

Project Settings – Build Trigger

- The Jenkins' build trigger is used to trigger automatic project execution. There are several ways to construct build triggers:
  - Build periodically
    - This mode uses the UNIX cron syntax, in which five fields are used to set the task period: *MINUTE HOUR DOM MONTH DOW*.
  - Poll SCM
    - This mode periodically queries code repositories to detect code changes. If the code changes, a build will be triggered.
  - Build after other projects are built



- Meaning of the five fields:
  - **MINUTE**: minute in each hour. The value ranges from **0** to **59**.
  - **HOUR**: hour in each day. The value ranges from **0** to **23**.
  - **DOM**: day in a month. The value ranges from **1** to **31**.
  - **Month**. The value ranges from **1** to **12**.
  - **DOW**: day in a week. The value ranges from **0** to **7**. The values **0** and **7** indicate Sunday.
- The Poll SCM mode periodically queries code repositories to detect code changes. If the code changes, a build will be triggered. The method of setting the interval is the same as that of "build periodically" mode.
- To use "Build after other projects are built" mode, you need to specify an upstream task of the current task. When the upstream task is executed, the downstream task is automatically triggered.

Project Settings – Build Environment

- In Jenkins, **Build Environment** is used to set and configure the environment required for application build. The environment can be configured through plugins or native methods.
- Functions of **Build Environment**:
  - Set environment variables.
  - Inject passwords or credentials.
  - Run scripts or commands.
  - Set file operation permissions.
  - Configure a build tool version.



Project Settings – Build Step

- A build step of Jenkins is a specific operation or task executed in an application build process, and is used to automatically complete work such as application build, test, and deployment. Specific procedure includes:
  - Code compilation
  - Running test
  - Application packaging
  - Static code analysis
  - Integrated deployment
  - Notification release

HUAWEI

- Code compilation: The build step can use a compiler to compile source code into an executable file, a library file, or an intermediate file in another form.
- Running test: The build step can execute test cases to verify the correctness and performance of the code. This includes various types of tests such as unit tests, integration tests, and end-to-end tests.
- Application packaging: The build step can package the built products as distributable formats (such as JAR, WAR, and Docker images) for further deployment and distribution.
- Static code analysis: The build step can perform static code analysis to check code style and identify potential errors and vulnerabilities, thereby enhancing code quality and maintainability.
- Integrated deployment: The build step can deploy the built products to the target environment, for example, deploy applications to the test server and production environment.
- Notification release: The build step can send notifications to team members and other related personnel by email or Slack.

Project Settings – Post Build

- Post-build operations of Jenkins are a supplement to application build, which are used to complete subsequent build tasks and improve the entire build process, including:
  - Release of built products
  - Notification of build results
  - Report generation
  - Cleanup
  - Integrated deployment

HUAWEI

- Release of built products: The built products are released in specified locations. For example, this operation can upload executable files, deployment packages, and documents to specified servers or repositories so that team members or systems can access and use them.
- Notification of build results: This operation can send build results, such as notifying team members and relevant personnel of the build status—whether it succeeded or failed—via email, Slack, or WeChat for subsequent actions.
- Report generation: After a build is complete, various reports are generated, such as test reports, coverage reports, and static code analysis reports, for subsequent analysis and decision-making.
- Cleanup: Clear resources such as temporary files and log files generated during the build process to ensure that the build environment is clean and resources are released.
- Integrated deployment: After the build is complete, perform some integrated deployment operations, for example, automatically deploy the build result to the test environment or production environment.

Jenkins Pipeline

- Pipeline is a key feature of Jenkins. It visualizes the entire project release process, enables quick fault detection during releases, and facilitates project deployment using a Jenkins file.
- The Jenkins pipeline is implemented using Groovy scripts, including:
  - **Pipeline**: defines a pipeline script.
  - **Agent**: specifies the position where steps and commands are executed in the entire pipeline or a specific phase. This part must be defined at the top layer of the pipeline block or can be redefined in a stage. However, the stage level is optional.
  - **Stages**: defines all phases of a task.
  - **Stage**: defines a stage of a task.
  - **Steps**: defines one or more steps executed in a given stage instruction.

HUAWEI

Jenkins Use Case

- Background: An enterprise uses Go to develop an application, which is accessed in HTTP mode. After the development is complete, engineers compile and package the application and deploy it on a service server. To ensure service availability, services are deployed in a cluster mode, with Nginx providing load balancing.
- Use the following code to simulate the application:

```
package main

import (
        "fmt"
        "net/http"
        "os"
)

func main() {
        http.HandleFunc("/", hostnameHandler)
        http.ListenAndServe(":8085", nil)
}

func hostnameHandler(w http.ResponseWriter, r *http.Request) {
        hostname, err := os.Hostname()
        if err != nil {
                fmt.Fprintf(w, "Error getting hostname: %v", err)
        } else {
                fmt.Fprintf(w, "Hostname: %s\n", hostname)
        }
}
```


HUAWEI

- For details, see section 8.4.1 in the *Lab Guide*.

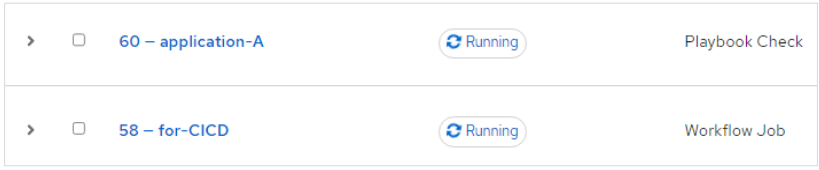
Implementation Logic of Service CI/CD

- Step 1: Write code.
  - The code includes the application code (written with Go) and the code (shell script) required for application deployment. The developed files need to be pushed to GitLab.
- Step 2: Compile and package code.
  - Complete application compilation.
- Step 3: Send code.
  - Send the compiled and packaged code to the business service.
- Step 4: Deploy the application.
  - Run the service.
- Step 5: Deploy Nginx.
  - Deploy Nginx as planned to provide load balancing for service clusters.

HUAWEI

Role of AWX in This Practice

- AWX in Jenkins CI/CD is used to automatically install the environment required for application running.
- In this practice, AWX is used to automatically install and deploy Nginx services. Therefore, Nginx is installed on a specified host using a playbook, and related network ports are enabled.



HUAWEI

Pipeline Script for Project Deployment

```
pipeline {
    agent any
    stages {
        stage('pull code') {                    ##### Pull the application script.
            steps {
                .......
            }
        }
        stage('application compilation') {                    ##### Compile and package the application.
            steps {
                .......
            }
        }
        stage('delete the old application') {                    ##### Delete and overwrite the old application.
            steps {
                .........
            }

        stage('stop the running application') {                ##### The application of the old version occupies the corresponding port. You need to stop it before
running the application of the new version.
            steps {
                .......
            }
        }
        stage('send the application') {                    ##### Send the compiled and packaged application to the application server.
            steps {
                ........
            }
        }
        stage(send startup script') {                ##### Start the application of the new version.
            steps {
                .......
            }
        }
        stage('call AWX'){                    ##### Call AWX to deploy the Nginx service.
            steps {
                ......
            }
        }
    }
}
```


HUAWEI

Contents

1. Core Component of CI/CD – Jenkins
2. Installing and Deploying Jenkins
3. Basic Jenkins Operations
4. **Dynamic Slave Deployment of Jenkins**

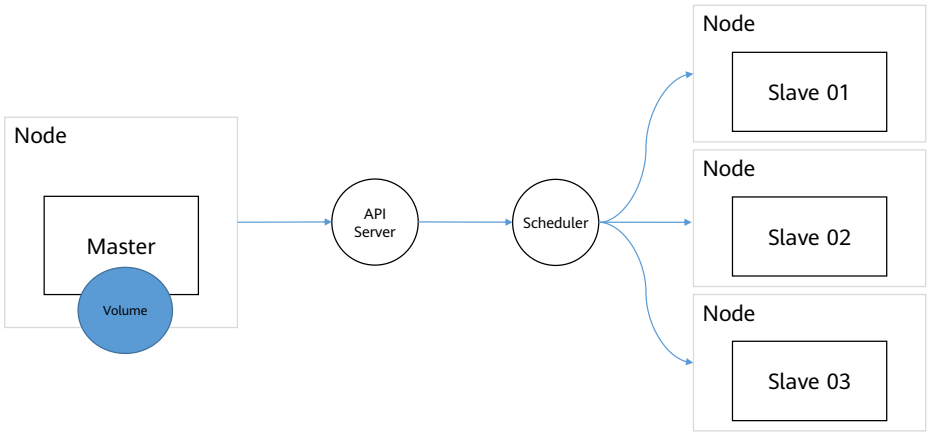
HUAWEI

Disadvantages of Jenkins Static Slaves

- Single point of failure risks
  - Once the master is faulty, the CI/CD process is terminated.
- Difficult to cope with complex and differentiated application build environments
  - A standard slave cannot support multiple application development languages at the same time.
- Uneven task assignment
  - Multiple tasks may wait in a queue for a busy slave, but an idle slave cannot be assigned to tasks and is always in the idle state.
- Resource waste
  - When no task is processed, a slave still occupies a large number of resources.

HUAWEI

Kubernetes-Based Dynamic Slave Implementation Logic



HUAWEI

- Slaves are automatically deleted when a task is complete, to release resources.

Key Parameters of Kubernetes

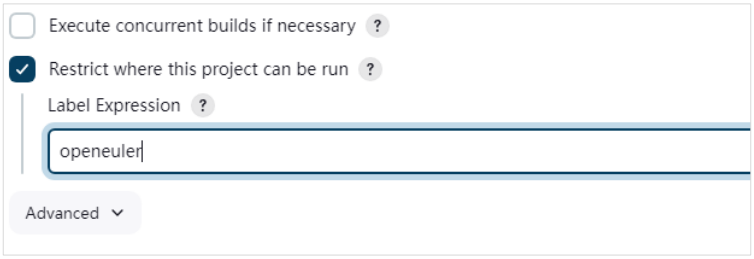
- Kubernetes address: Kubernetes API server address, which can be obtained from **.kube/config**.
- Kubernetes service certificate key: corresponds to the content of **certificate-authority-data** in **.kube/config**.
- Credentials can be in multiple forms, including usernames, passwords, key files, and keys.
- Jenkins address: address of Jenkins in a cluster, which can be obtained based on the actual name or namespace.
- Pod label: label of the pod where a slave is located. This label is used to collect the number of pods and limit the number of concurrent pods. The default value is **Jenkins=slave**.
- Pod template: configuration of the pod that carries a slave.
  - **Name**: name of a pod template, which is used to identify different slave pods. In a pipeline, different pod templates can be called based on their names.
  - **Namespace**: namespace where a slave pod runs. It is recommended that its value be the same as the namespace where the master resides.
  - **Label list**: slave label. Jenkins allocates different types of tasks to the corresponding slave based on labels.
  - **Container list**: information about containers in a pod.
    - **Name**: The name is customized but cannot be set to **jnlp**.
    - **Volume**: persistent storage information mounted to a pod.
    - **Service Account**: SA name of the master.

HUAWEI

- The API server obtained from **.kube/config** is the external address of Kubernetes. If Jenkins is deployed in Kubernetes, you can directly use the internal address.
- The container name in a slave pod cannot be set to **jnlp** because the slave agent name in the pod is **jnlp** by default. If the container name is set to this name, a conflict occurs.

Slave Configuration in a Free Style Project

- In a free-style project, Jenkins specifies a slave to execute a task by setting **Label Expression** under **Restrict where this project can be run**.



Configuration Call by a Slave in a Pipeline

```
podTemplate {
    node('openeuler') {
        stage('Run shell') {
            sh 'echo hello openeuler'
        }
        stage('Run kubectl') {
            sh 'kubectl version'
        }
    }
}
```

```
Started by user admin
[Pipeline] Start of Pipeline
.....
Agent hcie-openeuler-0n1fx is provisioned from template hcie-openEuler
---
apiVersion: "v1"
kind: "Pod"
metadata:
  labels:
    jenkins: "slave"
    jenkins/label-digest: "dccec450eba251e2c807d2ca3f9c4bc652684d18"
    jenkins/label: "openeuler"
  name: "hcie-openeuler-0n1fx"
  namespace: "devops"
spec:
  containers:
  .......
    image: "swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/slave:1.0"
    name: "jnlp-test"
.......
 image: "jenkins/inbound-agent:3192.v713e3b_039fb_e-1"
    name: "jnlp„
Running on hcie-openeuler-0n1fx in /home/jenkins/agent/workspace/test5
hello world
+ kubectl version
Client Version: version.Info{Major:"1", Minor:"23", GitVersion:"v1.23.1", .......
```


HUAWEI

Running Multiple Containers in a Slave

- Multiple containers can be defined in the same slave pod and share resources with each other.
- Different instructions can be executed for different containers.
- You need to run the shell command as the user "jenkins" to execute containers in the slave pod. If the user does not exist, customize a container image.
- If the user "jenkins" does not have the permission to execute other applications, you need to add the corresponding permission to the user.

```
podTemplate(containers: [
    containerTemplate(name: 'maven', image: 'maven:3.8.1-jdk-8', command: 'sleep', args: '99d'),
    containerTemplate(name: 'golang', image: 'golang:1.16.5', command: 'sleep', args: '99d')
  ]) {
    node(POD_LABEL) {
        stage('Get a Maven project') {
            git 'https://github.com/jenkinsci/kubernetes-plugin.git'
            container('maven') {
                stage('Build a Maven project') {
                    sh 'mvn -B -ntp clean install'
                }
            }
        }
        stage('Get a Golang project') {
            git url: 'https://github.com/hashicorp/terraform.git', branch: 'main'
            container('golang') {
                stage('Build a Go project') {
                    sh '''
                    mkdir -p /go/src/github.com/hashicorp
                    ln -s `pwd` /go/src/github.com/hashicorp/terraform
                    cd /go/src/github.com/hashicorp/terraform && make
                    '''
                }
            }
        }
    }
}
```


HUAWEI

- Source of cases: https://github.com/jenkinsci/kubernetes-plugin
- For example, in the docker in docker scenario, the user "jenkins" cannot run the docker command. To solve this problem, you need to customize a container image and add the user "jenkins" to the supplementary group corresponding to docker.

Introduction to Harbor

- Harbor is an open source container image repository used to store and distribute container images. It provides a secure and scalable platform for setting up a local container image repository.
- Harbor can be deployed in Kubernetes or containers.
- Harbor components include:
  - Proxy
  - Registry
  - Database
  - Core services
  - Job services
  - Log collector

HUAWEI

Jenkins Dynamic Slave Case

- Case description: Compile the application developed by Go and deploy it in Kubernetes in deployment mode.
- Procedure
  - Interconnect Jenkins with Kubernetes.
  - Prepare code.
  - Compile code.
  - Create a container image.
  - Deploy the application.

Slave Image Creation

- The Go language image does not have the user "jenkins". You need to manually create the user and the required home directory. When creating the user, set the user ID to **1000**.
- When creating a container in a Jenkins slave, you need to add the docker permission to the user "jenkins". Therefore, you need to manually create a docker user group and set the supplementary group of the user "jenkins" to **docker**.
- During service deployment, the user "jenkins" must have the Kubernetes permission. Therefore, you need to upload the Kubernetes authentication file **.kube/config** to the corresponding directory and specify the configuration file as this file when creating the Kubernetes resource object.

```
FROM swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/golang:1.17.3

RUN mkdir /home/jenkins \
    && useradd -u 1000 -d /home/jenkins \
    && chown 1000:1000 /home/jenkins
USER Jenkins
```

```
FROM swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/jenkins-docker:1.0

USER root
RUN echo "docker:x:987:jenkins" >> /etc/group
USER Jenkins
```


HUAWEI

Pipeline Configuration for Implementing the Case

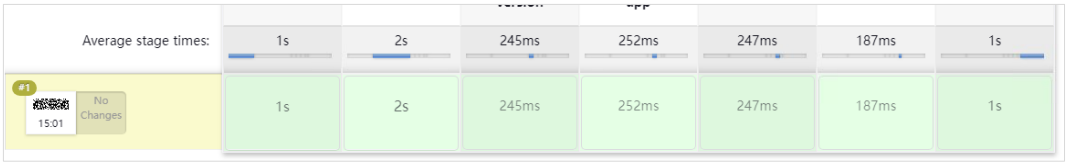
```
podTemplate(
    containers: [
        containerTemplate(name: 'golang', image: '172.16.0.14:8088/hcie-openeuler/golang:1.17.3', command: 'sleep', args: '99d'),
        containerTemplate(name: 'docker', image: '172.16.0.14:8088/hcie-openeuler/jenkins-docker:1.0', command: 'sleep', args: '99d')],
        volumes: [hostPathVolume(hostPath: '/run/docker.sock', mountPath: '/run/docker.sock', readOnly: false),
                  hostPathVolume(hostPath: '/usr/bin/kubectl', mountPath: '/usr/bin/kubectl', readOnly: false)]
) {
    node(Pod_LABEL) {
        stage('Generate an application.') {
            container('golang') {
                stage('Pull code.') {
                    git branch: '${tag}', credentialsId: '0ce4ba97-2d29-4cc3-bcc3-1d5e11022019', url: 'http://172.16.0.14/hcie-openeuler/jenkins-test.git'
                }
                stage('Compile code.') {
                    sh 'go build web.go'
                }
            }
        }
        stage('Deploy the application.'){
            container('docker') {
                stage('Generate an image and push it.') {
                    sh 'docker build -t ${HarborHost}/${harborRepo}/test:${tag} . && docker login -u ${harborUser} -p ${harborPasswd} ${HarborHost} && docker
push ${HarborHost}/${harborRepo}/test:${tag}'
                }
                stage('Deploy a deployment.') {
                    sh 'kubectl --kubeconfig ./config apply -f deploy.yml'
.....
```

Implement docker in docker and kubectl in docker.

Specify a container.

HUAWEI

Case Implementation Result




```
The recommended git tool is: NONE
using credential 0ce4ba97-2d29-4cc3-bcc3-1d5e11022019
.....
Cloning the remote Git repository
Cloning repository http://172.16.0.14/hcie-openeuler/jenkins-test.git
 .......
Commit message: "1.2"
First time build. Skipping changelog.
```

```
+ docker build -t 172.16.0.14:8088/hcie-openeuler/test:main .
.......
Sending build context to Docker daemon  6.312MB

Step 1/5 : FROM 172.16.0.14:8088/hcie-openeuler/openeuler:22.03lts
.....
Successfully built 3b66d0340afa
Login Succeeded
+ docker push 172.16.0.14:8088/hcie-openeuler/test:main
.........
```

```
+ kubectl --kubeconfig ./config apply -f deploy.yml
namespace/jenkins unchanged
deployment.apps/applicationa unchanged
service/applicationa unchanged
ingress.networking.k8s.io/applicationa unchanged
```


HUAWEI

Quiz

1. (True or False) In Jenkins' static slave mode, the number of slaves is capped, while in dynamic slave mode, there is no upper limit on the number of slaves.
   A. False
   B. True

HUAWEI

- A

Summary

- This course covers Jenkins and common CI/CD scenarios using Jenkins. By the end of this course, you will have mastered the implementation of CI/CD in automation management environments.

HUAWEI

Acronyms and Abbreviations

Acronym/Abbreviation	Full Spelling
CI	continuous integration
CD	continuous delivery/continuous deployment

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



OpenStack Cluster Management



Foreword

- This course describes the Kolla automation tool for deploying OpenStack in containers and integrating with external storage systems, along with managing OpenStack services.

Objectives

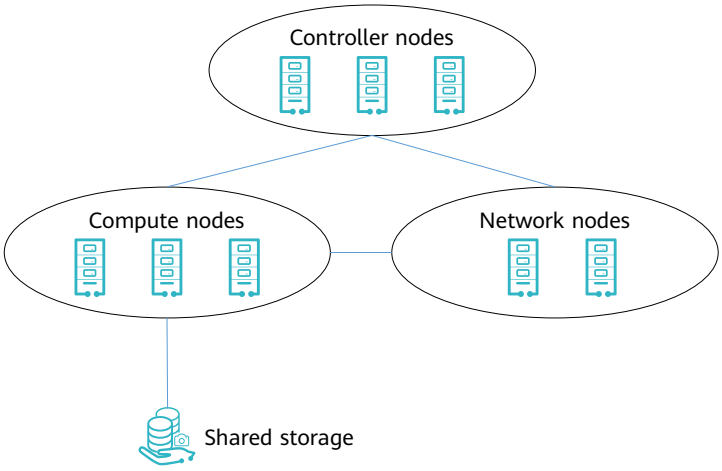
- On completion of this course, you will understand:
  - Usage of the Kolla automation tool
  - OpenStack storage resource management
  - OpenStack network resource management
  - OpenStack service management

Contents

1. **Kolla Introduction**
2. OpenStack Resource Management
3. OpenStack Service Management

HUAWEI

OpenStack Cluster Structure



HUAWEI

- An OpenStack cluster includes multiple control nodes for service active/standby redundancy or load balancing. User and management data has transitioned from local storage to shared storage. Multiple network nodes are deployed to enable redundancy.
- OpenStack supports both centralized and distributed shared storage.
- If distributed shared storage is employed, storage nodes can be merged with compute nodes for hyper-convergence.

OpenStack Cluster Deployment Tools

- Tools for deploying OpenStack clusters include:
  - DevStack
  - Kolla
  - oos
  - RPM
  - OpenStack Helm

HUAWEI

- Kolla, based on Docker and Ansible, is an OpenStack container deployment solution comprising two projects: Kolla and Kolla Ansible.
- DevStack now supports installing OpenStack on openEuler, validated on openEuler 20.03 LTS SP2 with upstream official CI providing quality assurance. For other openEuler versions, user testing is required (validated on the openEuler master branch as of April 25, 2022).
- **oos** is a command line tool provided by the openEuler OpenStack SIG. The **oos env** commands leverage an Ansible script to offer push-button deployment of OpenStack (all-in-one or multi-node cluster). The script will quickly deploy an OpenStack environment using RPM packages provided by openEuler.
- OpenStack Helm enables allows for OpenStack component components on Kubernetes. This project offers Helm Charts for various OpenStack components and a series of scripts for installation.

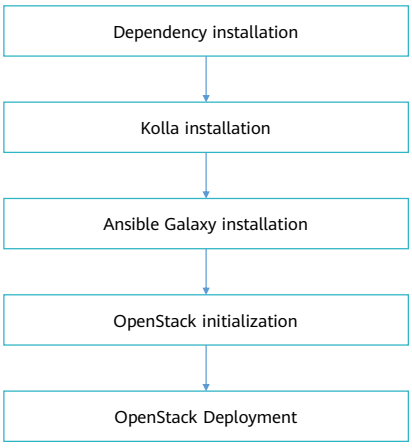
Kolla

- Kolla, based on Docker and Ansible, is an OpenStack container deployment solution comprising two container creation tools: Kolla and Kolla Ansible.
- Key advantages:
  - Simplified OpenStack deployment with a consistent process
  - Easy scaling and performance enhancement for OpenStack clusters based on containers
  - Customizable configurations satisfy diverse deployment needs
  - Component containerization that enhances system reliability and stability, ensuring swift restarts on failure
  - Active community backing with technical support, documentation, and tutorials

HUAWEI

OpenStack Deployment Process

- OpenStack deployment process using Kolla



HUAWEI

- Kolla integrates many tools to automated the entire deployment process.
- Dependencies include virtual environment dependencies and Kolla dependencies.
- This course mainly describes OpenStack initialization.

OpenStack Initialization – Kolla Host Files

- Kolla offers separate host file templates for all-in-one and multi-node deployment modes.
  - All-in-one deployment runs all OpenStack services on a single machine, ideal for simpler environments like development, testing, and learning. However, its performance and scalability are restricted.
  - Multi-node deployment distributes OpenStack services across multiple machines, providing better performance and scalability. This setup involves separate control and compute nodes, allowing flexibility to scale with added hardware and management effort.

HUAWEI

Kolla Host Files

- The Kolla host file maps main OpenStack roles to specific hosts:
  - Control: API and database services on an odd number of nodes
  - Network: Neutron agents and HAProxy or Keepalived.
  - Compute: Elastic Cloud Servers (ECSs) of users
  - Storage: storage components like cinder-volume, LVM, and Swift
  - Monitoring: services for monitoring various hosts
- Other roles are associated with the main roles for deployment on designated nodes.

HUAWEI

OpenStack Initialization – Configuration File

- Kolla provides the **globals.yml** configuration file, which includes:
  - Global configurations
  - OpenStack service installation schedule
  - Service configurations

HUAWEI

- The **global.yml** file includes options for components like Kolla, Docker, Keepalived, OpenStack, RabbitMQ, Neutron, external Ceph, Keystone, and Glance. These options collectively determine the deployment of OpenStack services by Kolla.

Global Network Parameters

- kolla_internal_vip_address
  - IP address for management traffic. If Keepalived is set up, it is the Keepalived floating address. If not, it is the address of **network_interface** of the management node.
- network_interface
  - Interface for management traffic.
- neutron_external_interface
  - Interface for Neutron to connect to external networks.
- neutron_plugin_agent
  - Type of Neutron plugin agent, supporting **openvswitch**, **ovn**, **linuxbridge**, and more.

HUAWEI

- The type of **neutron_external_interface** can be **vlan** or **flat** based on the specified network type during creation.

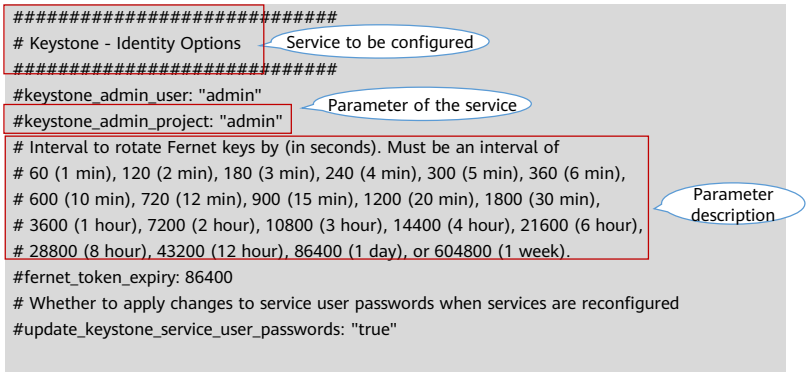
OpenStack Service Installation Schedule

- In **globals.yml**, you can schedule the services to be installed in either of the following ways:
  - Directly set the parameter to **yes** for the desired service, like installing the cinder service by setting **enable_cinder** to **yes**.
  - Use conditional logic to determine service installation based on specifications for other services. For instance, setting **enable_openstack_core** to **yes** will install Neutron if its configuration condition **enable_neutron: "{{ enable_openstack_core | bool }}"** is fulfilled.

HUAWEI

Service Configurations

- Service configurations refer to the specific parameters for individual services.
- These configurations reflected in **globals.yml** as follows:



Service Configuration Example

```
......
enable_cinder: "yes"
enable_cinder_backup: "yes"
......
########################
# External Ceph options
########################
......
ceph_cinder_user: "cinder"
ceph_cinder_keyring: "client.{{ ceph_cinder_user }}.keyring"
ceph_cinder_pool_name: "volumes"
ceph_cinder_backup_user: "cinder-backup"
ceph_cinder_backup_keyring: "client.{{ ceph_cinder_backup_user }}.keyring"
ceph_cinder_backup_pool_name: "backups"
################################
# Cinder - Block Storage Options
################################
cinder_backend_ceph: "yes"
.........
```


HUAWEI

Service Configurations and Logs

- Kolla maps important data of services to the host using **VolumeMount**. Configuration files are mapped to the **/etc/kolla/***service_name* directory on the host, while log files are mapped to the **/var/log/kolla/***service_name* directory on the host. For example, the mapping for Horizon-related data is as follows:

```
{
            "Type": "bind",
            "Source": "/etc/kolla/horizon",
            "Destination": "/var/lib/kolla/config_files",
            "Mode": "ro",
.....
},
        {
            "Type": "volume",
            "Name": "kolla_logs",
            "Source": "/var/lib/docker/volumes/kolla_logs/_data",
            "Destination": "/var/log/kolla",
            "Driver": "local",
.....
}
```


HUAWEI

c(1

Contents

1. Kolla Introduction
2. OpenStack Resource Management
   - **Compute Resource Management**
   - Storage Resource Management
   - Network Resource Management
3. OpenStack Service Management

HUAWEI

Addition of Compute Nodes

- In OpenStack, compute resources are primarily delivered by compute nodes.
- Kolla-Ansible supports addition of compute nodes with the following steps:
  - Add information about the new nodes to related configuration files, including the host inventory and **hosts** on deployment nodes.
  - Use the **kolla-ansible** commands with the **--limit** *host* option to confine operations to the specified node during node preparation, image pulling, and deployment. For instance, the following commands add **compute02** as a compute node:

```
kolla-ansible -i multinode bootstrap-servers --limit compute02
kolla-ansible -i multinode pull --limit compute02
kolla-ansible -i multinode deploy --limit compute02
```


HUAWEI

Removal of Compute Nodes

- OpenStack allows you to remove compute nodes as required.
- To remove a compute node:
  - Stop the nova-compute service on the node.

    ```
    openstack compute service set <host> nova-compute --disable
    ```

  - Migrate all server instances on the node.

    ```
    openstack server list --all-projects --host <host> -f value -c ID | while read server; do
    openstack server migrate --live-migration $server
    done
    ```

  - Stop all services on the node.

    ```
    kolla-ansible -i <inventory> stop --yes-i-really-really-mean-it [ --limit <limit> ]
    ```

  - Remove the node from the host inventory.

HUAWEI

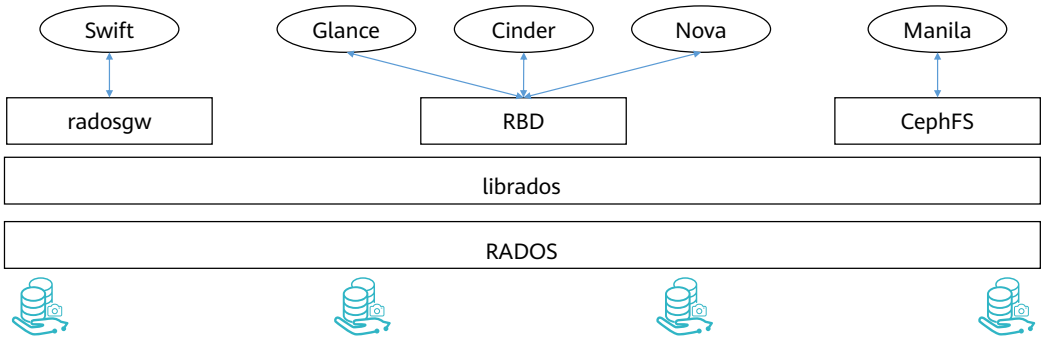
Contents

1. Kolla Introduction
2. **OpenStack Resource Management**
   - Compute Resource Management
   - **Storage Resource Management**
   - Network Resource Management
3. OpenStack Service Management

HUAWEI

OpenStack Storage Resources

- OpenStack offers storage services like block, file, and object storage for tenants and internal services. Ceph, an open source unified storage system, provides all three types of storage within a single cluster.



Ceph Information for OpenStack

- Ceph provides the following information to OpenStack:
  - Ceph configuration file
  - Keyring for the resource pool
  - Storage pool name
- This information can be specified through the Kolla configuration file. Kolla will synchronize the information to OpenStack during deployment or upgrade. Alternatively, you can manually add the information after installation.

HUAWEI

- The Ceph configuration file provides the Ceph Monitor (MON) information, cluster ID, and authentication method.
- The Keyring for the resource pool is used by OpenStack components for authentication with Ceph.
- If OpenStack is deployed through Kolla, Kolla is recommended for updating the information when new storage pools are added.

Ceph Information for Glance

- Create the **/etc/kolla/config/glance** directory on the Kolla deployment node and create **ceph.conf** in it.
- Save the keyring file of the authorized user for the resource pool in the same directory.
- If the Glance backend corresponds to multiple RBD resource pools, declare them in **global.yml**.

```
#####ceph.conf format
[global]
fsid = 1d89fec3-325a-4963-a950-xxxxxxx
keyring = /etc/ceph/ceph.client.glance.keyring
mon_initial_members = ceph-0
mon_host = 192.168.0.56
auth_cluster_required = cephx
auth_service_required = cephx
auth_client_required = cephx
```

```
##### Declaring Glance backend resource pools
glance_ceph_backends:
  - name: "rbd"
    type: "rbd"
    cluster: "ceph"
enabled: "{{ glance_backend_ceph | bool }}"
- name: "another-rbd"
  type: "rbd"
  cluster: "rbd1"
  enabled: "{{ glance_backend_ceph | bool }}"
```


HUAWEI

Ceph Information for Cinder

- On the Kolla deployment node, create the **/etc/kolla/config/cinder** directory and within it, create the **cinder-volume** and **cinder-backup** directories for storing information about Cinder backend and backup storage pools, respectively.
- Nova also needs to store information about Cinder backend resource pools for OpenStack compute nodes to access them.
- Save **the ceph.conf** file in both **/etc/kolla/config/cinder/cinder-volume** and **/etc/kolla/config/cinder/cinder-backup**.
- Save the keyring file for Cinder resource pools in **/etc/kolla/config/cinder/cinder-volume**, **/etc/kolla/config/cinder/cinder-backup**, and **/etc/kolla/config/nova/**. Save the keyring file for Cinder backup resource pools in **/etc/kolla/config/cinder/cinder-volume**.
- If the Cinder backend corresponds to multiple RBD resource pools, declare them in **global.yml**.

HUAWEI

- For the **ceph.conf** file format and how to declare Glance backend resource pools, see the previous page.

Ceph Information for Manila

- On the Kolla deployment node, create the **/etc/kolla/config/manila** directory and within it, save the **ceph.conf** file and the keyring file for Manila to access CephFS.
- Manila can access CephFS using the native or NFS interface of Ceph. Specify the interface to use when declaring the resource pool.

```
manila_ceph_backends:
    - name: "cephfsnative1"
      share_name: "CEPHFS1"
      driver: "cephfsnative"
       cluster: "ceph"
       enabled: "{{ enable_manila_backend_cephfs_native | bool }}"
       protocols:
          - "CEPHFS"
    - name: "cephfsnfs1"
       share_name: "CEPHFSNFS1"
       driver: "cephfsnfs"
       cluster: "ceph1"
       enabled: "{{ enable_manila_backend_cephfs_nfs | bool }}"
       protocols:
          - "NFS"
          - "CIFS"
```


HUAWEI

Example: Ceph RBD Pool as Cinder Backend Storage

- Requirements: Two Ceph resource pools, **volumes** and **openeuler**, store tenant cloud drives of the **__DEFAULT__** and **openeuler** types, respectively.
- Implementation procedures:
  - Create RBD resource pools **volumes** and **openeuler** in Ceph and create associated users.
  - Import the user keyring file to Kolla Ansible.
  - Create Ceph configuration files corresponding to the resource pools so that Cinder can connect to MON.
  - Declare the resource pools in the **global.yml** configuration file of Kolla.

HUAWEI

Implementation Procedures (1)

- After the keyring file and Ceph configuration file are imported to Kolla Ansible, files in related directories are as follows:

```
(venv) [root@controller ~]# ls /etc/kolla/config/cinder/
ceph.conf  cinder-backup  cinder-volume  openeuler.conf
(venv) [root@controller ~]# ls /etc/kolla/config/cinder/cinder-volume/
ceph.client.cinder.keyring  ceph.conf  openeuler.client.openeuler.keyring  openeuler.conf
(venv) [root@controller ~]# ls /etc/kolla/config/cinder/cinder-backup/
ceph.client.cinder-backup.keyring ceph.conf  openeuler.client.openeuler.keyring  openeuler.conf
(venv) [root@controller ~]# ls /etc/kolla/config/nova
ceph.client.cinder.keyring  openeuler.client.openeuler.keyring  test.txt
```


HUAWEI

Implementation Procedures (2)

- The section for declaring the **volumes** and **openeuler** storage pools in **global.yml** is as follows:

```
################################
# Cinder - Block Storage Options
################################
# Enable / disable Cinder backends
cinder_backend_ceph: "yes"
cinder_ceph_backends:
  - name: "rbd-1"
    cluster: "ceph"
    enabled: "{{ cinder_backend_ceph | bool }}"
  - name: "openeuler"
    cluster: "openeuler"
    enabled: "{{ cinder_backend_ceph | bool }}"
```


HUAWEI

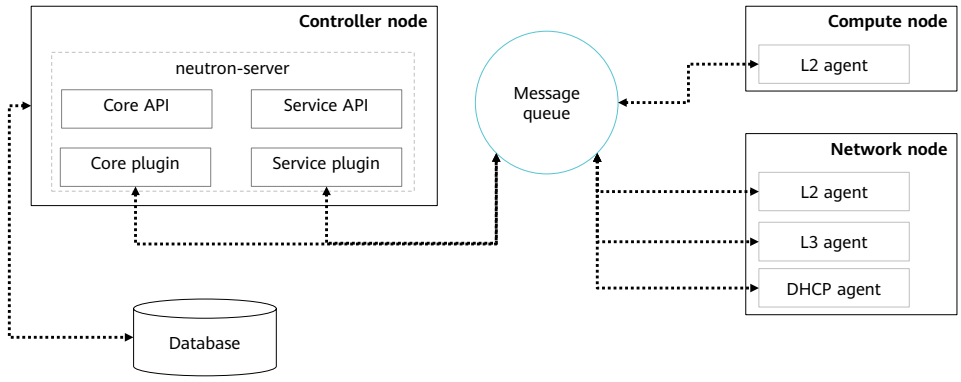
Contents

1. Kolla Introduction
2. **OpenStack Resource Management**
   - Compute Resource Management
   - Storage Resource Management
   - **Network Resource Management**
3. OpenStack Service Management

HUAWEI

OpenStack Network Resources

- OpenStack network resources, provisioned upon tenant application, facilitate communication between services and with external networks.



HUAWEI

- External networks include the Internet and local area networks (LANs) outside the cloud network.

Network Plugins

- OpenStack network plugins enable different network services. The core plugin, Modular Layer 2 (ml2), manages layer 2 network resources, including networks, subnets, and ports.
- OpenStack supports various core plugins, such as Open vSwitch (OVS), ovn, linuxbridge, and plugins for third-party network devices.
- OVS is recommended for small-scale private clouds, while ovn is suitable for large-scale hybrid or public clouds.

HUAWEI

Network Resource Types

- From the perspective of origins and usage, OpenStack classifies network resources into:
  - Project Network (also known as self-service network): used for internal services within a project or tenant, requiring a virtual router for external communication, usually created by the tenant administrator.
  - Provider network:
    - Layer 2 provider network: used to connect and map to existing layer 2 networks of data centers.
    - Routed provider network: used to map to existing layer 3 networks of data centers, including multiple layer 2 segments corresponding to the layer 2 provider network.

HUAWEI

- Existing networks generally refer to physical networks.

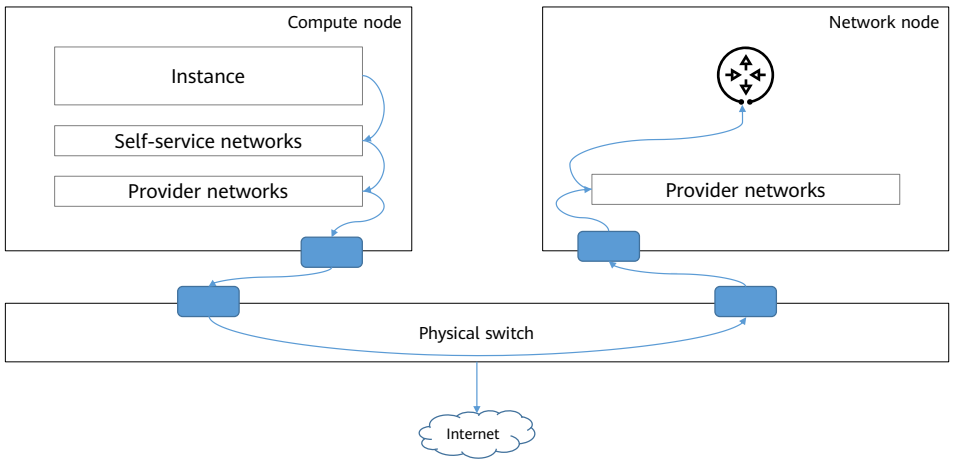
Network Resource Types

- From the perspective of isolation technologies, OpenStack classifies network resources into:
  - Flat
    - Instances are not isolated share the same network as the host.
  - Virtual local area network (VLAN)
    - Networks are isolated by VLANs on the physical network.
  - Generic routing encapsulation (GRE) and virtual extensible local area network (VXLAN)
    - Networks are isolated through tunnels as both GRE and VXLAN are tunneling encapsulation protocols.

HUAWEI

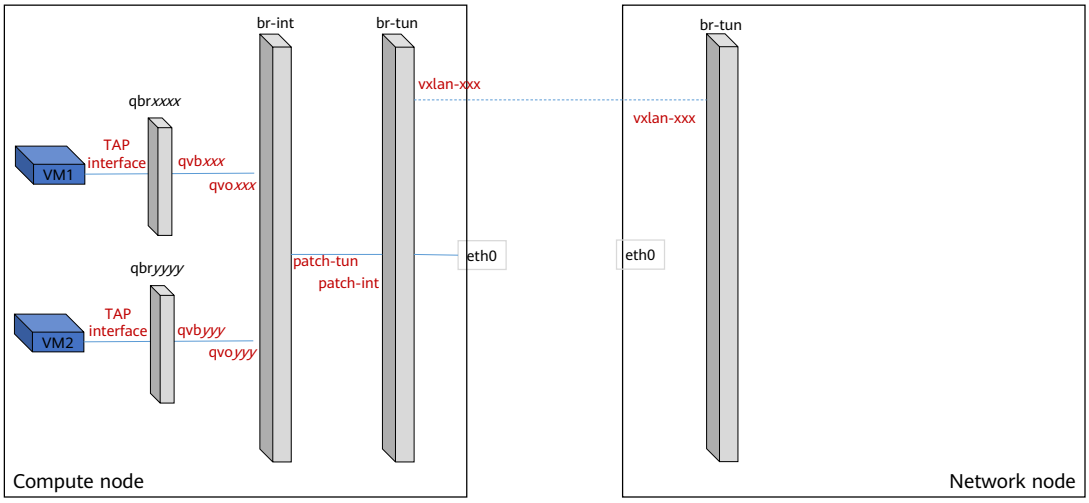
- The following practices are recommended in cloud computing:
  - Project networks are isolated through VXLAN tunnels.
  - Provider networks correspond to physical networks. For example, if a provider network is set to the VLAN type, VLAN configuration needs to be added to the physical switch. The same applies if the provider network is set to VXLAN.
- Both GENEVE and VXLAN encapsulate Ethernet frames within UDP packets, but GENEVE adds type-length-values (TLVs) to its header to indicate extensible data.
- By default, ovn uses GENEVE for network isolation.

OpenStack Network Traffic



HUAWEI

OVS-based Network Structure



HUAWEI

- This page shows only the overlay structure.

Basic OVS Operations

- Check OVS information: **ovs-vsctl show**

```
[root@compute01 ~]# docker exec -it openvswitch_vswitchd /bin/bash
(openvswitch-vswitchd)[root@compute01 /]# ovs-vsctl show
e33340ab-bc20-4933-bc9d-560976130a58
    Manager "ptcp:6640:127.0.0.1"
        is_connected: true
    Bridge br-int
        Controller "tcp:127.0.0.1:6633"
            is_connected: true
        fail_mode: secure
        datapath_type: system
        Port br-int
            Interface br-int
                type: internal
        Port qvo863a9c4c-a2
            tag: 4
            Interface qvo863a9c4c-a2
        Port patch-tun
```

- List all interfaces of a bridge: **ovs-vsctl list interface**

```
(openvswitch-vswitchd)[root@compute01 /]# ovs-vsctl list interface
_uuid               : 505f8678-a3fb-41ef-a77f-8ea2679f2695
admin_state         : up
bfd                 : {}
bfd_status          : {}
cfm_fault           : []
cfm_fault_status    : []
cfm_flap_count      : []
cfm_health          : []
cfm_mpid            : []
cfm_remote_mpids    : []
cfm_remote_opstate  : []
```


HUAWEI

OVS Port Types

- Common OVS port types include:
  - Normal: physical network interfaces recognized and mounted to OVS without configurable IP addresses.
  - Internal: virtual network interfaces created internally within OVS. When a bridge is created in OVS, a corresponding internal port with the same name is automatically created.
  - Patch: virtual ports that connect OVS bridges. Patch ports usually appear in pairs, each located on the connected bridges.
  - Tunnel: virtual ports that encapsulate and decapsulate traffic using tunneling technologies like GRE and VXLAN.

HUAWEI

Configurations for OVS-based Network Resources

- Set up the Neutron backend before configuring networks on the UI.

```
######/etc/neutron/plugins/ml2/ml2_conf.ini
[ml2]
type_drivers = flat,vlan,vxlan
tenant_network_types = vxlan
mechanism_drivers = openvswitch,l2population
extension_drivers = port_security

[ml2_type_vlan]
network_vlan_ranges =

[ml2_type_flat]
flat_networks = physnet1

[ml2_type_vxlan]
vni_ranges = 1:1000
```

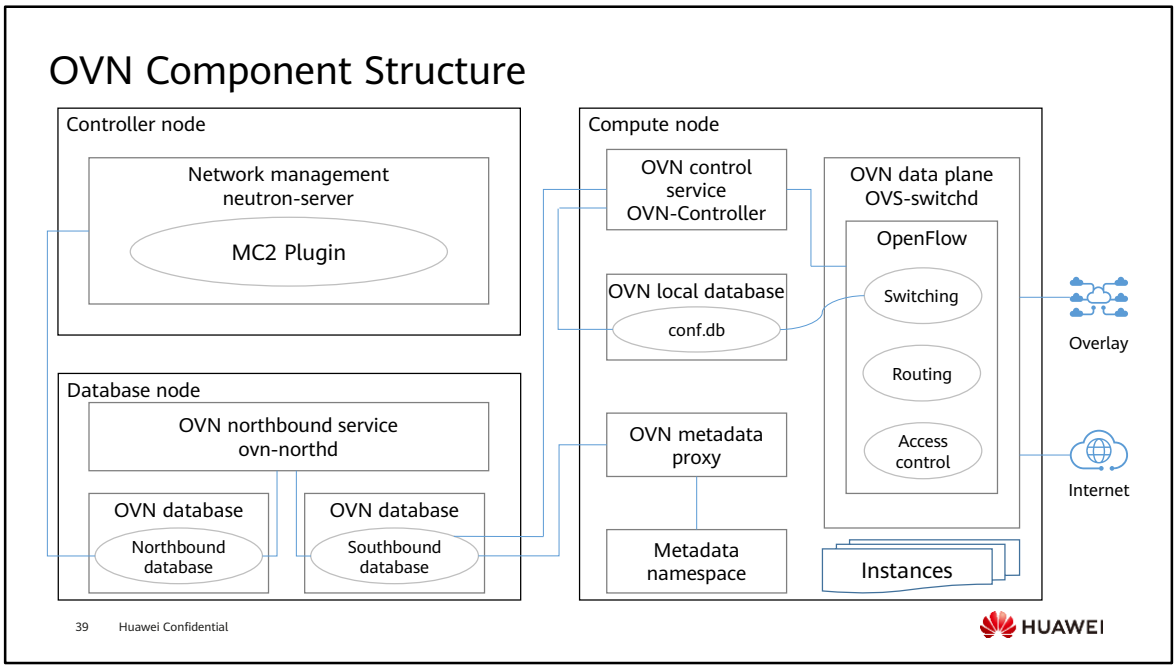
```
######/etc/neutron/plugins/ml2/openvswitch_agent.ini
[agent]
tunnel_types = vxlan
l2_population = true
arp_responder = true

[securitygroup]
firewall_driver = neutron.agent.linux.iptables_firewall.OVSHybridIptablesFirewallDriver

[ovs]
bridge_mappings = physnet1:br-ex
datapath_type = system
ovsdb_connection = tcp:127.0.0.1:6640
ovsdb_timeout = 10
local_ip = 192.168.1.133
```

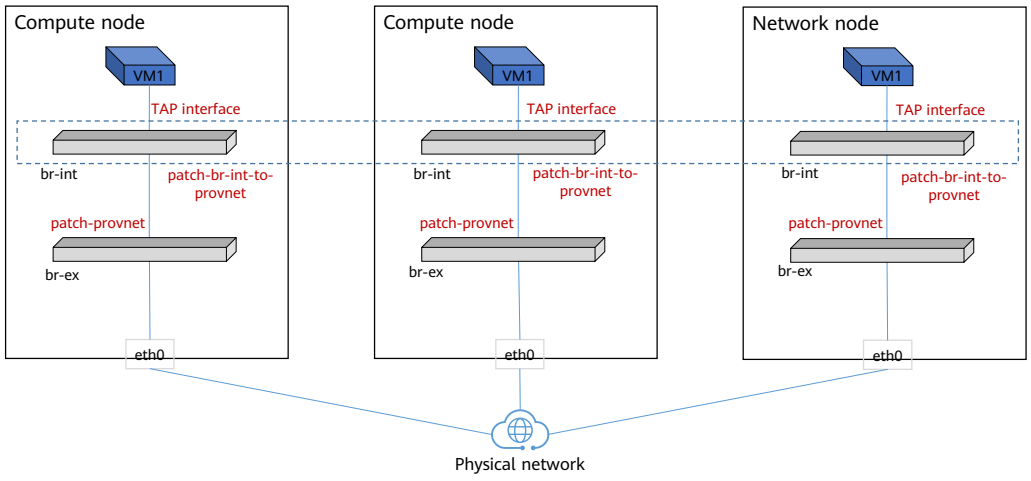

HUAWEI

- In **ml2_conf.ini**: **type_drivers** specifies the types of supported provider networks. **tenant_network_types** specifies the types of networks available for projects. If the provider network is set to **vlan**, you need to specify the physical network name and VLAN ID range in **network_vlan_ranges**. For example, **openstack:10:100** means the physical network is **openstack** and the range is 10 to 100.
- In **openvswitch_agent.ini**: **bridge_mappings**, the main configuration, maps the physical network in **ml2_conf.ini** to the actual network. For example, **physnet1:br-ex** means the **physnet1** physical network is mapped to the **br-ex** bridge, thus connecting the virtual network and physical network.



- The northbound database defines the network structure, while the southbound database specifies flow table configurations for implementing this structure.
- OVN-Controller on each compute node executes the implementation from the southbound database, acting as the agent in OVN similar to the ovs-agent in Neutron.

OVN-based Network Structure



HUAWEI

- This page shows only the overlay structure.
- The **br-int**'s of all compute nodes are connected through the GENEVE tunnel to form a large, distributed virtual switch.
- OVN supports distributed virtual routing. By setting **enable_neutron_provider_networks: yes** in the **global.yml** file, compute nodes directly connect to the physical network. This automatically creates a **br-ex** bridge on each compute node. When a network with an associated router is created in OpenStack, a pair of patch ports is established between **br-int** and **br-ex**. Instances destined for the physical network are forwarded directly through **br-ex**.

Basic OVS Operations – Northbound Database Information

- View the network structure: **ovn-nbctl show**
- List logical switches: **ovn-nbctl list Logical_Switch**
- List logical switch ports: **ovn-nbctl list Logical_Switch_Port**
- List logical routers: **ovn-nbctl list Logical_Router**
- List router ports: **ovn-nbctl list Logical_Router_Port**
- List network address translation (NAT) rules: **ovn-nbctl list NAT**
- List gateway nodes: **ovn-nbctl list Gateway_Chassis**

HUAWEI

Basic OVS Operations – Southbound Database Information

- List network virtual tunnel endpoint gateway (VTEP) gateways: **ovn-sbctl list Chassis**
- List network tunnels: **ovn-sbctl list Encap**
- List address sets: **ovn-nbctl list Address_Set**
- List connections: **ovn-sbctl list Connection**

HUAWEI

Configurations for OVN-based Network Resources

- Set up the Neutron backend before configuring networks on the UI.

```
######/etc/neutron/plugins/ml2/ml2_conf.ini
[ml2]
type_drivers = flat,vlan,vxlan,geneve
tenant_network_types = geneve
mechanism_drivers = ovn
extension_drivers = port_security
.......
[ml2_type_geneve]
vni_ranges = 1001:2000
max_header_size = 38
[ovn]
ovn_nb_connection = tcp:192.168.1.131:6641
ovn_sb_connection = tcp:192.168.1.131:6642
ovn_metadata_enabled = True
enable_distributed_floating_ip = False
ovn_emit_need_to_frag = True
```

```
######/etc/neutron/plugins/ml2/ovn_agent.ini
[ovn]
ovn_nb_connection = tcp:192.168.1.131:6641
ovn_sb_connection = tcp:192.168.1.131:6642

[ovs]
ovsdb_connection = tcp:127.0.0.1:6640
ovsdb_timeout = 10
```


HUAWEI

- The configuration files are read and interpreted by the neutron_server process.
- **ml2.conf** defines the connection methods for the OVN northbound and southbound databases and the support for GENEVE, the default tenant network type in OVN.
- In OVN, OVN-controller running on each node replaces the original agent. You only need to specify **mechanism_drivers** as **ovn** in the ml2 configuration file.
- In non-controller nodes, **neutron_ovn_agent** is responsible for connecting the OVN and OVS databases.

Contents

1. Kolla Introduction
2. OpenStack Resource Management
3. **OpenStack Service Management**
   - **Image Creation**
   - ECS Metadata
   - Shared Storage

HUAWEI

Guestfish

Zero-startup operations

Guestfish allows **direct** manipulation of file systems within a VM image without requiring VM startup. This is particularly useful for repairing corrupted VMs, inspecting files within VMs, and performing data recovery operations.

Flexibility

Guestfish enables versatile operations across **diverse file systems**, including viewing, editing, and copying files, executing commands, and more. This flexibility enhances the efficiency of working with VM images.

Integration

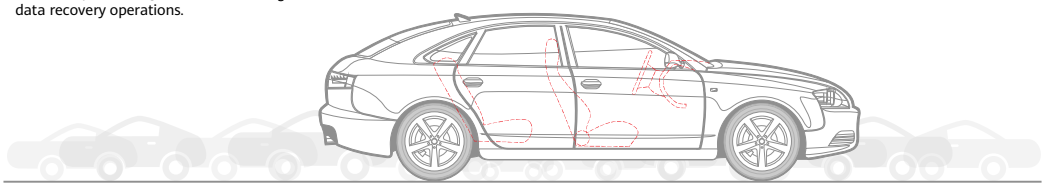
Guestfish is a part of the libguestfs toolkit, facilitating seamless integration with other libguestfs tools and APIs for an **effective** and **comprehensive** approach to managing VM images.

Extensibility

Guestfish provides a comprehensive suite of features and commands, supporting multiple file systems and VM image formats to cater to varying use cases.

Automation

As Guestfish is command line and scripting-friendly, it can be effortlessly integrated into **automation** workflows, automating VM image management operations.



HUAWEI

- Guestfish is a tool for manipulating VM images, part of the libguestfs toolkit. libguestfs is a set of utilities and libraries for managing VM images, providing a set of commands and an API to manipulate VM images directly.

Common Guestfish Options

- Common **guestfish** options used for editing images include:
  - **-a**: adds a drive image.
  - **-i**: automatically mounts file systems.
  - **-w**: mounts file systems as read/write.
  - **-r**: mounts file systems as read-only.
  - **--network**: enables network functionality.

Guestfish Commands

- After entering an image using **guestfish**, you can run two types of commands to edit the image:
  - Guestfish built-in commands, including **touch**, **vi**, and **mkdir**
  - **command** keyword for running external commands, for example, **command "yum install -y vim"**
- Example: Create directory **/etc/test** within an image, create file **test.conf** within that directory, and write content.

```
><fs> mkdir /etc/ceph
libguestfs: error: mkdir: /etc/ceph: File exists
><fs> mkdir /etc/test
><fs> touch /etc/test/test.conf
><fs> vi /etc/test/test.conf
><fs> command "cat /etc/test/test.conf"
this is a test
```


HUAWEI

Contents

1. Kolla Introduction
2. OpenStack Resource Management
3. **OpenStack Service Management**
   - Image Creation
   - **ECS Metadata**
   - Shared Storage

HUAWEI

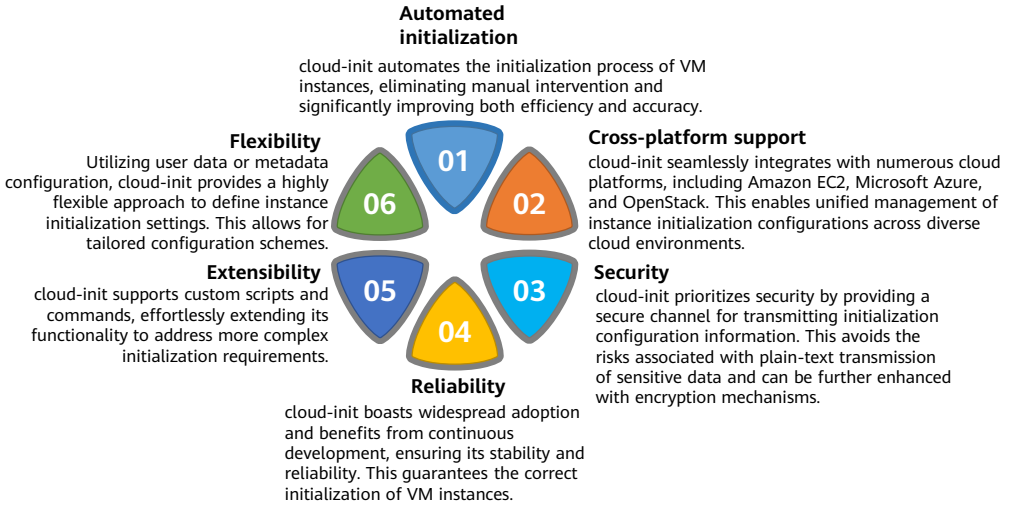
ECS Metadata

- ECS metadata **includes**:
  - Host name
  - Password
  - Network configuration
  - User data
- ECS metadata can be defined using **cloud-init**.

HUAWEI

- The host name and password are specified during ECS creation.
- User data can be modified through commands, shell scripts, and files.

cloud-init



HUAWEI

- cloud-init integrates with various cloud and virtualization platforms, including Amazon EC2, Microsoft Azure, and OpenStack. Upon VM instance startup, cloud-init reads user-provided configuration data and executes the specified initialization tasks. This automation ensures that VM instances are configured and launched precisely as required.
- Typically, users provide configuration information to cloud-init as user data or metadata. This data, often in specific formats, is seamlessly passed to the VM instance during the startup process. cloud-init parses this information and performs the initial setup accordingly.

Four Stages of cloud-init

- cloud-init customizes VM instances through a four-stage process, each handled by a dedicated service:
  - Local executed by cloud-init-local.service
  - Init executed by cloud-init.service
  - Config executed by cloud-config.service
  - Final executed by cloud-final.service

HUAWEI

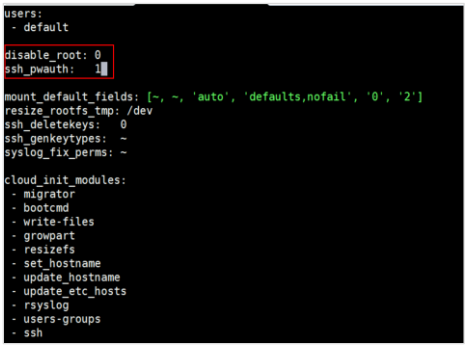
Stages of cloud-init

- The local stage is the first stage executed by cloud-init. At this point, the VM instance does not know how to configure its network interfaces. The task of cloud-init is to obtain configuration information from the config drive and write it to the **/etc/network/interfaces** file (or **/etc/sysconfig/network-scripts/ifcfg-xxx** for CentOS).
- If no config drive is present, all network interfaces default to DHCP mode. This is a crucial step, as the VM instance can only retrieve metadata after the network interfaces are properly configured.
- Under normal circumstances, the network configuration is complete and metadata retrieval is successful before the init, config, and final stages are executed. The cloud-init configuration file (**/etc/cloud/cloud.cfg**) defines the tasks to be executed in each of these three stages as modules.

HUAWEI

VM Management Using cloud-init – Password Management (1)

- After installing up Cloud-init, modify the **/etc/cloud/cloud.cfg** configuration file to enable password-based remote login for the **root** user.



HUAWEI

- Press **i**, set **disable_root** to **0** and **ssh_pwauth** to **1**.

VM Management Using cloud-init – Password Management (2)

- Prevent cloud-init from automatically disabling password-based logins by setting **lock_passwd** to **False**.

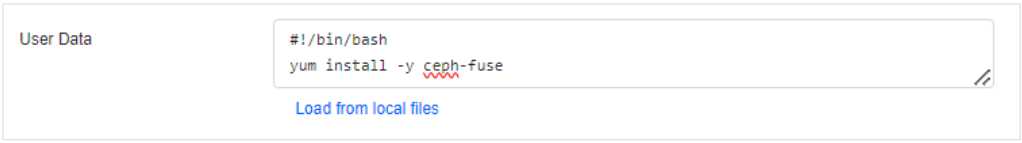
```
system_info:
  default_user:
    name: euleros
    lock_passwd: False
    gecos: Cloud User
    groups: [wheel, adm, systemd-journal]
    shell: /bin/bash
  distro: rhel
  paths:
    cloud_dir: /var/lib/cloud
    templates_dir: /etc/cloud/templates
  ssh_svcname: sshd

# vim:syntax=yaml
-- INSERT --
```


HUAWEI

VM Management Using cloud-init – Shell Script Execution

- During ECS creation, you can enter shell scripts to be executed in the **User Data** area.



HUAWEI

Contents

1. Kolla Introduction
2. OpenStack Resource Management
3. **OpenStack Service Management**
   - Image Creation
   - ECS Metadata
   - **Shared Storage**

HUAWEI

Manila

- Manila, an open source shared file system service born from the OpenStack community, empowers users to effortlessly share and access file storage within their OpenStack cloud environments.
  - **File system sharing**: Manila enables users to create and mount shared file systems across multiple VM instances, fostering seamless file sharing and collaboration.
  - **Versatile backend storage**: Integrated with a wide array of storage backends like NFS and CIFS, Manila grants users the freedom to select the optimal storage solution for their needs.
  - **Dynamic scalability**: Manila empowers users to dynamically create, expand, and fine-tune their shared file systems, ensuring optimal capacity and performance.
  - **Access control**: The robust access control of Manila allows for granular permission management, safeguarding data integrity and security.
  - **OpenStack integration**: As a core OpenStack project, Manila seamlessly integrates with other services such as Nova and Cinder, forging a cohesive and comprehensive cloud computing ecosystem.

HUAWEI

Basic Concepts of Manila

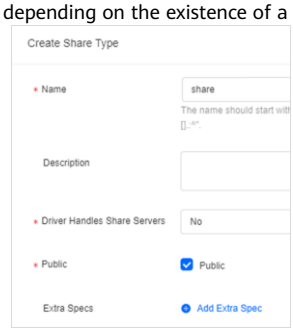
- Share: basic resource unit.
- Storage pool: Starting from OpenStack Kilo, the backend resources for Manila come from storage pools.
- Share type: abstraction of share characteristics, typically used for service levels.
- Share access rule: defines which users can access a share.
- Security service: authentication server, currently supporting AD, LDAP, and Kerberos.
- Share network: defined within the project where the share is created, associated with the network and subnet of the project. Share networks facilitate instance access to shares and incorporate security services for authentication protocols. Shares can leverage share networks to provide shared file systems to non-OpenStack services.
- Share server: transparent to users, conceptually similar to StorageClass of Kubernetes.

HUAWEI

- Share instance: A resource created on a designated backend, bound to a share. A share represents the abstraction while the instance embodies its realization. Usually, a one-to-one correspondence exists between them. However, a share can correspond to multiple instances during share migration or share disaster recovery.

Share Type Creation

- When creating a share type, you need to set the share server type.
  - If this parameter is set to Yes, the shared driver creates a shared server and manages the life cycle of the shared server.
  - If Shared Server is set to No, the administrator (rather than the shared driver) uses the network interface to manage bare metal storage without depending on the existence of a shared server.



HUAWEI

- If Share Server is set to No, you do not need to set the shared network.

File Share Creation

- When creating a file share, in addition to specifying the name and share type, you also need to set the share protocol. Manila supports the following share protocols:
  - NFS
  - CIFS
  - GlusterFS
  - HDFS
  - CephFS
  - MAPRFS

HUAWEI

- The specific protocols to use depend on the backend storage for Manila.

Shared Storage Mounting Using CephFS

- Grant necessary permissions and export the keyring file in Ceph.

```
ceph auth caps client.manila mon 'allow rwx' mgr 'allow rw'
ceph auth get-or-create --name client.manila --keyring /etc/ceph/ceph.client.manila.keyring client. client.test
>test.keyring
```

- Copy the generated keyring file and **ceph.conf** to the client VM, and mount the CephFS share using **ceph-fuse**.

```
ceph-fuse --name client.test --keyring manila/test.keyring --conf manila/ceph.conf --
client_mountpoint /volumes/_nogroup/volume_ID mount_directory
```

- Modify **/etc/fstab** for automatic mounting on system startup.

```
name=client.test,keyring=/root/manila/test.keyring,conf=/root/manila/ceph.conf,client_mountpoint=/
volumes/_nogroup/volume_ID /mnt/ceph fuse.ceph defaults,_netdev 0 0
```


HUAWEI

Quiz

1. (True or false) The all-in-one template of Kolla is used to deploy OpenStack container clusters.
   A. True
   B. False

HUAWEI

- B

Summary

- This course covers the core functionality of OpenStack components, exploring containerized deployments using Kolla, integration with external Ceph clusters, and network service configuration.

HUAWEI

Recommendations

- OpenStack official document:
  - https://www.openstack.org/
- openEuler official document:
  - https://www.openeuler.org/en/

HUAWEI

Acronyms and Abbreviations

- API (Application programming interface): A set of rules and specifications that allows different programs to communicate and interact with each other.
- CLI (Command line interface): A text-based interface for users to interact with computer programs. Users enter commands as text, which a program interprets and executes, with results displayed as text or graphics in the terminal.
- HTTP (Hypertext transfer protocol): The application-layer communication protocol used for transferring data between web servers and web browsers or other programs on clients.

HUAWEI

Acronyms and Abbreviations

- WSGI (Web server gateway interface): An interface between Python web applications or frameworks and web servers.
- Xen: An open source hypervisor using paravirtualization to manage CPU scheduling and memory allocation between VMs. It abstracts hardware but does not handle networking, storage, video, or other I/O.
- OVS (Open virtual switch or Open vSwitch): A production-quality, multilayer virtual switch under the Apache 2.0 license. It aims to automate configuration, management, and maintenance of large networks through programmability while supporting standard management interfaces and protocols.
- SNAT (Source network address translation): A mechanism where a router or firewall gateway translates the private IP address of an internal device to a public IP address when the device accesses the Internet. This allows devices sharing a single public IP to access the Internet.
- VLAN (Virtual local area network): A logical network built on a switched LAN using network management software, enabling communication across different subnets. It logically segments a physical LAN into multiple broadcast domains, allowing devices within a VLAN to communicate directly while isolating traffic between VLANs. This efficiently suppresses broadcast packets.

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

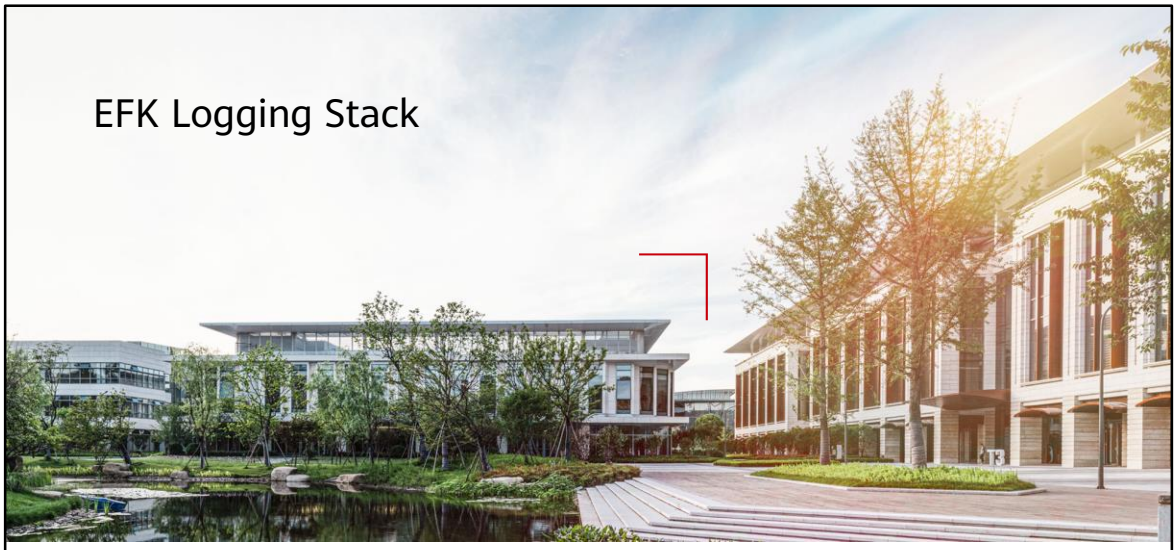
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



EFK Logging Stack



Foreword

- Monitoring and logging solutions are the cornerstones of the O&M infrastructure of any enterprise. Effective implementation of both hinges on addressing three key questions: how to collect, how to store, and how to visualize the data.
- While monitoring solutions are essential for identifying issues, they often fall short in pinpointing the root cause of complex failures in commercial environments. This is where logging solutions excel, providing the granular data needed to analyze and diagnose the underlying problem, ultimately leading to faster resolution.

HUAWEI

Objectives

- On completion of this course, you will understand:
  - Architecture of logging solutions
  - Common logging solutions, including their strengths and weaknesses
  - EFK logging solution
  - EFK installation and configuration in Kubernetes clusters
  - Basic functions and practical usage of EFK

HUAWEI

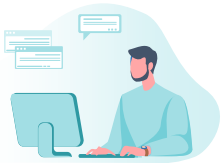
Contents

1. **Logging Solutions**
2. EFK Stack
3. EFK Installation on Kubernetes
4. Common EFK Functions

HUAWEI

Logs

- Logs are records generated by programs for developers to track program execution, detailing operations, errors, and system states at specific times and locations.
- Logs serve several crucial functions:
  - Troubleshooting: Logs provide real-time insights into system health, enabling swift identification and resolution of issues.
  - Data analysis: By analyzing service logs, organizations gain a holistic view of operations, including user demographics, geographic distribution, and popular resources, informing marketing and sales strategies.
  - Security and compliance: Logs are essential for meeting regulatory requirements, such as centralized storage and analysis of security device logs.
  - Internal threat detection: Analyzing user behavior logs is a common practice for monitoring intranet security and mitigating internal threats.
  - Intelligent O&M: In the era of big data, automated O&M relies on machine-generated data, including logs, to drive intelligent decision-making and streamline processes.



HUAWEI

Log Syntax

- Log files, regardless of format, adhere to a specific syntax akin to natural language grammar. Each log entry typically consists of several fields, including:
  - Timestamp
  - Entry type
  - Originating system or application
  - Severity, priority, or importance level
  - Associated operator or user
  - Log message (user actions, program results, etc.)



HUAWEI

Logging Framework



Log collection

Log storage

Log visualization

HUAWEI

Log Collection Methods

- Log collection methods fall into two categories:
  - Push: The client (log source device or application) actively sends logs to the log analysis system.
  - Pull: The log analysis system actively retrieves logs from the client.
- The pull method has significant limitations. Log analysis systems already consume substantial resources for processing large volumes of data, and pulling logs further exacerbates this issue. In contrast, the push method offers greater configurability and only consumes minimal resources on each client, resulting in superior performance.
- Common log collection methods include:
  - Agent-based collection
  - Syslog
  - Packet capture
  - API-based collection
  - Trace points in service applications
  - Container log collection



HUAWEI

Log Collection Methods – Agent-based Collection

- Agent-based collection involves deploying agents on client devices to actively push client logs.
- Popular open source log collection agents include Logstash and Filebeat.
- The usage of log collection agents is generally similar: install the agent on the client, configure the agent to specify the log files to be collected and the destination for sending the logs.
- When using agents for log collection, note the following:
  - Minimize permissions: Use a low-privilege user to start the agent, granting it read-only access to the logs to collect.
  - Limit script execution: While most agents can execute scripts or commands, minimize this usage.
  - Conserve system resources: When the client system resources are strained, temporarily pause log collection to ensure normal operation of the production system.
  - Reduce dependency on system libraries: Enterprise servers vary widely, with different underlying libraries. Excessive reliance on system libraries can lead to compatibility issues when agents are deployed at scale.

HUAWEI

- Agent-based log collection offers several advantages. Agents can send log data directly to log analysis systems or to other log processing components, which further process and forward the logs to the analysis system.

Log Collection Methods – Syslog

- In Linux, Syslog is the most common log collection method. It is a built-in service primarily used for system logs.
- A typical Syslog log format looks like this:

  ```
  <30> Dec 9 22:33:20 machine1 auditd[1834]:The audit daemon is exiting.
  ```

  - <30>: PRI (priority) section, a number enclosed in angle brackets
  - Dec 9 22:33:20 machine1: header section, containing the timestamp and host name
  - auditd[1834]: tag section, consisting of the process name and ID
  - The audit daemon is exiting: content section



- When using Syslog to send logs, consider the following:
  - Reliability: The default transmission method is UDP, which may result in log loss.
  - Buffer size: Adjust the transmission buffer size based on log volume, as a full buffer can also lead to log loss.
  - Large logs: Use TCP for logs exceeding 4 KB.

HUAWEI

- Most Linux OSs use Rsyslog instead of the original Syslog. Rsyslog is essentially an enhanced version of Syslog, with minor differences.
- Syslog-ng is gaining popularity as an open source alternative. It offers more powerful features and significantly higher sending rates than Rsyslog.

Log Collection Methods – Packet capture and API-based Collection

- Packet capture refers to the process of intercepting and recording packets on a specific network interface of a host or network device. By analyzing these captured packets, users can obtain valuable information, typically used for network troubleshooting.
- When it is necessary to retrieve internal program information or when log data cannot be stored locally, API-based collection is often employed. API-based collection requires custom development based on the specific data being collected. Different programs have varying internal mechanisms, resulting in different collection implementations. Pay attention to the following when performing API-based collection:
  - Avoid excessive collection frequency.
  - If the volume of logs is large, avoid retrieving the entire set of latest logs during each collection to minimize the impact on program performance.



HUAWEI

- Collecting logs through packet capture is uncommon due to the computational resources required for parsing captured packets, especially considering the typically large volume of log data. Compared to conventional log collection methods like agent-based collection, packet capture introduces unnecessary complexity. The strength of packet capture lies in capturing network traffic, commonly achieved by configuring mirror ports on switches and directing traffic to dedicated hardware for analysis.

Log Collection Methods – Trace Points in Service Applications

- Trace points involve injecting code into specific application workflows to collect relevant information.
- They are typically used to track application usage patterns, enabling continuous product optimization and providing data-driven insights for operations. Trace points gather information in two main areas:
  - User visits: This includes not only usage statistics (numbers of visits and visitors) but also path analysis, which maps user journeys through the various trace points within a product, aiding in optimization.
  - User behavior: By analyzing user actions based on demographics like gender, location, and age, services can deliver targeted content tailored to specific preferences.
- Two main approaches exist for implementing trace points:
  - Manual code injection: This offers greater data security but requires higher technical expertise. It is suitable for companies prioritizing data security and dealing with complex analysis scenarios.
  - Third-party tools: These provide mature technology and reliable analysis but may pose data security risks. They are ideal for companies focusing on data mining and product usability.



HUAWEI

- Example of trace point implementation: By embedding a trace point in an image, you can collect information about all users who click it. This data can then be analyzed to extract user characteristics, informing future marketing strategies.
- When implementing trace points, carefully consider their placement and methodology.

Log Collection Methods – Container log collection

- With the growing popularity of containers, container log collection has become a key concern for enterprises.
- Containers operate on a "multi-process + process isolation" principle, where a parent process launches child container processes. While the parent process can collect logs from its immediate children, it cannot access logs from deeper child processes. If a container has only one process, collecting logs is straightforward.
- However, with Kubernetes becoming the dominant container management platform, the presence of pods complicates log collection. Container engines alone cannot gather logs from application processes within pods. Two primary solutions address this:
  - Utilizing container engine APIs for log collection
  - Mounting log files from application processes and collecting them externally
- When collecting container logs, consider the following:
  - Implement log rotation to prevent excessive log file growth.
  - Avoid excessive log collection processes, especially one per container, as this wastes resources.
  - Ensure log collection does not consume excessive system resources.
  - If log collection and application processes share a container, carefully manage resource allocation to prevent the former from impacting the latter.

HUAWEI

- If the container engine is Docker, you can use Docker log drivers to collect logs.

Log Storage Methods – Database Storage

- Logs can be stored in databases:
  - Relational databases: Extract relevant information from log files as fields and store them in database tables. This allows easy retrieval, backup, and import of log data.
  - Non-relational databases: Store log files as key-value pairs or documents, which are easily utilized by object-oriented languages and enable fast queries in massive data.
- Advantages of database storage:
  - Ease of use: SQL queries provide convenient data manipulation.
  - Manageability: Databases offer access control and backup/recovery mechanisms.
  - Easy deployment: Existing databases can be used by simply adding log-specific tables or directories.
- Disadvantages of database storage:
  - Read/write overhead: Database operations can be time-consuming, impacting logging performance.
  - Query limitations: Large datasets or complex joins can slow down queries.
  - Deletion overhead: Deleting large volumes of historical logs can be time-consuming and may lead to large transactions.
- Suitable scenarios: products requiring log display for auditing or analysis; frontends requiring log data from backends for display; extracting common key information or performing correlation queries across multiple log types

HUAWEI

Log Storage Methods – Distributed Storage

- Single-machine capacity is often insufficient for storing large-scale logs, making distributed systems like Hadoop an ideal choice. Distributed systems offer unlimited capacity and scalability, significantly faster query and retrieval through optimized algorithms, and data safety through node replication.
- Advantages of distributed storage:
  - High scalability: Nodes can be easily added or removed to adjust capacity.
  - Massive storage capacity: The storage capacity typically ranges from gigabytes to petabytes.
  - High throughput: Parallel processing and streaming data access enable high throughput.
  - Fault tolerance: Node replication ensures data availability even in case of failures.
  - Low hardware requirements: Distributed storage systems can run on common commercial hardware.
- Disadvantages of distributed storage:
  - Secondary development: Limited native support in logging tools may require custom development for Hadoop integration.
  - Access latency: Optimized for high throughput, Hadoop may have higher access latency compared to other storage systems.
- Suitable scenarios: storage of very large log files; applications with less strict latency requirements
- Unsuitable scenarios: storage of numerous small files; applications requiring low-latency data; scenarios involving frequent multi-party read/write and arbitrary file modifications

HUAWEI

Log Storage Methods – File Retrieval System Storage

- For fast and efficient searching within large volumes of files, both query algorithms and file storage methods play crucial roles. Efficient storage can significantly improve query speed. Many search engines, like Elasticsearch, utilize inverted indexes. These indexes establish mappings between words and documents, organizing data around words rather than documents. With inverted indexes, searching involves directly looking up document indexes associated with keywords, returning all relevant documents quickly.
- Advantages of file retrieval systems:
  - Fast search: Inverted indexes significantly enhance search efficiency, especially with massive data.
  - Strong concurrency: Inverted indexes are lock-free, improving concurrent access.
  - Reduced overhead: Compression of inverted indexes saves CPU and drive I/O resources.
- Disadvantages of file retrieval systems:
  - Time-consuming index creation: Creating indexes for each keyword in every document can be time-consuming.
  - High maintenance cost: Modifying a document often requires rebuilding the index, increasing maintenance overhead.
- Suitable scenarios: storage and retrieval of petabyte-scale structured or unstructured data

HUAWEI

Log Storage Methods – Cloud Storage

- Cloud storage, deployed as clusters, leverages cloud computing and distributed file systems to enable collaboration among diverse storage devices, providing data storage and access services. Users can purchase or rent cloud storage from providers and access their data from anywhere with an Internet connection.
- Advantages of cloud storage:
  - Accessibility: Data can be conveniently accessed from any location with Internet connectivity.
  - Scalability: Storage capacity can be easily expanded by simply requesting expansion from the provider.
  - Cost reduction: For large-scale data storage and querying, cloud storage is often more cost-effective than on-premises solutions.
  - Disaster recovery: Cloud providers typically offer robust disaster recovery and backup capabilities.
- Disadvantages of cloud storage:
  - Network dependency: Access speed can be significantly affected by network conditions.
  - Security concerns: Data security may be less robust than on-premises solutions.
- While cloud storage can be deployed on public, private, or hybrid clouds, private clouds offer enhanced security. However, no cloud storage is entirely immune to risks. Users need to implement encryption and other security measures.

HUAWEI

Log Analysis and Visualization

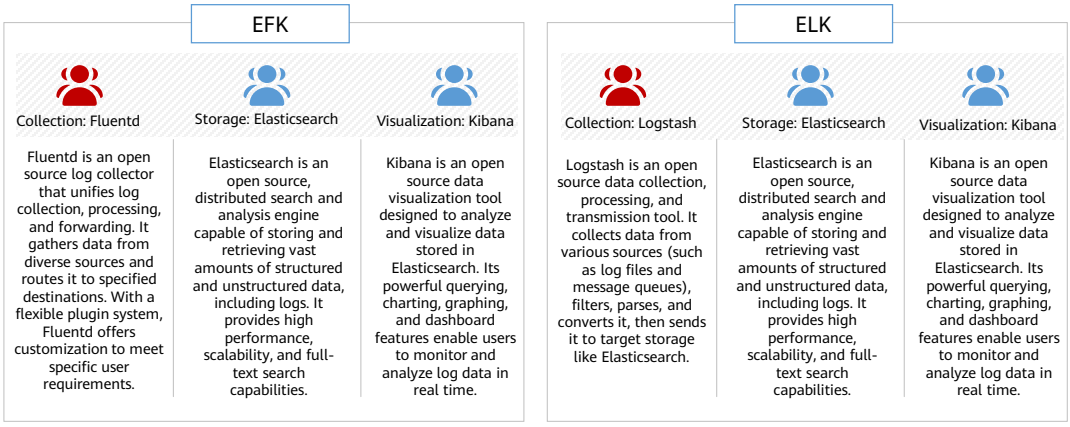
- Log analysis begins with centralizing all log data within an organization, addressing the issue of scattered logs. Then, custom parsing rules are defined to optimize analysis results. Common analysis methods include:
  - Clustering: Logs vary widely, even from the same device. Clustering automatically categorizes diverse events within logs.
  - Anomaly detection: Clustering and similar methods can quickly identify specific error codes. When service systems are monitored based on defined keywords, the appearance of a keyword may indicate a critical error, necessitating real-time log monitoring.
- Log visualization: This involves presenting logs in a more efficient, intuitive, clear, and convenient visual format. With diverse log types, visualization offers varied presentation styles. Real-time log data enables accurate analysis, and visualization can be combined with drill-down and search capabilities for easier data interaction. Two key concepts in log visualization are:
  - Data content: data to be displayed, such as temperature, precipitation, or network traffic data
  - Chart medium: type of chart used to represent the data



HUAWEI

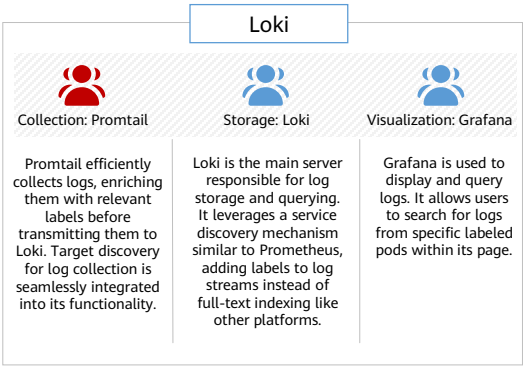
Common Logging Solutions (1)

- Common logging solutions include ELK (Elasticsearch, Logstash, and Kibana), EFK (Elasticsearch, Fluentd, and Kibana), Filebeat, and Loki.



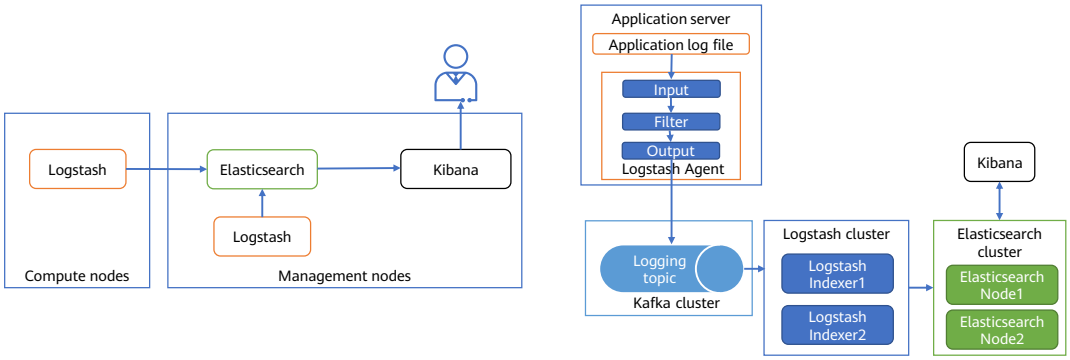
Common Logging Solutions (2)

- Filebeat, developed by Elastic, is a lightweight log transmission tool designed to collect and forward log data to central storage or analysis systems like Elasticsearch or Logstash. It monitors designated files and locations, reads log events, and transmits them to specified destinations. Filebeat supports various log formats and protocols while maintaining low resource consumption.
- Loki, an open source log aggregation system developed by Grafana Labs, focuses on storing, indexing, and querying log data. It utilizes stream processing and label indexing for efficient log querying and analysis. Tightly integrated with Prometheus, Loki supports the use of Prometheus Query Language (PromQL) for querying and filtering log data. It also provides tools and libraries for log collection and client endpoint management.



Comparison Between Common Logging Solutions – ELK

- The ELK stack is a well-established and popular log collection platform. Logstash collects and feeds logs into Elasticsearch for storage and indexing, while Kibana provides visualization and analysis tools.
- Disadvantages:
  - Logstash can be resource-intensive and has complex configuration syntax.



HUAWEI

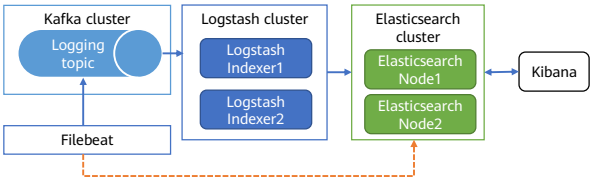
Comparison Between Common Logging Solutions – EFK

- Fluentd offers a streamlined logging solution as part of the EFK stack, addressing the resource-intensive nature and configuration complexity of Logstash. Unlike Logstash, Fluentd directly feeds logs into Elasticsearch for visualization in Kibana. However, the limitation of Fluentd lies in its ability to collect only console logs (accessible via the **logs** command), hindering its effectiveness in production environments where applications often generate non-console log files. While a workaround involves using sidecar containers to redirect file logs to the console (by running **tail -f**), this approach introduces significant management overhead.
- Advantages:
  - Fluentd features low resource consumption and simplified configuration syntax.
- Disadvantages:
  - Fluentd is limited to collecting console logs, unsuitable for capturing the full spectrum of logs in production environments.
  - The reliance on Elasticsearch increases maintenance complexity and resource consumption.



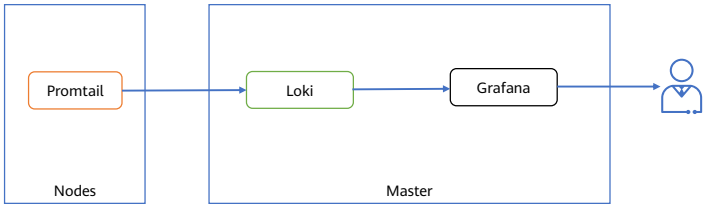
Comparison Between Common Logging Solutions – Filebeat

- In the ELK stack, Logstash is the primary log collector, responsible for both gathering and parsing log data. However, Logstash has a significant resource footprint, consuming considerable CPU, memory, and I/O resources. In contrast, Filebeat offers a lightweight alternative with negligible system resource usage.
- This lightweight nature makes Filebeat ideal for deployment as sidecar containers within Kubernetes pods, efficiently collecting custom log files generated by container applications. Additionally, Filebeat can be deployed as a DaemonSet across a Kubernetes cluster for gathering system logs and application console output.
- While Fluentd, Logstash, and Filebeat itself can directly output collected logs to Elasticsearch, a more robust and scalable approach involves utilizing Kafka as an intermediary buffer, which enhances log analysis and reduces Elasticsearch load.
- Advantages:
  - Log analysis is enhanced while the CPU and memory usage is reduced.
- Disadvantages:
  - The reliance on Elasticsearch increases maintenance complexity and resource consumption.



Comparison Between Common Logging Solutions – Loki

- While ELK, EFK, and Filebeat rely on Elasticsearch for data storage, which is known for its complexity and resource demands, Loki offers a lightweight alternative.
- Developed by Grafana Labs, Loki is a scalable, highly available, and multi-tenant log aggregation system.
- Instead of indexing the entire log content, Loki indexes only relevant metadata, resulting in simplified operations and reduced costs. Designed for Kubernetes, it seamlessly integrates with existing deployments to efficiently collect pod logs, automatically enriching them with Kubernetes metadata. This enables developers to quickly pinpoint specific logs using filters like namespaces and labels.
- Advantages:
  - Loki is lightweight, easy to use, and cost-effective.
- Disadvantages:
  - Loki has limited functionality compared to other logging solutions.



HUAWEI

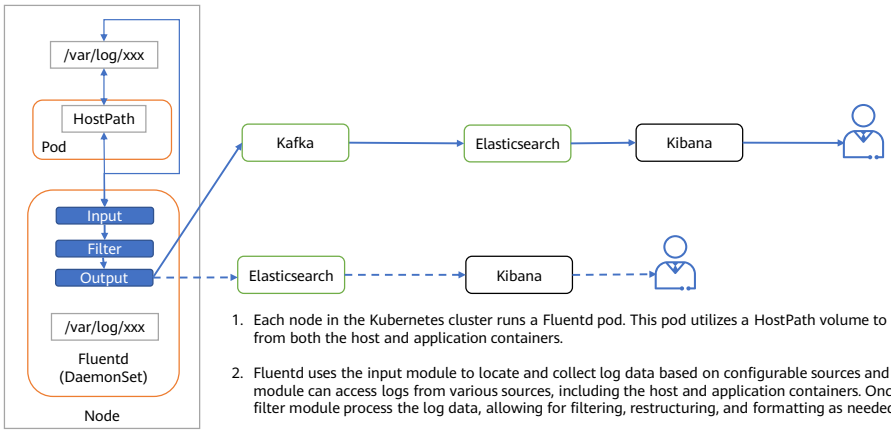
- Loki is the main server responsible for log storage and querying. It leverages a service discovery mechanism similar to Prometheus, adding labels to log streams instead of full-text indexing like other platforms.
- Promtail efficiently collects logs, enriching them with relevant labels before transmitting them to Loki. Target discovery for log collection is seamlessly integrated into its functionality.
- Grafana is used to display and query logs. It allows users to search for logs from specific labeled pods within its page.

Contents

1. Logging Solutions
2. **EFK Stack**
3. EFK Installation on Kubernetes
4. Common EFK Functions

HUAWEI

EFK Stack



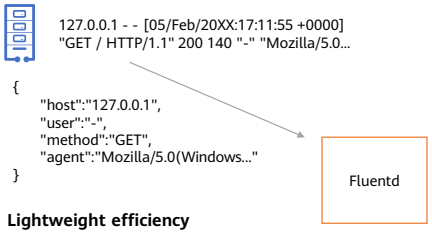
1. Each node in the Kubernetes cluster runs a Fluentd pod. This pod utilizes a HostPath volume to access log files from both the host and application containers.
2. Fluentd uses the input module to locate and collect log data based on configurable sources and match rules. This module can access logs from various sources, including the host and application containers. Once collected, The filter module process the log data, allowing for filtering, restructuring, and formatting as needed.
3. Fluentd uses the output module to send the processed log data to designated destinations, such as Elasticsearch clusters or Kafka queues, as defined in its configuration.

HUAWEI

- While Fluentd offers additional modules like parser, formatter, and buffer, this overview focuses on the core functionality of input, filter, and output for clarity.

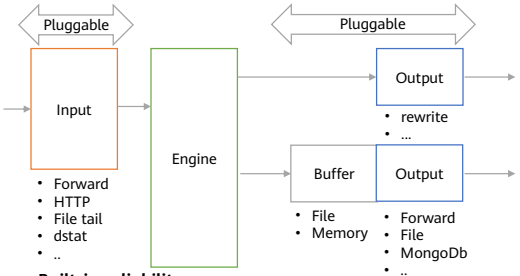
Fluentd Features

- **Unified logging with JSON**

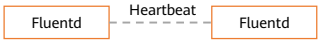


- **Lightweight efficiency**
- Built with C and Ruby, Fluentd operates with minimal resource consumption. Instances run smoothly on 30 to 40 MB of memory and can handle an impressive 13,000 events per second per core. For even tighter memory constraints (around 450 KB), Fluent Bit, a lightweight Fluentd forwarder, offers a compelling alternative.

- **Flexible plugin architecture**



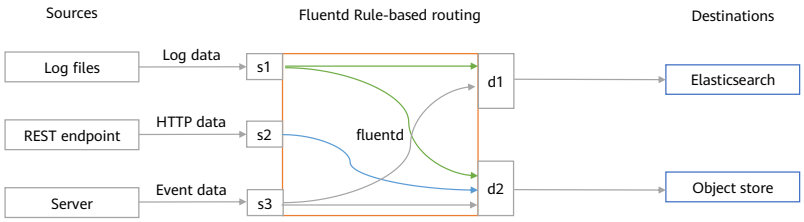
- **Built-in reliability**
- Data loss between nodes is mitigated through robust memory and file-based buffering. Fluentd ensures high availability with reliable failover mechanisms, keeping the log pipeline operational even under pressure.



HUAWEI

Fluentd Architecture

- Fluentd gathers logs from various sources, transforms them into a structured JSON format, and forwards them to designated destinations such as Elasticsearch, object storage, and over 300 other compatible services. The main steps are as follows:
  - Fluentd obtains log data from diverse sources.
  - Fluentd structures and tags the collected data.
  - Based on matched tags, Fluentd sends the processed log data to multiple destinations.



Elasticsearch Concepts (1)

- Elasticsearch is distributed document storage that does not organize information into rows and columns. Instead, it stores complex data structures serialized as JSON documents. In a multi-node Elasticsearch cluster, these documents are distributed across all nodes, ensuring accessibility from any node within the cluster.
- **Node**: A node is a single instance of Elasticsearch.
- **Cluster**: A cluster is a collection of Elasticsearch nodes working together.
- **Shard**: An index may hold massive data, potentially exceeding the hardware limitations of a single node. For example, an index for a billion documents can easily reach 1 TB, exceeding the storage capacity of a single node or making it too slow to handle search requests. To address this, Elasticsearch divides indexes into smaller components called shards. The number of shards is specified during index creation. Each shard functions as a self-contained index that can reside on any node within the cluster.
- **Replica**: Shards enhance data capacity by distributing data across multiple nodes. Replicas, which are complete copies of primary shards, further bolster availability and scalability. They handle overflow search requests during peak demand and ensure data redundancy by replacing failed primary shards.

HUAWEI

- After launching Elasticsearch on a server, you have a single node. Launching it on another server creates another node. You can even have multiple nodes on the same server by running multiple Elasticsearch processes.
- In a multi-node cluster, data is distributed across multiple servers, boosting performance and stability. With at least one replica shard per primary shard, Elasticsearch can continue operating and return any data even if one node fails. However, this requires ensuring fast and steady communication between nodes to avoid split-brain, where two parts of the cluster are unable to communicate and presume the other half failed.
- Shards are crucial for two main reasons:
  - Horizontal scaling: Shards allow you to distribute data horizontally.
  - Performance and throughput: Shards enable distributed and parallel operations across multiple shards (potentially on different nodes).
- Elasticsearch transparently manages shard distribution and document aggregation for search requests. In failure-prone network/cloud environments, a failover mechanism is highly recommended in case a shard or node goes offline. Elasticsearch allows creating replica shards (replicas) for each primary shard, ensuring data redundancy.
- You can adjust the number of replica shards at any time, as they can be created and removed dynamically. However, the number of primary shards an index is divided into must be decided before index creation. Too few shards limit scalability, while too many can impact performance. The default setting of 5 is a good starting point.

Elasticsearch Concepts (2)

- **Document**: In Elasticsearch, the basic unit of data for both indexing and searching is a document.
- **Type**: A type is a logical container for documents, similar to how a table contains rows in a relational database. The best practice is to use different types for documents with distinct structures. For example, you can define one type for "groups" formed at a gathering, and another type for "activities" that people participate in.
- **Index**: An index acts as a container for document types and their corresponding mappings. It is a large, independent collection of documents, all residing within the same group of files on drives. An index also contains the fields of all mapping types, along with various settings.
- **Mapping**: Mappings determine how data is processed before being indexed. They define how text is broken down into individual terms, which terms are excluded, and any additional processing steps, such as removing HTML tags. Essentially, mappings store the information required by the analysis chain.

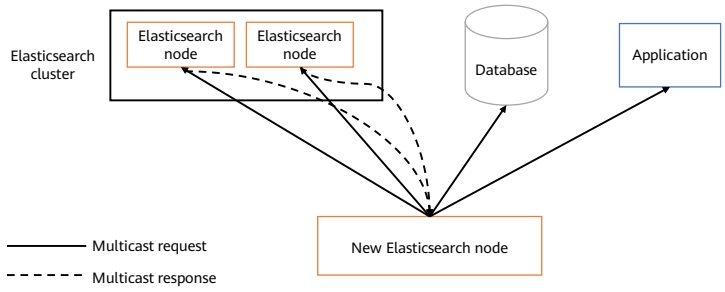
Elasticsearch	MySQL
Index	Database
Type	Table
Document	Row
Field	Column
Mapping configurations	Constraints (type and length) of column

HUAWEI

- A type is similar to a table in a MySQL database. Before version 6.x, an index can contain multiple types , much like a database contains multiple tables. Similar to how a database table has a schema defining the structure and constraints of each column, Elasticsearch has mappings that define the structure of documents within an index.
- Documents are raw data stored in the index. For example, each product can be represented as a separate document.
- Fields represent attributes within a document.
- Mappings control data types, attributes, indexing, and storage.

Working Principles of Elasticsearch – Startup

- When an Elasticsearch node starts, it uses a discovery process (configurable as either broadcast or unicast) to locate and connect to other nodes in the same cluster. Within the cluster, a single node is elected as the master node. This node is responsible for managing the cluster state, responding to changes in cluster topology, and distributing index shards across the appropriate nodes in the cluster.

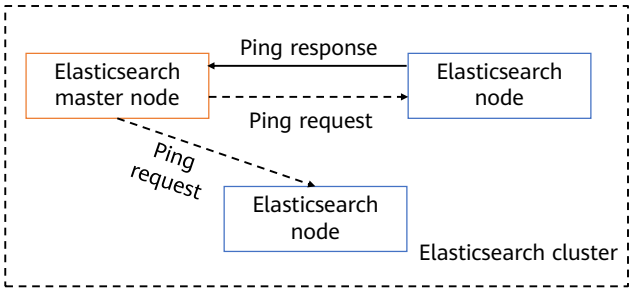


HUAWEI

- From the perspective of a user, master nodes in Elasticsearch are no different than any other node. This is unlike other distributed systems like databases. Users do not need to know which node is the master. They can interact with any node directly. Elasticsearch intelligently handles queries internally. For instance, a node can distribute sub-queries to other nodes in parallel, merge the results, and return them to the user, all without involving the master node. This is because Elasticsearch operates on a peer-to-peer architecture.
- The master node is responsible for managing the cluster state and performing recovery operations when needed.

Working Principles of Elasticsearch – Fault Detection

- When a cluster is operational, the master node monitors all available nodes to ensure they are functioning correctly. If a node fails to respond within a predefined timeframe, it is considered offline. This triggers an error handling process that rebalances the cluster and its shards.

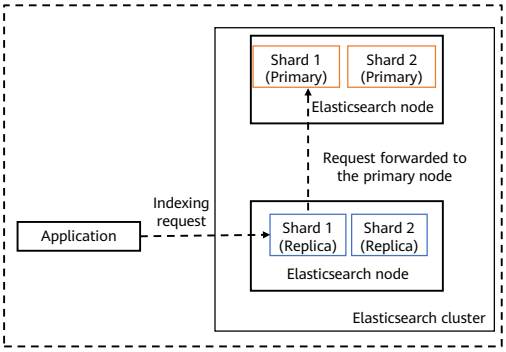


HUAWEI

- Rebalancing is crucial because shards residing on the offline node become inaccessible. To compensate, the remaining nodes assume responsibility for the missing data. Specifically, for each lost primary shard, a new primary shard is promoted from one of its replicas. The placement strategy for these new primary shards and replicas is configurable, allowing users to tailor the process to their specific needs.
- In the figure, the master node sends ping requests to other nodes and waits for responses. The unresponsive node will be removed from the cluster.

Working Principles of Elasticsearch – Indexing

- Elasticsearch provides four primary methods for indexing data:
- RESTful API: This versatile and widely adopted method uses simple HTTP requests, making it easily accessible through various tools.
- Bulk API and UDP Bulk API: Both methods enable the simultaneous indexing of multiple documents. The Bulk API relies on HTTP for communication, while the UDP Bulk API leverages the faster UDP protocol.
- Rivers: These Elasticsearch plugins operate directly on nodes to pull data from external sources.

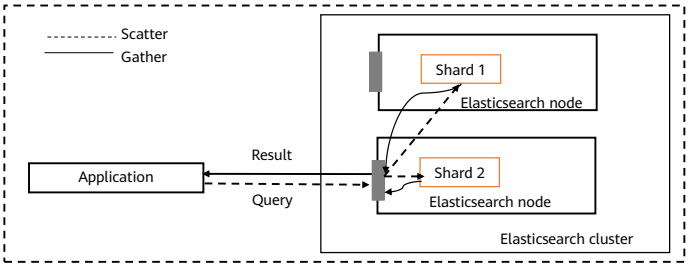


HUAWEI

- Elasticsearch offers a RESTful API to facilitate data input and query construction, the two most crucial user operations, ensuring seamless integration with any HTTP-compliant system.
- Indexing is restricted to primary shards. When an indexing request reaches a node lacking the relevant primary shard or possessing only replicas, it is automatically forwarded to the node hosting the correct primary shard.

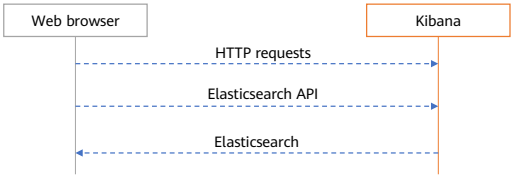
Working Principles of Elasticsearch – Querying

- The Elasticsearch API heavily relies on query the API. The Query Domain Specific Language (DSL) enables a wide range of search operations, from basic term and phrase matching to complex range queries, document filtering, and targeted document retrieval.
- A query is not a single-step process. It involves two distinct phases: scatter and gather. In the scatter phase, the query is distributed across multiple shards containing relevant documents. During the gather phase, results from these shards are collected, merged, sorted, processed further, and finally returned to the client.



Kibana Architecture

- Kibana features a web-based architecture, allowing users to access its intuitive interface through any browser for data visualization and analysis. It leverages the Elasticsearch API to query and retrieve data, offering seamless integration with Elasticsearch query syntax and filters for powerful data exploration and insights.
- The core components of Kibana include:
  - Web server: Powered by Node.js, the web server of Kibana efficiently manages HTTP requests and responses.
  - Client-side JavaScript: Utilizing modern frameworks like AngularJS and ReactJS, Kibana delivers a dynamic and interactive user experience.
  - Elasticsearch API: The tight integration with the Elasticsearch API enables comprehensive data querying and retrieval, supporting the full range of Elasticsearch query syntax and filtering capabilities.
  - Plugins: The extensible plugin system of Kibana allows developers to enhance its functionality and tailor it to specific needs.



HUAWEI

- Kibana is used in various scenarios, including:
  - Monitoring of server and network device health status, including CPU, memory, drive, and network metrics
  - Log analysis and statistical reporting, including log quantities, error types, and access sources
  - Analysis and visualization of service data, such as sales figures and user behavior
  - Security event analysis and monitoring for intrusions and exceptions
  - Data visualization through maps, charts, and other visual representations

Contents

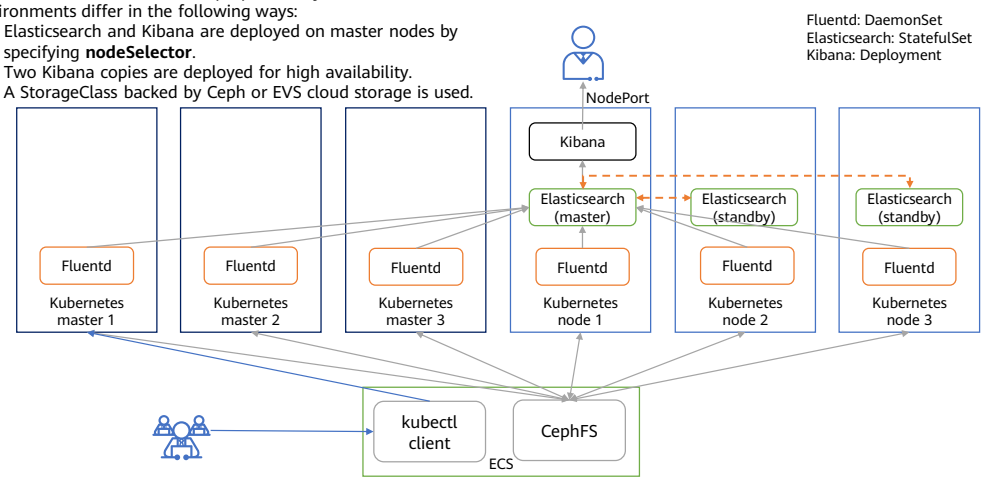
1. Logging Solutions
2. EFK Stack
3. **EFK Installation on Kubernetes**
4. Common EFK Functions

HUAWEI

EFK Installation on Kubernetes

This architecture is for exercise purposes only. Production environments differ in the following ways:

1. Elasticsearch and Kibana are deployed on master nodes by specifying **nodeSelector**.
2. Two Kibana copies are deployed for high availability.
3. A StorageClass backed by Ceph or EVS cloud storage is used.



HUAWEI

Installation Process



1. Deploying Elasticsearch
   1. Creating a service
   2. Creating a StatefulSet
2. Deploying Kibana
   1. Creating a ConfigMap
   2. Creating a service
   3. Creating a deployment
3. Deploying Fluentd
   1. Creating a ConfigMap
   2. Configuring RBAC
   3. Creating a DaemonSet

HUAWEI

Elasticsearch Cluster Creation – Service Settings

- Create a **Elasticsearch-svc.yaml** file for defining a headless service.
- In the file, define a service named **Elasticsearch** with the **app:Elasticsearch** label so that the service returns DNS A records for Elasticsearch pods that have the **app=Elasticsearch label**. Then, set **ClusterIP: None** to make the service headless. Finally, open ports 9200 and 9300 for interaction with the RESTful API and communication between nodes, respectively.

```
kind: Service
apiVersion: v1
metadata:
  name: Elasticsearch
  namespace: logging
  labels:
    app: Elasticsearch
spec:
  selector:
    app: Elasticsearch
  clusterIP: None
  ports:
    - port: 9200
      name: rest
    - port: 9300
      name: inter-node
```

```
$ kubectl create -f Elasticsearch-svc.yaml
service/Elasticsearch created
$ kubectl get services --namespace=logging
Output
NAME            TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)             AGE
Elasticsearch   ClusterIP   None         <none>        9200/TCP,9300/TCP   26s
```

- Elasticsearch is a stateful cluster service. This means that both the frontend service and the pods within the cluster need to know the actual IP addresses of each other.
- A Kubernetes StatefulSet assigns a stable identity and persistent storage to each pod. Since Elasticsearch requires stable storage to ensure data persists across pod reschedules and restarts, a StatefulSet is needed to manage its pods.

HUAWEI

- A headless service does not have a **clusterIP**. This forces the Kubernetes cluster DNS to resolve the service name directly to the IP addresses of the individual pods. This allows frontend services to access specific pods in the cluster by their stable DNS names.

Elasticsearch Cluster Creation – StatefulSet Settings (1)

- Create a StatefulSet object named **es**, and associate it with the previously created headless service using **serviceName=Elasticsearch**. This ensures each pod within the StatefulSet is accessible using a unique DNS address, for example: **es-[0,1,2].Elasticsearch.logging.svc.cluster.local**, where **[0,1,2]** corresponds to the assigned pod number.
- Specify the number of replicas for the pod. An odd number is recommended.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: es
  namespace: logging
spec:
  serviceName: Elasticsearch
  replicas: 3
  selector:
    matchLabels:
      app: Elasticsearch
  template:
    metadata:
      labels:
        app: Elasticsearch
...
```



HUAWEI

Elasticsearch Cluster Creation – StatefulSet Settings (2)

- Split-brain occurs when one or more Elasticsearch nodes lose communication with the rest of the cluster, potentially leading to multiple master nodes. To prevent this, you can set **discover.zen.minimum_master_nodes=***N*/2+1, where *N* represents the number of master-eligible nodes in the Elasticsearch cluster. For example, in a three-node cluster (*N*=3), the value for this setting should be 2. With this setting in place, if one node becomes temporarily disconnected from the cluster, the remaining two nodes can still elect a new master. This ensures the cluster continues to function even when the disconnected node attempts to rejoin.

```
spec:
    containers:
    - name: Elasticsearch
      image: swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/Elasticsearch:7.17.1
      resources:
        limits:
          cpu: 1000m
        requests:
          cpu: 100m
      ports:
      - containerPort: 9200
        name: rest
        protocol: TCP
      - containerPort: 9300
        name: inter-node
        protocol: TCP
      volumeMounts:
      - name: data
        mountPath: /usr/share/Elasticsearch/data
```

```
env:
        - name: cluster.name
          value: k8s-logs
        - name: node.name
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
        - name: cluster.initial_master_nodes
          value: "es-0,es-1,es-2"
        - name: discovery.zen.minimum_master_nodes
          value: "2"
        - name: discovery.seed_hosts
          value: "Elasticsearch"
        - name: ES_JAVA_OPTS
          value: "-Xms512m -Xmx512m"
        - name: network.host
          value: "0.0.0.0"
```


HUAWEI

- This section defines the pod within the StatefulSet, exposing ports 9200 and 9300. Note that **name** should be the same as the previously defined service. We then declare the persistent data directory through **volumeMount**. Then, we set some environment variables within the container:
- **cluster.name**: name of the Elasticsearch cluster, for example, **k8s-logs**.
- **node.name**: name of the node, obtained via **metadata.name**. This will resolve to **es-[0,1,2]**, depending on the designated order of the node.
- **discovery.seed_hosts**: discovery method for nodes to connect to each other within the Elasticsearch cluster. Due to the headless service we configured earlier, our pods have unique DNS domains **es-[0,1,2].Elasticsearch.logging.svc.cluster.local**, so we set this variable accordingly.
- **discovery.zen.minimum_master_nodes**: *N*/2+1, where *N* is the number of master-eligible nodes in the cluster. There are three Elasticsearch nodes. Therefore, set this parameter to 2 (rounded down to the nearest integer).
- ES_JAVA_OPTS: **-Xms512m -Xmx512m**. This sets the JVM to use 512 MB for both minimum and maximum heap size. Adjust the value according to cluster resource availability and needs.

Elasticsearch Cluster Creation – StatefulSet Settings (3)

- Init containers configure the Elasticsearch cluster. They run in sequence and must complete successfully before the main application container can start.
  - The **increase-vm-max-map** container increases the system limit on mmap counts. The default value is often too low and can lead to out-of-memory errors.
  - The **increase-fd-ulimit** container increases the maximum number of open file descriptors using the **ulimit** command.
  - You can add more init containers to perform additional initialization tasks as needed.

```
...
  initContainers:
  - name: increase-vm-max-map
    image: busybox
    command: ["sysctl", "-w", "vm.max_map_count=262144"]
    securityContext:
      privileged: true
  - name: increase-fd-ulimit
    image: busybox
    command: ["sh", "-c", "ulimit -n 65536"]
    securityContext:
      privileged: true
```



HUAWEI

- For more about **increase-vm-max-map**, see the official Elasticsearch document: https://www.elastic.co/guide/en/Elasticsearch/reference/current/vm-max-map-count.html.

Elasticsearch Cluster Creation – StatefulSet Settings (4)

- Define a volume template using **volumeClaimTemplates** in the StatefulSet to configure persistent storage for the data directory.

```
...
  volumeClaimTemplates:
  - metadata:
      name: data
      labels:
        app: Elasticsearch
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: harbor
      resources:
        requests:
          storage: 50Gi
```


HUAWEI

- This example uses **volumeClaimTemplates** to define a persistent storage template. Kubernetes uses this template to create a PersistentVolume for each pod. **accessModes** is set to **ReadWriteOnce**, meaning the volume can only be mounted by a single pod for reading and writing. A **harbor** StorageClass named will be used to automatically provision the PersistentVolume. Finally, each PersistentVolume is allocated 50 GB of storage. You can adjust this value as required.

Elasticsearch Cluster Creation – Deployment

- Run the **kubectl** command for deployment.

```
$ kubectl create -f Elasticsearch-statefulset.yaml
statefulset.apps/es created
```

- After the deployment is complete, you can check the resources created in the logging namespace:

```
$ kubectl get sts -n logging
NAME   READY   AGE
es     3/3     83m
$ kubectl get pods -n logging
NAME                    READY   STATUS    RESTARTS   AGE
es-0                    1/1     Running   0          83m
es-1                    1/1     Running   0          82m
es-2                    1/1     Running   0          81m
$ kubectl get svc -n logging
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)             AGE
Elasticsearch   ClusterIP   None            <none>        9200/TCP,9300/TCP   20h
```



HUAWEI

Elasticsearch Cluster Creation – Verification

- After the pods are deployed, you can check the health of the Elasticsearch cluster by sending a request to its RESTful API. Forward the local port 9200 to the corresponding port on an Elasticsearch node (such as **es-0**).

```
$ kubectl port-forward es-0 9200:9200 --namespace=logging
Forwarding from 127.0.0.1:9200 -> 9200
Forwarding from [::1]:9200 -> 9200
```

- Then, in another terminal window, execute the following request:

```
$ curl http://127.0.0.1:9200/_cluster/state?pretty
```

- If the cluster is running correctly, you will receive a response similar to the information on the right:

```
{
  "cluster_name" : "k8s-logs",
  "compressed_size_in_bytes" : 348,
  "cluster_uuid" : "QD06dK7CQgids-GQZooNVw",
  "version" : 3,
  "state_uuid" : "mjNIWXAzQVuxNNOQ7xR-qg",
  "master_node" : "IdM5B7cUQWqFgIHXBp0JDg",
  "blocks" : { },
  "nodes" : {
    "u7DoTpMmSCixOoictzHItA" : {
      "name" : "es-1",
      "ephemeral_id" : "ZlBflnXKRMC4RvEACHIVdg",
      "transport_address" : "10.244.4.191:9300",
      "attributes" : { }
    },
    "IdM5B7cUQWqFgIHXBp0JDg" : {
      "name" : "es-0",
      "ephemeral_id" : "JTk1FDdFQuWbSFAtBxdxAQ",
      "transport_address" : "10.244.2.215:9300",
      "attributes" : { }
    },
    "R8E7xcSUSbGbgrhAdyAKmQ" : {
      "name" : "es-2",
      "ephemeral_id" : "9wv6ke71Qqy9vk2LgJTqaA",
      "transport_address" : "10.244.40.4:9300",
      "attributes" : { }
...
```


HUAWEI

- The information on the right indicates that nodes **es-0**, **es-1**, and **es-2** are created in the **k8s-logs** Elasticsearch cluster, and **es-0** is the master node.

Kibana Creation – Service Settings

- For easier testing, set the service to the **NodePort** type and Kibana container port to 5601. The node port is randomly assigned. To make Kibana accessible over the internet, you need to bind an Elastic IP to the node.

```
apiVersion: v1
kind: Service
metadata:
  name: kibana-service
  namespace: logging
  labels:
    app: kibana
spec:
  ports:
  - port: 5601
  type: NodePort
  selector:
    app: kibana
```

Optional

```
apiVersion: v1
kind: Ingress
metadata:
  name: kibana-ingress
spec:
  rules:
  - host: kibana.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: kibana-service
            port:
              number: 5601
```

- Optionally, to make Kibana accessible through an Ingress, you can create an Ingress resource after creating the NodePort service. See the example on the right.
  - The example code defines an Ingress resource named **kibana-ingress**. It specifies that traffic addressed to the Kibana host name (**kibana.example.com**) with a path of **/** is routed to **kibana-service** on port 5601, where Kibana is running.

HUAWEI

- Before deploying the Ingress, ensure that the Kibana service is running with a correct service name and port in the Kubernetes cluster. Also, configure DNS records to resolve **kibana.example.com** to the IP address of the Kubernetes cluster or load balancer.
- You may need to adjust the Ingress configurations based on specific requirements, such as TLS encryption, path matching, and load balancing.

Kibana Creation – Deployment Settings

- As a headless Elasticsearch service is used as the backend, you need to configure the **Elasticsearch_HOSTS** environment variable to set the entry point of the Elasticsearch cluster.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kibana
  namespace: logging
  labels:
    app: kibana
spec:
  selector:
    matchLabels:
      app: kibana
  template:
    metadata:
      labels:
        app: kibana
```

```
spec:
    #nodeSelector:
    #  es: log
    containers:
    - name: kibana
      image: swr.cn-east-3.myhuaweicloud.com/hcie_openeuler/kibana:7.17.1
      resources:
        limits:
          cpu: 1000m
        requests:
          cpu: 1000m
      env:
      - name: Elasticsearch_HOSTS
        value: http://Elasticsearch:9200
      ports:
      - containerPort: 5601
```


HUAWEI

- The DNS server within the cluster will resolve this headless service name into the IP addresses of the corresponding Elasticsearch pods.

Kibana Creation – Deployment

- Run the **kubectl** command for deployment.

```
$ kubectl create -f kibana.yaml
service/kibana created
deployment.apps/kibana created
```

- After the deployment is complete, you can check the Kibana pod status.

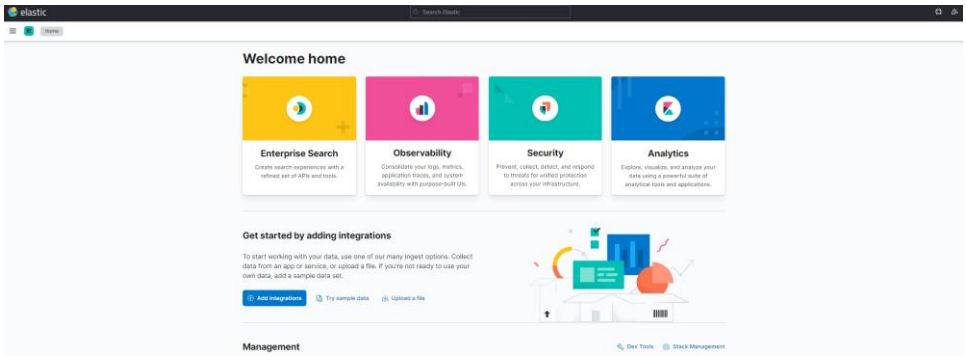
```
$ kubectl get pods --namespace=logging
NAME                      READY   STATUS    RESTARTS   AGE
es-0                      1/1     Running   0          85m
es-1                      1/1     Running   0          84m
es-2                      1/1     Running   0          83m
kibana-5c565c47dd-xj4bd   1/1     Running   0          80m
$ kubectl get svc -n logging
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)             AGE
Elasticsearch   ClusterIP   None            <none>        9200/TCP,9300/TCP   3h22m
kibana          NodePort    10.111.223.99   <none>        5601:31139/TCP      3h20m
```



HUAWEI

Kibana Creation – Verification

- Access the Kibana service for verification. If the service is normal, the following page is displayed.



HUAWEI

- The actual address for accessing Kibana can be obtained from the service or Ingress.

Fluentd Creation – Log Source Settings

- The following configuration sets container logging:
  - **id**: unique identifier for the log source. This ID can be used to filter and route structured log data later.
  - **type**: built-in Fluentd instruction to use. **tail** instructs Fluentd to continuously collect data by tailing the log file. Other options include **http** for collecting data via GET requests.
  - **path**: path to the log files for a **type** of **tail**. In this example, it points to **/var/log/containers/**, the directory where Docker stores standard output logs for containers running on Kubernetes nodes.
  - **pos_file**: checkpoint for the position in the current log file. If Fluentd restarts, it resumes log collection from this position.
  - **tag**: custom string used to match log sources with destinations or filters. Fluentd routes log data based on matching source and destination tags.

```
<source>
  @id fluentd-containers.log
  @type tail  # Built-in Fluentd input method to continuously obtain new
logs from the source file
  path /var/log/containers/*.log         # Path to the mounted Docker
container logs on the server
  pos_file /var/log/es-containers.log.pos
  tag raw.kubernetes.*                   # Tag for the logs
  read_from_head true
  <parse>                              # Parses multi-line logs into JSON.
    @type multi_format                   # Uses the multi-format-parser plugin.
    <pattern>
      format json                      # Uses the JSON parser.
      time_key time                      # Specifies the "time" field as the event
timestamp.
      time_format %Y-%m-%dT%H:%M:%S.%NZ  # Specifies the timestamp
format.
    </pattern>
    <pattern>
      format /^(?<time>.+) (?<stream>stdout|stderr) [^ ]* (?<log>.*)$/
      time_format %Y-%m-%dT%H:%M:%S.%N%:z
    </pattern>
  </parse>
</source>
```


HUAWEI

Fluentd Creation – Route Settings

- The following configuration specifies how Fluentd sends log data to Elasticsearch:
  - **match**: destination tag followed by a regular expression to match log sources. ****** indicates that all logs are captured and sent to Elasticsearch..
  - **id**: unique identifier for the destination.
  - **type**: output plugin to use. Built-in plugin Elasticsearch is used.
  - **log_level**: log level to capture. **info** indicates that logs at the INFO level or higher (INFO, WARNING, ERROR) are routed to Elasticsearch.
  - **host** and **port**: address of Elasticsearch. Authentication information can also be configured. In this example, Elasticsearch does not require authentication, so only **host** and **port** are necessary.
  - **logstash_format**: Elasticsearch uses inverted indexes to search log data efficiently. **true** indicates that Fluentd forwards structured log data in a format compatible with Logstash.
  - **Buffer**: allows for buffering when the destination is unavailable, such as during network outages or when Elasticsearch is down. Buffering also helps reduce drive I/O.

```
<match **>
@id Elasticsearch
@type Elasticsearch
@log_level info
include_tag_key true
type_name fluentd
host "#{ENV['OUTPUT_HOST']}"
port "#{ENV['OUTPUT_PORT']}"
logstash_format true

<buffer>
@type file
path /var/log/fluentd-buffers/kubernetes.system.buffer
flush_mode interval
retry_type exponential_backoff
flush_thread_count 2
flush_interval 5s
retry_forever
retry_max_interval 30
chunk_limit_size
"#{ENV['OUTPUT_BUFFER_CHUNK_LIMIT']}"
queue_limit_length
"#{ENV['OUTPUT_BUFFER_QUEUE_LIMIT']}"
overflow_action block
</buffer>
```


HUAWEI

Fluentd Creation – Filtering

- To avoid collecting excessive logs, you can leverage the filter module with pod labels to target specific logs. For example, you can configure Fluentd to gather logs from pods labeled with **logging=true**.

```
# Deletes useless attributes.

<filter kubernetes.**>
  @type record_transformer
  remove_keys
$.docker.container_id,$.kubernetes.container_image_id,$.kubernetes.pod_id,$.kubernetes.namespace_id,$.kubernetes.master_url,$.kubernetes.labels.pod-
template-hash
</filter>

# Keeps only the logs of pods with the "logging=true" label.

<filter kubernetes.**>
  @id filter_log
  @type grep
  <regexp>
    key $.kubernetes.labels.logging
    pattern ^true$
  </regexp>
</filter>
```


HUAWEI

Fluentd Creation – ConfigMap Settings

- Use a ConfigMap object to specify Fluentd configuration files. Create a **fluentd-configmap.yaml** file and place the following log source, routing, and filtering configurations within the ConfigMap:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: fluentd-config
  namespace: logging
data:
  system.conf: |-
      ...
  containers.input.conf: |-
    <source>
      ..
    </source>
    # Detects exceptions in log output and forwards them as a log
entry.
    <match raw.kubernetes.**>   # Matches log information with the
"raw.kubernetes.**" tag.
      ...
    </match>
    <filter **>  # Concatenates logs.
      ...
    </filter>
    # Adds Kubernetes metadata data.
    <filter kubernetes.**>
      ...
    </filter>
    # Repairs JSON fields in Elasticsearch.
    <filter kubernetes.**>
      ...
    </filter>
    # Removes some redundant attributes.
    <filter kubernetes.**>
      ...
    </filter>
    # Keeps only the logs of pods with the "logging=true" label.
    <filter kubernetes.**>
      ...
    </filter>
  ###### Listening configuration, generally used for log aggregation ######
  forward.input.conf: |-
    # Listens for messages sent via TCP.
    <source>
      ...
    </source>
  output.conf: |-
    <match **>
      @id Elasticsearch
      @type Elasticsearch
      logstash_prefix k8s  # Sets the index prefix to "k8s".
      ...
    </match>
```


HUAWEI

- In the example configuration file, only the Docker container log directory is configured. The collected data is processed and sent to the Elasticsearch:9200 service.

Fluentd Creation – RBAC Settings

- Fluentd requires specific permissions to collect resource data from the cluster and send it to the StatefulSet of Elasticsearch for storage. To enable this, configure role-based access control (RBAC) policies for Fluentd, utilizing ServiceAccounts, ClusterRoles, and ClusterRoleBindings.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: fluentd-es
  namespace: logging
  labels:
    k8s-app: fluentd-es
    kubernetes.io/cluster-service: "true"
    addonmanager.kubernetes.io/mode: Reconcile
```

```
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: fluentd-es
  labels:
    k8s-app: fluentd-es
    kubernetes.io/cluster-service: "true"
    addonmanager.kubernetes.io/mode: Reconcile
rules:
- apiGroups:
  - ""
  resources:
  - "namespaces"
  - "pods"
  verbs:
  - "get"
  - "watch"
  - "list"
```

```
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: fluentd-es
  labels:
    k8s-app: fluentd-es
    kubernetes.io/cluster-service: "true"
    addonmanager.kubernetes.io/mode: Reconcile
subjects:
- kind: ServiceAccount
  name: fluentd-es
  namespace: logging
  apiGroup: ""
roleRef:
  kind: ClusterRole
  name: fluentd-es
  apiGroup: ""
```


HUAWEI

Fluentd Creation – DaemonSet Settings

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-es
  namespace: logging
  labels:
    k8s-app: fluentd-es
    kubernetes.io/cluster-service: "true"
    addonmanager.kubernetes.io/mode: Reconcile
spec:
  selector:
    matchLabels:
      k8s-app: fluentd-es
  template:
    metadata:
      labels:
        k8s-app: fluentd-es
spec:
      serviceAccountName: fluentd-es
      containers:
      - name: fluentd-es
        image: quay.io/fluentd_Elasticsearch/fluentd:v3.4.1
        env:
        - name: FLUENTD_ARGS
          value: --no-supervisor –q
        resources:
          limits:
            memory: 500Mi
          requests:
            cpu: 100m
            memory: 200Mi
        volumeMounts:
        - name: varlog
          mountPath: /var/log
        - name: varlibdockercontainers
          mountPath: /data/docker/containers
          readOnly: true
        - name: config-volume
          mountPath: /etc/fluent/config.d
      nodeSelector:
        beta.kubernetes.io/fluentd-ds-ready: "true"
      tolerations:
      - operator: Exists
      terminationGracePeriodSeconds: 30
      volumes:
      - name: varlog
        hostPath:
          path: /var/log
      - name: varlibdockercontainers
        hostPath:
          path: /var/lib/docker/containers
      - name: config-volume
        configMap:
          name: fluentd-config
```


HUAWEI

Fluentd Creation – DaemonSet Setting Description

- You can specify the hosts for log collection by setting the **nodeSelector** property.

```
nodeSelector:
  beta.kubernetes.io/fluentd-ds-ready: "true"
```

- This ensures that Fluentd collects logs only from nodes that have the specified label. For example, in the output below, only **ydzs-node1** and **ydzs-node3** have the required label.

```
$ kubectl get nodes --show-labels
NAME          STATUS   ROLES    AGE    VERSION   LABELS
ydzs-master   Ready    master   170d   v1.16.2   beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=ydzs-
master,kubernetes.io/os=linux,node-role.kubernetes.io/master=
ydzs-node1    Ready    <none>   170d   v1.16.2   beta.kubernetes.io/fluentd-ds-
ready=true,beta.kubernetes.io/os=linux,es=log,kubernetes.io/arch=amd64,kubernetes.io/hostname=ydzs-node1,kubernetes.io/os=linux
ydzs-node2    Ready    <none>   170d   v1.16.2
beta.kubernetes.io/os=linux,com=youdianzhishi,es=log,kubernetes.io/arch=amd64,kubernetes.io/hostname=ydzs-node2,kubernetes.io/os=linux
ydzs-node3    Ready    <none>   169d   v1.16.2   beta.kubernetes.io/fluentd-ds-
ready=true,beta.kubernetes.io/os=linux,es=log,kubernetes.io/arch=amd64,kubernetes.io/hostname=ydzs-
node3,kubernetes.io/os=linux,monitor=prometheus
```

- To add a label to a node, run the following command:

```
kubectl label nodes nodename beta.kubernetes.io/fluentd-ds-ready=true
```


HUAWEI

- If a Kubernetes cluster is set up using kubeadm, the master nodes are typically tainted by default. To collect logs from these master nodes, you need to add a toleration.

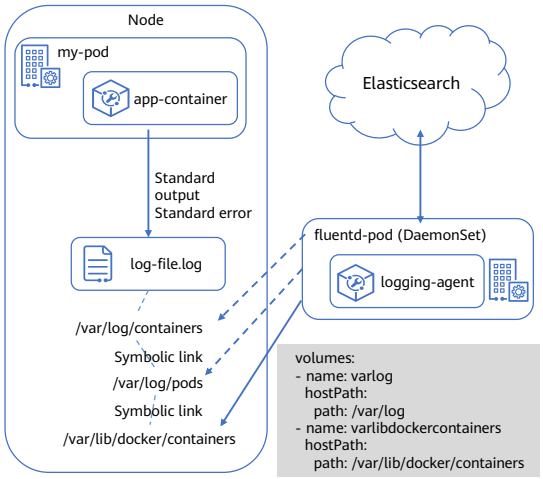
Fluentd Creation – Log Path Description

- Within a Kubernetes cluster, container logs follow a symbolic link chain: **/var/log/containers** > **/var/log/pods** > **/var/lib/docker/containers**
- Container logs are stored in the **/var/log/containers** directory by default. However, the log files in this directory are symbolic links pointing to files within **/var/log/pods**. These files, in turn, are also symbolic links, ultimately resolving to the actual log files stored within the Docker container storage directory at **/var/lib/docker/containers**.
- Both **varlog** and **varlibdockercontainers** must be mounted. Failing to mount the Docker container storage directory will result in the following error:

```
log unreadable. It is excluded and would be examined next time
```

- While the **path** field within the Fluentd ConfigMap declares the intent to read the log location, the actual log path is configured through the **volumes** field within the Fluentd DaemonSet.

```
path /var/log/containers/*.log
```



HUAWEI

Fluentd Creation – Deployment

- Create the ConfigMap object and DaemonSet.

```
$ kubectl create -f fluentd-configmap.yaml
configmap "fluentd-config" created
$ kubectl create -f fluentd-daemonset.yaml
serviceaccount "fluentd-es" created
clusterrole.rbac.authorization.k8s.io "fluentd-es" created
clusterrolebinding.rbac.authorization.k8s.io "fluentd-es" created
daemonset.apps "fluentd-es" created
```

- After the creation is complete, view the pod list to check whether the deployment is successful.

```
$ kubectl get pods -n logging
NAME                    READY   STATUS    RESTARTS   AGE
es-0                    1/1     Running   0          108m
es-1                    1/1     Running   0          107m
es-2                    1/1     Running   0          106m
fluentd-es-h4jl2        1/1     Running   0          100m
fluentd-es-vngmd        1/1     Running   0          100m
kibana-5c565c47dd-xj4bd 1/1     Running   0          103m
```

- After Fluentd is started, logs can be sent to Elasticsearch. However, no data is currently being collected because only pods with the **logging=true** label are included.



HUAWEI

Fluentd Creation – Verification (1)

- Use the following file to create a test application:

```
apiVersion: v1
kind: Pod
metadata:
  name: counter
  labels:
    logging: "true"  # Only logs of pods with this label are collected.
spec:
  containers:
  - name: count
    image: busybox
    args: [/bin/sh, -c,'i=0; while true; do echo "$i: $(date)"; i=$((i+1)); sleep 1; done']
```

- This application prints logs to standard output, allowing Fluentd to collect and display the log data. Run the **kubectl** command to create the pod:

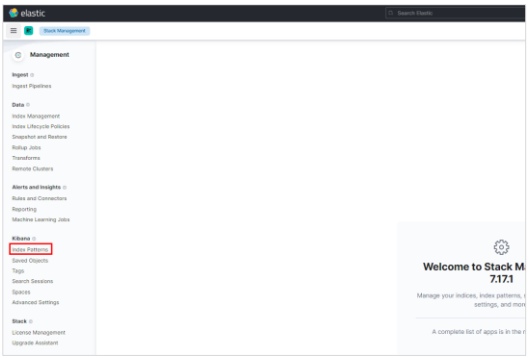
```
$ kubectl create -f counter.yaml
$ kubectl get pods
NAME                        READY   STATUS    RESTARTS   AGE
counter                     1/1     Running   0          9h
```

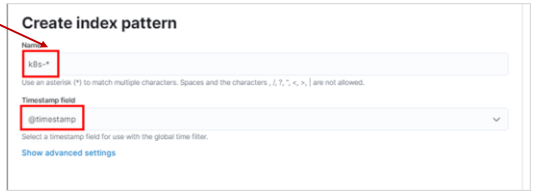

HUAWEI

Fluentd Creation – Verification (2)

- After the pod is created and running, return to the **Kibana Dashboard** page, import the index data on the **Stack Management** page, and view the data on the **Discover** page.

The **k8s** index prefix is defined in the Fluentd ConfigMap.







HUAWEI

Contents

1. Logging Solutions
2. EFK Stack
3. EFK Installation on Kubernetes
4. **Common EFK Functions**

HUAWEI

Example Application (1)

- With a system for collecting application logs established, let's delve into analyzing them with a practical example. The example application produces logs in the following JSON format:

```
{"LOGLEVEL":"WARNING","serviceName":"msg-processor","serviceEnvironment":"staging","message":"WARNING client connection terminated unexpectedly."}
{"LOGLEVEL":"INFO","serviceName":"msg-receiver-api":"msg-receiver-api","serviceEnvironment":"staging","volume":14,"message":"API received messages"}
{"LOGLEVEL":"ERROR","serviceName":"msg-receiver-api","serviceEnvironment":"staging","message":"ERROR Unable to upload files for processing"}
```

- The inherent parsability of JSON significantly simplifies log analysis. By transmitting structured logs to Elasticsearch, we can efficiently search based on specific field values instead of relying on text-based queries. While plain text logs can be structured, the process is tedious and requires customization for the unique format of each application. Therefore, a unified JSON logging format is highly recommended.
- The example application periodically generates diverse log messages, encompassing different severity levels (INFO, WARN, and ERROR). Each JSON line represents a single log message, collected and forwarded to Elasticsearch by Fluentd. The automatic JSON parsing plugin of Fluentd is used, which by default treats each log line as a **log** field and appends additional fields such as **tag** (identifying the container) and **stream** (indicating standard output or standard error output).

HUAWEI

Example Application (2)

- The following filter configuration in Fluentd enables structured JSON data parsing:

```
<filter kubernetes.**>
  @id filter_parser
  @type parser                # multi-format-parser plugin for multiple formats
  key_name log                # Field name to parse within the log record
  reserve_data true           # Retains original key-value pairs in parsed results.
  remove_key_name_field true  # Removes the "key_name" field after successful
parsing.
  <parse>
    @type multi_format
    <pattern>
      format json
    </pattern>
    <pattern>
      format none
    </pattern>
  </parse>
</filter>
```



- This configuration leverages the **json** and **none** plugins for structural parsing, effectively extracting JSON attributes into individual fields. After applying this configuration, you need to refresh the index fields of Kibana to ensure recognition of these new fields. This can be done through **Stack Management** > **Index Patterns** > **Refresh**.

HUAWEI

Example Application (3)

- The following is an example application deployed in the Kubernetes cluster:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dummylogs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: dummylogs
  template:
    metadata:
      labels:
        app: dummylogs
        logging: "true"  # This label needs to added to
pods for log collection.
    spec:
      containers:
      - name: dummy
        image: cnych/dummylogs:latest
        args:
        - msg-processor
```

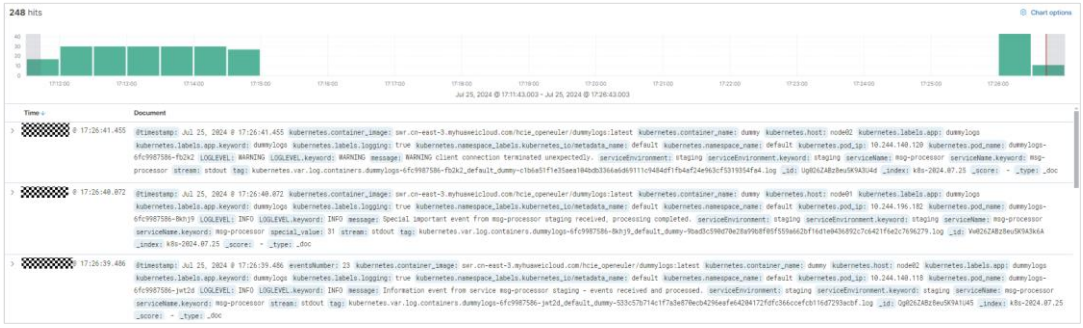
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dummylogs2
spec:
  replicas: 3
  selector:
    matchLabels:
      app: dummylogs2
  template:
    metadata:
      labels:
        app: dummylogs2
        logging: "true"  # This label needs to added to
pods for log collection.
    spec:
      containers:
      - name: dummy
        image: cnych/dummylogs:latest
        args:
        - msg-receiver-api
```

```
$ kubectl apply -f dummylogs.yaml
$ kubectl get pods -l logging=true
NAME                READY   STATUS
counter             1/1     Running
dummylogs- X        1/1     Running
dummylogs- X        1/1     Running
dummylogs- X        1/1     Running
dummylogs2-X        1/1     Running
dummylogs2-X        1/1     Running
dummylogs2-X        1/1     Running
```


HUAWEI

Example Application (4)

- After the deployment is complete, the **dummylogs** and **dummylogs2** applications start to generate logs of different levels. In normal cases, the collected logs are as follows:

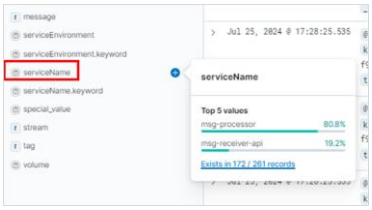


HUAWEI

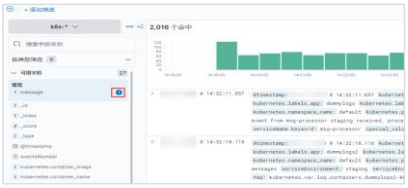
- Add the **beta.kubernetes.io/fluentd-ds-ready=true** label to the nodes where the applications are located.

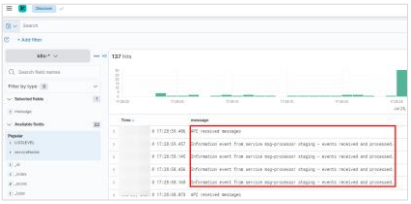
Log Analysis – Discover (1)

- The **Available fields** list lets you explore collected data. For example, click the **serviceName** field to see related information.



- By default, the log data display includes numerous attributes, making it hard to read. You can customize this by selecting specific fields to display.





HUAWEI

Log Analysis – Discover (2)

- Filter and analyze the specific log data you need.



- Alternatively, you can refine your search using Kibana Query Language (KQL).

serviceName:msg-processor OR serviceName:msg-receiver-api



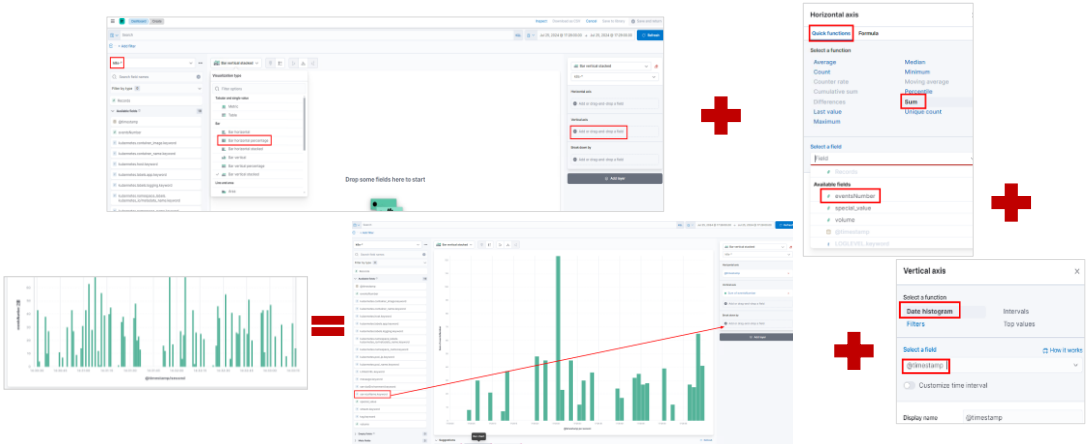


HUAWEI

- KQL simplifies interaction with Elasticsearch data. It supports index fields and autocompletion for efficient querying.
- KQL syntax:
  - Exact match: *field_name*:*match_value* (for example, **serviceName:msg-processor**)
  - Logical operators (case-insensitive): **AND**, **OR**, and **NOT** (for example, **serviceName:msg-processor OR serviceName:msg-receiver-api**)

Log Analysis – Dashboard

- On the Dashboard page, you can create various chart styles for log analysis and troubleshooting by configuring (either through clicking or dragging and dropping) options like **Horizontal axis**, **Vertical axis**, **Break down by**, and **Chart type**.



HUAWEI

Log Alerting

- While Prometheus effectively monitors application metrics in production, it cannot capture everything. Error logs often contain crucial information not reflected in standard metrics. To proactively identify and address potential issues, implementing error log monitoring and alerting is essential. ElastAlert, a powerful tool developed by Yelp, provides an effective solution for monitoring Elasticsearch logs.
- ElastAlert runs queries against Elasticsearch indexes at regular intervals, comparing the results against predefined thresholds. When the results exceed these thresholds, ElastAlert triggers alerts through various channels, including emails and messaging software. In this course, we will deploy ElastAlert within a Kubernetes cluster.



HUAWEI

Alerting Plugin – ConfigMap Settings

- ElastAlert configurations and rules are saved in ConfigMaps.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: elastalert-config
  namespace: logging
  labels:
    app: elastalert
data:
  elastalert_config: |-
    ---
    rules_folder: /opt/rules       # Directory for storing the rule files
    scan_subdirectories: false
    run_every:                     # Interval for querying data from Elasticsearch
      minutes: 1
    buffer_time:
      minutes: 15
    es_host: Elasticsearch
    es_port: 9200
    writeback_index: elastalert
    use_ssl: False
    verify_certs: True
    alert_time_limit:              # Time before a repeated alert can be sent
      minutes: 2880
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: elastalert-rules
  namespace: logging
  labels:
    app: elastalert
data:
  rule_config.yaml: |-
    name: dummylogs error      # Rule name, which is unique
    es_host: Elasticsearch
    es_port: 9200
    type: any                  # Alert type
    index: k8s-*               # Elasticsearch index
    filter:                    # Filter
    - query:
        query_string:
          query: "LOGLEVEL:ERROR"  # Alert condition
    alert:                     # Alert notification type
    - "email"
    smtp_host: smtp.qq.com
    smtp_port: 587
    smtp_auth_file: /opt/auth/smtp_auth_file.yaml
    email_reply_to: xxxxx@xx.com
    from_addr: xxxxx@xx.com
    email:                     # Recipient email address
    - "xxxxx@xx.com"
```


HUAWEI

Alerting Plugin – Deployment Settings

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: elastalert
  namespace: logging
  labels:
    app: elastalert
spec:
  selector:
    matchLabels:
      app: elastalert
  template:
    metadata:
      labels:
        app: elastalert
    spec:
      containers:
      - name: elastalert
        image: jertel/elastalert-docker:0.2.4
        imagePullPolicy: IfNotPresent
        volumeMounts:
        - name: config
          mountPath: /opt/config
        - name: rules
          mountPath: /opt/rules
        - name: auth
          mountPath: /opt/auth
```

```
        resources:
          limits:
            cpu: 50m
            memory: 256Mi
          requests:
            cpu: 50m
            memory: 256Mi
      volumes:
      - name: auth
        secret:
          secretName: smtp-auth
      - name: rules
        configMap:
          name: elastalert-rules
      - name: config
        configMap:
          name: elastalert-config
          items:
          - key: elastalert_config
            path: elastalert_config.yaml
```

- When using emails for alerting, you need to specify a file for the **smtp_auth_file** parameter (**smtp_auth_file.yaml**), which contains the user name and password:

```
user: "xxxxx@xx.com" # Sender email address
password: "ewwghfhdvjwnbjea" # Email address authorization code
```



HUAWEI

Alerting Plugin – Deployment

- Use the **smtp_auth_file.yaml** file to create a secret resource object.

```
$ kubectl create secret generic smtp-auth --from-file=smtp_auth_file.yaml -n logging
```

- Create the **elastalert** application.

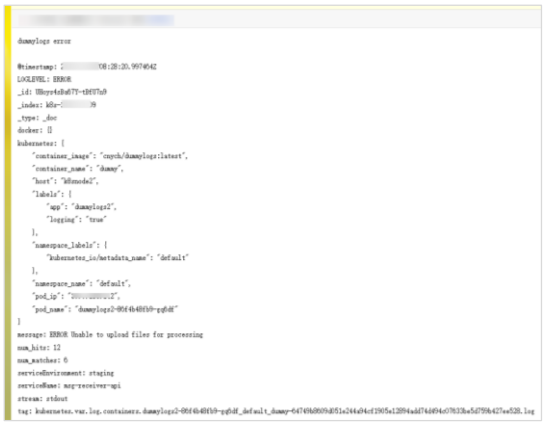
```
$ kubectl apply -f elastalert.yaml
$ kubectl get pods -n logging -l app=elastalert
NAME                          READY   STATUS    RESTARTS   AGE
elastalert-64ccfbffcf-gd6xz   1/1     Running   0          102s
$ kubectl logs -f elastalert-64ccfbffcf-gd6xz -n logging
Elastic Version: 7.6.2
Reading Elastic 6 index mappings:
Reading index mapping 'es_mappings/6/silence.json'
Reading index mapping 'es_mappings/6/elastalert_status.json'
Reading index mapping 'es_mappings/6/elastalert.json'
Reading index mapping 'es_mappings/6/past_elastalert.json'
Reading index mapping 'es_mappings/6/elastalert_error.json'
Deleting index elastalert_status.
New index elastalert created
Done!
```

- The preceding information indicates that the **elastalert** application is deployed successfully.

HUAWEI

Alerting Plugin – Testing

- The example application generates ERROR logs at a certain frequency. The recipient email address should receive the following alert information:



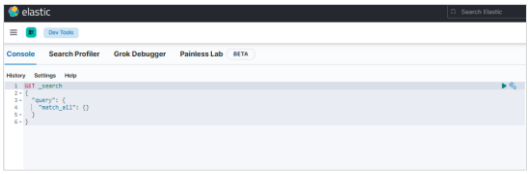


HUAWEI

- You can also configure ElastAlert to send alerts to messaging software or mobile phones. Furthermore, you can install an ElastAlert Kibana plugin for visualization on Kibana. For details, visit https://elastalert.readthedocs.io/en/latest/.

Kibana Dev Tools

- Kibana Dev Tools allow you to create (PUT), delete (DELETE), update (POST), and retrieve (GET) Elasticsearch data.



- Check information about the nodes: GET /_cat/nodes?v
- Get a summary of all indexes: GET /_cat/indices?v
- Get detailed information for all indexes: GET _all
- Check the overall health status of the cluster: GET /_cat/health?v



HUAWEI

Elasticsearch Command Examples (1)

- Inserting data:

```
PUT /index/type/id
```

- Example:

```
POST /ecommerce/product/13
{
  "base_price" : 24.99,
  "discount_percentage" : 0,
  "quantity" : 1,
  "manufacturer" : "Champion Arts",
  "tax_amount" : 0,
  "product_id" : 11238,
  "category" : "Women's Clothing",
  "sku" : "ZO0489604896",
  "taxless_price" : 24.99,
  "unit_discount_amount" : 0,
  "min_price" : 11.75,
  "discount_amount" : 0,
  "created_on" : "2XXX-12-25T21:59:02+00:00",
  "product_name" : "Denim dress - black denim",
  "price" : 24.99,
  "taxful_price" : 24.99,
  "base_unit_price" : 24.99
}
```

- Using **match_all** to perform a query that matches all documents:

```
GET /ecommerce/product/_search
{
  "query":{
    "match_all": {

    }
  }
}
```

- Using **match** to perform a query that matches documents based on a specific condition:

```
GET /ecommerce/product/_search
{
  "query":{
    "match": {
      "category" : "Clothing"
    }
  }
}
```


HUAWEI

- When inserting data into Elasticsearch, if you specify an index and type that do not exist, they will be automatically created by default.

Elasticsearch Command Examples (2)

- Exact match query using **match_phrase**:

```
GET /ecommerce/product/_search
{
  "query":{
    "match_phrase": {
      "category":"Women's Clothing"
    }
  }
}
```

- Sorting results with **sort**:

```
GET /ecommerce/product/_search
{
  "query":{
    "match": {
      "category" : "Clothing"
    }
  },
  "sort":[
    {
      "price":{
        "order":"desc"
      }
...
```

- Pagination with **size**:

```
GET /ecommerce/product/_search
{
  "query":{
    "match": {
      "category" : "Clothing"
    }
  },
  "sort":[
    {
      "price":{
        "order":"desc"
      }
    }
  ],
  "from":0, # Starting document index
  "size":2 # Number of documents per page
}
```


HUAWEI

Elasticsearch Command Examples (3)

- Returning specific fields with **_source**:

```
GET /ecommerce/product/_search
{
  "query":{
    "match_all": {


    }
  },
  "sort":[
    {
      "price":{
        "order":"desc"
      }
    }
  ],
  "_source":["price","min_price","base_price"] # Returns only price-related data.
}
```

- Conditional query:

```
GET /ecommerce/product/_search
{
  "query":{
    "bool": {
      "must": [
        {
          "match": {
            "category": "Women's Clothing"
          }
        }
      ],
      "filter": [
        {
          "range": {
            "price": {
              "gte": 20,
              "lte": 50
            }
          }
        }
      ]
    }
  }
}
```

Searches for items whose name contains **Women's Clothing** and price is between 20 and 50.

HUAWEI

Elasticsearch Command Examples (4)

- Aggregation query:

```
GET /ecommerce/_search
{
  "aggs":{
    "group_by_category":{
      "terms": {
        "field": "category.keyword"
      }
    }
  }
}
```

Calculates the item count for each clothes category.

```
"aggregations" : {
  "group_by_category" : {
    "doc_count_error_upper_bound" : 0,
    "sum_other_doc_count" : 0,
    "buckets" : [
      {
        "key" : "Women's Clothing",
        "doc_count" : 2
      },
      {
        "key" : "Women's Shoes",
        "doc_count" : 2
      },
      {
        "key" : "Men's Clothing",
        "doc_count" : 1
      }
    ]
  }
}
```

```
GET /ecommerce/_search
{
  "aggs":{
    "group_by_category":{
      "terms": {
        "field": "category.keyword"
      },
      "aggs": {
        "avg_price": {
          "avg": {
            "field": "price"
          }
        }
      }
    }
  }
}
```

Calculates the average price and item count for each clothes category.

```
"aggregations" : {
  "group_by_category" : {
    "doc_count_error_upper_bound" : 0,
    "sum_other_doc_count" : 0,
    "buckets" : [
      {
        "key" : "Women's Clothing",
        "doc_count" : 2,
        "avg_price" : {
          "value" : 24.989999771118164
        }
      },
      {
        "key" : "Women's Shoes",
        "doc_count" : 2,
        "avg_price" : {
          "value" : 87.48999786376953
        }
      },
      {
        "key" : "Men's Clothing",
        "doc_count" : 1,
        "avg_price" : {
          "value" : 11.989999771118164
        }
      }
    ]
  }
}
```



HUAWEI

- Aggregation results are displayed in the **aggregations** section at the bottom of the query results.
- To group and count by category (counting the number of products in each category):
  - **group_by_category**: This is a custom name for the aggregation.
  - **field**: This field requires the **.keyword** suffix. The category field is of the **text** type, which is not indexed by default. However, after text analysis, the extracted keywords are indexed. Therefore, you need to use **category.keyword** for accurate aggregation.

Quiz

1. (Single-answer question) When you create the StatefulSet for Elasticsearch, which of the following fields prevents split-brain in the Elasticsearch cluster?
   A. discovery.seed_hosts
   B. discovery.zen.minimum_master_nodes
   C. discovery.zen.maxmum_master_nodes
   D. cluster.initial_master_nodes

HUAWEI

- Answer: B

Summary

- This course describes:
  - Logging solution framework (collection, storage, and visualization)
  - Common logging solutions and their advantages and disadvantages
  - EFK logging solution architecture
  - EFK installation on Kubernetes
  - Common EFK functions

HUAWEI

Recommendations

- Elasticsearch official document:
  - https://www.elastic.co/guide/en/Elasticsearch/reference/current/index.html
- Fluentd official document:
  - https://docs.fluentd.org/
- Kibana official document:
  - https://www.elastic.co/guide/en/kibana/current/index.html
- ElastAlert official document:
  - https://elastalert.readthedocs.io/en/latest/

HUAWEI

Acronyms and Abbreviations

- UDP (User Datagram Protocol): A simple protocol for transmitting datagrams at the transport layer
- TCP (Transmission Control Protocol): A core protocol in TCP/IP that divides data into packets for transmission via the Internet Protocol. It also verifies and reassembles received packets into complete messages.
- HTTP (Hypertext Transfer Protocol): An application layer protocol enabling communication between web browsers (or other clients) and web servers.
- API (application programming interface): A set of rules and specifications that allows different software applications to communicate with each other.
- SQL (Structured Query Language): A programming language for querying and managing relational databases, used for tasks like data retrieval, manipulation, and updating.
- JSON (JavaScript Object Notation): A language-independent data description format.
- REST (Representational State Transfer): A software architectural style with a set of design principles and constraints.

HUAWEI

Acronyms and Abbreviations

- HTML (Hypertext Markup Language): A system for marking up text documents to be published on the World Wide Web.
- EVS (Elastic Volume Service): A distributed and scalable block storage service.
- ECS (Elastic Cloud Server): A computing server consisting of CPU, memory, storage, and an operating system image, offering on-demand provisioning and flexible scalability.
- DNS (Domain Name System): A functional unit that provides the domain name service on the Internet.
- NFS (Networked File System): A file sharing service where a dedicated server manages storage space through a file system.
- RBAC (role-based access control): An access control model and policy where permissions are associated with roles. Users gain access to resources by being assigned to appropriate roles. Roles are created to perform specific tasks, and users are assigned to roles based on their responsibilities and qualifications.

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

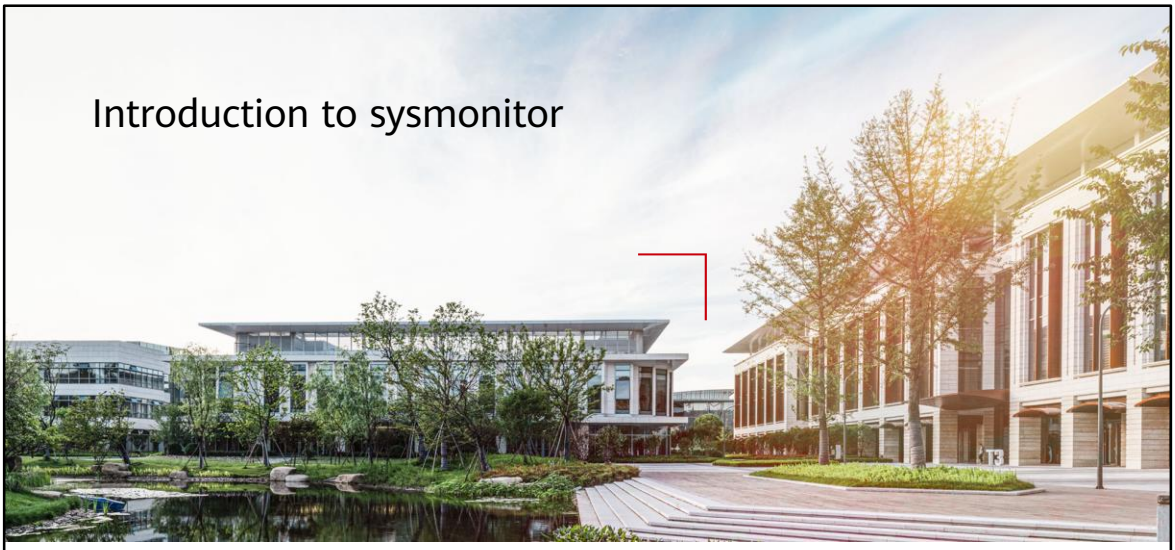
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Introduction to sysmonitor



Foreword

- During system running, system call errors may occur due to various reasons. By monitoring system call behaviors, O&M personnel can detect and record the errors in a timely manner, which facilitates troubleshooting and reduces costs. This course describes how to install and configure sysmonitor (system O&M software), and how to use its monitoring functions.

Objectives

- Upon completion of this course, you will be familiar with sysmonitor regarding:
  - Installation and usage
  - NIC status monitoring
  - System resource monitoring
  - Key process monitoring
  - Custom monitoring

HUAWEI

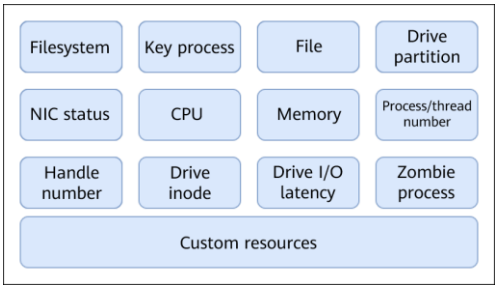
Contents

1. **Introduction to sysmonitor**
2. sysmonitor Installation, Configuration, and Monitoring

HUAWEI

Introduction to sysmonitor

- sysmonitor is a system O&M and monitoring utility. It detects exceptions during OS running and records them in system logs (**/var/log/sysmonitor.log**). Specifically, it can monitor the usage of system resources (such as drives, CPUs, and memory), the number of processes, threads, and handles, filesystem exceptions, as well as network interface card (NIC) and file operations (which are also recorded in logs). It can also monitor key processes and restore them when they are abnormal. In addition, sysmonitor allows users to define monitoring on custom resources.



HUAWEI

- You can run the **systemctl start|stop|restart|reload sysmonitor** command to start, stop, restart, and reload the service.

Application Scenarios

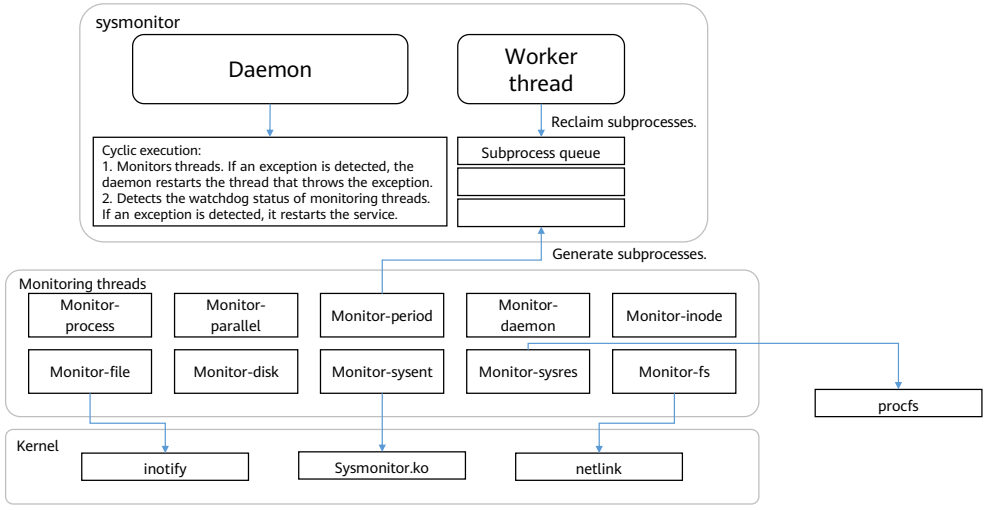
sysmonitor assists in monitoring the OS running status and locating faults when exceptions are detected.

- **NIC monitoring:** The status or IP address of a NIC may change due to manual interventions or exceptions during system running, causing services or the OS to malfunction. sysmonitor tracks NIC starts and stops and IP address additions and deletions, and logs these operations and their operators. The logs allow O&M personnel to determine the time and operator of NIC changes.
- **System resource monitoring:** sysmonitor monitors the usage of system resources and logs the resource usage beyond a configured threshold, helping O&M personnel locate system resource exceptions.
- **Key process monitoring:** The normal running of key service processes is critical. However, processes inevitably break down due to system- or human-related factors. sysmonitor detects process exceptions and quickly recovers key processes to ensure the normal running of services.
- sysmonitor also allows users to **customize the monitoring framework** to extend monitoring capabilities.

HUAWEI

- ext3/ext4 filesystem monitoring: A fault in the filesystem may trigger I/O operation errors, which further cause OS faults. Filesystem fault detection can detect the faults in real time so that system administrators or users can rectify them in a timely manner. If the **errors=remount-ro** mounting option is added to the ext3 or ext4 filesystem, exception information will be recorded in the **sysmonitor.log** file when the filesystem is faulty.
- File monitoring: If key system files are deleted accidentally, the system may run abnormally or even break down. Through file monitoring, users can learn about the deletion of key files or the addition of malicious files in the system in a timely manner, so that administrators and users can rectify faults in a timely manner. The file monitoring configuration item can be added to **/etc/sysmonitor/file.d**.
- Drive partition monitoring: The system periodically monitors the drive partitions mounted to the system. When the drive partition usage is greater than or equal to the configured alarm threshold, the system records a drive space alarm. When the drive partition usage falls below the configured alarm recovery threshold, a drive space recovery alarm is recorded. The configuration file is **/etc/sysmonitor/disk**.
- CPU monitoring: The system monitors the global CPU usage or the CPU usage in a specified domain. When the CPU usage exceeds the configured alarm threshold, the system runs the configured log collection command. The configuration file is **/etc/sysmonitor/cpu**.
- Memory monitoring: monitors the system memory usage and records logs when the memory usage exceeds or falls below the threshold. The configuration file is **/etc/sysmonitor/memory**.
- System handle monitoring: monitors the number of system file handles (FDs). When the total number of system FDs exceeds or is less than the threshold, a log is recorded. The configuration file is **/etc/sysmonitor/sys_fd_conf**.

sysmonitor Architecture



HUAWEI

Contents

1. Introduction to sysmonitor
2. **sysmonitor Installation, Configuration, and Monitoring**

HUAWEI

Installing and Configuring sysmonitor (1)

sysmonitor runs as a systemd service on the system background. Commonly used commands:

- Start the monitoring service.

```
systemctl start sysmonitor
```

- Stop the monitoring service.

```
systemctl stop sysmonitor
```

- Restart the monitoring service.

```
systemctl restart sysmonitor
```

- After the configuration file of monitoring items is modified, reload the monitoring service for the modification to take effect.

```
systemctl reload sysmonitor
```


HUAWEI

- By default, logs are split and dumped to prevent the **sysmonitor.log** file from getting to large. Logs are dumped to a drive directory. In this way, a certain number of logs can be retained.
- The configuration file is **/etc/rsyslog.d/sysmonitor.conf**. Because this rsyslog configuration file is added, after sysmonitor is installed for the first time, restart the rsyslog service to make the sysmonitor log configuration take effect.

Installing and Configuring sysmonitor (2)

- The configuration file **/etc/sysconfig/sysmonitor** of sysmonitor specifies whether to enable monitoring and defines the monitoring period of each monitoring item. After the configuration is modified, restart or reload the service for the modification to take effect.
  - **DISK_MONITOR**: Whether to enable drive partition monitoring.
  - **DISK_MONITOR_PERIOD**: Drive monitoring period, in seconds.
- Configuration item examples

Configuration Item	Description	Mandatory	Default Value
PROCESS_MONITOR	Whether to enable key process monitoring. The value can be **on** or **off**.	No	on
PROCESS_MONITOR_PERIOD	Monitoring period on key processes, in seconds.	No	3s
PROCESS_RECALL_PERIOD	Interval for recalling a key process after the process fails to be recovered, in minutes. The value can be an integer ranging from 1 to 1440.	No	1min
PROCESS_RESTART_TIMEOUT	Timeout interval for recovering a key process from an exception, in seconds. The value can be an integer ranging from 30 to 300.	No	90s
PROCESS_ALARM_SUPRESS_NUM	Number of alarm suppression times when the key process monitoring configuration uses the alarm command to report alarms. The value is a positive integer.	No	5

HUAWEI

- For details about the configuration items, refer to https://gitee.com/openeuler/docs/blob/stable2-22.03_LTS_SP2/docs/en/docs/sysmonitor/sysmonitor-usage.md#configuration-overview.

NIC Status Monitoring (1)

- Introduction

  During system running, the NIC status or IP address may change due to manual interventions or exceptions. Users can monitor the NIC status and IP address changes to detect exceptions in a timely manner and locate exception causes.

- Configuration file

  The configuration file is **/etc/sysmonitor/network**. Example:

```
#dev event
eth1 UP
```


HUAWEI

NIC Status Monitoring (2)

- Configuration item description

Configuration Item	Description	Mandatory	Default Value
dev	NIC name	Yes	None
event	Event to be monitored. The value can be **UP**, **DOWN**, **NEWADDR**, or **DELADDR**. - UP: The NIC is up. - DOWN: The NIC is down. - NEWADDR: An IP address is added. - DELADDR: An IP address is deleted.	No	If this item is empty, **UP**, **DOWN**, **NEWADDR**, and **DELADDR** are monitored.

HUAWEI

- After modifying the configuration file for NIC monitoring, run **systemctl reload sysmonitor** for the new configuration to take effect.
- The **UP** and **DOWN** status of virtual NICs cannot be monitored.
- Ensure that each line in the NIC monitoring configuration file contains less than 4096 characters. Otherwise, a configuration error message will be recorded in the monitoring log.
- By default, all events of all NICs are monitored. That is, if no NIC monitoring is configured, the **UP**, **DOWN**, **NEWADDR**, and **DELADDR** events of all NICs are monitored.
- If a NIC is configured but no event is configured, all events of the NIC are monitored by default.
- The events of route addition can be recorded five times per second. You can change the number of times by setting **NET_RATE_LIMIT_BURST** in **/etc/sysconfig/sysmonitor**.

NIC Status Monitoring (3)

- Exception logs
  - If a NIC event is detected, information similar to the following is displayed in the **/var/log/sysmonitor.log** file, which records the IP address change event of the lo NIC and information about the process (including its parent process) that operates the NIC.

```
info|sysmonitor[127]: lo: ip[::1] prefixlen[128] is added, comm: (ostnamed)[1046], parent comm: systemd[1]
info|sysmonitor[127]: lo: device is up, comm: (ostnamed)[1046], parent comm: systemd[1]
```

  - If a route event is detected, information similar to the following is displayed in the **/var/log/sysmonitor.log** file:

```
info|sysmonitor[881]: Fib4 replace table=255 192.168.122.255/32, comm: daemon-init[1724], parent comm: systemd[1]
info|sysmonitor[881]: Fib4 replace table=254 192.168.122.0/24, comm: daemon-init[1724], parent comm: systemd[1]
info|sysmonitor[881]: Fib4 replace table=255 192.168.122.0/32, comm: daemon-init[1724], parent comm: systemd[1]
info|sysmonitor[881]: Fib6 replace fe80::5054:ff:fef6:b73e/128, comm: kworker/1:3[209], parent comm:  kthreadd[2]
```


HUAWEI

Drive Partition Monitoring (1)

- Introduction
  - The system periodically monitors the drive partitions mounted to the system. When the drive partition usage is greater than or equal to the configured alarm threshold, the system records a drive space alarm. When the drive partition usage falls below the configured alarm recovery threshold, a drive space recovery alarm is recorded.
- Configuration file
  - The configuration file is **/etc/sysmonitor/disk**. Example:

```
DISK="/var/log"  ALARM="90" RESUME="80"
DISK="/" ALARM="95" RESUME="85"
```


HUAWEI

Drive Partition Monitoring (2)

- Configuration item description

Configuration Item	Description	Mandatory	Default Value
DISK	Mount directory	Yes	None
ALARM	Integer indicating the drive space alarm threshold	No	90
RESUME	Integer indicating the drive space recovery threshold	No	80

HUAWEI

- After modifying the configuration file for drive space monitoring, run **systemctl reload sysmonitor**. The new configuration takes effect after a monitoring period.
- If a mount directory is configured repeatedly, the last configuration item takes effect.
- The value of **ALARM** must be greater than that of **RESUME**.
- Only the mount point or the drive partition of the mount point can be monitored.
- When the CPU usage and I/O usage are high, the **df** command execution may time out. As a result, the drive usage cannot be obtained.
- If a drive partition is mounted to multiple mount points, an alarm is reported for each mount point.

Drive Partition Monitoring (3)

- Exception logs
  - The drive space of the **/var/log** directory is monitored by default. The alarm threshold is 90%, and the recovery threshold is 80%.
  - If a drive space alarm is detected, information similar to the following is displayed in the **/var/log/sysmonitor.log** file:

    ```
    warning|sysmonitor[127]: report disk alarm, /var/log used:90% alarm:90%
    ```

  - If a drive space recovery alarm is detected, information similar to the following is displayed in the **/var/log/sysmonitor.log** file:

    ```
    info|sysmonitor[127]: report disk recovered, /var/log used:4% resume:10%
    ```


HUAWEI

Key Process Monitoring (1)

- Introduction
  - Key processes in the system are periodically monitored. When a key process exits abnormally, sysmonitor automatically attempts to recover the key process. If the recovery fails, alarms can be reported. The system administrator can be promptly notified of the abnormal process exit event and whether the process is restarted. Fault locating personnel can locate the time when the process exits abnormally from logs.
- Configuration file
  - The configuration file is **/etc/sysmonitor/process**. Each process or module corresponds to a configuration file. Example:

```
USER=root
NAME=irqbalance
RECOVER_COMMAND=systemctl restart irqbalance
MONITOR_COMMAND=systemctl status irqbalance
STOP_COMMAND=systemctl stop irqbalance
```


HUAWEI

- The permission on the configuration file needs to be changed to 600.

Key Process Monitoring (2)

Configuration Item	Description	Mandatory	Default Value
NAME	Process or module name	Yes	None
RECOVER_COMMAND	Recovery command	No	None
MONITOR_COMMAND	Monitoring command If the command output is 0, the process is normal. If the command output is greater than 0, the process is abnormal.	No	**pgrep -f** $(which *xxx*) *xxx* is the process name configured in the **NAME** field.
STOP_COMMAND	Stopping command	No	None
USER	User name User for executing the monitoring, recovery, and stopping commands or scripts	No	If this item is left blank, the **root** user is used by default.
CHECK_AS_PARAM	Parameter transfer switch If this item is **on**, the return value of **MONITOR_COMMAND** is transferred to the **RECOVER_COMMAND** command or script as an input parameter. If this item is set to **off** or other values, the function is disabled.	No	None
MONITOR_MODE	Monitoring mode The value can be **parallel** or **serial**.	No	serial
MONITOR_PERIOD	Monitoring period - Parallel monitoring period - This item does not take effect when the monitoring mode is **serial**.	No	3
USE_CMD_ALARM	Alarm mode If this item is set to **on** or **ON**, alarms are reported using the alarm reporting command.	No	None
ALARM_COMMAND	Alarm reporting command	No	None
ALARM_RECOVER_COMMAND	Alarm recovery command	No	No

HUAWEI

•

Key Process Monitoring (3)

- Exception logs

  sysmonitor monitors the irqbalance service by default.

  - **RECOVER_COMMAND** configured

    If a process or module exception is detected, information similar to the following is recorded in the **/var/log/sysmonitor.log** file:

    ```
    info|sysmonitor[127]: irqbalance is abnormal, check cmd return 1, use "systemctl restart irqbalance" to recover
    ```

    If the process or module recovers, information similar to the following is recorded in the **/var/log/sysmonitor.log** file:

    ```
    info|sysmonitor[127]: irqbalance is recovered
    ```

  - **RECOVER_COMMAND** not configured

    If a process or module exception is detected, information similar to the following is recorded in the **/var/log/sysmonitor.log** file:

    ```
    info|sysmonitor[127]: irqbalance is abnormal, check cmd return 1, recover cmd is null, will not recover
    ```

    If the process or module recovers, information similar to the following is recorded in the **/var/log/sysmonitor.log** file:

    ```
    info|sysmonitor[127]: irqbalance is recovered
    ```


HUAWEI

Custom Monitoring (1)

- Introduction

  Users can customize monitoring items. The monitoring framework reads the content of the configuration file, parses the monitoring attributes, and calls the monitoring actions to be performed. The monitoring module provides only the monitoring framework. It is not aware of what users are monitoring or how to monitor, and does not report alarms.

- Configuration files

  The configuration files are stored in **/etc/sysmonitor.d/**. Each process or module corresponds to a configuration file. Example:

```
MONITOR_SWITCH="on"
TYPE="periodic"
EXECSTART="/usr/sbin/iomonitor_daemon"
PERIOD="1800"
```


HUAWEI

Custom Monitoring (2)

- Configuration item description

Configuration Item	Description	Mandatory	Default Value
MONITOR_SWITCH	Monitoring switch	No	Off
TYPE	Custom monitoring item type **daemon**: background execution **periodic**: periodic execution	Yes	None
EXECSTART	Monitoring command	Yes	No
ENVIRONMENTFILE	Environment variable file	No	None
PERIOD	If the type is **periodic**, this item is mandatory and sets the monitoring period. The value is an integer greater than 0.	Yes when the type is **periodic**	None

- Exception logs
  - If a monitoring item of the daemon type exits abnormally, information similar to the following is recorded in the **/var/log/sysmonitor.log** file:

```
info|sysmonitor[127]: custom daemon monitor: child process[11609] name unetwork_alarm exit code[127],[1] times.
```


HUAWEI

- The absolute path of the configuration file or environment variable file cannot contain more than 127 characters. The environment variable file path cannot be a soft link path.
- The length of the **EXECSTART** command cannot exceed 159 characters. No space is allowed in the key field.
- The execution of the periodic monitoring command cannot time out. Otherwise, the custom monitoring framework will be affected.
- Currently, a maximum of 256 environment variables can be configured.
- Custom monitoring of the daemon type checks whether the reload command is delivered or whether the daemon process exits abnormally every 10 seconds. If the reload command is delivered, the new configuration is loaded 10 seconds later. If a daemon process exits abnormally, the daemon process is restarted 10 seconds later.
- If the content of the **ENVIRONMENTFILE** file changes, for example, an environment variable is added or the environment variable value changes, restart the sysmonitor service for the new environment variable to take effect.
- Users are advised to set the permission on the configuration files in the **/etc/sysmonitor.d/** directory to 600. If **EXECSTART** is only an executable file, users are advised to set the permission on the executable file to 550.
- After a daemon process exits abnormally, sysmonitor reloads the configuration file of the daemon process.

Quiz

1. (Single-answer question) Which is the default alarm threshold for drive partition monitoring?

   A. 80

   B. 90

   C. 100

   D. 60

HUAWEI

- Answer:
  - B 90

Summary

- This course described how to use and configure the sysmonitor service, and introduced the functions of monitoring NICs, drive partitions, key processes, and custom resources.

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



A-Ops Intelligent O&M Platform



Foreword

- Modern enterprise IT systems have grown in scale and complexity, and conventional, labor-intensive O&M has become obsolete. This course will introduce A-Ops, an intelligent O&M platform that implements quick troubleshooting and efficient O&M at lower costs.

Objectives

- On completion of this course, you will understand:
  - Essential functions of A-Ops
  - A-Ops deployment and host management methods
  - Vulnerability management based on A-Ops
  - Configuration tracing based on A-Ops

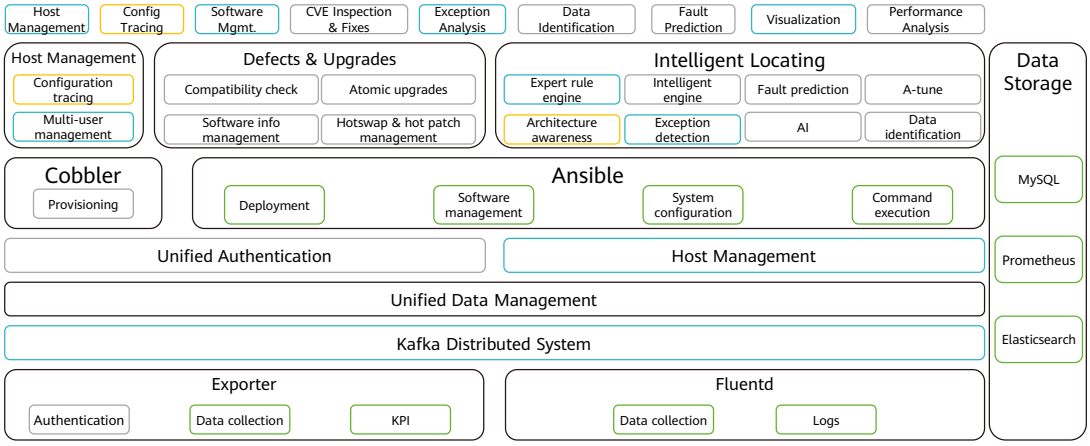
Contents

1. **Introduction to A-Ops**
2. Asset Management on A-Ops
3. Vulnerability Management on A-Ops
4. Configuration Tracing on A-Ops

HUAWEI

A-Ops Architecture

- A-Ops is an intelligent O&M tool that provides configuration tracing, architecture awareness, and the basic capabilities of fault locating. It also supports quick troubleshooting and reduces O&M costs.



HUAWEI

A-Ops Modules

Module	Description
aops-ceres	Client of the A-Ops service. Collects host data and manages other data collectors (such as gala-gopher). Responds to the commands delivered by the management center over SSH and processes the requirements and operations of the management center.
aops-zeus	A-Ops management center, which interacts with other modules. The default port number is 11111. Provides basic host management functions, such as adding and deleting hosts and host groups.
aops-hermes	Provides a visualized interface for A-Ops to display data information to users.
aops-apollo	Vulnerability management module of A-Ops. The default port number is 11116. Identifies clients, and periodically obtains security notices released by the openEuler community and updates them to the vulnerability database. Detects vulnerabilities in the system and software by comparing the vulnerabilities with those in the vulnerability database.
aops-vulcanus	Basic tool library of A-Ops. Except the aops-ceres and aops-hermes modules, other modules must be installed and used together with this module.
aops-tools	A-Ops database deployment script, which can be directly invoked to install MySQL and Elasticsearch.
gala-ragdoll	Configuration tracing module of A-Ops. Uses Git to monitor and record configuration file changes. The default port number is 11114.
dnf-hotpatch-plugin	DNF plugin. Enables the DNF tool to identify hot patch information and provide hot patch scanning and repair.

HUAWEI

- **gala-gopher** is an eBPF-based light-load probe framework. It provides a cloud observation engine in bare metal, VM, and container scenarios to accelerate service innovation.
- **gala-ragdoll** is an OS-level configuration management service. It provides a unified, traceable, and trusted OS configuration O&M portal to manage OS configurations in a cluster and masks configuration differences of various OS types.
- **aops-agent** is the A-Ops client that collects host data, responds to commands delivered by **aops-zeus** (original **aops-manager**) on the server, and manages A-Ops plugins. The repository has been renamed **aops-ceres** and migrated to a new repository.
- **aops-manager** is the basic service layer of A-Ops. It manages hosts and users and interacts with other A-Ops modules. This repository (renamed **aops-zeus** and migrated to a new repository) stores the architecture design document of the A-Ops project. You can refer to this document to learn more about the functions of this module in the A-Ops project.
- **aops-utils** is an A-Ops development tool package that encapsulates some common functions and methods. The repository has been renamed **aops-vulcanus** and migrated to a new repository.
- **cve-manager** is the vulnerability management module of A-Ops. It manages cold and hot patches for managed hosts, including vulnerability inspection, fixing, and rollback. The repository has been renamed **aops-apollo** and migrated to a new repository.
- **aops-web** is the web service of A-Ops that provides a visualized user interface (UI). The repository has been renamed **aops-hermes** and migrated to a new repository.
- **aops-tools** is an auxiliary script repository of A-Ops. It provides one-click deployment scripts for some services including MySQL and Elasticsearch. This

repository has been merged into **aops-vulcanus**.

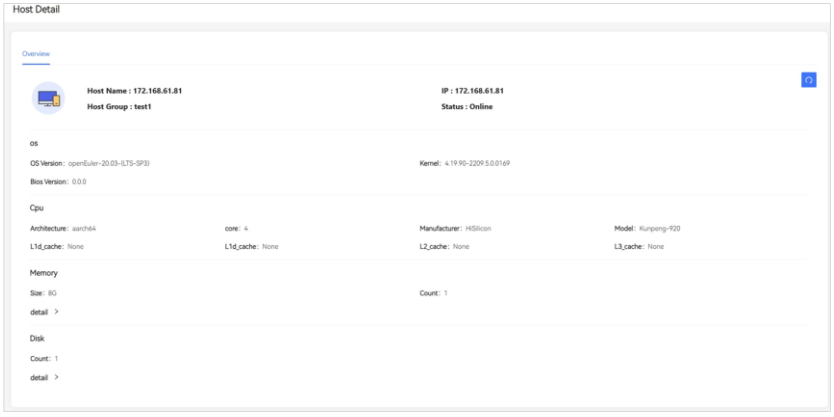
Contents

1. Introduction to A-Ops
2. **Asset Management on A-Ops**
3. Vulnerability Management on A-Ops
4. Configuration Tracing on A-Ops

HUAWEI

A-Ops Asset Management

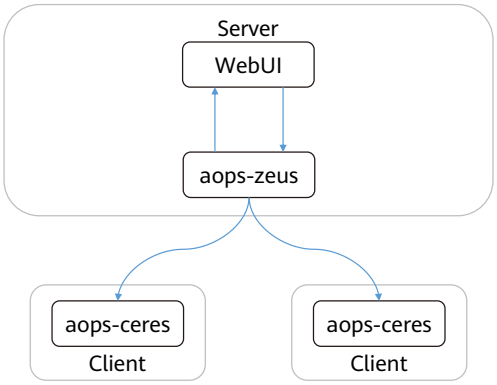
- A-Ops uses the aops-ceres module to collect host data, manage other data collectors, and manage information assets on hosts.



HUAWEI

- Data collectors include gala-gopher.

Logical Architecture of A-Ops Asset Management



HUAWEI

A-Ops Deployment – Server Configuration (1)

- Three hosts are used: two act as servers and the other as a client managed by A-Ops. In addition, the update repository needs to be configured in the repository configuration.
- Deployment:
  - Host A: Deploy basic services including MySQL, Redis, and Elasticsearch as well as aops-zeus and aops-hermes. Recommended memory: > 8 GB
  - Host B: Deploy the aops-apollo service to provide vulnerability management capabilities. Recommended memory: > 6 GB
  - Host C: Deploy the aops-ceres service so that the host can be managed by A-Ops.

HUAWEI

A-Ops Deployment – Server Configuration (2)

- To use the asset management function, deploy the aops-zeus, aops-hermes, MySQL, and Redis services.
  - **Deploy MySQL on host A.** Under the **mysql** section in the MySQL configuration file **/etc/my.cnf**, add **bind-address** and set it to the local IP address, and restart MySQL.

```
[mysqld]
bind-address=192.168.1.1
log_timestamps=SYSTEM  # Ensure that the MySQL log time is the system time.
```

  - Set the access permission of the root user of the MySQL database.

```
[root@localhost ~] mysql
mysql> show databases;
mysql> use mysql;
mysql> select user,host from user;
+---------------+-----------+
| user          | host      |
+---------------+-----------+
| root          | localhost |
| mysql.session | localhost |
| mysql.sys     | localhost |
+---------------+-----------+
3 rows in set (0.00 sec)
mysql> update user set host = '%' where user='root';
mysql> flush privileges;
mysql> create database aops default character set utf8mb4 collate utf8mb4_unicode_ci;  // Create the aops database.
mysql> exit
```

The value localhost indicates that only the local host can connect to the MySQL database. The external network and local software client cannot connect to the MySQL database.

Allow the root user to access the database using any IP address.

HUAWEI

- Before deployment, disable the firewall and SELinux on each host.
- Install MySQL: **yum install mysql-server -y**

A-Ops Deployment – Server Configuration (3)

- **Deploy Redis on host A.** In the **/etc/redis.conf** configuration file, bind the local IP address and then start the Redis service.

```
bind 127.0.0.1 192.168.1.1
```

- **Deploy aops-zeus on host A.** In the **/etc/aops/zeus.ini** configuration file, change the service addresses to the actual ones and then start aops-zeus.

```
[zeus]
ip=192.168.1.1 // Change the IP address to that of the aops-zeus host.
port=11111

[uwsgi]
wsgi-file=manage.py
daemonize=/var/log/aops/uwsgi/zeus.log
http-timeout=600
harakiri=600
processes=2                    // Generate a specified number of workers or processes.
gevent=100                     // Number of gevent asynchronous cores

[mysql]
ip=192.168.1.1  // Change the IP address to that of the MySQL host.
port=3306
database_name=aops
engine_format=mysql+pymysql://@%s:%s/%s
pool_size=100
pool_recycle=7200
```

```
[agent]
default_instance_port=8888

[redis]
ip=192.168.1.1  // Change the IP address to that of the Redis host.
port=6379

[apollo]
ip=192.168.1.2  // Change the IP address to that of the aops-apollo host. It is recommended that aops-apollo and aops-zeus be deployed separately. If vulnerability management is not used, you do not need to configure this item.
port=11116
```


HUAWEI

- Install aops-zeus: **yum install aops-zeus -y**

A-Ops Deployment – Server Configuration (4)

- Initialize the aops-zeus database.

```
cd /opt/aops/scripts/deploy
bash aops-basedatabase.sh init zeus
```

- **Deploy aops-hermes on host A.** Modify the **/etc/nginx/aops-nginx.conf** configuration file as follows, and start the apos-hermes service.

```
location / {
    try_files $uri $uri/ /index.html;
    if (!-e $request_filename){
        rewrite ^(.*)$ /index.html last;
    }
}
location /api/ {
    proxy_pass http://192.168.1.1:11111/;
}
  location /api/domain {
    proxy_pass http://192.168.1.1:11114/;
    rewrite ^/api/(.*) /$1 break;
}

location /api/vulnerability {
    proxy_pass http://192.168.1.1:11116/;
    rewrite ^/api/(.*) /$1 break;
}
```

Ensure that Nginx still uses index.html as the entry when the front-end route changes.

Change the IP address to that of the aops-zeus host.

Change the IP address to that of the gala-ragdoll host.

Change the IP address to that of the aops-apollo host.

HUAWEI

- yum install aops-hermes

Deploying aops-ceres (1)

- As the client of A-Ops, aops-ceres exchanges data with the A-Ops management center over HTTP and collects host information, manages tools (such as gala-gopher), and responds to and processes commands delivered by the management center.
  - Deploy aops-ceres by modifying the **/etc/aops/ceres.conf** configuration file.

```
[gopher]
;Default path to the gala-gopher configuration file. If you need to change the path, ensure that the file path is correct.
config_path=/etc/gala-gopher/gala-gopher.conf

;aops-ceres log collection configuration
[log]
;Level of the logs to be collected, which can be set to DEBUG, INFO, WARNING, ERROR, or CRITICAL
log_level=INFO
;Location for storing collected logs
log_dir=/var/log/aops
;Maximum size of a log file
max_bytes=31457280
;Number of backup logs
backup_count=40
```


HUAWEI

Deploying aops-ceres (2)

- Register with aops-zeus. To identify users and prevent APIs from being invoked randomly, aops-ceres uses tokens to authenticate users, reducing the pressure on the deployed hosts.
  - Change the values of the data items to the actual values by referring to the **/opt/aops/register_example.json** template file.

```
{

  "ssh_user": "root", // User name for remotely logging in to the local host
  "password": "password", // User password
  "ssh_port":22,  // Remote login port
  "zeus_ip": "192.168.1.1", // IP address of the host where aops-zeus is running
  "zeus_port": 11111, // aops-zeus port
  "host_name": "host_name", // Host name to be registered
  "host_group_name": "aops", // Existing host group name
  "management": false, // Whether to register as a management host
  "access_token": "token-string" // Management user token obtained after login
}
```

```JSON
{

  "ssh_user": "root", // User name for remotely logging in to the local host
  "password": "password", // User password
  "ssh_port":22,  // Remote login port
  "zeus_ip": "192.168.1.1", // IP address of the host where aops-zeus is running
  "zeus_port": 11111, // aops-zeus port
  "host_name": "host_name", // Host name to be registered
  "host_group_name": "aops", // Existing host group name
  "management": false, // Whether to register as a management host
  "access_token": "token-string" // Management user token obtained after login
}
```

Note: Ensure that aops-zeus is running on the target host, for example, 192.168.1.1, and the registered host group exists.

HUAWEI

- Before the registration, obtain the token from the management user on aops-zeus and run the **register** command to register the token with aops-zeus. Then, the registration success result is displayed on the GUI. In addition, save the local host information to the aops-zeus database.

Deploying aops-ceres (3)

- Run: **aops-ceres register** [**-f** *<FILE>*] [**-d** *<register_host_info>*]
- The registration result is displayed on the GUI. If the registration is successful, the token character string is saved to a specified file. If the registration fails, locate the fault based on the message and log (**/var/log/aops/aops.log**). Example:

Registration succeeded:

```
[root@localhost ~]# aops-ceres register -f register.json
Register Success
```

Registration failed. The following shows an aops-zeus start failure.

```
[root@localhost ~]# aops-ceres register -f register.json
Register Fail
[root@localhost ~]#
```

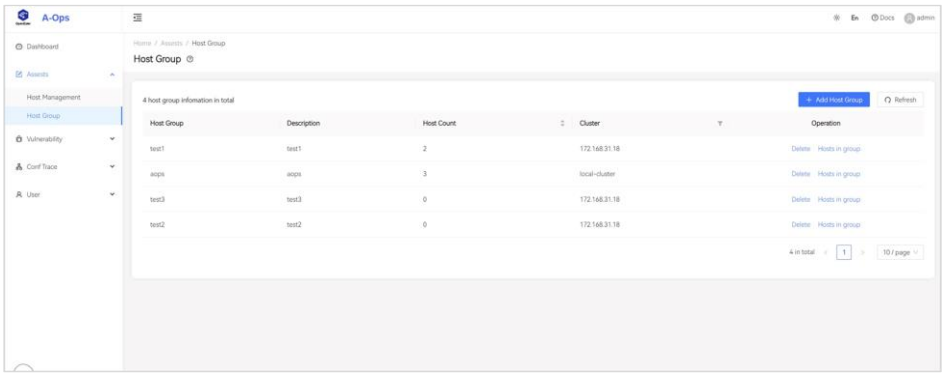
Log content:

```
2022-09-05 16:11:52,576 ERROR command_manage/register/331: HTTPConnectionPool(host='192.168.1.2', port=11111): Max retries exceeded with url: /manage/host/add (Caused by NewConnectionError('<urllib3.connection.HTTPConnection object at 0x7ff0504ce4f0>: Failed to establish a new connection: [Errno 111] Connection refused'))
```


HUAWEI

Asset Management

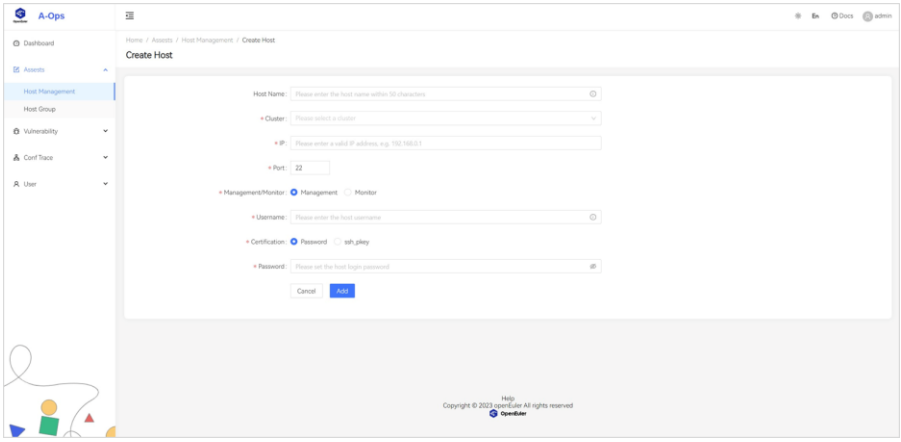
- Asset management includes host group management and host management. Before registering a host on the frontend, add it into a host group for batch management.



HUAWEI

Adding a Host

- You can use the host password or key for authentication when adding a host.

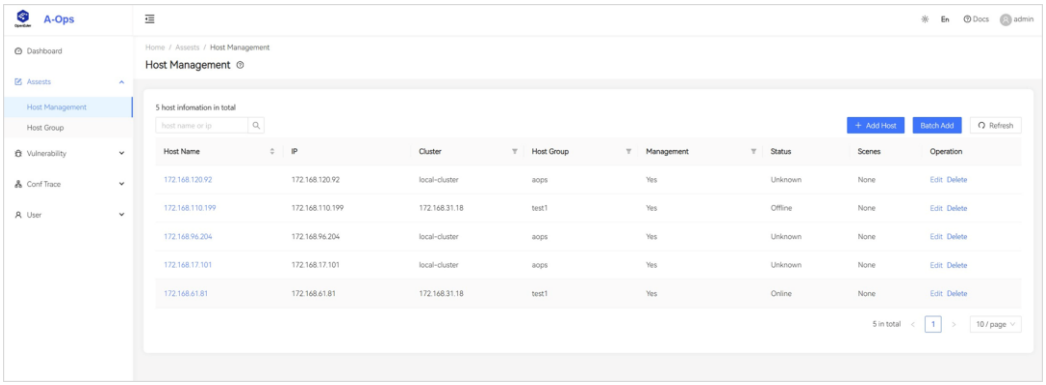


HUAWEI

- The following operations are supported:
  - Adding host groups
  - Deleting host groups
  - Viewing all host groups
  - Viewing information about hosts in each host group
- There is no essential difference between management nodes and monitoring nodes.

Host List

- The managed nodes are displayed in the host list.



HUAWEI

Contents

1. Introduction to A-Ops
2. Asset Management on A-Ops
3. **Vulnerability Management on A-Ops**
4. Configuration Tracing on A-Ops

HUAWEI

Vulnerability Management Overview

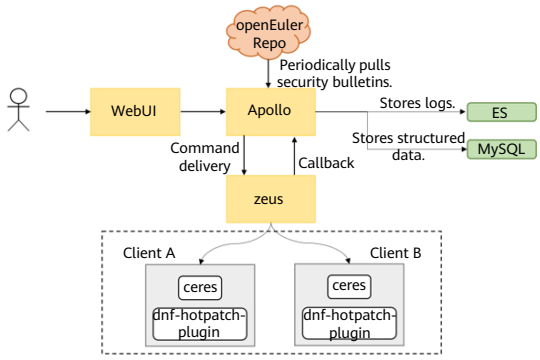
- Pain points in fixing cluster OS vulnerabilities:
  - O&M personnel cannot detect released vulnerabilities in a timely manner.
  - The cluster environment cannot detect the health status of all hosts.
  - Services are interrupted during kernel upgrade using cold patches.
  - Cluster operations are error-prone.
- The aops-apollo service and hotpatch plugin are developed against the preceding pain points.
  - dnf-hotpatch-plugin: single-node vulnerability scanning, hot patching
  - aops-apollo: batch scanning, cold and hot patch repair

HUAWEI

-

Vulnerability Management Framework

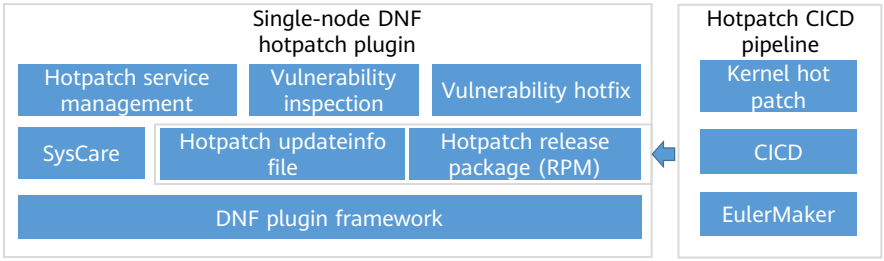
- Server design:
  - aops-hermes (WebUI) is responsible for front-end interaction.
  - aops-apollo implements the main logic of vulnerability management.
  - aops-zeus interacts with clients through SSH.
  - MySQL and Elasticsearch databases store service information
- Client design:
  - aops-ceres executes client commands.
  - dnf-hotpatch-plugin scans and fixes vulnerabilities on a single node.



HUAWEI

Single-Node Hotpatch Plugin

- Based on the DNF plugin framework
- Obtains hot patch release packages based on detected CVEs.
- Fixes kernel vulnerabilities without restarting processes or the host.
- Provides single-node inspection, local vulnerability fixing, and hotpatch service management.



HUAWEI

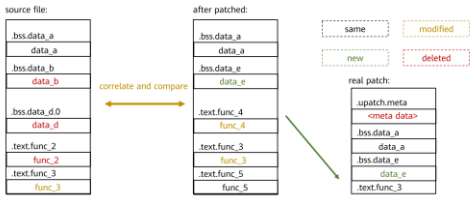
- DNF is a Linux software package manager that manages RPM packages. DNF can query software package information, obtain required software packages from a specified software repository, install and uninstall the software packages by automatically processing dependencies, and update the system to the latest available version.
- After installing dnf-hotpatch-plugin, run **dnf** commands to use aops-apollo functions related to hot/cold patches, such as hot patch scanning (**dnf hot-updateinfo**), setting and querying (**dnf hotpatch**), and applying (**dnf hotupgrade**).

SysCare: Unified Environment for Kernel- and User-space Hot Patches

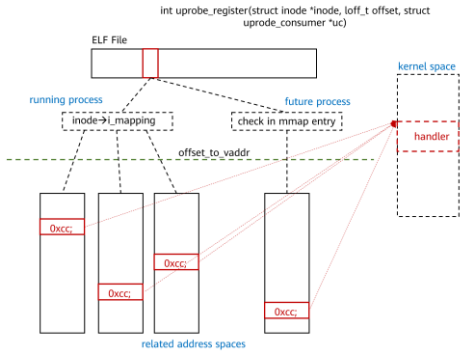
- **Service management**: patch state machine transition and patch rollback



- **Patch creation**: RPM package creation and



- **Binding patches to ELF files**: uprobe breakpoint creation in inode->i_mapping of ELF files based on the **uprobe mechanism**



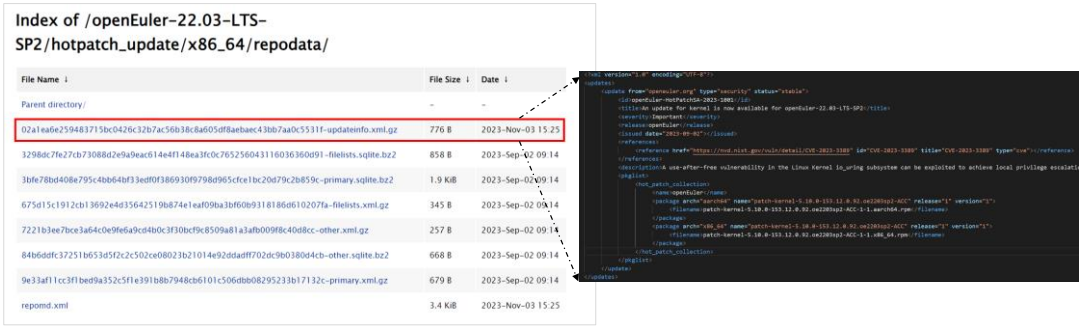
- **Patch validation**: adds a uprobe breakpoint at a function; calls the uprobe handler when the function is running; maps the patch to the address space of the current process

HUAWEI

- SysCare is a system-level hot patch service that unifies kernel- and user-space hot patching. It provides online hot repair for OSs to automatically repair bugs and vulnerabilities of base system components, such as kernels, user-space services, and dynamic libraries.
- Based on source code of system components and problems, you can create RPM patch packages that contain the patch file, dependency information, and configuration information. The RPM patch packages can be uploaded to the patch repository. The cluster daemon periodically queries the patch repository and performs hot fixes on CVEs and software errors in the system, ensuring a secure, stable, and efficient system.
- Target files take different naming formats according to the system or platform. In Unix and x86_64 Linux, the Executable and Linkable Format (ELF) is used. The ELF format provides two perspectives. For the assembler and linker, an ELF file is a set of sections described by the Section Header Table. For the loader, an ELF file is a set of segments described by the Program Header Table.
- SysCare provides patch creation, activation, and uninstallation operations in addition to letting you create and manage kernel- and user-space hot patches.
  - SysCare provides one-click patch creation. It unifies the process of making kernel- and user-space hot patches, and masks patch creation details and differences between kernel- and user-space patches.
  - SysCare also provides unified patch management interfaces for users to install, activate, uninstall, and query patches.
- User-space probes (uprobes) can dynamically collect debugging and performance data of functions of applications without causing interruption.

Single-Node Hotpatch Plugin – Implementation Logic (1)

(1) Obtain the **updateinfo** file from the hotpatch repository of the openEuler community.



HUAWEI

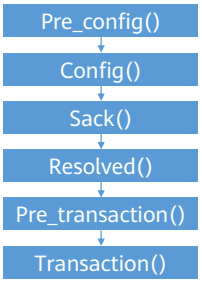
- openEuler community hot patches:
- Hot patch creation: https://gitee.com/openeuler/hotpatch_meta
- Hot patch release repository: https://repo.openeuler.org/

Single-Node Hotpatch Plugin – Implementation Logic (2)

(2) Parse the **updateinfo** file.



Associate a CVE and hot patch package based on the **updateinfo** file.



Interface invoking sequence in the DNF plugin framework

HUAWEI

Using the Single-Node Hotpatch Plugin (1)

- **Perform CVE inspection.**

```
[root@openEuler tmp]# dnf hot-updateinfo list cves
Last metadata expiration check: 0:00:11 ago
CVE-2023-3389 Important/Sec. kernel-5.10.0-153.18.0.94.oe2203sp2.x86_64 patch-kernel-5.10.0-153.12.0.92.oe2203sp2-ACC-1-1.x86_64
```

- **Hot fix the CVE.**

```
[root@openEuler tmp]# dnf hotupgrade --cve CVE-2023-3389
Last metadata expiration check: 0:00:23 ago
Dependencies resolved.
================================================================================
 Package                  Architecture   Version      Repository          Size
================================================================================
Installing:
 patch-kernel-5.10.0-153.12.0.92.oe2203sp2-ACC
                          x86_64         1-1          hotpatch_update     13 k

Transaction Summary
================================================================================
Install  1 Package

Total download size: 13 k
Installed size: 25 k
Is this ok [y/N]: y
Downloading Packages:
patch-kernel-5.10.0-153.12.0.92.oe2203sp2-ACC-1-1.x86_64.rpm  92 kB/s |  13 kB     00:00
--------------------------------------------------------------------------------
Total                                                          44 kB/s |  13 kB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Installing       : patch-kernel-5.10.0-153.12.0.92.oe2203sp2-ACC-1-1.x86_64 1/1
  Running scriptlet: patch-kernel-5.10.0-153.12.0.92.oe2203sp2-ACC-1-1.x86_64 1/1
  Verifying        : patch-kernel-5.10.0-153.12.0.92.oe2203sp2-ACC-1-1.x86_64 1/1

Installed:
  patch-kernel-5.10.0-153.12.0.92.oe2203sp2-ACC-1-1.x86_64

Complete!
Apply hot patch succeed: kernel-5.10.0-153.12.0.92.oe2203sp2/ACC-1-1.
```


HUAWEI

Using the Single-Node Hotpatch Plugin (2)

- **Inspect fixed CVEs.**

```
[root@openEuler tmp]# dnf hot-updateinfo list cves --installed
Last metadata expiration check: 0:04:50 ago
CVE-2023-3389 Important/Sec. - patch-kernel-5.10.0-153.12.0.92.oe2203sp2-ACC-1-1.x86_64
```

- **Inspect unfixed CVEs.**

```
[root@openEuler tmp]# dnf hot-updateinfo list cves
Last metadata expiration check: 0:03:29 ago
```

- **Check the status of the CVE hot patch.**

```
[root@openEuler tmp]# dnf hotpatch --list cves
Last metadata expiration check: 0:00:54 ago
CVE-id        base-pkg/hotpatch                                  status
CVE-2023-3389 kernel-5.10.0-153.12.0.92.oe2203sp2/ACC-1-1/vmlinux ACTIVED
```

- **Accept the hot patch so that it takes effect after the system restarts.**

```
[root@openEuler tmp]# dnf hotpatch --accept kernel-5.10.0-153.12.0.92.oe2203sp2/ACC-1-1/vmlinux
Last metadata expiration check: 0:02:08 ago
Gonna accept this hot patch: kernel-5.10.0-153.12.0.92.oe2203sp2/ACC-1-1/vmlinux
accept hot patch 'kernel-5.10.0-153.12.0.92.oe2203sp2/ACC-1-1/vmlinux' succeed
```

- **Check the status of the CVE hot patch again.**

```
[root@openEuler tmp]# dnf hotpatch --list cves
Last metadata expiration check: 0:02:33 ago
CVE-id        base-pkg/hotpatch                                  status
CVE-2023-3389 kernel-5.10.0-153.12.0.92.oe2203sp2/ACC-1-1/vmlinux ACCEPTED
```


HUAWEI

Vulnerability Management Deployment Overview

- Deployment:
  - Host A: Deploy basic services including MySQL, Redis, and Elasticsearch as well as aops-zeus and aops-hermes. Recommended memory: > 8 GB
  - Host B: Deploy the aops-apollo service to provide vulnerability management capabilities. Recommended memory: > 6 GB
  - Host C: Deploy the aops-ceres service so that this host can be managed by A-Ops.
- Required package:
  - Install the update package (matching the OS version) that includes full feature updates and optimizations.
- Configuration modification:
  - Modify the configurations of basic services such as MySQL and services such as aops-zeus and aops-apollo. Ensure that the IP addresses are correctly configured for mutual access.
- Starting services

HUAWEI

- For details, see "Deploying A-Ops" in the openEuler community.

Deploying Vulnerability Management (1)

- The CVE management module is implemented based on the asset management module. Before deploying the CVE management module, deploy the asset management module and then deploy aops-apollo.
- The aops-apollo service requires MySQL, Elasticsearch, and Redis databases.
  - Add the Elasticsearch repository.

```
echo "[aops_elasticsearch]
name=Elasticsearch repository for 7.x packages
baseurl=https://artifacts.elastic.co/packages/7.x/yum
gpgcheck=1
gpgkey=https://artifacts.elastic.co/GPG-KEY-elasticsearch
enabled=1
autorefresh=1
type=rpm-md" > "/etc/yum.repos.d/aops_elascticsearch.repo"
```

  - **Deploy Elasticsearch on host A.** Modify the **/etc/elasticsearch/elasticsearch.yml** configuration file and restart the Elasticsearch service.

```
node.name: node-1
network.host: 192.168.1.1
http.port: 9200
cluster.initial_master_nodes: ["node-1"]
http.cors.enabled: true
http.cors.allow-origin: "*"
```


HUAWEI

- yum install elasticsearch-7.14.0-1

Deploying Vulnerability Management (2)

- **Deploy aops-apollo on host B.** In the **/etc/aops/apollo.ini** configuration file, change the service addresses to the actual ones and then start aops-apollo.

```
[apollo]
ip=192.168.1.2// Change the IP address to that of the aops-apollo host.
port=11116
host_vault_dir=/opt/aops
host_vars=/opt/aops/host_vars

[zeus]
ip=192.168.1.1              // Change the IP address to that of the aops-zeus host.
port=11111

[hermes]
ip=192.168.1.1              // Change the IP address to that of the aops-hermes host,
for example, host A.
port=80                     // Change it to the aops-hermes service port.

[cve]
cve_fix_function=yum
# value between 0-23, for example, 2 means 2:00 in a day.
cve_scan_time=2

[mysql]
ip=192.168.1.1 // Change it to the actual IP address of host A.
port=3306
database_name=aops
engine_format=mysql+pymysql://@%s:%s/%s
pool_size=100
pool_recycle=7200

[elasticsearch]
ip=192.168.1.1 // Change it to the actual IP address of host A.
port=9200
max_es_query_num=10000000

[redis]
ip=192.168.1.1 // Change it to the actual IP address of host A.
port=6379

[uwsgi]
wsgi-file=manage.py
daemonize=/var/log/aops/uwsgi/apollo.log
http-timeout=600
harakiri=600
processes=2
gevent=100
```


HUAWEI

- yum install aops-apollo

Deploying Vulnerability Management (3)

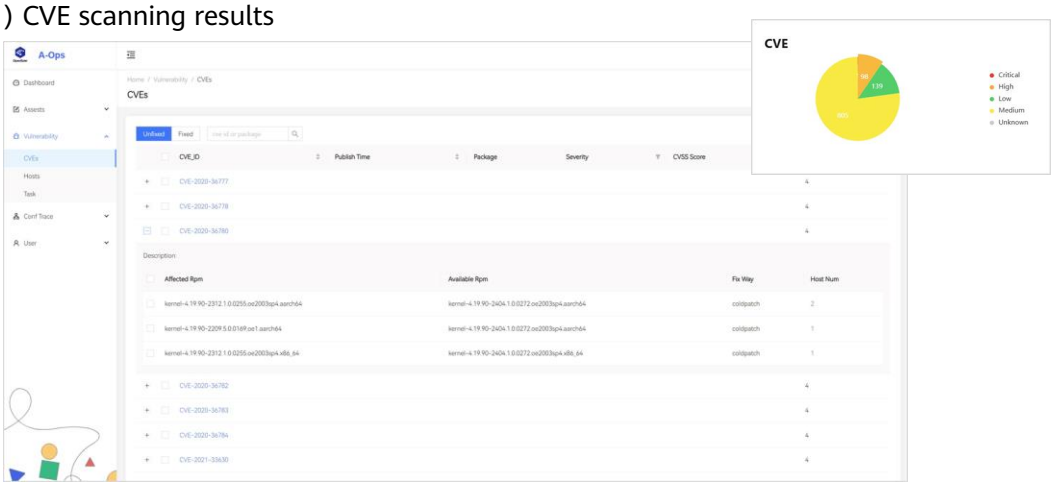
- Initialize the aops-apollo database.

```
cd /opt/aops/scripts/deploy
bash aops-basedatabase.sh init apollo
```


HUAWEI

Vulnerability Management Example (1)

(1) CVE scanning results

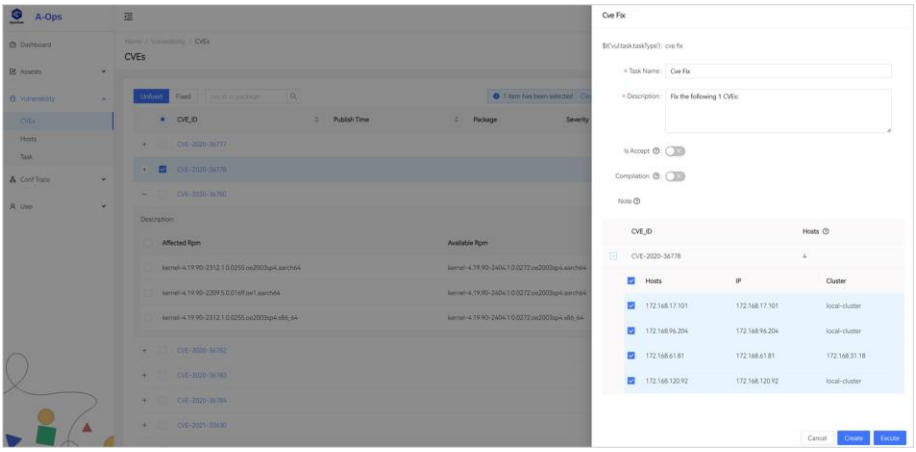


HUAWEI

- The vulnerability management module consists of three pages:
  - **CVEs**: query basic info about hosts and the number of CVEs (filtered by host group or Repo name, sorted by the number of CVEs and last scan time, and queried by host name), last CVE scan time on the host, and saved repositories; add and delete repositories; scan host vulnerabilities; display host details by clicking a host name; and export current data
  - **Hosts**
  - **Tasks**

Vulnerability Management Example (2)

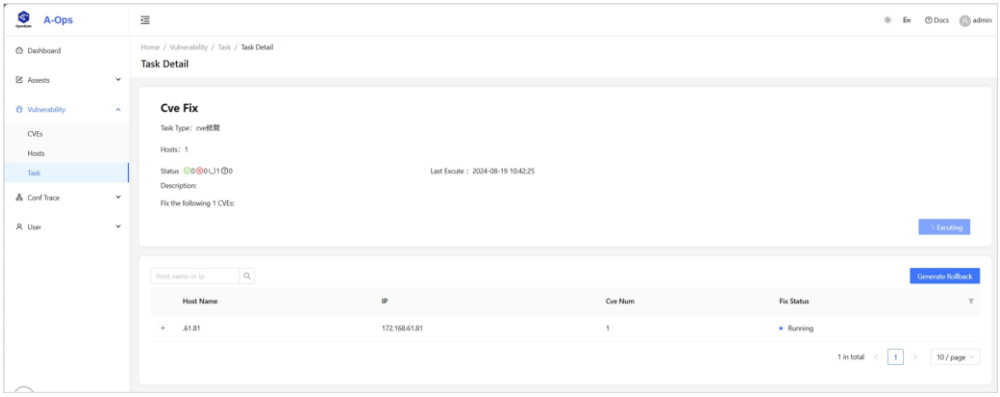
(2) Select CVEs and software packages to be fixed.



HUAWEI

Vulnerability Management Example (3)

(3) CVE fixing process



HUAWEI

Contents

1. Introduction to A-Ops
2. Asset Management on A-Ops
3. Vulnerability Management on A-Ops
4. **Configuration Tracing on A-Ops**

HUAWEI

Introduction to Configuration Tracing

Scenarios

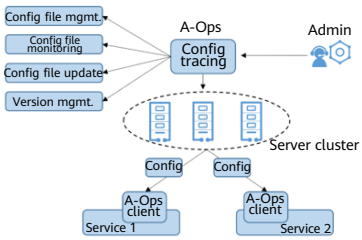
- Low reliability and error-prone of manual operations
- No historical modification records for configuration tracing
- Unable to locate faults caused by configuration changes quickly and accurately

Benefits

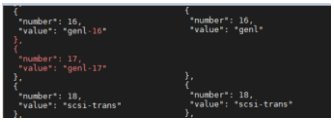
- Baseline management capabilities for cluster hosts configuration in a unified and scenario-based manner.
- A unified portal for changing configurations, ensuring all change operations are trusted and reliable.
- Cluster configuration changes are achieved in minutes.
- Configuration files are managed by version, and historical changes are recorded for configuration tracing and quick fault location.
- Lower labor costs and improved efficiency of locating configuration problems.

Solution

Configuration management:



Configuration difference comparison:



HUAWEI

- The configuration tracing of A-Ops is used to detect and record changes in the configuration file of the target host, which provides support for locating faults caused by incorrect file configurations.

Deploying Configuration Tracing

- A-Ops configuration tracing depends on the host management module and the gala-ragdoll service. Before deploying gala-ragdoll, ensure that host management has been properly configured.
  - **Deploy gala-ragdoll.** In the **/etc/ragdoll/gala-ragdoll.conf** configuration file, change the IP address of **collect_address** in the **collect** section to the IP address of host B, **collect_api** to the actual API, and **collect_port** to the actual port. Then start the gala-ragdoll service.

```
[git]
git_dir = "/home/confTraceTest"
user_name = "user_name"
user_email = "user_email"

[collect]
collect_address = "http://192.168.1.X"    // Change the IP address to that of the gala-ragdoll host.
collect_api = "/manage/config/collect"    // The value is an example. Change it to the actual API.
collect_port = 11111                      // Change it to the aops-zeus service port.

[sync]
sync_address = "http://192.168.1.X"
sync_api = "/demo/syncConf"
sync_port = 11114

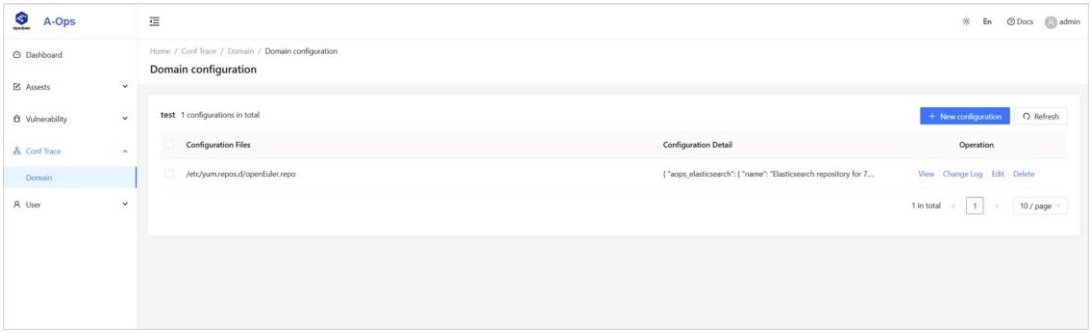
[ragdoll]
port = 11114
```


HUAWEI

- yum install gala-ragdoll python3-gala-ragdoll

Configuration Tracing Example (1)

(1) Set the configuration baseline of a domain.

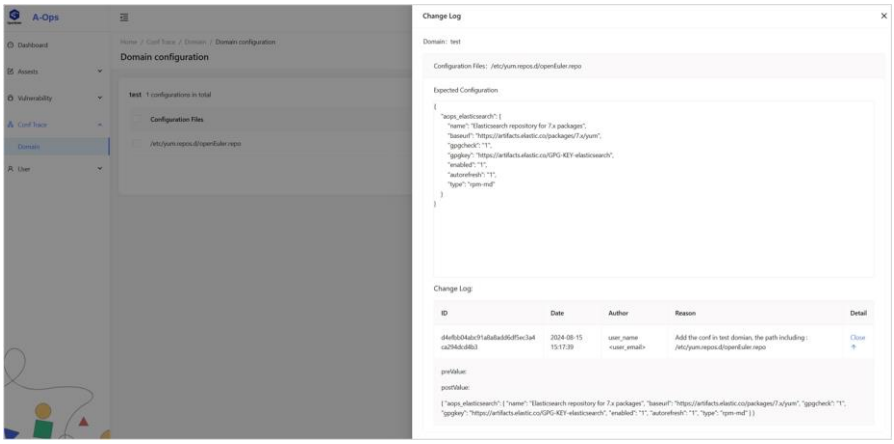


HUAWEI

- Available functions: adding domains, adding nodes to a domain, adding configurations in a domain, querying expected/current configurations, deleting configurations, and verifying configurations.

Configuration Tracing Example (2)

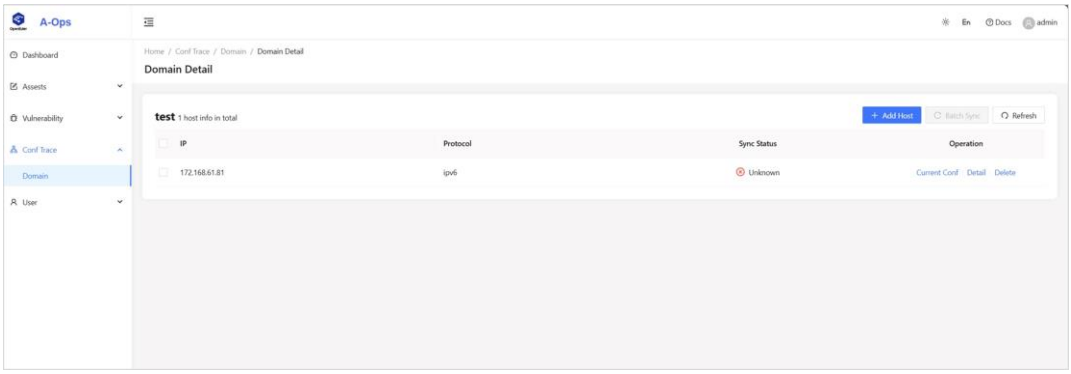
(2) Query baseline configuration change records.



HUAWEI

Configuration Tracing Example (3)

(3) Check service configuration synchronization.



HUAWEI

Quiz

1. (Single-answer question) Which of the following underlying technologies is used by the A-Ops client to apply and manage hot patches and implement the hotpatch repair capability of the A-Ops vulnerability management module?
   A. kpatch
   B. Upatch
   C. SysCare
   D. kGraft

HUAWEI

- Answer:
- C

Summary

- This course introduced the basic O&M methods of A-Ops and principles of vulnerability management and configuration tracing, helping trainees perform O&M on services.

HUAWEI

Acronyms and Abbreviations

- UI: user interface on which users interact with the system.
- CVE: Common Vulnerabilities and Exposures. It is a program that assigns unique identifiers to publicly disclosed vulnerabilities.

HUAWEI

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

